Nullexit: Unified Project Document

Omar Alaaeldein

July 12, 2026

Contents

1 Project Directory Tree 2

2 README.md 4

3 agent.md 10

4 devref.md 17

5 diagrams.md 62

6 toggle.sh 67

7 recover.sh 84

8 setup.sh 92

9 scripts/toggle-linux.sh 93

10 scripts/recover-linux.sh 105

11 scripts/setup-linux.sh 108

12 docker-compose.yml 109

13 scripts/common.sh 112

14 scripts/watcher.sh 119

15 scripts/crypto.sh 123

16 .aiignore 123

17 .gitignore 124

18 .host_ips 124

19 Recover-Gateway.applescript 124

20 Toggle-Gateway.applescript 125

21 benchmarks/benchmark.sh 125

22 benchmarks/test_load.py 126

23 black_list.txt **127**

24 cloud-init.yaml **128**

25 docker/tor/Dockerfile **129**

26 docker/tor/entrypoint.sh **129**

27 launchd/com.nullexit.wake-recovery.plist **130**

28 post-rules.txt **131**

29 scripts/Dockerfile.routing-fix **131**

30 scripts/Dockerfile.rule-compiler **131**

31 scripts/Toggle-Gateway.bat **132**

32 scripts/check_circuits.py **134**

33 scripts/diagnose-host-leak.sh **135**

34 scripts/dns-proxy.py **145**

35 scripts/fix-docker-bridge-collision.sh **146**

36 scripts/fix-ssh-delay.sh **151**

37 scripts/generate_tex.py **152**

38 scripts/logger/go.mod **154**

39 scripts/logger/main.go **154**

40 scripts/pf.conf **158**

41 scripts/reinstall-host-tailscale.sh **158**

42 scripts/routing-fix.sh **168**

43 scripts/rule-compiler/go.mod **172**

44 scripts/rule-compiler/main.go **172**

45 scripts/setup-common.sh **182**

46 scripts/unlock-files.sh **188**

47 tor-bridges.txt **189**

48 white_list.txt **189**

1 Project Directory Tree

```
nullexit/
|-- .aiignore
|-- .gitignore
|-- .host_ips
|-- LICENSE
|-- README.md
|-- Recover-Gateway.applescript
|-- Toggle-Gateway.applescript
|-- agent.md
|-- benchmarks
|   |-- benchmark.sh
|   `-- test_load.py
|-- black_list.txt
|-- cloud-init.yaml
|-- devref.md
|-- diagrams.md
|-- docker
|   `-- tor
|       |-- Dockerfile
|       `-- entrypoint.sh
|-- docker-compose.yml
|-- launchd
|   `-- com.nullexit.wake-recovery.plist
|-- post-rules.txt
|-- recover.sh
|-- scripts
|   |-- Dockerfile.routing-fix
|   |-- Dockerfile.rule-compiler
|   |-- Toggle-Gateway.bat
|   |-- check_circuits.py
|   |-- common.sh
|   |-- crypto.sh
|   |-- diagnose-host-leak.sh
|   |-- dns-proxy.py
|   |-- fix-docker-bridge-collision.sh
|   |-- fix-ssh-delay.sh
|   |-- generate_tex.py
|   |-- logger
|   |   |-- go.mod
|   |   `-- main.go
|   |-- pf.conf
|   |-- recover-linux.sh
|   |-- reinstall-host-tailscale.sh
|   |-- routing-fix.sh
|   |-- rule-compiler
|   |   |-- go.mod
|   |   `-- main.go
|   |-- setup-common.sh
|   |-- setup-linux.sh
|   |-- toggle-linux.sh
|   |-- unlock-files.sh
|   `-- watcher.sh
```

```
|-- setup.sh
|-- toggle.sh
|-- tor-bridges.txt
`-- white_list.txt
```

2 README.md

```
# nullexit: Tailscale + Cloudflare WARP Docker Gateway

> **Last updated:** July 11, 2026  [Architecture & Flow Diagrams ](./diagrams.md)  [Full Dev Reference ](./devref.md)

**nullexit** is a chained network gateway that routes all Tailscale exit-node traffic through a Cloudflare WARP VPN tunnel  double-encrypting every packet, hiding your ISP metadata, and providing network-wide DNS ad-blocking (AdGuard Home) and kernel-level IP threat blocking (‘ipset‘/‘iptables‘) for every device on your mesh.

---

## 1. Prerequisites
- Docker and Docker Compose installed.
- A Tailscale account.
- **macOS:** Install the standalone Tailscale CLI via Homebrew (‘brew install tailscale‘) and register ‘tailscaled‘ as a system service (‘brew services start tailscale‘). No ‘.app‘ GUI install required.
- **OS & Python:** Developed and tested against **macOS 26.5.2**. The project has zero external Python dependencies (no ‘requirements.txt‘) and explicitly targets the default Apple-provided **Python 3.9.6** (‘/usr/bin/python3‘). While the scripts could function on newer versions, ‘nullexit‘ explicitly avoids experimenting with or modifying the built-in system Python environment to prevent unexpected OS-level side effects.

## 2. Installation & Setup

Run the interactive setup script for your OS. It downloads ‘wgcf‘, generates fresh Cloudflare WARP WireGuard keys, prompts for a Tailscale Auth Key, and writes your ‘.env‘.

‘‘‘bash
./setup.sh           # macOS
./scripts/setup-linux.sh  # Linux
‘‘‘

### Reference ‘.env‘
‘‘‘env
WIREGUARD_PRIVATE_KEY=<Generated>
WIREGUARD_PUBLIC_KEY=<Generated>
WIREGUARD_ADDRESSES=<Generated>
TS_AUTHKEY=tskey-auth-...
GATEWAY_RULE_PROFILE=medium        # light | medium | heavy
# Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds and have a healthy path.
GATEWAY_MSS=1120
# Set to false to run as a ’Headless Server’ (Docker acts as an exit node, but the host Mac/Linux’s own internet is NOT hijacked).
GATEWAY_HIJACK_HOST=true
GATEWAY_USE_EXIT_NODE=true
WARP_FAIL_THRESHOLD=6              # consecutive warp=off polls before auto-shutdown (default 6 = 30s)
KILL_SWITCH=false                  # enforce strict PF lock that breaks SSH if VPN fails
# On campus/enterprise networks (WPA2-Enterprise / 802.1x), AP Client Isolation blocks direct
# LAN traffic between devices. Tailscale attempts a local P2P upgrade anyway, which causes
# macOS gvproxy to SNAT-mangle the reply’s source port  poisoning the phone’s WireGuard
# endpoint and blackholing 100% of its traffic. Setting this to false drops those local
# hole-punch packets so the phone safely falls back to Tailscale DERP relays.
# watcher.sh auto-detects 802.1x and AP isolation on every Wi-Fi change and overrides
# this setting automatically  you only need to set this manually if auto-detection fails.
TAILSCALE_ALLOW_LAN_P2P=false     # auto-managed; true only on trusted hotspots/home networks
ADGUARD_USER=admin                 # Username for AdGuard Home (defaults to admin if not set)
ADGUARD_PASSWORD=nullexit          # Shared password for AdGuard Home and Tor ControlPort (defaults to nullexit if not set)
BLOCKED_COUNTRIES="kp il"          # dynamically block country IP ranges via ipdeny.com
‘‘‘

## 3. Deploy the Gateway

**macOS (Recommended):**
‘‘‘bash
./toggle.sh
‘‘‘
Running it again stops the gateway and restores your network. Handles the full lifecycle: Colima VM, containers, DNS hijacking, Tailscale exit-node routing, rule compilation, and sleep prevention.

**Linux / Manual:**
‘‘‘bash
docker compose up -d
‘‘‘

## 4. Post-Deployment
```

~~~

Approve the Exit Node
1. Go to [Tailscale Admin Machines](https://login.tailscale.com/admin/machines).
2. Find the `tailscale` device `...` **Edit route settings** enable as Exit Node.

Verify Traffic Flow
On any Tailscale client device, select this gateway as the exit node, then visit api.ipify.org or run:
```bash
curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
# expect: warp=on
```

AdGuard DNS Setup

To ensure client devices correctly route DNS through the AdGuard container (and do not bypass it via `tailscaled`'s internal resolver), you **must** configure your Tailscale Admin Console:
1. Go to the **DNS** tab in the [Tailscale Admin Console](https://login.tailscale.com/admin/dns).
2. Enable **MagicDNS**.
3. Under **Nameservers**, add a **Custom** nameserver and enter the gateway's Tailscale IPv4 address (run `tailscale ip -4` on the gateway machine to find it, e.g., `100.x.x.x`).
4. Click the **three dots (...)** next to the nameserver you just added and enable **"Use with exit node"**.
5. Delete any other Global Nameservers (like Google or Cloudflare).
6. Toggle ON **Override local DNS**.

Traffic will now be correctly forced into the AdGuard sinkhole, and mobile devices will be blocked from bypassing it via encrypted DNS (DoH/DoT).

> [!WARNING]
> **Disable "Private DNS" on mobile devices.** Android's Private DNS (DoT/DoH) bypasses port 53 interception entirely. Go to `Settings  Connections  More connection settings  Private DNS  Off`.

> [!WARNING]
> **Bandwidth doubling:** Streaming 1GB of video on a remote device consumes 1GB download + 1GB upload on the host's network. Be mindful on metered connections.

AdGuard Dashboard
Navigate to `http://<gateway-tailscale-ip>:3000` credentials: **`admin` / `nullexit`**.

Access Any Mesh Device From Anywhere (SFTP/SSH)

Once the gateway exit node is online, **any device on your Tailscale mesh can reach any other mesh device, anywhere in the world** as long as both devices are on the mesh and the target device has an SSH server running. No port forwarding, no public IPs, no firewall holes. You can SSH into any device using its MagicDNS hostname (e.g. `ssh username@my-macbook`) or its assigned Tailscale IP (`100.x.x.x`).

> **Security Tip (Zero Local Attack Surface):** It is highly recommended to disable macOS's native "Remote Login" and "Screen Sharing" in System Settings to prevent exposing those ports (22, 5900) to physical Wi-Fi networks (which scanners can fingerprint). The gateway enforces `tailscale up --ssh=true` and the `pf` Kill-Switch automatically blocks local Wi-Fi access to these ports. Since both MagicDNS and Tailscale IPs route through the exact same encrypted tunnel, they are equally securebut **MagicDNS is recommended simply for ease of use.**

Extreme OPSEC: The Secret Tor-over-WARP Proxy
`nullexit` includes a completely isolated, headless Tor infrastructure proxy designed for terminals and hacking tools (e.g., `curl`, `sqlmap`, `nmap`).
To protect against local malware fingerprinting:
1. **Procedural Ports:** The proxy ports are randomized into the ephemeral range (10000-65000) on every single boot using a strict kernel socket collision-avoidance check (`netstat`).
2. **Honey-Port Tripwire:** The gateway spawns a fake trap port. If any malware attempts a sequential port-scan on `localhost` to find the proxy, it trips the Honey-Port, instantly triggering a macOS desktop notification and a critical log alert.

> [!NOTE]
> **Threat Model limitation:** Port randomization and the Honey-Port only defeat blind, generic port scanners. They do not protect against targeted local malware that runs `lsof -i` or reads the `/tmp/nullexit-ports.env` file directly. To actually prevent malicious local processes from reaching the proxy, you must run a host-level outbound firewall (like LuLu 4.3.0+ or Little Snitch) with **localhost/loopback filtering explicitly enabled** to gate access by process identity.

To use the Tor proxy, run `cat /tmp/nullexit-ports.env` to find your `$TOR_SOCKS_PORT` for the current session, and point your hardened browser (LibreWolf/Tor Browser) or CLI tool to `127.0.0.1:$TOR_SOCKS_PORT`.

> [!WARNING]
> **Preventing DNS Leaks (SOCKS5 vs. SOCKS5-Hostname):**
> When routing command-line tools through Tor, you **must** force the tool to delegate DNS resolution to the Tor proxy. Using plain SOCKS5 (e.g., `socks5://` or curl's `--socks5` flag) will resolve hostnames *locally* using the host's DNS settings (AdGuard/WARP) before establishing the connection.
~~~

```
    While this DNS traffic is encrypted inside the WARP tunnel, the destination domain name is still leaked to AdGuard and Cloudflare, undermining your anonymity.
>
> Use these correct protocols and configurations:
> - **‘curl‘:** Use ‘--socks5-hostname‘ or the ‘socks5h://‘ scheme (e.g., ‘curl -x socks5h://127.0.0.1:$TOR_SOCKS_PORT https://check.torproject.org‘). Do **not** use ‘--socks5‘ or ‘socks5://‘.
> - **‘sqlmap‘:** Pass the proxy URL using the ‘socks5h://‘ scheme to ensure remote resolution: ‘--proxy="socks5h://127.0.0.1:$TOR_SOCKS_PORT"‘.
> - **‘nmap‘:** Nmap’s built-in ‘--proxies‘ option resolves hostnames locally. To safely scan targets without DNS leaks, tunnel it using ‘proxychains-ng‘ configured with ‘proxy_dns‘ enabled (add ‘socks5 127.0.0.1 <PORT>‘ to ‘/etc/proxychains.conf‘ or a local config, then run ‘proxychains4 nmap <args>‘).


#### Transparent .onion Browsing & Dark Web Access
In addition to the randomized SOCKS5 proxy port, ‘nullexit‘ provides seamless, transparent ‘.onion‘ domain browsing for **all devices** on your Tailscale mesh:
- **Zero-Config DNS:** AdGuard Home automatically intercepts queries for ‘.onion‘ domains and resolves them to Tor’s virtual mapping subnet.
- **Tailscale Auto-Routing:** The virtual subnet is mapped to the ‘198.18.0.0/15‘ benchmark range (RFC 2544). Since this range is not a private IP subnet, Tailscale’s Exit Node mode automatically routes it to the gateway (bypassing local LAN exclusions).
- **Transparent NAT Proxying:** Kernel firewall rules in the gateway redirect all TCP traffic destined for ‘198.18.0.0/15‘ directly into Tor’s transparent port (‘9040‘).
- **Exit Node Exclusion:** Supports the ability to exclude specific countries from being used as Tor exit nodes via the ‘TOR_EXCLUDE_EXIT_NODES‘ environment variable in ‘.env‘.

##### Enable in Web Browsers
To prevent accidental DNS leaks, web browsers block ‘.onion‘ lookups by default. To browse dark web sites natively:
- **Firefox:** Go to ‘about:config‘, search for ‘network.dns.blockDotOnion‘, and set it to ‘false‘. Type any onion address (e.g. ‘duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion‘) directly into the URL bar and it will load.
- **Mobile Browsers (iOS/Android):** Connect to your Tailscale exit node, open Firefox (with blockDotOnion disabled), and browse onion sites natively on the go.

#### Hardening: Using obfs4 Tor Bridges
If you want to disguise your Tor traffic from the WARP exit node provider, you can optionally enable Tor bridges:
1. Obtain private ‘obfs4‘ bridge lines from [bridges.torproject.org](https://bridges.torproject.org) or the official Telegram bot.
2. Create a file named ‘tor-bridges.txt‘ in the root of the ‘nullexit‘ folder and paste your bridge lines inside it (one per line).
3. Set ‘TOR_USE_BRIDGES=true‘ in your ‘.env‘ file and restart the gateway. The proxy will refuse to start if the bridges file is empty or missing.

#### Example: Browse your Android phone’s files from your Mac


1. **On your Android phone**, install [Termux](https://termux.dev/) and [Termux:Boot](https://f-droid.org/packages/com.termux.boot/) from F-Droid.
2. In Termux, install and start the SSH server:
   ‘‘‘bash
   pkg install openssh
   sshd
   ‘‘‘
   Termux defaults to **port 8022** (ports below 1024 require root on Android).
3. Find your phone’s Tailscale IP:
   ‘‘‘bash
   tailscale ip -4   # e.g. 100.87.42.87
   ‘‘‘
4. **Stop Android from killing Termux**  do ALL of these (Samsung One UI is especially aggressive):
   - **Settings  Apps  Termux  Battery  Unrestricted**
   - **Device Care  Battery  Background usage limits  Sleeping apps / Deep sleeping apps**  remove Termux if listed
   - In Termux, run ‘termux-wake-lock‘ to hold a persistent wakelock
5. **On your Mac/PC**, open [Cyberduck](https://cyberduck.io/) (or any SFTP client: WinSCP, FileZilla, FE File Explorer on iOS) and connect to:
   - **Server:** ‘100.87.42.87‘ (your phone’s Tailscale IP)
   - **Port:** ‘8022‘
   - **Protocol:** SFTP (SSH File Transfer Protocol)
   - **Username:** (your Termux username, usually ‘u0_a123‘  run ‘whoami‘ in Termux to check)
   - **Password:** (your Termux password  set with ‘passwd‘ in Termux if you haven’t already)

That’s it  you can now browse, download, and upload files to your phone from any device on the mesh, no matter where either device is physically located. This works identically for any mesh device: Mac Mac, Windows  Linux, phone  NAS, etc.

> **Troubleshooting:** If the connection hangs for ~30 seconds before failing, your phone’s SSH server isn’t running. Open Termux and run ‘sshd‘ again. If Android keeps killing it despite Unrestricted battery, the ‘termux-wake-lock‘ command is your fix.
```

~~~markdown

---

## 5. Multi-Layer Firewall & Kill-Switch

### Layer 1  DNS Sinkhole (AdGuard Home)
Blocks ads and tracking domains before they are downloaded. In automated Lighthouse tests on ad-heavy sites:
- **Time to Interactive:** ~22% faster (51.0s  41.8s)
- **Speed Index:** ~20% faster (15.3s  12.8s)

Customize blocking via `black_list.txt` and `white_list.txt`. Rule profiles (`light` / `medium` / `heavy`) are set in `.env`.

### Layer 2  Kernel IP Firewall (`ipset`/`iptables`)
On every startup, the `rule-compiler` fetches threat-intelligence feeds (Spamhaus DROP, Feodo Tracker, Emerging Threats, CINS) and compiles **~16,700 unique malicious IPs/CIDRs** into the kernel `FORWARD` chain  blocking both outbound C2 connections and inbound attack traffic. Zero configuration required.

### Layer 3  Geo-IP Blocking
Dynamically blocks all traffic to and from specific countries using live IP ranges from `ipdeny.com`.
To add countries to your blocklist, open your `.env` file and set the `BLOCKED_COUNTRIES` variable with 2-letter ISO country codes (e.g. `BLOCKED_COUNTRIES="kp il cn ru"`), then restart the gateway.

### Layer 4  macOS PF Kill-Switch
A native macOS Packet Filter (`pf`) kill-switch that enforces a strict default-deny policy on your physical Wi-Fi interface (`en0`). When `KILL_SWITCH=true` is set in your `.env`, it drops all outgoing traffic except the Cloudflare WARP endpoints and Tailscale DERP relays.

- **Failsafe Design:** If the VPN tunnel crashes or Docker dies, your Mac completely loses internet access rather than leaking your real IP.
- **Remote-Access Safe:** Carefully designed to whitelist Tailscale's control plane (`192.200.0.0/24`), `utun*` tunnel traffic, and local LAN subnets. This guarantees that your remote SSH sessions and local AirDrop will survive even when the kill switch engages.
- **Setup Requirement:** The toggle scripts need permission to manipulate the network and firewall in the background without prompting you for a password. You must configure your passwordless sudo config by running exactly:
  ```bash
  echo "$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /usr/bin/dscacheutil, /usr/bin/killall, /usr/bin/pkill, /bin/kill, /sbin/route, /sbin/ifconfig, /usr/bin/true, /opt/homebrew/bin/brew, /usr/local/bin/brew, /usr/bin/python3 -I $PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3 -I $PWD/scripts/dns-proxy.py, /usr/local/bin/python3 -I $PWD/scripts/dns-proxy.py" | sudo tee /etc/sudoers.d/nullexit
  ```

---

## 6. Toggle Scripts

### macOS  `toggle.sh`
Fully automated lifecycle script. Also supports a native `.app` launcher:
```bash
osacompile -o "Toggle Gateway.app" Toggle-Gateway.applescript
```

Optional `.env` settings:
- `GATEWAY_BYPASS_PING=true`  proceed even if pre-flight connectivity checks fail.
- `GATEWAY_USE_EXIT_NODE=false`  DNS-only mode, skip exit-node routing.
- `GATEWAY_HIJACK_HOST=false`  skip DNS hijacking on the host (provides VPN/adblocking only to external Tailscale peers).
- `GATEWAY_MSS=1360`  override TCP Maximum Segment Size for the tunnel (default is calculated automatically).
- `HOST_LEAK_PROBE=false`  disable background host-egress leak prober (default: true).
- `HOST_MTU=1200`  override the host Tailscale interface MTU. Defaults to 1200 to fit inside the 1280-byte WARP tunnel.
- `KILL_SWITCH=true`  enforce strict network lock that breaks SSH if the VPN fails.
- `STOP_COLIMA_ON_EXIT=true`  fully shut down the Colima VM on toggle-off (saves battery on dedicated hosts).
- `TAILSCALE_ALLOW_LAN_P2P=true`  allow direct Tailscale LAN P2P connections between devices on the same network.
  - **Default: `false`.** On campus or enterprise Wi-Fi (WPA2-Enterprise / 802.1x), AP Client Isolation blocks local device-to-device traffic. When Tailscale still attempts a local P2P upgrade, macOS's `gvproxy` layer SNAT-mangles the reply's source port. WireGuard's endpoint roaming treats this as a legitimate address change and updates the phone's outbound path to the newly-mangled port  which has no listener. This blackholes 100% of the phone's traffic while the DERP relay path is abandoned. Setting this to `false` preemptively drops all RFC1918-destined Tailscale UDP from the gateway so the phone receives a clean failure and safely falls back to DERP.
  - **Auto-managed:** `watcher.sh` automatically detects WPA2-Enterprise via `system_profiler SPAirPortDataType` (the `airport` binary was removed in macOS 14.4) and probes for AP isolation by pinging the default gateway (`route -n get 1.1.1.1`) on every network change. It writes the detected
~~~

```
    value to ‘.lan_p2p_detected‘ in the repo root, which ‘routing-fix.sh‘ re-reads every 30 seconds  ** no restart required.** Only set this manually if auto-detection fails. See ‘devref.md 36‘ for the full SNAT endpoint poisoning analysis and ‘devref.md 40‘ for the ‘airport‘ removal background. *(Note: Direct P2P between the gateway container and the host Mac itself is always permitted via the Docker bridge to bypass DERP relays, regardless of this setting).*
  - Set to ‘true‘ on trusted home networks or Windows/mobile hotspots (no AP isolation) for a faster, lower-latency direct path.
- ‘WARP_FAIL_THRESHOLD=3‘  number of consecutive failed checks before forcing a gateway teardown (default: 6 checks = 30s).
- ‘WARP_ENDPOINT_1‘ / ‘WARP_ENDPOINT_2‘  override the Cloudflare WARP WireGuard endpoints.

### Linux  ‘scripts/toggle-linux.sh‘
‘‘‘bash
./scripts/toggle-linux.sh   # toggle ON/OFF
./scripts/recover-linux.sh  # recovery tool
‘‘‘

### Sleep Prevention (macOS)
When the gateway is ON, ‘caffeinate -i‘ runs in the background to prevent idle system sleep, keeping Docker and all services alive even on battery. The display can still sleep normally.

### Auto-Recovery (macOS  post-wake / post-roam)
Install the launchd LaunchAgent so the gateway self-heals after sleep/wake and Wi-Fi roams:
‘‘‘bash
cp ./launchd/com.nullexit.wake-recovery.plist ~/Library/LaunchAgents/
sed -i ’’ "s|__NULLEXIT_HOME__|$(pwd)|" ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
‘‘‘
See ‘devref.md 11.16‘ for the full deep dive.

---

## 7. Networking Notes

- **Port conflicts:** None. Tailscale uses dynamic UDP ports; WARP uses outbound UDP:2408. The gateway binds ‘0.0.0.0:41642/udp‘ on the host for Tailscale’s WireGuard listener  this is intentional and required to receive inbound hole-punch packets from peers on the internet. All other internal ports (AdGuard :5335, SOCKS5 :1080) are bound to ‘127.0.0.1‘ only and are not reachable from the LAN.
- **Tailscale P2P vs DERP:** On the same Wi-Fi, Tailscale establishes a fast P2P connection. On cellular (CGNAT), it always falls back to a DERP relay (~80-200ms). See ‘devref.md 5.1‘.
- **AirDrop / AirPlay:** Unaffected  the gateway only touches standard Wi-Fi/Ethernet interfaces, not ‘awdl0‘.
- **VPN coexistence:** Do **not** run a local VPN client (WARP, Mullvad, NordVPN) simultaneously with the exit node. Both fight for the default route and will blackhole your connection.
- **Banking & financial sites:** May intermittently block logins through WARP. Banks use anti-fraud databases that flag datacenter IP ranges (like Cloudflare’s ‘104.28.x.x‘) as VPN/proxy traffic. Success fluctuates depending on which WARP IP you land on. If a site blocks you, either disable the exit node temporarily for that session or use the bank’s mobile app (which often uses different fraud detection).
- **MSS clamping (Bufferbloat):** Double-tunnelling adds ~120-160 bytes of overhead per packet. This is now automatically handled natively via the macOS ‘pf‘ firewall, which universally applies TCP MSS Clamping to all outbound packets to strictly prevent MTU fragmentation and bufferbloat. No manual ‘.env‘ configuration is required. Additionally, the host’s Tailscale interface MTU is explicitly lowered to 1200 (configurable via ‘HOST_MTU‘ in ‘.env‘) to guarantee packets fit inside the container’s WARP tunnel without fragmenting.

---

## 8. Privacy & Security Hardening

The stack shifts trust across four layers: WARP hides traffic from your ISP, AdGuard blocks tracking at DNS, Tailscale encrypts device-to-device, and ‘ipset‘ blocks known malicious IPs at the kernel.

For higher security, the gateway supports these drop-in upgrades:

| Layer | Default | Hardened |
|---|---|---|
| VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, no-logs, anonymous payment) |
| DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH |
| Coordination | Tailscale (US) | Headscale (self-hosted) |
| Auth | Persistent OAuth keys | Ephemeral auth keys |
| Content | WireGuard asymmetric | WireGuard + Pre-Shared Keys (PSK) |

#### Python Runtime Airgap (Isolated Mode)
**Never** install third-party ‘pip‘ packages (like ‘requests‘ or ‘pyyaml‘) into the Python environment that ‘nullexit‘ uses. All python scripts in this project (e.g., the ‘dns-proxy.py‘ daemon) are strictly engineered to run on the zero-dependency Python Standard Library. To strictly isolate the environment from supply-chain attacks, ‘nullexit‘ executes these scripts using **Python’s Isolated Mode** (‘python3 -I‘). This deliberately blinds the execution environment to any malicious user-installed ‘pip‘ packages on the host, permanently air-gapping the gateway from PyPI.

```

```
See ‘devref.md 8‘ for the full threat model, Snowden programme analysis, and trust assumptions.

---

## 9. Debugging

- **‘output.log‘**  All stderr from ‘toggle.sh‘, ‘recover.sh‘, ‘setup.sh‘, and the rule-compiler is written here. The WARP Watcher (‘WARP_FAIL_THRESHOLD‘) and the Host Leak Probe (‘HOST_LEAK_PROBE‘) also write all their events here  ‘LEAK‘, ‘ROTATE‘, ‘HOST-PROBE failed‘, and WARP shutdown notices. Check this first.
- **‘bash scripts/diagnose-host-leak.sh‘**  One-shot host-routing diagnostic. Runs 8 checks and classifies your state into one of four known scenarios (System Extension conflict, SOCKS5 fallback, IPv6 leak, route-table freeze) or OK, and prints the exact fix command. Check 5 uses ‘route -n get 1.1.1.1‘ to ask the kernel which interface real traffic actually resolves through (more accurate than ‘netstat -rn | grep default‘ on macOS). Pass ‘--fix‘ to apply the remediation automatically. Pass ‘--watch‘ (or ‘--watch 30‘) to run a full baseline diagnostic then continuously monitor warp/IPv6/default-route for leaks, alerting on any state change.
- **‘bash scripts/host-leak-probe.sh‘**  Sub-second host-egress prober (enabled automatically via ‘HOST_LEAK_PROBE=true‘ in ‘.env‘). Polls ‘cdn-cgi/trace‘ every 300ms directly from the host NIC  not via ‘docker exec‘  so it catches flash-leaks invisible to the in-container WARP Watcher. All state changes, curl errors, and probe timeouts land in ‘output.log‘. Grep with ‘grep LEAK output.log‘.
- **‘bash scripts/fix-docker-bridge-collision.sh‘**  One-shot fix for Docker bridge IP conflicts. If your local Wi-Fi router assigns you an IP in the ‘172.17.0.0/16‘ range, it will violently collide with Docker’s default internal bridge, permanently killing your internet. This script detects the collision and auto-patches your Colima VM to use a safe subnet (e.g. ‘10.200.0.1/24‘).
- **‘devref.md‘**  Complete architecture reference, routing deep-dives, and full resolved-issues log. Read this before modifying any routing or firewall logic.

> [!CAUTION]
> **Two infinite routing loops are possible.** (1) If a hotspot device providing internet to the gateway host is *also* using the gateway as its exit node, packets bounce forever between the two. (2) When the host’s default route is redirected to ‘utun*‘, WARP endpoint packets can loop back into the tunnel. Both are documented in ‘devref.md 11.10‘ and ‘11.18‘ and are automatically handled by ‘toggle.sh‘.

---

## 10. Unified Project Document (Quine)

The project includes a ‘generate_tex.py‘ script that compiles the entire source code, configurations, and documentation into a single, unified LaTeX document (‘nullexit_unified.tex‘ / ‘.pdf‘).

Funnily enough, this document acts as a "project-level quine." It encapsulates the entirety of its own source code, its documentation, and the exact Python code required to generate itself. It serves as a self-contained, long-term archiving strategyproviding everything needed to reconstruct and understand the ‘nullexit‘ system from a single file.

---

## 11. Acknowledgements
- **[SyameimaruKoa](https://github.com/SyameimaruKoa):** Dual-stack TCP MSS clamping, ‘SIGHUP‘ state-tracking in the routing sidecar, and ‘TS_AUTH_ONCE‘ integration.

## 12. License
GNU Affero General Public License v3. See [LICENSE](./LICENSE).

## 13. Changelog

- **July 10, 2026**  **Bug fixes:**
  1. Resolved a race condition where Wi-Fi roaming caused the host IP to leak as the raw ISP IP (‘132.x‘) after every network switch. The WARP Watcher was firing nuclear shutdown while ‘recover.sh --post-wake‘ was still intentionally recreating the warp container. Fixed by adding a ‘/tmp/nullexit-warp-inhibit.marker‘ that pauses the watcher’s failure counter during post-wake recovery. See ‘devref.md 25‘.
  2. Resolved a startup race condition where the pre-flight connectivity check ‘[1/3] Gateway reachable via Tailscale‘ would fail because the host’s ‘tailscaled‘ had not yet propagated the netmap immediately after ‘tailscale up --reset‘. Upgraded the check to a 5-attempt retry loop with a 1-second delay. See ‘devref.md 26‘.
  3. Resolved a concurrency collision where the network changes from a nuclear ‘recover.sh‘ teardown triggered ‘watcher.sh‘ to run ‘recover.sh --post-wake‘ in parallel, corrupting container states. Implemented a shared lifecycle lock (‘/tmp/nullexit-toggle.lock‘) across both scripts to safely skip post-wake logic during teardowns. See ‘devref.md 27‘.
  4. Resolved an infinite auto-recovery loop after a tunnel failure shutdown. When the WARP Watcher triggered nuclear ‘recover.sh‘ to teardown the gateway, the active-state marker file ‘/tmp/nullexit-gateway-active.marker‘ was leaked on disk. The resulting network configuration changes from the teardown then triggered ‘watcher.sh‘, which saw the marker and immediately restarted the gateway, looping indefinitely. Fixed by explicitly deleting the marker file at the start of nuclear ‘recover.sh‘. See ‘devref.md 28‘.
  5. Resolved a pre-flight check false negative during cold boots where the ‘[1/3] Gateway reachable via Tailscale‘ check would fail. Because gluetun blocks UDP STUN traffic to the public internet, Tailscale is forced to use a slower DERP-assisted handshake which takes ~30-35 seconds on cold boots
```

```
. Increased the retry loop limit to 45 attempts (45s) to give the handshake sufficient time to complete. See ‘devref.md 29‘.
  6. Resolved a startup race condition during restarts where ‘docker compose build‘ would fail with DNS resolution errors because physical Wi-Fi was power-cycled during teardown and Colima booted before the host obtained a DHCP lease. Implemented a unified ‘wait_for_dhcp_settle‘ helper (configured with a 30s timeout to tolerate macOS DHCP lease latency) in ‘scripts/common.sh‘ and added it to the start of ‘toggle.sh‘. See ‘devref.md 30‘.
  7. Resolved a bug where enabling the PF Kill-Switch during SOCKS5 fallback mode blocked all host traffic (including ‘tailscaled‘ daemon connections to the control plane) because SOCKS5 does not route non-TCP/system traffic. Removed kill-switch activation from the SOCKS5 fallback path in ‘toggle.sh‘. See ‘devref.md 31‘.
  8. Resolved a bug where host-side ‘tailscale up‘ calls passed the ‘--reset‘ flag, which aggressively cleared macOS Tailscale credentials and logged the user out. Removed the ‘--reset‘ flag from all host-side invocations to preserve authenticated sessions during exit-node state toggling. See ‘devref.md 32‘.
  9. Resolved a bug where container connectivity check failures in ‘toggle.sh‘ and ‘recover.sh‘ were discarded to ‘/dev/null‘. Redirected stderr of Docker exec checks to ‘output.log‘ to prevent diagnostic blind spots during tunnel outages. See ‘devref.md 33‘.
- **July 6, 2026**  Edge-case security and stability hardening: DNS Watcher interface independence, robust lock-file PID verification, fail-closed crypto integrity check, strict ‘iptables‘ pinning for tailscale0tun0, Docker port binding to ‘127.0.0.1‘ to prevent LAN exposure, routing verification via ‘netstat -nr‘. See ‘devref.md 12‘.
- **July 4, 2026**  Colima bridged networking, SOCKS5 proxy migrated natively to Tailscale, headless prompt crashes fixed, proxy bypass domains added to fix LAN P2P. Python dependencies completely eliminated: ‘logger‘ and ‘rule-compiler‘ ported to blazing-fast Go multi-stage Docker builds. Added ‘crypto.sh‘ to enforce cryptographic signature validation on all core bash scripts. Massively refactored ‘toggle.sh‘ and ‘recover.sh‘ to eliminate ~200 lines of duplicated code by migrating network and process logic to ‘scripts/common.sh‘. Added a concurrency mutex to ‘toggle.sh‘, resolved unbound ‘TS_AUTHKEY‘ check crashes on setup, and integrated Go validation to the CI pipeline.
- **July 3, 2026**  **Critical Security Updates:** Addressed the overnight silent IP leak and improved geo-blocking. Introduced ‘scripts/host-leak-probe.sh‘ (a sub-second host-egress leak detector) and a statistical threshold auto-shutdown watcher. Added ‘unlock-files.sh‘ to safely reset file permissions. Resolved numerous startup regressions and system bugs.
- **July 2, 2026**  Two infinite routing loops resolved (‘devref.md 11.18‘): host-VM tunnel loop (WARP bypass routes now via gateway IP + manual ‘utun*‘ redirection) and Docker Compose subnet takeover (‘10.200.1.0/24‘ lock). ‘--accept-routes=true‘ now explicit on all ‘tailscale up --reset‘ calls.
- **July 2, 2026**  Auto-recovery daemon documented in 6 above (‘scripts/watcher.sh‘ + launchd plist). See ‘devref.md 11.16‘.
- **July 2, 2026**  Linux scripts (‘scripts/toggle-linux.sh‘, ‘recover-linux.sh‘, ‘setup-linux.sh‘) documented in 6.
- **July 1, 2026**  ‘recover.sh --post-wake‘ ships; wired to watcher daemon. Triggered on every sleep/wake or network-state change.
- **June 28, 2026**  8 internal scripts moved to ‘scripts/‘. User-facing files (‘toggle.sh‘, ‘recover.sh‘, ‘setup.sh‘, ‘docker-compose.yml‘) stay at repo root.
- **June 28, 2026**  All hardcoded install paths removed. ‘SCRIPT_DIR‘-relative resolution and ‘__NULLEXIT_HOME__‘ placeholder used throughout.

### Cryptographic Integrity Verification

nullexit uses a built-in cryptographic integrity checker designed to provide tamper-*evidence* against accidental edits, logic bugs, or non-privileged malware. During setup, a 256-bit ‘NULLEXIT_SEED‘ is injected into your ‘.env‘ file. The core entrypoint scripts (‘toggle.sh‘, ‘recover.sh‘, etc.) are hashed using HMAC-SHA256, and their signatures are stored in ‘.signatures‘.

*(Note: This is not a strict boundary against an attacker who already has write access to the repo, as they could simply read the seed and re-sign the scripts. It is designed as a strict footgun-catcher).*

If any script is modified without authorization, the gateway will loudly fail to boot. If you make intentional edits to the bash scripts, you must re-sign them by running:
‘‘‘bash
./scripts/crypto.sh --sign
‘‘‘
```

3 agent.md

```
# nullexit  Detailed Agent Analysis

> Complete per-file breakdown of every function, constant, duplication, and bug found.
> **Note (July 2026):** ‘sync-rules.py‘ and ‘logger.py‘ have been fully ported to Go (‘scripts/rule-compiler/main.go‘ and ‘scripts/logger/main.go‘) to dramatically improve performance and reduce container footprint via multi-stage Alpine builds. The analysis below reflects their previous Python states.

##  Important Agent Instruction
- **NEVER use ‘cat << EOF‘ or terminal redirections to write or modify files.** You must always use your native file-editing tools (‘replace_file_content‘ or ‘multi_replace_file_content‘). Using ‘cat EOF‘
```

```
    leads to duplicated text, broken markdown headers, and structural corruption (which previously destroyed the section numbering in ‘devref.md‘).
- **Always run ‘bash scripts/crypto.sh --sign‘ after making any modifications to the core bash scripts.** This is required to update the cryptographic HMAC-SHA256 signatures; otherwise, the startup integrity checks will fail and block execution.
- **Always run ‘git diff‘ and print the changes to the user *before* requesting or executing any git commit command.** The user must be shown the diff so they can review and understand the changes. Based on this diff, formulate a highly precise, detailed, and descriptive commit message (explaining exactly what was changed and why) rather than a generic summary, and present it to the user alongside the diff.
- **Always run ‘git status‘ before ‘git diff‘ to identify all modified and untracked files, ensuring no files (such as new scripts or configuration assets) are missed before formulating commit diffs and staging.**
- **Always use the ‘--restart‘ flag when asked to restart the toggle (e.g., run ‘./toggle.sh --restart‘).**
- **NEVER execute any ‘sudo‘ commands (especially ‘sudo route‘, ‘sudo dscacheutil‘, or any network-modifying commands) without explicitly explaining to the user what the command does and why it is needed FIRST. Do not assume implicit permission. Wait for the user’s approval before running it.**
- **When fixing an error and using logs to debug, always show the user the reasoning and the specific logs that support this theory.**
- **When the user tells you to "push", this means read git diffs and prepare commit messages that correspond to these changes (do not ignore any changes). If the changes can be atomized into multiple commits for easier understanding, do so.**
- **When the user says "latex this", it means you should run ‘python3 scripts/generate_tex.py‘ to regenerate the unified LaTeX document.**
- **NEVER apply changes to running gateway containers directly via ‘docker compose up‘ or ‘docker compose restart‘.** Recreating or restarting containers (especially the ‘warp‘ container which hosts the network namespace) breaks the shared network stack for all other services (‘adguardhome‘, ‘tailscale‘, ‘routing-fix‘), severing host connectivity and freezing macOS DNS. If you need to apply changes or rebuild containers, cleanly stop the gateway first using ‘./toggle.sh‘, or trigger ‘recover.sh --post-wake‘ to rebuild and re-verify the network stack safely in the correct dependency order. Do NOT run ‘./toggle.sh --restart‘ or any network-rebuilding commands yourself if the execution could sever host network access; instead, explain the changes and ask the USER to run ‘./toggle.sh --restart‘ (or ‘./toggle.sh‘ stop and start) themselves to safely apply the modifications.

##  How to Verify Gateway is Working
Whenever modifications are made to the gateway scripts, routing, or containers, execute these verification checks in order:
1. **Container Health:** Run ‘docker compose ps‘ to ensure all containers (‘warp‘, ‘adguardhome‘, ‘routing-fix‘, ‘tailscale‘) are ‘running‘ and ‘healthy‘.
2. **DNS Hijacking Status:** Run ‘networksetup -getdnsservers "Wi-Fi"‘ (or appropriate network service) and ensure it is set exclusively to the gateway’s Tailscale static IP (typically ‘100.100.21.8‘).
3. **DNS Query Interception:** Run ‘dig @100.100.21.8 google.com‘ (using the gateway static IP from ‘.gateway_ip‘) to ensure AdGuard Home is active, intercepting, and resolving queries.
4. **Double-Tunnel Egress:** Run ‘curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp‘ to verify the egress is routed through Cloudflare WARP (‘warp=on‘).
5. **Host Leak Monitoring:** Run ‘tail -n 30 output.log‘ and verify the background watchers and host leak prober are reporting healthy status without ‘LEAK‘ warnings.

---

## Part 1: macOS Main Scripts

### toggle.sh (1281 lines, 56KB)

**Functions (27 total):**

| # | Function | Lines | Purpose |
|---|----------|-------|---------|
| 1 | ‘run_with_timeout()‘ | L3064 | Run a command with a timeout watchdog; kills after N seconds |
| 2 | ‘run_gui_cmd()‘ | L6880 | Run a GUI command as the logged-in console user |
| 3 | ‘write_gateway_active_marker()‘ | L8890 | Write timestamp to ‘/tmp/nullexit-gateway-active.marker‘ |
| 4 | ‘clear_gateway_active_marker()‘ | L9295 | Remove gateway active marker + TUNNEL_FAILED_CLOSED.marker |
| 5 | ‘start_sleep_prevention()‘ | L108154 | Start ‘caffeinate‘ in background with shutdown trap to revert DNS |
| 6 | ‘stop_sleep_prevention()‘ | L157167 | Stop caffeinate process via PID file |
| 7 | ‘start_dns_watcher()‘ | L173191 | Background loop to re-hijack DNS every 30s for Wi-Fi roaming |
| 8 | ‘stop_dns_watcher()‘ | L193203 | Stop DNS watcher via PID file |
| 9 | ‘start_warp_watcher()‘ | L220279 | Background WARP liveness monitor; triggers recover.sh after N consecutive failures |
| 10 | ‘stop_warp_watcher()‘ | L281291 | Stop WARP watcher via PID file |
| 11 | ‘start_host_leak_probe()‘ | L297310 | Start host-egress leak prober (300ms polling) |
| 12 | ‘stop_host_leak_probe()‘ | L312322 | Stop host leak probe via PID file |
| 13 | ‘cleanup_handler()‘ | L325382 | Error/signal handler: resets DNS, stops all watchers, tears down containers |
| 14 | ‘restart_tailscaled_daemon()‘ | L449463 | Restart tailscaled via brew services (user then system fallback) |
| 15 | ‘disconnect_tailscale_host()‘ | L465477 | Disconnect host Tailscale with retry on failure |
| 16 | ‘get_active_service()‘ | L480508 | Detect active macOS network service name (e.g., "Wi-Fi") |
```

```
| 17 | `add_warp_bypass_routes()` | L524546 | Add static routes for WARP endpoints (162.159.192.1, 162.159.193.1) |
| 18 | `remove_warp_bypass_routes()` | L548552 | Remove WARP bypass routes |
| 19 | `setup_exit_node_routing()` | L558577 | Override default route to point through Tailscale utun interface |
| 20 | `reset_dns()` | L588595 | Reset DNS to 1.1.1.1 on ACTIVE_SERVICE + EN0_SERVICE |
| 21 | `cleanup_network_state()` | L600645 | Nuclear cleanup: disable proxies, flush DNS cache, flush routes, power-cycle Wi-Fi, kill sharing services |
| 22 | `enable_socks_proxy()` | L663682 | Enable system-wide SOCKS5 proxy on macOS |
| 23 | `disable_socks_proxy()` | L684689 | Disable SOCKS5 proxy |
| 24 | `stop_local_dns_proxy()` | L702710 | Kill local Python DNS proxy |
| 25 | `start_local_dns_proxy()` | L713768 | Start Python DNS proxy (UDP:53  TCP:5354) |
| 26 | `force_dns_to_gateway()` | L780820 | Force DNS to a specific IP with 3-retry verification loop |
| 27 | `is_gateway_active()` | L833854 | Check if gateway is running (containers, DNS, SOCKS proxy) |

**Constants:**

| Constant | Value | Line |
|----------|-------|------|
| `LOG_FILE` | `$PWD/output.log` | L8 |
| `PID_FILE` | `/tmp/nullexit-caffeinate.pid` | L105 |
| `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L169 |
| `WARP_WATCHER_PID_FILE` | `/tmp/nullexit-warp-watcher.pid` | L170 |
| `HOST_LEAK_PROBE_PID_FILE` | `/tmp/nullexit-host-leak-probe.pid` | L171 |
| `SOCKS_PROXY_PORT` | `1080` | L661 |
| `PATH` export | `/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH` | L396 |
| Gateway active marker | `/tmp/nullexit-gateway-active.marker` | L89 |
| TUNNEL_FAILED_CLOSED marker | `$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker` | L94 |
| WARP endpoints | `162.159.192.1`, `162.159.193.1` | L535-544 |
| DNS fallback | `1.1.1.1` | L592 |
| DNS search domain | `ts.net` | L794 |
| `LOCK_FILE` | `/tmp/nullexit-toggle.lock` | L62 |

---

### recover.sh (566 lines, 23KB)

**Functions (7 total):**

| # | Function | Lines | Purpose |
|---|----------|-------|---------|
| 1 | `step()` | L40 | Print bold step header |
| 2 | `ok()` | L41 | Print green success message |
| 3 | `warn()` | L42 | Print yellow warning message |
| 4 | `fail()` | L43 | Print red failure message |
| 5 | `die()` | L44 | Print red error and exit |
| 6 | `run_with_timeout()` | L5674 | Run command with timeout (bash-native, no GNU coreutils) |
| 7 | `restart_tailscaled_daemon()` | L118129 | Restart tailscaled daemon via brew services |

**Constants:**

| Constant | Value | Line |
|----------|-------|------|
| Color codes | `RED`, `GREEN`, `YELLOW`, `BOLD`, `NC` | L37-38 |
| `SCRIPT_DIR` | `$(dirname "${BASH_SOURCE[0]}")` | L50 |
| `PATH` export | `/opt/homebrew/bin:/usr/local/bin:$PATH` | L77 |
| `PID_FILE` | `/tmp/nullexit-caffeinate.pid` | L387 |
| `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L407 |
| `HOST_LEAK_PROBE_PID_FILE` | `/tmp/nullexit-host-leak-probe.pid` | L423 |
| WARP endpoints | `162.159.192.1`, `162.159.193.1` | L329-336 |
| TUNNEL_FAILED_CLOSED marker | `$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker` | L51 |

**Duplicated logic from toggle.sh:**
*(Note: As of July 2026, **ALL** of the below duplicated logic has been successfully extracted to `scripts/common.sh`!)*
- `run_with_timeout()`  simpler subshell version vs toggle's PID-tracking version
- `restart_tailscaled_daemon()`  near-identical (uses warn()/ok() vs echo)
- Active network service detection  inline at L217-233 (toggle has `get_active_service()` function)
- EN0 service resolution  inline at L230-233 (same as toggle L515-516)
- WARP bypass routes  inline at L329-337 (toggle has `add_warp_bypass_routes()`)
- Exit node routing setup  inline at L340-345 (toggle has `setup_exit_node_routing()`)
- DNS cache flushing  L296-302 (same as toggle L611-616)
- Wi-Fi power-cycling  L442-461 (similar to toggle L626-637)
- Proxy disabling  L282-288 (same as toggle L604-608)
- Sharing services reset  L586 (same as toggle L642)
- PID file stop pattern  L387-399, L407-419, L423-435 (same as toggle L157-167, L193-203, L312-322)
- KILL_SWITCH check  L194 (same as toggle L141, L469)
- .gateway_ip reading  L138-142, L209-213 (same pattern as toggle L1080)

---

```

```
### setup.sh (410 lines, 18KB)

**Functions (4 total):**

| # | Function | Lines | Purpose |
|---|----------|-------|---------|
| 1 | ‘step()‘ | L10 | Print bold blue step header |
| 2 | ‘ok()‘ | L11 | Print green success message |
| 3 | ‘warn()‘ | L12 | Print yellow warning message |
| 4 | ‘die()‘ | L13 | Print red error and exit |

**Constants:**

| Constant | Value | Line |
|----------|-------|------|
| Color codes | ‘RED‘, ‘GREEN‘, ‘YELLOW‘, ‘BLUE‘, ‘BOLD‘, ‘NC‘ | L7-8 |
| ‘RECOMMENDED_TS_VERSION‘ | ‘1.98.5‘ | L71 |
| ‘ADGUARD_PASSWORD‘ | ‘nullexit‘ | L215 |
| Default .env values | ‘GATEWAY_RULE_PROFILE=medium‘, ‘GATEWAY_MSS=1120‘, etc. | L240-248 |

---

## Part 2: Linux Scripts

### scripts/toggle-linux.sh (931 lines, 39KB)

**Functions (17 total):**

| # | Function | Line | Purpose |
|---|----------|------|---------|
| 1 | ‘run_with_timeout()‘ | L19 | Runs command with timeout watchdog, kills on expiry |
| 2 | ‘run_gui_cmd()‘ | L57 |  macOS code  uses ‘stat -f ’%Su’ /dev/console‘, DEAD CODE on Linux |
| 3 | ‘start_sleep_prevention()‘ | L74 | Uses ‘systemd-inhibit‘ (Linux) to block sleep |
| 4 | ‘stop_sleep_prevention()‘ | L116 | Kills sleep prevention process via PID file |
| 5 | ‘start_dns_watcher()‘ | L130 | Background daemon polling DNS every 30s, re-hijacks if changed |
| 6 | ‘stop_dns_watcher()‘ | L149 | Kills DNS watcher process via PID file |
| 7 | ‘cleanup_handler()‘ | L162 | Trap handler for ERR/INT/TERM/HUP  restores DNS, kills bg processes |
| 8 | ‘disconnect_tailscale_host()‘ | L273 | Disconnects host Tailscale from mesh |
| 9 | ‘get_active_service()‘ | L283 | Gets active network interface via ‘ip route‘ (Linux) |
| 10 | ‘reset_dns()‘ | L304 | Restores DNS via ‘resolvectl revert‘ (Linux) |
| 11 | ‘cleanup_network_state()‘ | L314 | Flushes DNS cache, routes, power-cycles network interface |
| 12 | ‘enable_socks_proxy()‘ | L361 | Stub  prints message that Linux global SOCKS is DE-specific |
| 13 | ‘disable_socks_proxy()‘ | L367 | Stub  prints skip message |
| 14 | ‘stop_local_dns_proxy()‘ | L383 | Kills Python DNS proxy processes |
| 15 | ‘start_local_dns_proxy()‘ | L394 | Starts Python DNS proxy (UDP:53  TCP:5354) + hijacks DNS |
| 16 | ‘force_dns_to_gateway()‘ | L461 | 3-attempt loop to set + verify DNS via ‘resolvectl‘ |
| 17 | ‘is_gateway_active()‘ | L495 | Checks if containers running or DNS was hijacked |

**Shared constants with macOS toggle.sh:**

| Constant | Value | Both |
|----------|-------|------|
| ‘SOCKS_PROXY_PORT‘ | ‘1080‘ | toggle-linux L359, toggle.sh L661 |
| ‘PID_FILE‘ | ‘/tmp/nullexit-caffeinate.pid‘ | toggle-linux L71, toggle.sh L105 |
| ‘DNS_WATCHER_PID_FILE‘ | ‘/tmp/nullexit-dns-watcher.pid‘ | toggle-linux L128, toggle.sh L169 |
| ARP threshold | ‘15‘ devices | toggle-linux L518, toggle.sh L863 |
| Colima memory | ‘0.6‘ (600MB) | toggle-linux L578, toggle.sh L928 |
| Colima swap | ‘400MB‘ | toggle-linux L587, toggle.sh L937 |
| Tailscale wait loop | ‘60 iterations  1s‘ | toggle-linux L613, toggle.sh L1010 |
| NoState stuck threshold | ‘40 consecutive‘ | toggle-linux L628, toggle.sh L1025 |

---

### scripts/recover-linux.sh (257 lines, 10KB)

**Functions (6 total):**

| # | Function | Line | Purpose |
|---|----------|------|---------|
| 1 | ‘step()‘ | L24 | Print formatted step header |
| 2 | ‘ok()‘ | L25 | Print green success message |
| 3 | ‘warn()‘ | L26 | Print yellow warning message |
| 4 | ‘fail()‘ | L27 | Print red failure message |
| 5 | ‘die()‘ | L28 | Print red error + exit 1 |
| 6 | ‘run_with_timeout()‘ | L33 | Simplified timeout (immediate SIGKILL, no PID tracking) |

---

### scripts/setup-linux.sh (416 lines, 18KB)

**Functions (4 total):**
```

```

| # | Function | Line | Purpose |
|---|----------|------|---------|
| 1 | ‘step()‘ | L10 | Print formatted step header |
| 2 | ‘ok()‘ | L11 | Print green success message |
| 3 | ‘warn()‘ | L12 | Print yellow warning message |
| 4 | ‘die()‘ | L13 | Print red error + exit 1 |

**~95% identical to macOS setup.sh.** Only differences:
1. Desktop shortcuts (L366-391 Linux ‘.desktop‘ files vs macOS ‘.app‘ via ‘osacompile‘)
2. Final instructions text
3. .env template: macOS adds ‘GATEWAY_USE_EXIT_NODE‘, ‘WARP_FAIL_THRESHOLD‘, ‘HOST_LEAK_PROBE‘, ‘KILL_SWITCH‘  Linux omits these

---

## Part 3: Utility Scripts

### scripts/diagnose-host-leak.sh (722 lines, 38KB)

**Purpose:** Comprehensive host-routing diagnostic. 8 checks classify host IP leaks. Supports ‘--fix‘ and ‘--watch‘ modes.

**Duplicated logic:**
- ‘resolve_active_service()‘  identical pattern as toggle.sh, recover.sh, fix-docker-bridge-collision.sh
- ‘run_with_timeout()‘  reimplemented timeout
- Color codes  RED/GREEN/YELLOW/BOLD/NC with tty detection
- WARP check via ‘cloudflare.com/cdn-cgi/trace‘
- ‘export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"‘
- .gateway_ip reading pattern

### scripts/fix-docker-bridge-collision.sh (402 lines, 18KB)

**Purpose:** Fixes Docker bridge subnet collisions with ‘10.200.1.0/24‘.

**Duplicated logic:**
- ‘resolve_active_iface()‘  same interface detection as diagnose-host-leak.sh
- Color codes, step()/ok()/warn()/fail()/die() formatting helpers

### scripts/fix-ssh-delay.sh (63 lines, 2.7KB)

**Purpose:** One-shot fix for ~20s SSH delay. Standalone, minimal duplication (uses hardcoded / instead of color functions).

### scripts/host-leak-probe.sh (110 lines, 5.5KB)

**Purpose:** Sub-second host-egress leak prober at 300ms intervals.

**Duplicated logic:**
- WARP check via cdn-cgi/trace (same curl + awk as toggle.sh WARP Watcher)
- Color codes (hardcoded; status updates and warnings are conditioned on TTY detection to prevent log pollution)
- LOG_FILE = ‘$PROJECT_ROOT/output.log‘

### scripts/reinstall-host-tailscale.sh (740 lines, 31KB)

**Purpose:** Complete uninstall + reinstall of host Tailscale.

**Duplicated logic:**
- ‘with_timeout()‘  yet another bash timeout implementation (polling-based, different from diagnose and toggle versions)
- Color codes + logging functions
- System Extension detection logic (similar to diagnose-host-leak.sh)

### scripts/routing-fix.sh (266 lines, 10.5KB)

**Purpose:** Runs INSIDE warp container. Maintains routing table 200, FORWARD rules, country blocking, IP blocklists, tunnel health checks.

**Duplicated logic:**
- (Fixed) WARP_ENDPOINT defaults are now centralized in common.sh
- WARP health check via cdn-cgi/trace (another instance)
- iptables dual-backend pattern (for-loop over ‘iptables iptables-legacy‘)

### scripts/watcher.sh (269 lines, 13KB)

**Purpose:** Long-running launchd daemon for post-wake/post-roam recovery.

**Key function  ‘detect_lan_p2p_mode()‘:**
Added July 10, 2026. Called on startup and on every network state change (Listener 2 NET: events). Determines whether the current Wi-Fi network safely allows direct Tailscale LAN P2P connections:
```

```
1. **WPA2-Enterprise check:** ‘system_profiler SPAirPortDataType‘  reads the ‘Security:‘ field for the current association. Enterprise = 802.1x = always AP-isolated  sets ‘allow=false‘.
2. **AP isolation probe:** ‘route -n get 1.1.1.1‘  finds default gateway IP, then ‘ping -c 3 -t 1 -q "$gw"‘  if 0 responses on a non-empty network, AP isolation is active  sets ‘allow=false‘.
3. **Output:** Writes ‘’true’‘ or ‘’false’‘ to ‘.lan_p2p_detected‘ in the repo root.
4. **Consumer:** ‘routing-fix.sh‘ reads this file every 30 seconds and enforces/relaxes the RFC1918 DROP rule accordingly  no restart required.

**Duplicated logic:**
- PATH export (similar to toggle.sh/recover.sh)
- Marker file pattern (‘/tmp/nullexit-gateway-active.marker‘)

### scripts/dns-proxy.py (92 lines, 2.9KB)   Clean, standalone

### scripts/logger.py (144 lines, 5.9KB)   Clean, standalone

---

## Part 4: Cross-Cutting Issues

### Functions Identical Across macOS  Linux

| Function | macOS toggle | macOS recover | macOS setup | Linux toggle | Linux recover | Linux setup |
|----------|:-----------:|:------------:|:-----------:|:------------:|:-------------:|:----------:|
| ‘run_with_timeout()‘ |  |  |  |  identical |  identical |  |
| ‘stop_local_dns_proxy()‘ |  |  |  |  identical |  |  |
| ‘start_local_dns_proxy()‘ |  |  |  | ~95% similar |  |  |
| ‘is_gateway_active()‘ |  |  |  | ~80% similar |  |  |
| ‘step/ok/warn/die‘ |  |  |  |  identical |  identical |

### Platform-Adapted Functions (same logic, different OS commands)

| Function | Linux Command | macOS Command |
|----------|--------------|---------------|
| ‘get_active_service()‘ | ‘ip route get 1.1.1.1 \| awk ’{print $5}’‘ | ‘route get default‘ + ‘networksetup -listnetworkserviceorder‘ |
| ‘reset_dns()‘ | ‘resolvectl revert‘ | ‘networksetup -setdnsservers‘ |
| ‘cleanup_network_state()‘ | ‘ip link set dev down/up‘ | ‘networksetup -setairportpower off/on‘ + proxy disable |
| ‘force_dns_to_gateway()‘ | ‘resolvectl dns‘ + ‘resolvectl domain ~ts.net‘ | ‘networksetup -setdnsservers‘ + ‘-setsearchdomains ts.net‘ |
| ‘start_sleep_prevention()‘ | ‘systemd-inhibit --what=sleep‘ | ‘caffeinate -i‘ |
| ‘enable/disable_socks_proxy()‘ | Stub (no-op on Linux) | ‘networksetup -setsocksfirewallproxy‘ |
| DNS cache flush | ‘resolvectl flush-caches‘ | ‘dscacheutil -flushcache‘ + ‘killall -HUP mDNSResponder‘ |

### Features Missing From Linux Scripts

These exist in macOS toggle.sh but NOT in toggle-linux.sh:

1. ‘write_gateway_active_marker()‘ / ‘clear_gateway_active_marker()‘
2. ‘restart_tailscaled_daemon()‘
3. ‘start_warp_watcher()‘ / ‘stop_warp_watcher()‘  WARP liveness monitor
5. ‘add_warp_bypass_routes()‘ / ‘remove_warp_bypass_routes()‘ / ‘setup_exit_node_routing()‘
6. ‘LOG_FILE‘ structured logging (timestamps to output.log)
7. MSS clamping injection (step 4c)
8. Rule count reporting
9. ‘--post-wake‘ mode in recover
10. Container teardown on failure in cleanup_handler

---



## Part 6: Constant Duplication Map

| Value | Files |
|-------|-------|
| ‘162.159.192.1‘ (WARP endpoint) | docker-compose.yml, routing-fix.sh (now dynamically loaded via .env/common.sh!) |
| ‘5335‘ (AdGuard DNS port) | docker-compose.yml, AdGuardHome.yaml, post-rules.txt |
| ‘5354‘ (mapped DNS port) | docker-compose.yml, dns-proxy.py |
| ‘41642‘ (Tailscale WireGuard) | docker-compose.yml, routing-fix.sh, post-rules.txt |
| ‘1080‘ (SOCKS5 port) | docker-compose.yml, toggle.sh |
| ‘1120‘ (MSS clamp) | post-rules.txt, .env |
| ‘admin:nullexit‘ (AdGuard creds) | AdGuardHome.yaml (bcrypt) |
| ‘10.200.1.0/24‘ (Docker subnet) | (Fixed) no longer duplicated |
| ‘America/New_York‘ (timezone) | docker-compose.yml (4 services) |
| ‘/tmp/nullexit-caffeinate.pid‘ | (Fixed) centralized in common.sh |
| ‘/tmp/nullexit-dns-watcher.pid‘ | (Fixed) centralized in common.sh |
| ‘/tmp/nullexit-host-leak-probe.pid‘ | toggle.sh, recover.sh |
```

```
| ‘/opt/homebrew/bin:...‘ (PATH) | (Fixed) centralized in common.sh |

---

## Part 7: Refactoring Outcome (Implemented July 2026)

**Decision: "Scripts-only" architecture**
Instead of introducing a new ‘lib/‘ directory, all shared code was moved into the existing ‘scripts/‘ directory to keep the project hierarchy flat and simple.

### 1. ‘scripts/common.sh‘ (288 lines, 10KB)
Extracts the fundamental primitives and networking logic used across orchestrator scripts:
- **Formatters:** ‘step()‘, ‘ok()‘, ‘warn()‘, ‘fail()‘, ‘die()‘
- **Timeout Wrapper:** ‘run_with_timeout()‘ (Canonical version with PID tracking, graceful SIGTERM, and SIGKILL fallback)
- **Daemon Lifecycle:** ‘restart_tailscaled_daemon()‘, ‘stop_pidfile_daemon()‘ (collapsed caffeinate, DNS watcher, WARP watcher, and host leak prober teardown logic)
- **Network Interface/Routing:** ‘get_active_service()‘, ‘get_en0_service()‘, ‘bounce_wifi_interfaces()‘, ‘add_warp_bypass_routes()‘, ‘remove_warp_bypass_routes()‘
- **Proxy/DNS Cleanup:** ‘disable_all_proxies()‘, ‘flush_dns_cache()‘, ‘reset_sharing_services()‘
- **Configuration Helpers:** ‘read_adguard_ip()‘, ‘is_kill_switch_enabled()‘, ‘get_warp_endpoint_1()‘, ‘get_warp_endpoint_2()‘ (which centralizes the hardcoded 162.159.192.1 / .193.1 into .env logic)

### 2. ‘scripts/setup-common.sh‘ (~350 lines)
Extracts the heavy, duplicated installation logic shared between ‘setup.sh‘ (macOS) and ‘scripts/setup-linux.sh‘:
- Docker and Colima prerequisite checks
- ‘wgcf‘ binary download and WARP key generation
- ‘.env‘ configuration file generation
- Interactive Tailscale Auth Key and AdGuard Rule Profile prompts

### Sourcing Pattern
- macOS root scripts (‘toggle.sh‘, ‘recover.sh‘, ‘setup.sh‘) source via:
  ‘source "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/common.sh"‘
- Linux sibling scripts (‘scripts/toggle-linux.sh‘, etc.) source via:
  ‘source "$(dirname "${BASH_SOURCE[0]}")/common.sh"‘

## Agent Verbs / Protocols

### ‘/sweep‘ (or ‘sweep the gateway‘)
When the user invokes this verb, the agent MUST perform a full end-to-end connectivity, health, security, and performance audit of the gateway:
1. **WARP Validation:** Run ‘curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp=‘ (must equal ‘on‘).
2. **Tor Proxy & Control Validation:**
   - Source the dynamically generated ports from ‘/tmp/nullexit-ports.env‘ (or identify them using ‘docker compose ps‘).
   - **SOCKS5 Check:** Run ‘curl -s --socks5-hostname 127.0.0.1:$TOR_SOCKS_PORT https://check.torproject.org/api/ip‘ to confirm the SOCKS5 proxy successfully connects and routes through the Tor network.
   - **Control Port Check:** Query the Tor Control Port using netcat: ‘echo "PROTOCOLINFO" | nc 127.0.0.1:$TOR_CONTROL_PORT‘. Verify it returns a standard ‘250-PROTOCOLINFO‘ response, confirming the Control Port is active and communicating.
   - **Circuit & Node Lookup:** Query the Tor Control Port for the active path (‘GETINFO circuit-status‘). For each active built circuit, parse the relay fingerprints, lookup their IP addresses (‘GETINFO ns/id/<fingerprint>‘), and resolve their country codes using Tor's local GeoIP database (‘GETINFO ip-to-country/<IP>‘). Include the list of active circuits, showing the role (Guard/Middle/Exit), nickname, IP, and country of each hop in the final sweep report.
   - **Transparent Onion Validation:** Run ‘nslookup duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion‘ to verify it resolves to an IP within ‘198.18.0.0/15‘ benchmark range. Verify transparent dark web routing by running ‘curl -sI -H "Host: duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion" http://<resolved-ip>/‘ (should return HTTP 301/200).
3. **DNS Leak Validation:**
   - Audit the upstream resolver currently utilized by the host. Run ‘curl -s https://edns.ip-api.com/json‘ to fetch details of the DNS resolver processing the client's requests.
   - Verify that the resolver IP and organization returned belong to Cloudflare (e.g., AS13335) or match the WARP tunnel's exit range, and that **no** DNS traffic leaks to the user's raw physical ISP DNS.
4. **Performance Benchmark Audit:**
   - Run the python benchmark script ‘python3 benchmarks/test_load.py‘.
   - Report the overall load speed and the ratio of successfully fetched vs. blocked/failed subresources for the targeted domains to ensure ad-blocking and MTU encapsulation are performing correctly.
5. **Tailscale Mesh Validation:**
   - Run ‘ping -c 1 100.100.21.8‘ to ensure the gateway is reachable.
   - Run ‘tailscale status‘ to identify all active/online nodes in the mesh.
   - For every online peer returned in the status output, execute a ping check (‘ping -c 1 <peer-ip>‘) to verify bidirectional connectivity and confirm healthy routing to all active mesh devices.
6. **Log Audit:** Run ‘tail -n 100 output.log | grep -iE "(error|fail|warn)"‘ to catch any underlying component failures or warnings.

### ‘/latex‘ (or ‘generate latex‘)
```

```
When the user invokes this verb, the agent MUST run ‘python3 -I scripts/generate_tex.py‘ to regenerate the documentation source code (‘nullexit_unified.tex‘). The agent MUST NOT attempt to compile the ‘.tex‘ file into a PDF (the user handles PDF compilation locally).
```

4 devref.md

```
# nullexit  Development Reference & Resolved Issues

> **Last updated:** July 10, 2026
> **Purpose:** Provide any LLM or developer with complete project understanding, debugging history, and resolved issues so they can make informed changes without re-reading every file.
> **Diagrams:** See [‘diagrams.md‘](./diagrams.md) for system architecture, toggle.sh flowcharts, monitoring layer, traffic sequence, recover.sh decision tree, and the full failureself-healing map.

---

## 1. What Is nullexit?

nullexit is a **Tunnel-in-Tunnel VPN gateway** that chains **Tailscale** (mesh VPN) through **Cloudflare WARP** (exit VPN). It runs on a macOS host inside Docker containers managed by **Colima** (lightweight Linux VM). The goal: any device on the user’s Tailscale mesh can use this gateway as an exit node, and all traffic exits through Cloudflare WARP  achieving double encryption, ISP invisibility, and network-wide ad-blocking via **AdGuard Home**.

### Traffic Flow
‘‘‘
Phone/Laptop  Tailscale mesh (WireGuard)  Mac host  Docker container
   Tailscale decrypts  Gluetun re-encrypts  WARP tun0  Cloudflare  Internet
‘‘‘

### SOCKS5 Failover Path (if exit node is unavailable)
‘‘‘
DNS:  Browser  host 127.0.0.1:53  Python proxy  TCP:5354  AdGuard  WARP DNS
TCP:  Apps  macOS SOCKS5 proxy (127.0.0.1:1080)  container  tun0  WARP  Internet
‘‘‘

---

## 2. Architecture

### Containers (all share ‘warp‘’s network namespace via ‘network_mode: service:warp‘)
| Container | Image | Role |
|-----------|-------|------|
| ‘warp‘ | ‘qmcgaw/gluetun:v3.41.1‘ | Gluetun WireGuard client  Cloudflare WARP. Owns the network namespace. Strict firewall. |
| ‘tailscale‘ | ‘tailscale/tailscale:v1.98.4‘ | Advertises as exit node on the mesh. Also provides the built-in SOCKS5 proxy fallback via ‘TS_SOCKS5_SERVER=:1080‘. |
| ‘routing-fix‘ | ‘alpine:3.20‘ | Sidecar that maintains routing tables + iptables rules every 30 seconds. |
| ‘rule-compiler‘| ‘golang:1.22-alpine‘ / ‘alpine:3.20‘ | One-shot startup container that compiles 500k+ DNS block rules in <2s and immediately exits. |
| ‘adguardhome‘ | ‘adguard/adguardhome:v0.107.77‘ | DNS sinkhole for ads/trackers. Listens on port 5335. Upstream DNS: ‘127.0.0.1:53‘ (through WARP). |

### Port Mappings (on host via Colima)
| Host Port | Container Port | Protocol | Purpose |
|-----------|----------------|----------|---------|
| 5354 | 5335 | TCP+UDP | AdGuard DNS |
| (Tailscale IP):3000 | 3000 | TCP | AdGuard web UI (Internal to Mesh) |
| 41641 | 41641 | UDP | Tailscale WireGuard direct |
| 1080 | 1080 | TCP | SOCKS5 proxy |

### Host Components
- **Colima VM**  Linux VM running Docker (600MB RAM + 400MB swap)
- **tailscaled**  Standalone Tailscale daemon via ‘brew install tailscale‘ + ‘brew services start tailscale‘ (NOT the GUI ‘.app‘)
- **macOS network settings**  DNS, SOCKS proxy, routing all manipulated by ‘toggle.sh‘

---

## 3. Key Files

| File | Lines | Purpose |
|------|-------|---------|
| ‘toggle.sh‘ | ~1265 | **Main script.** Detects state, toggles gateway ON/OFF. Handles DNS hijacking, Tailscale exit node, SOCKS proxy, sleep prevention, timeouts, cleanup. |
```

```
| `setup.sh` | 406 | **One-time setup.** Installs deps (Docker, Tailscale, wgcf), generates WARP keys, writes `.env`, configures AdGuard via API, starts containers. |
| `docker-compose.yml` | 194 | Service definitions for all 5 containers. |
| `scripts/routing-fix.sh` | 201 | Maintains routing tables (table 200 for SOCKS5, table 52 for CGNAT) and FORWARD iptables rules in both nftables and legacy backends. Loads and enforces the IP blocklist via `ipset`. Runs in a 30-second loop. |
| `post-rules.txt` | 18 | Gluetun iptables rules loaded at container start. FORWARD accept for tailscale0, NAT MASQUERADE on tun0, DNS redirect to port 5335, IPv6 drop, TCP MSS clamping. |

| `scripts/dns-proxy.py` | 91 | Local DNS proxy. UDP:53  TCP:5354 with proper 2-byte length prefix. Fallback when exit node is unavailable. |
| `scripts/rule-compiler/` | ~800 | Unified threat compiler (ported to Go from Python). **DNS pipeline:** fetches remote blocklists, deduplicates against AdGuard native filters, optimizes subdomains (~50% reduction), compiles to AdGuard syntax. **IP pipeline:** fetches threat-intel feeds (Spamhaus, Feodo, ET, CINS), strips private/Tailscale ranges, outputs atomic `ipset` restore file. Built dynamically in Docker multi-stage build. |
| **`adguard/work/`** |  | **Bind-mounted container data folder. Off-limits from outside Docker  READ-ONLY-EXCEPT-VIA-`sync-rules`. See README 6.** |
| `recover.sh` | ~557 | Nuclear recovery script (gain: use `--post-wake` for non-destructive refresh after sleep/wake or Wi-Fi roam  keeps the gateway live while still re-hijacking DNS + refreshing Tailscale exit-node + force-recreating warp if unhealthy). Resets DNS on ALL services, disables proxies, flushes routes, stops containers, kills sleep prevention, power-cycles Wi-Fi. Cryptographically verified. |
| `scripts/diagnose-host-leak.sh` | 580+ | **One-shot host-routing diagnostic.** Runs 7 checks (Tailscale, SOCKS5, CDN-cgi/trace, IPv6 leak, default route, output.log hints, gateway IP), classifies into ONE of three known leak scenarios (A=SOCKS5 fallback, B=IPv6 leak, C=route freeze) or OK, writes a timestamped report to `host-leak-diagnostic-<UTC>.txt`, and prints a ready-to-run fix on stdout. Pass `--fix` to apply the matched remediation and re-verify egress. Pass `--watch` (or `--watch 30`) to run a full baseline then continuously monitor warp/IPv6/default-route every N seconds, alerting on any state change. See 10.30. |
| `scripts/host-leak-probe.sh` | ~110 | **Continuous sub-second host-egress prober.** Launched automatically by `toggle.sh` when `HOST_LEAK_PROBE=true` in `.env` (default). Polls `cdn-cgi/trace` every 300ms directly from the host NIC via `curl`  not via `docker compose exec`  so it detects flash-leaks invisible to the in-container WARP Watcher. Logs only on state change (`LEAK`, `ROTATE`, `HOST-PROBE failed/timeout`) to `output.log`. curl errors (`2>>output.log`) also land there. Stopped gracefully (SIGTERM) by `toggle.sh` STOP path, `cleanup_handler`, and `recover.sh`. See 10.30.1. |
| `scripts/fix-docker-bridge-collision.sh` | ~290 | **One-shot fix for 9.13.** Detects Docker's `172.17.0.0/16` bridge colliding with the host Wi-Fi subnet (the symptom that produces `default 172.17.0.1 UGScg en0` in `netstat -rn`), picks a non-colliding `docker.bip` from a small candidate set, idempotently edits `~/.colima/default/colima.yaml` with a timestamped backup, restarts Colima, rebinds Wi-Fi DHCP, verifies the new default route is no longer Docker's bridge, and re-runs `toggle.sh` + `scripts/diagnose-host-leak.sh`. Supports `--dry` and `--skip-toggle`. |
| `scripts/watcher.sh` | 194 | **Long-running post-wake + post-roam daemon.** Launched by launchd LaunchAgent; subscribes to `com.apple.powermanagement` events via `log stream` and to network-state changes via `scutil n.watch`. Both fire `recover.sh --post-wake`. Single-instance lock + SCRIPT_DIR-relative resolve of `RECOVER`. See 10.29. |
| `scripts/common.sh` | ~40 | **Shared script primitives.** Centralizes formatted echo statements (`step`, `ok`, `warn`, `die`) and the safe background `run_with_timeout` execution wrapper. Sourced by all orchestrator scripts across macOS and Linux. |
| `scripts/setup-common.sh` | ~350 | **Shared installation logic.** Centralizes logic for verifying Docker, installing Tailscale/wgcf, generating `.env`, and prompting for auth keys. Sourced by `setup.sh` and `setup-linux.sh`. |
| `scripts/toggle-linux.sh` | ~900 | **Linux sibling of `toggle.sh`.** Sources `common.sh`. Native Linux equivalents for every macOS-specific primitive  `nmcli radio wifi off/on` instead of `networksetup -setairportpower`, `ip link set ... down/up` fallback, systemd-resolved wiring. Paired with `.desktop` shortcuts. |
| `scripts/recover-linux.sh` | ~250 | **Linux sibling of `recover.sh`.** Sources `common.sh`. Same nuclear semantics (flush, restart, verify)  but using `nmcli` / `ip link` instead of macOS `networksetup`. |
| `scripts/setup-linux.sh` | ~80 | **Linux sibling of `setup.sh`.** Sources `setup-common.sh`. Distro detection (apt/dnf/pacman), installs Linux-specific dependencies, creates `.desktop` shortcuts. |
| `scripts/Toggle-Gateway.bat` | ~300 | **Windows Toggle helper.** Moved from root to `scripts/`. Helps invoke the framework on Windows if applicable. |
| `scripts/reinstall-host-tailscale.sh` | ~740 | **Nuke-and-reinstall tool for host Tailscale.** Completely purges Tailscale, its settings, macOS Background Items (`.btm` database), and stale System Extensions. Wipes ALL Login Items via `sfltool reset-login-items` (requires sudo). Useful for crisis recovery but destructive to other login items. |
| `scripts/fix-ssh-delay.sh` | ~100 | **Fixes SSH connection delays.** One-shot script to address SSH delays caused by DNS or routing issues, useful in crisis situations. |
| `scripts/logger/` | ~100 | **Internal logger piped from `scripts/routing-fix.sh`** inside the `warp` container. Ported to Go from Python. Built via Docker multi-stage build. |
| `black_list.txt` | 71 | Custom domains to block (ads, trackers, telemetry). Supports `$important` modifier. |
| `white_list.txt` | 222 | Domains to force-allow (YouTube, Apple services, etc.). Always wins over blocks. |
| `.env` | ~14 | WARP WireGuard keys, Tailscale auth key, rule profile. **Contains secrets & NULLEXIT_SEED.** |
| `.gateway_ip` | 1 | Static gateway Tailscale IP (fallback for dynamic resolution). |
| `scripts/unlock-files.sh` | ~20 | **One-shot stale-permission fix.** Uses atomic rename (`cp` + `mv`) to replace locked inodes (mode `000`/`0444` from old `chmod` decisions) with fresh writable ones  no
```

```
‘chmod‘ called. |
| ‘scripts/crypto.sh‘ | ~50 | **Cryptographic integrity enforcement.** Uses ‘NULLEXIT_SEED‘ from ‘.env‘ to sign core bash scripts (HMAC-SHA256) and strictly verify them on start to prevent manipulation. Pass ‘--sign‘ or ‘--verify‘. |
| ‘scripts/pf.conf‘ | 35 | **macOS native firewall ruleset.** Enforces the default-deny kill-switch, allows local LAN / Tailscale traffic, and applies TCP MSS Clamping (‘max-mss 1160‘) to prevent WireGuard MTU fragmentation. |

---

## 4. toggle.sh Flow

### State Detection
‘is_gateway_active()‘ returns true if EITHER containers are running OR host DNS is not ‘1.1.1.1‘. This dual check prevents the script from starting when it should stop (e.g., containers crashed but DNS is still hijacked).

### START Path (gateway OFF  ON)
1. **Reset DNS to 1.1.1.1**  Prevents deadlocks during startup
2. **Disconnect host Tailscale**  ‘tailscale down‘ (prevents exit-node routing during container boot)
3. **Compile DNS rules**  ‘docker compose run --rm rule-compiler‘ (Go binary inside multi-stage Docker build)
4. **Boot Colima VM**  ‘colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged‘ if not running
   - **Why Bridged Networking?** The ‘--network-mode bridged‘ and ‘--network-address‘ flags are critical. By default, Colima creates an isolated network. Bridged mode forces the VM to receive its own IP address directly from your local router’s DHCP server, placing it on the same physical subnet as your host machine. This bypasses macOS network isolation and is absolutely required for peer-to-peer (P2P) connections, mDNS, and local LAN service discovery to function properly across the container boundary.
   - **Enterprise Wi-Fi Fallback & Dynamic Roaming:** ‘toggle.sh‘ dynamically queries ‘system_profiler SPAirPortDataType‘ on boot. If it detects a **WPA2-Enterprise** connection (802.1X), it forces Colima to fall back to ‘--network-mode shared‘. This prevents aggressive network intrusion systems on corporate/university Wi-Fi switches from terminating the host’s Wi-Fi connection due to "MAC spoofing" (detecting multiple MAC addresses originating from a single authenticated port). P2P connections are impossible on these networks anyway due to AP Client Isolation.
     - **Dynamic Roam Downgrading:** ‘toggle.sh‘ writes its selected network mode to ‘/tmp/nullexit_colima_mode.txt‘. When roaming between networks (e.g., Home to University), ‘recover.sh‘ actively checks this state against the new network’s security requirement (‘.lan_p2p_detected‘). If a mismatch is detected (e.g., bridged Colima on an Enterprise network), ‘recover.sh‘ automatically forces a full ‘toggle.sh --restart‘ in the background to safely transition the Virtual Machine’s network mode and protect the host from de-authentication.
5. **Configure VM swap**  400MB swap file inside the VM to prevent OOM
6. **Clean corrupted AdGuard config**  Remove empty ‘AdGuardHome.yaml‘
7. **Start containers**  ‘docker compose up -d‘
8. **Wait for gateway Tailscale**  Poll ‘tailscale status‘ for "offers exit node" (up to 60s, abort at 40 consecutive NoState)
9. **Resolve gateway IP**  From ‘.gateway_ip‘ or ‘docker compose exec tailscale tailscale ip -4‘
10. **Connect host to mesh**  Verify ‘tailscaled‘ is running (auto-start if needed), then ‘tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true --exit-node=‘ (‘--accept-routes=true‘ must be explicit  ‘--reset‘ reverts it to ‘false‘ otherwise, silently preventing the default route from switching to ‘utun*‘)
11. **Pre-flight checks**  [1/3] ‘tailscale ping‘ gateway, [2/3] ‘dig +tcp‘ AdGuard DNS, [3/3] WARP container internet
12. **If all pass**  Set exit node + hijack DNS to gateway IP (single server, no fallback)
13. **If any fail**  Enable SOCKS5 proxy + local DNS proxy as fallback
14. **Final DNS re-force**  ‘force_dns_to_gateway‘ called again after exit-node transition (tailscaled clobbers DNS briefly)
15. **Enable sleep prevention**  ‘caffeinate -i‘ in background to prevent idle sleep

### STOP Path (gateway ON  OFF)
1. Disconnect host Tailscale (‘tailscale down‘)
2. ‘docker compose down -t 5‘
3. Reset DNS to 1.1.1.1
4. Stop local DNS proxy
5. **Stop sleep prevention**  Kill caffeinate process
6. Full network state cleanup (proxies off, DNS cache flush, route flush, Wi-Fi power cycle)

### Safety Mechanisms
- ‘run_with_timeout()‘  Pure-bash watchdog for every system command (15s120s)
- ‘cleanup_handler()‘  Trap on ERR/INT/TERM/HUP restores DNS to 1.1.1.1, kills caffeinate, and tears down Tailscale
- ‘force_dns_to_gateway()‘  Sets DNS and VERIFIES via ‘networksetup -getdnsservers‘ (3 attempts with backoff)
- ‘SUCCESS_RUN‘ flag  Cleanup only runs if script didn’t complete successfully
- ‘start_sleep_prevention()‘ / ‘stop_sleep_prevention()‘  Manages ‘caffeinate -i‘ via PID file at ‘/tmp/nullexit-caffeinate.pid‘

---

## 5. Routing & Firewall Architecture (Inside Container)
```

```

### IP Rules (‘ip rule show‘)
| Priority | Match | Table | Purpose |
|----------|-------|-------|---------|
| 99 | ‘to 100.64.0.0/10‘ | 52 | CGNAT range  Tailscale routes (injected by routing-fix) |
| 100 | ‘from 172.18.0.2‘ | 200 | Container’s own traffic (SOCKS5 proxy)  tun0 |
| 101 | all | 51820 | Gluetun’s WireGuard fwmark  tun0 |

### Table 200 (SOCKS5 proxy traffic)
- ‘162.159.192.1 via $DOCKER_GW dev eth0‘  WARP endpoint exception (prevents tunnel loop)
- ‘default dev tun0‘  Everything else through WARP

### iptables (TWO separate stacks!)
- **nftables backend** (‘iptables‘)  Used by Gluetun’s ‘post-rules.txt‘. FORWARD policy DROP.
- **legacy backend** (‘iptables-legacy‘)  Used by Tailscale’s ‘ts-forward‘. FORWARD policy ACCEPT.
- Both stacks apply to every packet. FORWARD RELATED,ESTABLISHED must exist in BOTH.

### post-rules.txt (loaded by Gluetun)
‘‘‘
FORWARD: ACCEPT for tailscale0 in/out
INPUT: ACCEPT for tailscale0
NAT POSTROUTING: MASQUERADE on tun0
MANGLE: TCP MSS clamp to 1180
NAT PREROUTING: Redirect DNS (53)  5335 on tailscale0 and eth0
ip6tables: DROP FORWARD on tailscale0
‘‘‘

---

### 5.1 Tailscale Local Network Discovery (DERP Bypass)
By default, Gluetun’s strict NAT swallows all of Tailscale’s UDP hole-punching packets, forcing Tailscale to fall back to incredibly slow DERP relay servers (often adding 500ms+ latency).

To fix this, we use ‘FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8‘ in ‘docker-compose.yml‘ (on the ‘warp‘ container). This creates ‘ip rule‘ bypasses (Priority 99, Table 199) that route all packets destined for private IP ranges *outside* the WARP tunnel directly to the host’s ‘eth0‘.
- **172.16.0.0/12** is especially critical because many modern Wi-Fi routers (and Docker itself) assign IPs in this block (e.g., ‘172.17.x.x‘).
- This bypass allows Tailscale’s UDP packets to reach devices on the same Wi-Fi directly, establishing a fast peer-to-peer connection while the actual web traffic inside the tunnel remains fully routed through WARP.

---

### 5.2 Firewalling & Per-Device Access Control
Because every mesh device’s traffic passes through the ‘warp‘ container’s network namespace, this namespace serves as the ultimate choke point for network-wide firewall rules.

**Where to Put Rules**
The right place to add firewall rules is ‘scripts/routing-fix.sh‘. It runs a 30-second loop inside the namespace and already handles re-injecting rules that Gluetun resets on reconnect. **Do not use ‘post-rules.txt‘ for dynamic rules**, as Gluetun flushes and resets it upon VPN reconnects.

**The Dual iptables Backend Constraint (CRITICAL)**
The container runs two simultaneous iptables backends: ‘iptables‘ (for the nftables backend used by Gluetun) and ‘iptables-legacy‘ (for Tailscale). Every rule **must** be added to both stacks, or it will silently fail for certain traffic.

**Avoiding Duplication**
Because ‘scripts/routing-fix.sh‘ runs in a loop, you must always check for a rule’s existence with ‘-C‘ before appending with ‘-A‘. Otherwise, your rules will duplicate every 30 seconds.

**Per-Device Identification & Filtering**
Each mesh device has a stable ‘100.x.x.x‘ Tailscale IP, which is visible as the ‘src‘ IP in the ‘FORWARD‘ chain. This is how you identify devices for filtering.
* **Domain-level:** AdGuard Home (port 3000, credentials ‘admin/nullexit‘) provides a REST API that supports per-client blocking rules based on their Tailscale IP.
* **IP-level:** Use ‘iptables‘ in ‘scripts/routing-fix.sh‘ (e.g., ‘iptables -I FORWARD -s 100.x.x.x -d <blocked_ip> -j DROP‘).
* **Country-level:** Add ‘ipset‘ to the ‘routing-fix‘ apk install step in ‘docker-compose.yml‘, and load CIDR ranges from ‘ipdeny.com‘ into an ipset, then block that set in the ‘FORWARD‘ chain.

## 6. macOS Quirks to Remember

1. **Colima SSH tunnel is TCP-only**  UDP ports (like 5354/udp) are NOT accessible from host. Use ‘dig +tcp‘ or the Python DNS proxy (UDPTCP converter).
2. **‘docker compose exec -T‘ injects ‘\r‘**  Always ‘tr -d ’\r’‘ when capturing output for comparisons or ‘networksetup‘ commands.
3. **‘brew services‘ can get stuck**  Fix with ‘sudo brew services restart tailscale‘. Common state: ‘brew services list‘ shows ‘started‘ but ‘tailscale status‘ shows "Tailscale is stopped."
```

```
4. **No ‘timeout‘ command**  macOS lacks GNU ‘timeout‘. Use the ‘run_with_timeout()‘ bash function.
5. **‘sudo -v‘ prompts even with NOPASSWD**  Use ‘sudo -n‘ (non-interactive) everywhere.
6. **DNS is scoped to en0**  macOS scutil DNS resolver is scoped to en0 (Wi-Fi). DNS changes on other interfaces (like USB Ethernet) are ignored unless en0’s service is also updated. The script always sets DNS on BOTH ‘$ACTIVE_SERVICE‘ and ‘$EN0_SERVICE‘.
7. **tailscaled clobbers DNS during exit-node transition**  ‘force_dns_to_gateway‘ is called a second time at the end of the ENABLE branch to counteract this.
8. **‘tailscale up --reset‘** resets all unspecified flags to defaults  Every ‘--reset‘ call MUST also pass ‘--accept-dns=false‘ explicitly or tailscaled re-enables DNS management.
9. **‘docker compose ps‘ CWD pitfall**  ‘docker compose‘ commands require a ‘docker-compose.yml‘ in the current working directory. Use ‘docker ps‘ for raw container listing from any CWD; ‘docker compose ps‘ requires the project directory.

---

## 7. Environment Variables

### ‘.env‘ File
| Variable | Purpose |
|----------|---------|
| ‘WIREGUARD_PRIVATE_KEY‘ | WARP WireGuard private key (from ‘wgcf generate‘) |
| ‘WIREGUARD_PUBLIC_KEY‘ | WARP WireGuard public key |
| ‘WIREGUARD_ADDRESSES‘ | WARP WireGuard address (e.g., ‘172.16.0.2/32‘) |
| ‘TS_AUTHKEY‘ | Tailscale auth key for the container |
| ‘NULLEXIT_SEED‘ | 256-bit seed for HMAC-SHA256 script integrity signing (generated by ‘setup.sh‘) |
| ‘GATEWAY_RULE_PROFILE‘ | Rule compilation tier: ‘light‘, ‘medium‘, ‘heavy‘ |
| ‘GATEWAY_BYPASS_PING‘ | (Optional) ‘true‘ to proceed even if pre-flight checks fail |
| ‘GATEWAY_USE_EXIT_NODE‘ | (Optional) ‘false‘ to skip exit node, DNS-only mode |
| ‘GATEWAY_HIJACK_HOST‘ | (Optional) ‘false‘ to skip DNS hijacking on the host (VPN/adblocking for Tailscale peers only) |
| ‘GATEWAY_MSS‘ | (Optional) TCP MSS clamp value (default 1120); 1180 for speed on healthy paths |
| ‘WARP_FAIL_THRESHOLD‘ | (Optional) Consecutive ‘warp=off‘ polls before auto-shutdown (default 6 = 30s; 3 = 15s) |
| ‘WARP_ENDPOINT_1‘ | (Optional) Override Cloudflare WARP WireGuard endpoint IP 1 (default ‘162.159.192.1‘) |
| ‘WARP_ENDPOINT_2‘ | (Optional) Override Cloudflare WARP WireGuard endpoint IP 2 (default ‘162.159.193.1‘) |
| ‘KILL_SWITCH‘ | (Optional) ‘true‘ to enforce strict PF lock  drops all host traffic if VPN fails. Breaks SSH if VPN dies. |
| ‘HOST_LEAK_PROBE‘ | (Optional) ‘false‘ to disable the 300ms host-egress leak prober (default: ‘true‘) |
| ‘STOP_COLIMA_ON_EXIT‘ | (Optional) ‘true‘ to fully shut down the Colima VM on toggle-off (saves battery on dedicated hosts) |
| ‘ADGUARD_USER‘ | (Optional) AdGuard Home web UI username (default: ‘admin‘) |
| ‘ADGUARD_PASSWORD‘ | (Optional) Shared password for AdGuard Home and Tor ControlPort (default: ‘nullexit‘) |
| ‘BLOCKED_COUNTRIES‘ | Space-separated 2-letter ISO codes to block via ipdeny.com CIDR ranges (e.g. ‘"kp il cn ru"‘) |

---

## 8. Diagnostic Commands

### Container Health
‘‘‘bash
docker ps --format ’{{.Names}} {{.Status}}’
docker compose exec -T tailscale tailscale status
docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace
‘‘‘

### SOCKS5 Proxy
‘‘‘bash
curl --socks5-hostname 127.0.0.1:1080 --max-time 5 -s https://www.cloudflare.com/cdn-cgi/trace
‘‘‘

### AdGuard DNS
‘‘‘bash
dig +tcp @127.0.0.1 -p 5354 google.com +short +timeout=5
‘‘‘

### Firewall & Routing
‘‘‘bash
# nftables FORWARD chain
docker compose exec -T warp iptables -L FORWARD -v --line-numbers
# Legacy FORWARD chain
docker compose exec -T warp iptables-legacy -L FORWARD -v --line-numbers
# IP rules + routing tables
docker compose exec -T warp ip rule show
docker compose exec -T warp ip route show table 199
docker compose exec -T warp ip route show table 200
‘‘‘

```

~~~~
### Host macOS
```bash
tailscale status
brew services list | grep tailscale
networksetup -getdnsservers Wi-Fi
networksetup -getsocksfirewallproxy Wi-Fi
sysctl net.inet.ip.forwarding
```

---

## 9. Development Issue Tracker

This section documents issues encountered during development, their status, and resolutions.

### 9.1 Exit Node Return Path (`rx 0`)  RESOLVED
**Symptom:** Gateway showed `tx 936 rx 0`  transmitted but never received return traffic.

**Root Cause:** `docker-compose.yml` included `FIREWALL_OUTBOUND_SUBNETS=100.64.0.0/10`. Gluetun created a strict routing rule (`ip rule add to 100.64.0.0/10 lookup 199`, priority 99) that forced all Tailscale CGNAT traffic to bypass the VPN via the Docker bridge. Return packets were sent to the macOS host instead of back through `tailscale0`.

**Fix:** Removed `FIREWALL_OUTBOUND_SUBNETS` entirely. `scripts/routing-fix.sh` now injects `lookup 52`, and return packets correctly flow back into `tailscale0`. The SOCKS5 proxy was repurposed as a bulletproof failover.

### 9.2 Gluetun Resets nftables FORWARD Rules
**Symptom:** FORWARD RELATED,ESTABLISHED rule disappears after Gluetun health check.

**Fix:** `scripts/routing-fix.sh` re-adds to both iptables backends every 30 seconds. Legacy backend (policy ACCEPT) acts as safety net.

### 9.3 docker compose exec `\r` Injection
**Fix:** All `docker compose exec` output piped through `tr -d '\r'`.

### 9.4 Colima VM Memory / Swap Thrashing Latency
**Symptom:** After running nullexit flawlessly for over 24 hours straight to test stability, the 0.5GB VM RAM configuration began experiencing severe network latency on the Tailscale mesh later in the day.

**Root Cause:** Services (like AdGuard, Tailscale, Docker logs) slowly accumulate cache and state over extended periods. Under the tight 512MB physical RAM limit, this slow creep forced the Linux kernel to aggressively swap memory to the 512MB SSD swap file. The constant disk I/O thrashing blocked process execution, causing DNS resolutions and packet forwarding to delay significantly (making the internet feel "very slow").

**Proof (Empirical Observation):** While in this OOM-thrashing state, direct DNS queries through the Tailscale mesh (`dig @100.100.21.8 google.com`) would intermittently hang or take several seconds to resolve. After restarting with the 600MB configuration, the exact same query immediately dropped to a lightning-fast **41ms** response time. (Note: Querying the mapped host port via `127.0.0.1:5354` will always time out due to Gluetun's strict leak-prevention firewall blocking non-VPN ingress traffic).

**Fix:** Increased VM base physical RAM to 600MB (`--memory 0.6`) and reduced the swap file to 400MB. This provides enough native RAM headroom for long-running state caches without forcing heavy I/O swapping. Subdomain deduplication still reduces blocklists by ~60%. Memory profiles (`light`/`medium`/`heavy`) let users trade off coverage for memory.

### 9.5 tailscaled Daemon Broken State
**Symptom:** `brew services list` shows `started` but `tailscale status` shows "stopped".

**Fix:** `sudo brew services restart tailscale`. Toggle script now detects and auto-restarts.

### 9.6 Stderr Invisibly Swallowed
**Symptom:** Failures were silent due to stderr redirected to `output.log`.

**Fix:** Tailscale wait loop now shows output. Critical commands show errors inline.

### 9.7 Missing iptables in routing-fix Container
**Root Cause:** `alpine:3.20` does not have iptables installed. Commands failed silently with `2>/dev/null || true`.

**Fix:** Updated `docker-compose.yml` to `apk add --no-cache iptables iproute2` before `scripts/routing-fix.sh`.

### 9.8 DOCKER_GW Variable Parsing Bug
**Root Cause:** `ip route show default` printed two lines; `awk '{print $3}'` picked up both, causing route commands to fail.

**Fix:** Added `head -1` to the parsing chain.

~~~~

```
### 9.9 Native SSH & SFTP Security (Zero Local Attack Surface)
**Context:** macOS "Remote Login" (‘sshd‘) and "File Sharing" (‘smbd‘) open ports on the local network (e.g. ‘172.x.x.x‘), exposing the host to brute-force attacks on public Wi-Fi.

**Implementation:** The script strictly enforces ‘tailscale up --ssh=true‘ whenever the mesh state is reset. This empowers the user to completely disable native macOS Remote Login and File Sharing. This provides an incredible zero-trust security advantage: even if an attacker steals your Mac username and password, they **cannot** access your files remotely. Disabling the native services completely closes the listening ports on the local Wi-Fi interface (‘172.x.x.x‘), rendering the Mac inaccessible to local network logins. Meanwhile, Tailscale intercepts port 22 traffic exclusively over the encrypted mesh and authenticates cryptographically based on your Tailscale SSO identity. Stolen Mac passwords are mathematically useless without first bypassing your Tailscale Two-Factor Authentication and registering a device onto the mesh.

**Cross-Platform File Exchange:** To easily browse and exchange files securely, simply use an SFTP client and connect to your Mac’s MagicDNS name on port 22. Recommended clients: **WinSCP** or **FileZilla** (Windows), **Cyberduck** (macOS), and **FE File Explorer** or **Solid Explorer** (iOS/Android).

### 9.10 Changing macOS Local Hostname
**Context:** Users may wish to change their Mac’s computer name/hostname in macOS System Settings.
**Effect:** Changing the Mac’s hostname will **not** break Nullexit (which relies on internal routing/localhost). However, Tailscale will automatically detect this change and update the machine name on the Tailnet.
- The underlying Tailscale ‘100.x.x.x‘ IPv4 address will remain exactly the same.
- The **MagicDNS name** will change to match the new hostname. Users must remember to update their SFTP/SSH clients to use the new MagicDNS address.

### 9.11 Logging Architecture
For debugging, logs are strictly segmented based on the component’s lifecycle:
- **‘output.log‘ (Host-side):** Contains all standard error (‘stderr‘) and verbose output from the host scripts (‘toggle.sh‘, ‘recover.sh‘, ‘setup.sh‘). Since the ‘rule-compiler‘ container is ephemeral and deleted after running, its logs (and any Python errors from ‘scripts/sync-rules.py‘) are extracted and appended to this file before deletion.
  - **Log Rotation Policy:** On startup, ‘toggle.sh‘ checks if ‘output.log‘ exceeds **50MB** (‘52,428,800‘ bytes). If it does, it performs a metadata-only rename (‘mv output.log output.log.old‘), preserving history while resetting the active log to 0 bytes. This rename-based rotation avoids the heavy disk I/O and CPU overhead of rewriting a massive file line-by-line.
  - **Egress Probe Logging:** The background host-egress leak prober checks if stdout is a TTY (‘[ -t 1 ]‘) and will only print live status updates (using carriage return ‘\r‘) or duplicate console warnings in interactive mode. It automatically detects macOS/BSD vs. Linux environments to dynamically construct timestamps with date and time (omitting milliseconds on macOS to prevent high CPU utilization from spawning sub-second processes).
- **‘docker logs <container>‘ (Guest-side):** The persistent containers (‘warp‘, ‘tailscale‘, ‘routing-fix‘, ‘adguardhome‘) use the Docker ‘json-file‘ logging driver with a strict ‘max-size‘ (1m-10m) to prevent VM disk exhaustion. Use standard ‘docker logs‘ to view them.

### 9.12 Chrome Remote Desktop Connection Failures
**Context:** When attempting to access a remote machine via Chrome Remote Desktop (CRD), the connection fails or hangs if Tailscale is active **on the remote device**. The connection works fine even if Tailscale (and the exit node) is active **on the local client/viewer device**, but only if Tailscale is disabled on the remote machine.

**Why:** Having Tailscale active on the remote machine creates virtual network interfaces and DNS/routing modifications that conflict with Chrome Remote Desktop’s WebRTC bindings and signaling traffic.

**Workaround:**
- **Use Tailscale Native RDP/RustDesk (Recommended):** Connect directly to the remote device’s Tailscale IP (‘100.X.Y.Z‘) via an RDP or RustDesk client. This is faster and avoids third-party relay conflicts.
- **Disable Tailscale on the Remote Device:** If you must use CRD, disable Tailscale on the remote machine before connecting.

### 9.13 Docker Default Bridge vs Host Wi-Fi Subnet Collision (The 172.17.0.0/16 Subnet Overlap)
**Context:** When attempting to ping a local Windows client (‘172.17.22.187‘) or the default gateway (‘172.17.0.1‘) directly over Wi-Fi, the Mac host returns ‘No route to host‘ (displaying a ‘REJECT‘ / ‘!‘ flag in the routing table). Tailscale on the client also drops offline shortly after starting when using the exit node, failing to establish direct P2P connections and falling back to slow DERP relays.

**Root Cause & Subnet Overlap Mechanism:**
1. **Docker Default Subnet:** By default, the Docker daemon initializes its default bridge network (‘docker0‘) on the **‘172.17.0.0/16‘** IP range.
2. **Subnet Conflict:** The host’s physical Wi-Fi network happens to be assigned the exact same **‘172.17.0.0/16‘** range by the local network router.
3. **Routing Clash:** Colima launches a Linux VM on the Mac host to run the Docker engine. Because Docker inside that VM creates ‘docker0‘ and claims the ‘172.17.0.0/16‘ subnet, any packet sent to ‘172.17.x.x‘ from within the containers (such as Tailscale local discovery) is captured by the VM’s internal virtual bridge interface (which is down/empty) and is blackholed. On the Mac host, macOS dynamically marks routes as ‘REJECT‘ (‘!‘) when local ARP resolution fails, causing local pings to immediately abort with ‘No route to host‘.
```

~~~
4. **Tailscale Relay Saturation**: Because local peer-to-peer discovery is blackholed, Tailscale falls back to public DERP relay servers (e.g. `relay "tor"` in Toronto). When exit-node routing is enabled, all client web traffic is routed through the relay. Public relays impose strict rate limits and throttle high-volume traffic, resulting in packet drops. This saturation drops Tailscale's control packets, causing the connection to time out and showing the client as offline.

**Fix:**
The canonical script for this is `scripts/fix-docker-bridge-collision.sh`. **One command applies the entire fix:**

```bash
bash scripts/fix-docker-bridge-collision.sh        # apply
bash scripts/fix-docker-bridge-collision.sh --dry  # preview changes without applying
```

It auto-detects the host's actual Wi-Fi subnet (no more guessing whether the campus DHCP handed out `172.x`, `10.x`, or `192.168.x`), picks a non-conflicting `docker.bip` from a small candidate set (defaulting to `172.26.0.1/24` if no overlap is matched), backs up `~/.colima/default/colima.yaml` with an ISO-8601 timestamp, edits the file **in place** (idempotent  replaces an existing `docker.bip` if present, appends under an existing `docker:` block, or creates a fresh block), restarts Colima, rebinds Wi-Fi DHCP, verifies the new default route is no longer Docker's bridge, and re-runs both `./toggle.sh` and `scripts/diagnose-host-leak.sh` to print the post-fix verdict.

For reference, the underlying manual fix is: edit the Colima configuration file (`~/.colima/default/colima.yaml`) on your Mac to define:
```yaml
docker:
  bip: 172.26.0.1/24   # any /24 that does NOT overlap your Wi-Fi subnet
```
Then, restart Colima to apply the new subnet inside the VM:
```bash
colima restart
```
This forces Docker to use `172.26.x.x` for its default bridge, freeing the physical `172.17.x.x` range and letting pings and direct Tailscale peer-to-peer connections flow normally.

---


### 9.14 Cloudflare WARP 24-Second Startup Delay (UDP Session Rate-Limiting)
**Context:** When running `toggle.sh` to turn the gateway ON, `docker compose up -d` often hangs for exactly 24-25 seconds waiting for the `warp` container to become `healthy`. Users might mistakenly blame the Go `rule-compiler` processing 400k+ rules for this delay, as Docker prints `rule-compiler Exited 24.3s` at the end of the wait.
**Root Cause:** The `rule-compiler` actually finishes its work in <2 seconds. The 24-second blockage is entirely due to Cloudflare WARP's edge server rate-limiting. WireGuard is a stateless protocol with no "disconnect" packet. When the toggle is turned OFF, the old UDP session state remains active on Cloudflare's backend for 20-30 seconds. When the toggle is instantly turned back ON, Docker starts a new WireGuard connection from a new ephemeral UDP source port but uses the **same static cryptographic key** (from `.env`). Cloudflare detects this as a potential replay attack or key-sharing abuse, and intentionally drops all packets (rate-limits) for the new connection until the old session's ghost state fully expires (~20-25 seconds).
**Result:** Gluetun's healthcheck fails repeatedly every 2 seconds until Cloudflare lifts the penalty box, at which point traffic flows and Docker unblocks the rest of the startup sequence. This is a known, expected behavior of reusing static keys rapidly on Cloudflare's network and cannot be bypassed.

### 9.15 Background Sudo Failures
**Context:** The `--post-wake` script is intentionally designed to skip the `sudo -v` interactive prompt because it is launched by `watcher.sh` running via `launchd` in the background (which has no TTY and cannot accept password input). Thus, it relies entirely on `sudo -n` (non-interactive sudo) for all its internal routing changes.
**Problem:** If the user has not configured the `/etc/sudoers.d/nullexit` file properly, any automated background wake event will silently fail: `sudo -n` commands drop out, the WARP bypass routes fail to apply, and the `warp` container loses internet connectivity.

### 9.16 Tailscale Hijacking the Default Route (PHYSICAL_GW Bug)
**Problem:** When `--post-wake` runs, it clears the routing table using `sudo route -n flush` to ensure a clean slate after the Wi-Fi card roams or wakes up. Because it skips bouncing the Wi-Fi interface (to make the reconnect faster), macOS's DHCP client does not automatically rebuild the physical default route. To counteract this, the script must manually restore the physical default route. Originally, the script tried to capture the physical gateway via `route get default`. However, because the gateway is already running and Tailscale is active, the default route is often already pointing to `utunX`. When this happens, `route get default` does not output a `gateway:` field (it only outputs the `interface:`), causing the captured `PHYSICAL_GW` variable to be empty.
When `PHYSICAL_GW` is empty, the physical default route is never restored, causing the `en0` interface to be isolated from the physical network. The WARP bypass routes then fall back to targeting `-interface en0` (instead of routing via the gateway), which breaks WireGuard's remote connections completely.
**Fix:** The script now bypasses the OS routing table entirely and directly queries the hardware DHCP lease (`ipconfig getpacket en0`) to reliably extract the physical router IP, ensuring the physical route is correctly restored even when Tailscale is active.
~~~

```

### 9.17 Tailscale MagicDNS Bypassing the AdGuard PREROUTING Intercept
**Context:** The ‘routing-fix.sh‘ script applies an iptables NAT rule (‘PREROUTING -i tailscale0 -p udp --dport 53 -j REDIRECT‘) to intercept all incoming DNS queries from connected Tailscale peers and force them into the AdGuard container on port 5335.
**Problem:** When remote clients (like Android or iOS) have "Use Tailscale DNS settings" turned on, they query Tailscale's MagicDNS IP (‘100.100.100.100‘). This packet enters the exit node, but the ‘tailscaled‘ daemon running *inside* the gateway container intercepts the ‘100.100.100.100‘ packet internally. ‘tailscaled‘ then generates a brand new, local DNS request to the upstream DNS server to resolve the query. Because this new request is generated *locally* by the daemon (not arriving externally via the ‘tailscale0‘ interface), it bypasses the ‘PREROUTING‘ chain completely. The request glides straight out the WARP tunnel to Cloudflare/Google, entirely bypassing the AdGuard sinkhole.
**Fix:** The only way to fix this blindspot is to force ‘tailscaled‘ to use AdGuard as its upstream. In the Tailscale Admin Console, the gateway's Tailscale IPv4 address (e.g., ‘100.x.x.x‘) must be added as the sole Custom Global Nameserver, and "Override local DNS" must be enabled. Additionally, to block Android devices from bypassing this via Private DNS (DNS-over-TLS), outbound Port 853 traffic is explicitly REJECTED in both ‘routing-fix.sh‘ and ‘post-rules.txt‘.

### 9.18 Network Bufferbloat & MTU Optimizations (Double-VPN UDP Crash)
**Symptom:** When running aggressive UDP bandwidth tests (such as macOS ‘networkQuality‘, which uses QUIC/HTTP3), the ‘nullexit‘ tunnel completely crashes or exhibits severe lag (6,000ms+ bufferbloat).

**Root Cause:** The bug is a cascading MTU mismatch. Tailscale generates a UDP packet. Cloudflare WARP (also WireGuard) receives the packet, wraps it in another layer of encryption, pushing the packet size beyond the standard 1500 byte limit. Unlike TCP, UDP packets cannot be resized dynamically via MSS negotiation, resulting in massive IP packet fragmentation. The Apple Virtualization framework (‘vz‘), which Colima uses for bridged networking, silently drops heavily fragmented UDP packets under high load. This blackholes the entire UDP upload stream, instantly dropping the connection.

**Fix (Partial):** To prevent MTU fragmentation for standard web traffic, **TCP MSS Clamping** is strictly enforced via the macOS ‘pf‘ firewall. By adding ‘scrub out all max-mss 1160‘ to ‘scripts/pf.conf‘, macOS unilaterally shrinks all outgoing TCP headers to fit comfortably inside the double-encrypted tunnel, bypassing Path MTU Discovery blackholes. ‘TS_DEBUG_MTU=1200‘ was also added to ‘docker-compose.yml‘ to minimize internal fragmentation.
**Unresolved:** The only way to solve the UDP crash bug natively is to artificially cap the maximum tunnel bandwidth using SQM (‘fq_codel‘). Because a static bandwidth cap would artificially throttle high-speed networks, ‘nullexit‘ leaves the bandwidth uncapped, optimizing perfectly for TCP while accepting UDP fragmentation under extreme edge-case load tests.
*Why not Dynamic SQM (Autorate)?* Implementing an EMA/PID-controlled autorate daemon (like ‘sqm-autorate‘) requires a constant 100ms polling loop to measure the kernel packet queue, which severely degrades laptop battery life by preventing deep C-state CPU idling. Furthermore, macOS's native firewall (‘dummynet‘) only supports ‘fq_codel‘ and does not support the newer Linux ‘CAKE‘ algorithm, which is the only algorithm that offers kernel-native, event-driven ‘autorate-ingress‘ (zero polling penalty). Therefore, dynamic SQM is computationally hostile to battery-powered macOS hosts.

## 10. Resolved Issues (from README)

These bugs and edge cases were discovered and resolved during development.

### 10.1 DNS Proxy: TCP Wire Format Mismatch (socat / dnsmasq)

**Problem:** When the Tailscale data plane is unavailable (DERP relay only), the script falls back to a local DNS proxy that forwards queries from host UDP:53 to Docker's TCP:5354 (AdGuard). The ‘socat‘ and ‘dnsmasq‘ approaches both failed.

- **socat** forwards raw bytes  it does not prepend the 2-byte length prefix that DNS-over-TCP requires. AdGuard parses the stream incorrectly and returns garbage.
- **dnsmasq** tries UDP first to reach the upstream (‘127.0.0.1#5354‘), but Colima's SSH tunnel only forwards TCP. With a single upstream server, dnsmasq doesn't fall back to TCP even with ‘--timeout=1‘.

**Solution:** Replaced both with a **Python DNS proxy** (‘scripts/dns-proxy.py‘, ~25 lines) that properly handles the DNS-over-TCP wire format: reads a UDP query from the host, prepends the 2-byte length prefix, sends it over TCP to AdGuard, strips the prefix from the response, and sends it back over UDP.

*Architectural Note: Why is the DNS proxy in Python while the SOCKS5 proxy was moved to a compiled Go Docker container?*
The SOCKS5 proxy handles heavy data streaming (e.g., video, downloads). Python introduces massive CPU/battery overhead for raw socket streaming, which is why we rely on the compiled Go implementation built directly into the ‘tailscaled‘ daemon via ‘TS_SOCKS5_SERVER=:1080‘.
Conversely, the DNS proxy only handles intermittent UDP queries (a few bytes per minute) generated solely by the local host machine. The Python overhead for this is effectively 0.001 Watts. We deliberately kept ‘dns-proxy.py‘ in Python because it runs natively on the macOS/Linux hostrewriting it in Go would force users to install a Go compiler (‘brew install go‘) on their host machine for zero noticeable battery gain. (Note: If this architecture is ever adapted to serve as a public DNS resolver for thousands of users or for massive automated web-scraping clusters, ‘dns-proxy.py‘ must be rewritten in Go to survive the concurrent packet flood).

### 10.2 Exit Node Routing Conflict & The SOCKS5 Failover
```

~~~~

Problem: For days, Tailscale's exit node refused to return traffic back to the client (`rx 0`), leading to the assumption that Tailscale's userspace forwarding was bypassing `iptables` and ignoring the WARP `tun0` interface. In a desperate attempt to fix this, a **SOCKS5 proxy** (`scripts/socks5-proxy.py`) was introduced as a replacement.

The Real Root Cause: Tailscale's userspace forwarding was *not* the problem. The issue was an environment variable in Gluetun (`FIREWALL_OUTBOUND_SUBNETS=100.64.0.0/10`) that instructed the VPN container to hijack all Tailscale CGNAT traffic and forcibly route it out the unencrypted Docker bridge (`eth0`). This blackholed all return packets back to the client.

Solution: Removed the offending `FIREWALL_OUTBOUND_SUBNETS` variable, immediately restoring the Tailscale exit node. Instead of removing the SOCKS5 proxy, we repurposed it into a **bulletproof failover** for when restrictive Wi-Fi networks block Tailscale UDP ports.

Architecture after the fix:
```
PRIMARY (Exit Node):
DNS/TCP/UDP: Apps  Tailscale Exit Node  gateway container  tun0  WARP  internet

FAILOVER (SOCKS5 - if Tailscale is blocked by local Wi-Fi):
DNS: Browser  127.0.0.1:53  Python proxy  TCP:5354  AdGuard  WARP
TCP: Apps  SOCKS5:1080  gateway container  tun0  WARP  internet
Mesh: SSH/ping 100.x.x.x  utun5  WireGuard  peers (independent of proxy)
```

10.3 Pre-Flight Check 1: Wrong `tailscale ping` Flag

Problem: Check 1 of the pre-flight connectivity checks used `--c 1` (double dash) but `tailscale ping` accepts `-c 1` (single dash). The invalid flag caused the command to exit with code 1, marking the check as FAIL even though the gateway was reachable.

Fix: Changed `--c 1` to `-c 1`.

10.4 Pre-Flight Check 1: DERP Relay Exit Code

Problem: Even with the correct `-c 1` flag, `tailscale ping` exits with code 1 when the connection goes through a DERP relay (`direct connection not established`) even though a pong was successfully received (28ms via DERP(tor)). The check treated relayed connections as failures.

Fix: Changed the check from relying on exit code to piping stdout through `grep -q "pong"`. This accepts relayed connections as valid (they work fine for the exit node use case), while still failing on true timeouts or unreachable peers.

10.5 Container Default Route Bypasses WARP

Problem: Inside the WARP container's network namespace, the main routing table's default route was via `eth0` (Docker bridge), not `tun0` (WARP tunnel). While gluetun uses policy routing (rule 101 table 51820 `tun0`) for most traffic, traffic originating from the container's own IP (`172.18.0.2`) hits rule 100 (`from 172.18.0.2 lookup 200`), whose default was also via `eth0`. This caused the SOCKS5 proxy's outbound connections to bypass WARP.

Fix: Updated the `routing-fix` container to:
1. Change the main table's default route to `default dev tun0`
2. Change table 200's default route to `default dev tun0`
3. Add a specific route for the WARP WireGuard endpoint (`162.159.192.1`) through `eth0` via the Docker gateway in table 200, preventing a tunnel loop
4. Re-assert all routes every 30 seconds (gluetun may reset them on health checks)
5. Dynamically detect the Docker subnet from `ip route show dev eth0` instead of hardcoding `172.18.0.0/16`

10.6 IPv6 Exit-Node Leak

Problem: Tailscale supports IPv6, but WARP/gluetun is IPv4-only. IPv6 packets forwarded through the exit node would leak through the Docker bridge unencrypted.

Fix: Added `ip6tables -A FORWARD -i tailscale0 -j DROP` to `post-rules.txt`, blocking all IPv6 forwarded traffic from the Tailscale interface.

10.7 Hardcoded Docker Subnet in Routing Fix

Problem: The `routing-fix` container hardcoded `172.18.0.0/16` and `172.18.0.1` for the Docker bridge subnet and gateway. If Docker's bridge network changed (after `docker network prune`, Colima restart, or on a different machine), these routes would be wrong.

Fix: Detection is now dynamic using `ip route show`:
- `DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')`
- `DOCKER_GW=$(ip route show default | awk '{print $3}')`

This extracts the actual subnet and gateway directly from the routing table, working with any subnet size (`/16`, `/24`, `/20`, etc.).
~~~~

```

### 10.8 SOCKS5 Proxy Threading Race Condition

**Problem:** The ‘forward‘ function in ‘scripts/socks5-proxy.py‘ called ‘select.select([src, dst], [], [])‘ which raised ‘ValueError: file descriptor cannot be a negative integer (-1)‘ when one thread closed a socket while the other thread was selecting on it.

**Fix:** Added ‘ValueError‘ exception handling around ‘select.select()‘ and ‘recv()‘ calls. When a file descriptor becomes negative, the thread breaks out of the forwarding loop cleanly instead of crashing.

### 10.9 Docker Healthcheck: Multi-line Python in CMD Array

**Problem:** The socks-proxy healthcheck used a ‘CMD‘ array with a multi-line Python string. YAML collapsed the newlines, causing the ‘#‘ comment to comment out the rest of the code and the ‘assert‘ statement to be mangled into ‘as‘.

**Fix:** Changed from ‘["CMD", "python3", "-c", "..."]‘ to ‘["CMD-SHELL", "python3 -c '...'"]‘ with a single-line Python expression. Uses a real SOCKS5 handshake (sends ‘[5, 1, 0]‘, expects ‘[5, 0]‘) instead of a simple TCP port check.

### 10.10 Sudo Credential Caching Removed

**Problem:** The script used ‘sudo -v‘ at startup to cache sudo credentials, plus a background ‘SUDO_KEEPER_PID‘ loop refreshing them every 60 seconds. This required interactive password entry even with NOPASSWD sudoers configured, because ‘sudo -v‘ itself prompts.

**Fix:** Removed the entire ‘sudo -v‘ + ‘SUDO_KEEPER_PID‘ section. All privileged commands now use ‘sudo -n‘ (non-interactive), which works silently because the user has NOPASSWD rules in ‘/etc/sudoers.d/nullexit‘ for the specific commands needed (‘networksetup‘, ‘python3‘, ‘dscacheutil‘, ‘killall‘, ‘pkill‘, ‘route‘, ‘ifconfig‘).

### 10.11 macOS Application Firewall Silently Blocking AirDrop & Continuity

**Problem:** Users of complex network extensions (Tailscale, LuLu, Docker, etc.) often run into an issue where AirDrop, AirPlay, and Universal Clipboard silently fail, even with the gateway inactive and Wi-Fi/Bluetooth correctly configured. The internal Apple Wireless Direct Link (‘awdl0‘) interface appears healthy, but connections never establish.

**Root Cause:** The built-in macOS Application Firewall has a specific list of explicitly blocked applications. Apple's own core networking services (e.g., ‘/usr/libexec/sharingd‘, ‘/usr/libexec/rapportd‘, ‘/usr/libexec/avconferenced‘) can be silently added to the "Block incoming connections" listeither through accidental "Deny" clicks on permission prompts, execution of aggressive privacy-hardening scripts, or a known macOS bug that retains strict blocks even after toggling the "Block all incoming connections" setting off. Since these are background daemons, macOS provides no visible error.

**Fix:** The firewall rules must be cleared via the terminal (or System Settings > Network > Firewall > Options):
‘‘‘bash
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/sharingd
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/rapportd
‘‘‘
Once unblocked, ‘sharingd‘ immediately resumes broadcasting mDNS/Bonjour discovery packets, and AirDrop restores instantly without requiring a reboot.

### 10.12 Hard Reboots Leave Stale Docker Socket in Colima VM

**Problem:** If the macOS host machine is subjected to a hard reboot, sudden power loss, or a kernel panic, the Colima VM running in the background is abruptly killed. Upon next boot, running ‘toggle.sh‘ resulted in the contradictory output:
‘Colima is already running.‘ followed by ‘Cannot connect to the Docker daemon at unix://~/.colima/default/docker.sock.‘

**Root Cause:** The hard crash prevents the inner Docker daemon from executing its graceful shutdown routines. It leaves behind a corrupted ‘docker.sock‘ file inside the VM. On boot, the outer VM spins up (hence "already running"), but the inner Docker daemon sees the stale socket, assumes another instance is running, and crashes.

**Fix:** Added an Auto-Healing routine directly into ‘toggle.sh‘ (Step 4b). The script now explicitly tests the Docker daemon with ‘docker ps‘. If the daemon is unresponsive despite Colima reporting as active, it assumes a stale socket and automatically runs ‘colima restart‘ in the background to cleanly rebuild the VM and socket before proceeding.

### 10.13 Graceful Shutdowns Leave DNS Hijacked

**Problem:** The gateway overrides the macOS host's DNS settings (via ‘networksetup‘) to route queries to the AdGuard container. Because macOS persists ‘networksetup‘ changes to disk, shutting down the Mac while the gateway is running causes the Mac to boot up later with hijacked DNS. Since the Colima VM and Docker containers don't auto-start on boot, the user would have no internet access until they manually run the toggle script.
```

~~~

Root Cause: The `toggle.sh` script exits after the gateway starts. There was no background daemon listening for macOS system shutdown signals to revert the network settings.

Fix: Wrapped the background `caffeinate` process (used to prevent system sleep) in a bash `trap` command. The trap listens for `SIGTERM`, `SIGINT`, and `SIGHUP`. When the user initiates a graceful shutdown, macOS sends `SIGTERM` to the background trap, which instantly executes `recover.sh` to flush the host DNS back to `1.1.1.1` right before the machine powers off.
Note: We explicitly use `recover.sh` instead of `toggle.sh` because during a shutdown, the OS limits process cleanup time and is already independently sending termination signals to Docker and Colima. Trying to run `docker compose down` inside a shutdown trap creates a race condition that can hang the script until macOS forcefully `SIGKILL`s it. `recover.sh` abandons container management and surgically flushes the macOS network settings in milliseconds, guaranteeing completion before the OS pulls the plug.

10.14 Wi-Fi Edge Packet Loss via QUIC (UDP) Fragmentation

Problem: Applications that heavily utilize HTTP/3 (QUIC) over UDPsuch as Facebook, Instagram, and Google servicesexperience massive stuttering and stalled image loading when the client device is far away from the router (weak Wi-Fi signal). The issue did not occur when close to the router.

Root Cause: The `nullexit` gateway uses a double-encrypted tunnel (Tailscale + WARP). To prevent packets from fragmenting when entering these tunnels, a firewall rule in `post-rules.txt` uses `--set-mss 1120` to safely clamp TCP packets. However, because QUIC runs entirely over UDP, it bypasses the TCP MSS handshake completely. QUIC blasts maximum-MTU (1280 byte) UDP packets. The inner `tailscale0` interface (MTU 1200) forces these large UDP packets to fragment into two pieces. When the client is on the edge of Wi-Fi range (where 5-10% packet loss is common), losing *either* UDP fragment destroys the *entire* image frame. This exponential amplification of packet loss stalled QUIC streams.

Fix & Trade-offs: Appended an `iptables` rule to `post-rules.txt` that explicitly blocks `UDP --dport 443` (QUIC). When modern web apps detect QUIC is blocked, they instantly fall back to HTTP/2 (TCP 443). Because TCP is routed through the MSS clamp, the packets are resized *before* they leave, entirely eliminating IP fragmentation and restoring high-speed loading over weak Wi-Fi.
Consequences: This adds a minor latency penalty (TCP 3-way handshake) compared to QUIC's zero-RTT connection. It may also force WebRTC video calls (Google Meet/Discord) to fall back to TCP TURN servers, slightly inflating real-time video latency. However, in a constrained double-tunnel mesh, the absolute stability gained from eliminating fragmentation vastly outweighs these trade-offs.

10.15 Threat Model: Local Proxy Discovery

The Tor SOCKS proxy and Honey-Port Tripwire only bind to `127.0.0.1` (loopback). The real security boundary here is "can an unprivileged local process reach the loopback interface," not "does the malware know the port number."

The port-randomization and the Honey-Port trap only catch blind or generic port-scanners that do not already know where to look. They **do not protect** against a process that directly reads the `/tmp/nullexit-ports.env` file, greps the `toggle.sh` memory space, or runs `lsof -i` to inspect open socket bindings.

Because modifying file ownership on `/tmp/nullexit-ports.env` via `chown`/`chmod` to restrict read-access introduces an unavoidable Time-Of-Check to Time-Of-Use (TOCTOU) race condition (the file is created user-owned before privilege escalation, meaning file descriptors can be snatched), it is deliberately left as a standard user-owned file.

The actual, proper mitigation for preventing a malicious local process from reaching the proxy is not hiding the portit is running a host-level outbound firewall with strict localhost/loopback filtering enabled (e.g., LuLu 4.3.0+ or Little Snitch). These firewalls gate access by cryptographic process identity (binary signature) at the kernel/network-extension level, rather than by port secrecy.

Verifying Localhost Filtering (The Rogue Binary Test)

To empirically validate that the host firewall properly intercepts local proxy hijacking, you can compile and execute a completely unknown, un-whitelisted "rogue" binary to attempt a connection to the proxy port:

```c
// lulu-test.c
#include <stdio.h>
#include <stdlib.h>
#include <arpa/inet.h>

int main(int argc, char *argv[]) {
    int port = atoi(argv[1]);
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in server = { .sin_family = AF_INET, .sin_port = htons(port) };
    server.sin_addr.s_addr = inet_addr("127.0.0.1");

    printf("Rogue binary attempting connection to 127.0.0.1:%d...\n", port);
    if (connect(sock, (struct sockaddr *)&server, sizeof(server)) < 0) {
        printf("Connection BLOCKED by firewall.\n");
~~~

```
        return 1;
    }
    printf("Connection SUCCESSFUL (Firewall bypassed!).\n");
    return 0;
}
‘‘‘
Compile with ‘gcc -o lulu-test lulu-test.c‘. When executed against the ‘$TOR_SOCKS_PORT‘, a properly configured host firewall (with "Allow Loopback" disabled in LuLu Preferences) will immediately pause the socket execution and generate a desktop alert for the unrecognized binary signature. This physically stops targeted local malware from exfiltrating data through the Tor tunnel, proving the threat model mitigation.

---

## 11. Deep Dive: Exit Node Return Path Analysis (June 26, 2026)

This section preserves the detailed packet-level analysis that led to identifying and fixing the ‘rx 0‘ bug.

### The exit node was probably never going to work in this container topology

Here’s the packet path traced by reading ‘ip rule show‘ and the routing tables live:

**Outbound (Phone-1  internet):**
1. Phone-1 sends packet to gateway via Tailscale mesh
2. Gateway’s tailscale0 (kernel TUN, ‘TS_USERSPACE=false‘) decrypts  packet enters FORWARD chain
3. Packet has src=‘100.87.42.87‘ (Phone-1), dst=some internet IP
4. FORWARD chain (nftables): Rule 1 matches (‘-i tailscale0 -j ACCEPT‘)
5. Routing decision for the forwarded packet. Which ip rule matches?
   - Rule 98: ‘to 172.18.0.0/16‘  no (dst is internet)
   - Rule 99: ‘to 100.64.0.0/10‘  no (dst is internet)
   - Rule 100: ‘from 172.18.0.2‘  **NO** (src is ‘100.87.42.87‘, not the container IP!)
   - Rule 101: ‘not fwmark 0xca6c lookup 51820‘  **YES** (no fwmark on forwarded packets)
6. Table 51820: ‘default dev tun0‘  packet goes out through WARP
7. NAT POSTROUTING: ‘MASQUERADE on tun0‘  src becomes WARP IP
8. Packet exits through WARP to internet  (in theory)

**Return (internet  Phone-1):**
1. Return packet arrives on tun0, conntrack de-MASQUERADEs dst back to ‘100.87.42.87‘
2. Routing decision for dst=‘100.87.42.87‘:
   - Rule 99: ‘to 100.64.0.0/10 lookup 199‘  **YES**
3. Table 199: ‘100.64.0.0/10 via 172.18.0.1 dev eth0‘  **sends it to the Docker gateway!**
4. **This is wrong.** The packet should go to tailscale0 (to be re-encrypted and sent to Phone-1), but table 199 sends it out eth0 to the Docker bridge host.

**Table 199 is the smoking gun.** It has one route: ‘100.64.0.0/10 via 172.18.0.1 dev eth0‘. This sends ALL Tailscale CGNAT return traffic out the Docker bridge to the host  not back through tailscale0. The return packet leaves the container’s network namespace entirely, hits the host’s routing stack, and gets dropped because the host has no idea what to do with a raw packet for ‘100.87.42.87‘.

### Why table 199 exists and why it’s wrong for exit node traffic

Table 199 + the CGNAT rule (pref 99) was designed so that the **container itself** can reach other Tailscale peers (e.g., for mesh management, DERP relay). For that use case, routing CGNAT traffic via the Docker bridge to the host (which has its own tailscaled) makes sense.

But for **forwarded exit node traffic**, the return packet needs to go back through tailscale0 inside the container  not out to the host. The CGNAT rule intercepts the return packet before it can be routed back to tailscale0 through the main table or Tailscale’s own routing (table 52, which is at pref 5270).

### The SOCKS5 path works because it dodges all of this

SOCKS5 proxy creates connections **from the container’s own IP** (‘172.18.0.2‘). So:
- ip rule 100 (‘from 172.18.0.2 lookup 200‘) matches
- Table 200 has ‘default dev tun0‘
- Return traffic comes back to ‘172.18.0.2‘ on tun0, conntrack de-NATs, proxy sends response to client

No FORWARD chain involved. No CGNAT rule involved. No table 199. It just works.

### Summary of findings

| Finding | Status | Evidence |
|---------|--------|----------|
| Table 199 CGNAT route hijacks exit node return traffic | **Fixed** | Changed ‘lookup 199‘ to ‘lookup 52‘ in ‘scripts/routing-fix.sh‘. Return packets now route via tailscale0. |
| FORWARD RELATED,ESTABLISHED missing | **Fixed** | Installed ‘iptables‘ via apk in ‘docker-compose.yml‘. Rule now injects successfully. |
| DOCKER_GW variable parsing error | **Fixed** | Added ‘head -1‘ in ‘scripts/routing-fix.sh‘. |
| ‘FIREWALL_OUTBOUND_SUBNETS‘ was the root cause | **Fixed** | Removed from ‘docker-compose.yml‘. Gluetun no longer hijacks CGNAT routing. |
```

```
| Host tailscaled error state from aggressive ‘tailscale down‘ | **Mitigated** | Toggle script now detects and auto-restarts. |

### 11.1 Double-Tunneling MSS Clamping (Stalling Bug)

**Problem:** Traffic double-wrapped through Tailscale (WireGuard) and WARP (WireGuard) suffered 120 bytes of MTU overhead (60 + 60). The default MSS clamp of 1180 was still slightly too large, causing large packets to fragment or drop, which resulted in mysterious web page stalls on strict-MSS endpoints.

**Fix:** Lowered the TCP MSS clamp in ‘post-rules.txt‘ to ‘1120‘ to guarantee double-tunneled payloads fit safely within standard internet MTUs.

### 11.2 Linux Native Wi-Fi Bouncing

**Problem:** The generated ‘scripts/recover-linux.sh‘ incorrectly retained the macOS ‘networksetup‘ commands, which would crash and fail to bounce Wi-Fi on Linux hosts.

**Fix:** Swapped the macOS logic for native Linux commands. The script now attempts ‘nmcli radio wifi off/on‘ first, and gracefully falls back to ‘ip link set wl... down/up‘ if NetworkManager is unavailable.

### 11.3 TS_AUTH_ONCE Cloud Footgun & Ephemeral Keys

**Decision:** The compose file uses ‘TS_AUTH_ONCE=true‘ to prevent authentication loops. However, on headless cloud VPS deployments, if a standard auth key expires (usually 90 days), the container will silently drop off the mesh. We documented this trade-off explicitly in ‘docker-compose.yml‘ and recommended the use of **Tailscale Ephemeral Keys** for cloud deployments, which automatically clean themselves up and bypass this issue.

### 11.4 Hardcoded AdGuard Credentials

**Decision:** AdGuard is deployed with the hardcoded credentials ‘admin / nullexit‘. This is perfectly safe behind a NAT on a local macOS laptop. However, if deployed on a cloud host with exposed ports, this becomes a severe security risk. We added a stark warning comment in ‘docker-compose.yml‘ to ensure cloud users change this before deployment.

### 11.5 Routing Fix: 30-second polling vs Netlink Socket

**Decision:** Instead of building a complex Netlink socket listener (‘ip monitor‘) to react instantly to iptables and routing flushes from Gluetun, we retained the 5-second ‘sleep‘ loop in ‘scripts/routing-fix.sh‘.
- **Reasoning:** Building a netlink listener in pure bash on Alpine is incredibly brittle and complex (especially for iptables events). The 30-second polling loop is bulletproof. The only trade-off is a potential 1-4 second stutter if Gluetun reconnects, which was deemed an acceptable edge case for absolute reliability.

### 11.6 Cache Poisoning and URL Rot in scripts/sync-rules.py

**Problem:** If a remote blocklist URL went permanently offline (404 Not Found), the Python ‘urllib‘ request would return the 404 HTML string. The script would aggressively regex-parse this HTML, find no domains, and overwrite the healthy local cache with a 0-domain file. This effectively disabled ad-blocking until the URL was fixed.

**Fix:** Implemented a defensive programming sanity check in ‘scripts/sync-rules.py‘. Before overwriting the cache, the script now verifies that the compiled domain count is greater than 10 (and 1 for IP feeds). If it falls below this threshold (indicating a catastrophic 404 failure across multiple URLs), it aborts the cache overwrite, raises a ‘ValueError‘, and keeps the previous day’s healthy cache intact.

### 11.7 Cellular Networks & PMTUD Blackholes (The 1280 Byte Limit)

**Experiment:** Conducted a live packet sweep from the PC-1 host to Phone-1 connected via a cellular network + Tailscale DERP relay using ‘ping -D -s <size>‘.

**Result:**
- Packets with a payload of ‘1252‘ bytes (+ 28 bytes IP/ICMP headers = **1280 bytes total**) succeeded perfectly with ~60ms latency.
- Packets with a payload of ‘1253‘ bytes (**1281 bytes total**) and above resulted in a silent timeout.

**Analysis:** The cellular network (or the Tailscale DERP relay) enforces a strict MTU of 1280 bytes. Critically, it acts as a "PMTUD Blackhole"  it silently drops packets larger than 1280 bytes instead of returning an ICMP "Fragmentation Needed" warning. If the exit node tries to send a standard 1500-byte internet packet to the phone over this link, it vanishes, causing web pages to stall permanently.

**Conclusion:** This empirically validates the absolute necessity of the TCP MSS clamping rule in ‘post-rules.txt‘. By artificially clamping the MSS to ‘1120‘ (or ‘1180‘), we ensure the TCP payload + headers + double-WireGuard overhead never exceeds the strict 1280-byte ceiling of the cellular mesh link.

```

```
### 11.8 Gluetun nf_tables Parser Crash (Bash Interpolation Failure)

**Problem:** Attempting to make the TCP MSS dynamically configurable by using a standard bash variable inside ‘post-rules.txt‘ (‘iptables ... --set-mss ${GATEWAY_MSS:-1120}‘) caused the Gluetun container to catastrophically crash during startup. Gluetun's internal parser executes ‘post-rules.txt‘ line by line without a shell, meaning the variable was never expanded. It literally attempted to inject the string ‘${GATEWAY_MSS:-1120}‘ into iptables, resulting in a fatal ‘bad value for option "--set-mss"‘ error.

**Fix:** Removed the bash variable from ‘post-rules.txt‘ and reverted it to a pure hardcoded integer. To retain dynamic ‘.env‘ configuration, we moved the interpolation logic to ‘toggle.sh‘. Right before Docker starts, the host script parses ‘GATEWAY_MSS‘ from the ‘.env‘ file and uses a cross-platform ‘sed‘ command to dynamically overwrite the integer directly inside ‘post-rules.txt‘. This perfectly mimics manual configuration and completely bypasses Gluetun's strict parser.

### 11.9 Seamless Wi-Fi Roaming & The macOS DNS Wipeout Loophole

**Problem:** The Tailscale, WireGuard, and Colima NAT stacks are natively designed for connectionless roaming and will seamlessly survive when the macOS host switches Wi-Fi networks (e.g., roaming from home Wi-Fi to a cellular hotspot). However, the internet completely breaks upon network transition. This occurs because macOS intentionally wipes out all custom DNS settings (‘networksetup -setdnsservers‘) and reverts to the new network's default DHCP nameserver whenever the active Wi-Fi BSSID changes. Since the Mac is locked to the exit node, its DNS requests are swallowed by WARP and dumped onto the public internet, which cannot route private DHCP IPs. This results in a total DNS failure.

**Solution:** Implementing a silent background "DNS Watcher" daemon (‘nullexit-dns-watcher‘) in ‘toggle.sh‘. When the gateway starts, it spawns a background polling loop that checks the active Wi-Fi DNS every 30 seconds. If macOS resets the DNS during a network change, the watcher instantly detects the drift and forcefully re-injects ‘100.100.21.8‘ via ‘networksetup‘. When the gateway stops, the watcher process is cleanly killed. This allows true, seamless roaming across physical networks without ever dropping the gateway state.

### 11.10 The Infinite Recursive Routing Loop (Hotspot Paradox)

**Problem:** A critical network topology crash occurs if the host Mac connects to a Mobile Hotspot (e.g., a Windows PC or Phone) that is *also* connected to the Tailscale mesh and has "Use Exit Node" enabled. Because the Mac is hosting the Exit Node, it routes its internet traffic to the Hotspot. The Hotspot intercepts the Mac's traffic on its way to the cellular tower and routes it *back* to the Exit Node (the Mac) via Tailscale. The Mac receives it, tries to send it out again, and the Hotspot intercepts it again. This creates an infinite, recursive encryption loop that instantly destroys network bandwidth and drops all connections.

**Fix:** This is a physical routing paradox that cannot be resolved in software. The upstream physical router providing internet to the Mac **must** be allowed to route its traffic directly to the physical WAN interface. The user must manually disable "Use Exit Node" on the upstream device providing the hotspot.

**Advanced Workaround:** If the upstream hotspot is a Windows machine and the user explicitly wants it to remain connected to the Exit Node, a permanent static route can be injected into the Windows routing kernel to forcefully bypass the Tailscale WFP interception for WARP packets. By binding the route to the physical adapter's Interface Index (IF), it becomes a permanent, roaming-aware bypass.

On Windows (Admin Command Prompt):
‘‘‘cmd
route print  (find Interface List, note the IF number for the active Wi-Fi adapter, e.g. 15)
route -p add 162.159.192.1 mask 255.255.255.255 0.0.0.0 IF 15
route -p add 162.159.193.1 mask 255.255.255.255 0.0.0.0 IF 15
‘‘‘

On Linux (Root Terminal):
‘‘‘bash
ip route add 162.159.192.1 dev wlan0
ip route add 162.159.193.1 dev wlan0
‘‘‘

On macOS (Root Terminal):
‘‘‘bash
route add -host 162.159.192.1 -interface en0
route add -host 162.159.193.1 -interface en0
‘‘‘

*(Note: If the upstream router is an iPhone or Android device, you cannot inject static routes without jailbreak/root. You must simply disable "Use Exit Node" in the mobile Tailscale app).*

This punches a tiny hole straight through the paradox, letting the Mac's WARP packets escape to the physical internet while keeping the rest of the upstream router's traffic securely trapped in the Tailscale Exit Node.

### 11.11 AdGuard Filter Redundancy & Memory Optimization

```

```
**Problem:** AdGuard Home does not perform cross-list deduplication in memory. When it loads its own native subscription filters (e.g., `AdGuard DNS filter`) alongside our massive `compiled_rules.txt`, overlapping rules are loaded twice, wasting significant RAM in the Colima VM. The UI would show ~500k rules, representing the sum of all lists rather than unique domains.

**Fix:** Updated `scripts/sync-rules.py` to intelligently cross-reference AdGuard's configuration and deduplicate our list against it.
1. The script now reads `adguard/conf/AdGuardHome.yaml` to dynamically fetch the exact URLs of any enabled native AdGuard filters.
2. The parsing engine was upgraded to normalize basic AdGuard syntax (`||domain^`) back into raw base domains during the build process.
3. The script subtracts the native AdGuard domains from our custom blocklist *before* compiling, immediately purging ~83,000 completely redundant rules.
4. The normalized base domains then pass through our subdomain optimizer, which squashes an additional ~50% of the remaining rules.

This reduces the final compiled output from ~325k to ~281k rules on the `heavy` profile, saving ~45k rules from being loaded into memory twice and reducing the overall RAM footprint of the `adguardhome` container.

### 11.12 Kernel-Level IP Blocklist (Threat Intelligence Firewall)

**Problem:** DNS sinkholing cannot stop malware or botnets that bypass DNS entirely by hardcoding direct IP addresses (a common technique for C2 communication). These connections pass straight through AdGuard unseen.

**Fix:** Added a second compilation pipeline to `scripts/sync-rules.py` that runs on every startup alongside the DNS pipeline.
1. The compiler concurrently fetches four curated threat-intelligence feeds: **Feodo Tracker** (abuse.ch botnet C2 IPs), **Spamhaus DROP** (IPs allocated to criminal organizations), **Emerging Threats** (Proofpoint  active C2 and scanners), and **CINS** (active brute-force sources).
2. All entries are normalized via Python's `ipaddress` module. Any IP or CIDR that overlaps with RFC1918 private ranges, loopback, link-local, or the Tailscale CGNAT range (`100.64.0.0/10`) is stripped to prevent accidentally locking users out of their own LAN or mesh.
3. The cleaned list (16,721 unique IPs/CIDRs) is written as an `ipset restore` file using an **atomic swap pattern**: `create_new  populate  swap with live  destroy_new`. This guarantees the live `block_malicious` ipset is never empty during a reload.
4. `scripts/routing-fix.sh` watches the file's mtime every 30 seconds. On change it runs `ipset restore` and idempotently re-injects `FORWARD DROP` rules for both `src` and `dst` into both iptables backends (nftables + legacy).
5. `docker-compose.yml` mounts `adguard/work/userfilters/` into `routing-fix` as `/userfilters:ro` and adds `rule-compiler: service_completed_successfully` to its `depends_on`, eliminating the race condition where routing-fix could start before the file existed.

**Memory cost:** The entire 16,721-entry ipset costs only ~1.6 MiB of kernel memory  negligible in the 600MB VM budget.

### 11.13 Standalone Tailscale Daemon Freeze after macOS Sleep/Wake

**Problem:** When the macOS host goes to sleep and wakes up, the network interfaces and routing tables are rebuilt. The standalone `tailscaled` daemon (installed via Homebrew) occasionally fails to handle this transition and becomes completely frozen/unresponsive. In this state, any command like `tailscale down` or `tailscale status` hangs indefinitely, and all DNS requests sent to the local Tailscale resolver (`100.100.21.8`) time out, causing a total internet blackout on the host.

**Fix:** Enhanced the Tailscale verification and teardown logic in `toggle.sh` and `recover.sh`:
1. **Unresponsive Daemon Detection:** Instead of assuming the daemon is healthy just because the Unix socket `/var/run/tailscaled.socket` exists, the scripts now actively run a status check with a 5-second timeout (`run_with_timeout 5 tailscale status`). If the check times out (exit code 143), the daemon is classified as unresponsive.
2. **Auto-Recovery Loop:** If the daemon is unresponsive, the script now automatically attempts to restart the service using `brew services restart tailscale` (falling back to `sudo -n brew services restart tailscale`).
3. **Graceful Retry:** Once restarted, the script pauses for 3 seconds for the daemon to initialize before proceeding with connection or teardown commands.

### 11.14 macOS Sharing Services (AirDrop/AirPlay) Freeze on Network Transitions

**Problem:** When the gateway is turned on or off, the scripts perform network configuration cleanups which include flushing the routing tables (`route -n flush`), restarting the `mDNSResponder` daemon (`killall -HUP mDNSResponder`), and power-cycling the Wi-Fi interface (`setairportpower` or `ifconfig en0 down/up`). While AirDrop BLE discovery continues to function, the actual file transfers stall at "Waiting..." indefinitely.

**Root Cause:** The rapid tear-down and rebuild of the local routing table and Wi-Fi interface causes Apple's core sharing and connection daemons (`sharingd` and `rapportd`) to lose their socket bindings to the mDNS/Bonjour discovery interface. Instead of reconnecting gracefully, they enter a wedged state and try to route peer-to-peer (AWDL) IPv6/TCP transfer traffic into the default gateway (the Tailscale interface `utun`) instead of the direct wireless link, causing the TLS verification handshake to time out.

```

~~~
**Fix:** Integrated an automatic sharing services reset into both `toggle.sh` and `recover.sh`:
1. The script now automatically runs `sudo -n killall sharingd rapportd` at the end of both the **START** path and the **STOP** path (as well as inside `recover.sh`).
2. Killing these daemons forces macOS to immediately relaunch them, binding them fresh to the newly initialized network interfaces and routing tables, allowing AirDrop to work seamlessly while the gateway is active.

### 11.15 Unlocking Mode-Locked Output Files Without `chmod` ( `unlock-files.sh`)

**The one-command fix:** `bash scripts/unlock-files.sh`  replaces every known locked inode in the project with a fresh writable one. No `chmod`. Covers `.env` (mode `000`), `adguard/work/userfilters/compiled_rules.txt`, `ip_blocklist.ipset`, `cache/*.txt`, `cache/ip/*.txt`, and `adguard/work/data/filters/*.txt` (mode `0444`). Run it once after setup or after pulling an old repo; `toggle.sh` and `sync-rules.py` will then work without permission errors.

**Context:** The nullexit project has a strict project-wide policy of `no chmod from scripts` (see README 6). This rule exists because every prior `chmod 0444` (post-write tamper-proof lock) and `chmod 000` (post-toggle `.env` lock) call from `scripts/sync-rules.py` / `toggle.sh` could leave a file permanently unreadable on disk if the script exited early, was killed mid-flight, or simply set a restrictive mode without ever being re-flipped. We've stripped every `chmod` from the codebase. But files **written by older versions of those scripts** are still on disk in those restrictive modes (`0444` for `compiled_rules.txt`, `ip_blocklist.ipset`, `data/filters/<id>.txt`; `000` for `.env` after end-of-toggle lock). Re-running `toggle.sh` or `scripts/sync-rules.py` against those leftovers hits `PermissionError: [Errno 13] Permission denied` immediately.

**Why the stale permissions also block `mv`/`rm` of the parent folder:** The restrictive modes (`000` on `.env`, `0444` on output files) don't just block writes  they prevent the kernel from unlinking the file's dentry during a `mv` or `rm` of the **containing directory** (`adguard/work/`). macOS enforces this at the VFS layer: you own the directory, but you can't delete a directory that contains entries you can't write to. This made the entire `adguard/work/` tree unmovable/undeletable until the locked inodes inside it were replaced. `unlock-files.sh` resolves this permanently by swapping each locked inode for a fresh `0644` one via atomic rename (`cp` + `mv`), which operates on the directory entry rather than the file contents  no `chmod` called, no permission bypass needed.

**Problem:** You (the owner) cannot `cat`, `cp`, `rm`, `> redirect`, or any normal POSIX write against a `mode=000` file even though you own it  the basic mode bits block ALL access for owner, group, and other. The script does not call `chmod` and you want a permanent fix that never relies on a chmod from any future run either.

**Solution:** Replace the locked inode with a fresh readable one. `mv` is atomic; mode bits live in the inode, not the directory entry. Two flavors cover every case we hit on a real machine:

**Flavor A  locked file is mode `000` (owner can't even READ it):**
NOPASSWD sudoers (`/etc/sudoers.d/nullexit`) already includes `/usr/bin/python3`. So we read via `sudo python3` (which has the DAC override root has) and pipe the bytes around as base64 in user space, where the redirect happens with the user's normal umask = `0644`:
```bash
sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb').read()).decode())" > /tmp/snap.b64
base64 -d < /tmp/snap.b64 > <FILE>.new
mv -f <FILE>.new <FILE>
ls -la <FILE>           # mode is now 0644
```
The old `mode=000` inode is unlinked by `mv`; the new `mode=0644` inode (created by base64-decode via the calling user's umask) takes the path. No chmod ever called. The parent directory just needs to be writable by you (it always is, since you own it).

**Flavor B  locked file is mode `0444` (you can READ but cannot WRITE; `scripts/sync-rules.py` needs to re-write):**
You own the file and can read it, you just cannot truncate/overwrite it. The simplest path is to rename it aside and let the writer create a fresh inode from scratch (which gets the umask-default `0644`):
```bash
mv <FILE> /tmp/old.<FILE>.$$
python3 scripts/sync-rules.py    # now writes <FILE> cleanly at mode 0644
```
Or apply Flavor A if you'd prefer to preserve the contents while resetting the mode.

**Why this is the canonical recovery, not a hack:**
- Mode bits live in the inode, not the path. `mv -f` replaces the path's inode reference; the old locked inode drops out of the directory entirely (dentry eviction) and loses its last link, so the kernel reclaims it. The new inode (whatever it is: a freshly-written one, or a base64-decoded copy) gets the path.
- The only prerequisite to apply Flavor B's `mv` is: you own the parent directory (so you can unlink entries from it) AND you have a NOPASSWD-capable binary that can read the byte stream if mode prohibits user read.
- This pattern generalizes. Any time a script ends up producing a "locked file that the next run can't update" situation, the same trick applies without a single chmod anywhere.

**Empirical verification (user machine, July 1, 2026):**
- `.env` was `mode=000` (owner $USER, i.e. you): Flavor A unblocked it in 3 commands, no chmod.
  - Before: `----------  1 $USER  staff  899 Jun 28 07:07 .env`
~~~

```
  - After:  ‘-rw-r--r--@ 1 $USER  staff  899 Jul  1 20:39 .env‘
  - ‘docker compose config‘ immediately picked up ‘TS_AUTHKEY‘ + ‘WIREGUARD_PRIVATE_KEY‘ substitution.
- ‘compiled_rules.txt‘, ‘ip_blocklist.ipset‘, ‘data/filters/1782645604.txt‘ were ‘mode=0444‘: Flavor B (‘mv‘-aside) unblocked all three.
  - scripts/sync-rules.py then ran clean: 279587 block rules + 171 allow rules + 16710 IP entries compiled; new files came out at ‘0644‘.
- ‘toggle.sh‘ then went end-to-end in 48s (warp / routing-fix / adguardhome / tailscale all healthy, gateway mesh-joined, DNS hijack verified single-server).

**When this trick is appropriate vs. inappropriate:**
-  You own the parent directory and the file (typical desktop scenario).
-  The lock is a leftover from a removed ‘chmod‘ call that you want off disk forever.
-  NOPASSWD sudo includes at least one interpreter (‘python3‘ on this system) that can be used to read mode-0 files. If it does not, add it first: ‘<USER> ALL=(ALL) NOPASSWD: /usr/bin/python3‘.
-  The file is owned by another user (root, another account) and the lock is intentional  respect it; do not bypass another user’s security boundary.
-  The parent directory is owned/writable only by another user  escalate properly via sudo, or stop and ask the user.
-  You do not actually own the file and there is a legitimate reason for its lock state  restore the file via a trusted source rather than circumventing the perms.

**Reusable cookbook:**

‘‘‘bash
# Quick diagnostic: figure out which flavor you need.
WHOAMI=$(whoami)
ls -la <FILE>
stat -f ’mode=%Sp owner=%Su group=%Sg’ <FILE>
# mode=--------- or mode=----r----- : you cannot even read it  Flavor A.
# mode=r--r--r-- but writer fails  : you are blocked from write but can read  Flavor B.
# mode=rw-r--r--                    : not actually locked at all; unrelated write failure.

# Flavor A (mode=000):
sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open(’<FILE>’,’rb’).read()).decode())" > /tmp/snap.b64
base64 -d < /tmp/snap.b64 > <FILE>.new && mv -f <FILE>.new <FILE> && rm -f /tmp/snap.b64

# Flavor B (mode=0444, file is readable):
mv <FILE> /tmp/old.<FILE>.$(date +%s)
# (let the original writer recreate <FILE>; new file lands at mode 0644)

# Verify after either flavor:
ls -la <FILE>             # mode should be 0644
<your normal validation command>
‘‘‘


### 11.16 Gateway Breakage on Lid Close / Wi-Fi Roam  Post-Wake + Post-Roam Auto-Recovery

**Context / Symptom.** Closing the lid (sleep/wake) OR losing + reconnecting Wi-Fi (e.g. in an elevator, caf, or roaming between APs) leaves the gateway in a partly-dead state. Symptoms range from a ~30s window of dead DNS to a permanent outage that only full ‘recover.sh‘ or ‘toggle.sh‘ fixes. The root cause is that nullexit has **no daemon watching macOS sleep/wake or network-state-change events** the existing DNS Watcher inside ‘toggle.sh‘ only runs in-process and is suspended along with the rest of the user shell on lid close, so the gateway cannot self-heal after either event.

**Diagnosis.** Two distinct failure modes share the same root gap.

* **(A) Lid close  sleep  wake.** ‘caffeinate -i‘ (used by ‘toggle.sh‘) blocks *idle* sleep but **does not block forced sleep from lid close**. Closing the lid hard-suspends Colima VM + every container + every Tailscale-quit-shells-except-tailscaled. On wake the VM clock needs ~5-30s to resync via NTP, during which TLS cert validation fails: Cloudflare WARP ‘162.159.192.1:2408‘ rejects the handshake  gluetun’s healthcheck (‘interval: 2s, retries: 15‘) eventually flips unhealthy  Docker ‘unless-stopped‘ restarts the ‘warp‘ container (~30s gap). ‘mDNSResponder‘’s in-memory cache still holds pre-sleep entries (stale A records for tailnet hosts) so the first dozen DNS queries after wake time out. ‘tailscaled‘ on the host often keeps its stale DERP relay preference and routes packets through a dead relay for another 30-60s.
* **(B) Wi-Fi roam / loss in elevators, captive portals.** WARP is a UDP tunnel to Cloudflare’s anycast edge. Carrier NATs and captive-portal networks silently invalidate the UDP binding on roam and on hotspot-bound re-association. macOS ‘networksetup‘ *should* preserve DNS settings on the same Wi-Fi service across a roam, but most captive-portal networks DHCP-replace DNS to the captive-portal resolver **during the captive-portal dance itself**, before the user has authenticated  so even DNS hijack survives a clean roam and quietly breaks on captive-portal handoff.

**Resolution.** A single **launchd LaunchAgent** (‘com.nullexit.wake-recovery‘) runs a long-running shell daemon (‘scripts/watcher.sh‘) that listens for both event sources and, on each fire, executes ‘bash recover.sh --post-wake‘  a *lighter-touch* recovery mode that’s the opposite of the existing ‘--nuclear‘ semantics: it keeps the gateway live while still refreshing every stale subsystem.

#### Apple Power Management Primer (for the unfamiliar)

```

```
macOS exposes several relevant surfaces; the right primitive depends on what you actually need to detect. None of these are obvious and none of them have a standard splash screen in the docs.

* **`caffeinate <flags>`** creates I/O Kit power assertions via `IOPMAssertionCreateWithName`. It does NOT block forced sleep (lid close, low-battery shutdown, sleep timer firing on a per-set Energy Settings policy). Flags:
  * `-i`  PreventIdleSleep (the only one `toggle.sh` uses; protects against the OS going idle while the user is active  but a clamshell close still hard-puts it to sleep)
  * `-s`  PreventSystemSleep (only honored on AC power; the user may be on battery)
  * `-u`  DeclareUserActive (resets the idle timer every 30s by default; equivalent to keeping the cursor moving)
  * `-d`  PreventDisplaySleep
  * `-m`  PreventDiskIdleSleep
  * There is NO `-l` (lid-block) flag in any modern macOS. Lid close is a kernel-firmware-level event that ignores user-space assertions.
  * To prove any of this on live hardware: `pmset -g assertions` lists every active assertion with type / process / reason / timeout.
* **`pmset`** is the read-side of the same system:
  * `pmset -g log | grep -E 'Sleep|Wake|DarkWake'` shows the recent timeline.
  * `pmset -g history` is the per-day summary.
  * `pmset -g assertions` is the live assertion list (matches `-i -s -d -u` to processes).
* **`launchd` does NOT have a native "on-wake" hook.** Not in any version of macOS as of Sonoma. The canonical recipe is `StartCalendarInterval` with a sub-minute cadence and let launchd queue missed intervals for replay-on-wake, or use a third-party daemon (Sleepwatcher). For our use case a long-running shell daemon listening to the unified log is more responsive than a polling agent.
* **`com.apple.system.powermanagement.*` Darwin notifications** (`willSleep`, `didWake`, `didDim`, `didUndim`) are the C-level signal. From a shell, they are best reached through the unified log with `log stream --predicate`  see the watcher implementation.
* **`scutil n.watch`** is the canonical CLI for live-following network-state changes. Pairs with `n.add State:/Network/Global/IPv4` to get a single tap that fires on **any** IPv4 link/route/DNS/address change across **all** interfaces. This is what `scripts/watcher.sh`'s network-change listener uses.
* **`com.apple.networkChange` Darwin notification** is the C-level equivalent but has no shell-friendly listener; use `scutil` instead.
* **`SCNetworkReachability`** is the Objective-C API used by SOCKS5/HTTP clients to detect route changes per target. Never a fit for shell scripts.

#### The Fix (component-by-component)

1. **`recover.sh --post-wake`** (new flag, **non-destructive** by design). Adding `--post-wake` to the existing nuclear recovery script inverts the semantics. The default mode still tears down Tailscale, resets DNS to empty, stops caffeinate, runs `docker compose down`, power-cycles Wi-Fi. The new mode does:
  * Skip Tailscale disconnect (we WANT it to keep the mesh connection)
  * **Re-hijack** DNS to the gateway Tailscale IP read from `.gateway_ip` (instead of resetting to empty)  applies to both `ACTIVE_SERVICE` and `EN0_SERVICE` because the system resolver is scoped to en0
  * **Refresh** the exit-node preference with `tailscale up --reset --ssh=true --accept-dns=false --exit-node=\"$TS_IP\" --exit-node-allow-lan-access=true` (the exact flags `toggle.sh` START uses). This re-asserts DERP mapping without dropping the mesh
  * Inspect `docker inspect --format '{{.State.Health.Status}}' nullexit-warp-1` and **only** `docker compose up -d --force-recreate warp` if gluetun is unhealthy  this single targeted recreate nudges the UDP NAT binding back to Cloudflare without disturbing Tailscale or AdGuard
  * Run the 10.27 sharingd-reset step (existing fix from `toggle.sh` START path) so AirDrop doesn't freeze on the new IP lease
  * Verify the gateway with `dig +tcp @127.0.0.1 -p 5354 google.com` (AdGuard via TCP through Colima's SSH tunnel) and a 4-second `tailscale ping` to the gateway Tailscale IP, instead of the default mode's direct-to-1.1.1.1 check
  * Crucially: **skip `sudo -v`** because the LaunchAgent has no TTY to prompt on; all privileged calls use `sudo -n` and lean on `/etc/sudoers.d/nullexit` NOPASSWD entries (`networksetup`, `dnsmasq`, `dscacheutil`, `killall`, `pkill`).
2. **`toggle.sh`** writes and clears `/tmp/nullexit-gateway-active.marker` (an ISO-8601 UTC timestamp) at the bottom of the START path and the top of the STOP path / `cleanup_handler`. Two new helpers: `write_gateway_active_marker` and `clear_gateway_active_marker`. The watcher uses this file as its **only signal** that the gateway is live: if `toggle.sh` was stopped (or never started), the watcher no-ops. This prevents waking-up-only-on-counterfactual events from churning DNS.
3. **`scripts/watcher.sh`** (long-running daemon, sibling-style source file). Two backgrounded pipelines:
  * **Wake listener**: `log stream --predicate 'composedMessage CONTAINS[c] "didWake" OR composedMessage CONTAINS[c] "Wake from Sleep" OR composedMessage CONTAINS[c] "Waking from" OR composedMessage CONTAINS[c] "DarkWake"' --style compact --info` piped through `while read; do run_recover "WAKE: ..."; done`. `[c]` makes the predicate case-insensitive (the unified log writes phrases in mixed case across versions).
  * **Network listener**: `(echo "n.add State:/Network/Global/IPv4"; echo "n.watch"; while true; do sleep 86400; done) | scutil 2>/dev/null` piped through a `case` match on `n.state*` / `SCEventUpdate*`.
  * **`run_recover`** checks the marker, debounces via `/tmp/nullexit-watcher.last-recovery` (default 10s, configurable via `NULLEXIT_DEBOUNCE_SECONDS`), and shells out to `bash recover.sh --post-wake`. The 10s debounce prevents a single wake event (which typically emits 2-3 log lines) from triggering multiple recoveries.
  * `trap ... TERM INT`  `kill 0` cleanly tears down the process group on launchctl `bootout` so the `sleep 86400` inside the scutil pipe also dies.
  * `wait` blocks indefinitely while both pipelines feed it.
4. **`launchd/com.nullexit.wake-recovery.plist`** (LaunchAgent in the user domain  runs as the console user at every login without needing sudo).
```

```
  * ‘Label: com.nullexit.wake-recovery‘
  * ‘ProgramArguments: ["/bin/bash", "<absolute-path-to-your-nullexit-install>/scripts/watcher.sh"]‘
  * ‘RunAtLoad: true‘  starts immediately on ‘launchctl load‘ (no need to log out and back in)
  * ‘KeepAlive: { SuccessfulExit: false, Crashed: false }‘  **don’t** restart on clean SIGTERM exit (so ‘launchctl bootout‘ doesn’t get stuck in a relaunch loop). Also **don’t** restart on hard crash: we observed that macOS Sonoma+’s sleep/wake suspend-resume path accumulates orphan watcher.sh PIDs because SIGCONT does not reliably clean up the prior instance’s listener grandchildren, and a ‘KeepAlive.Crashed=true‘-driven relaunch actively races with the still-live prior process. The script enforces single-instance on its own via a ‘flock‘-style PID-file lock, so a relaunch-on-crash would be skipped by the lock and produce   0 listener coverage until the live instance happened to die. Trade-off: a real crash leaves the user without auto-recovery until they run ‘launchctl load -w‘ manually  acceptable because (a) gateway still works without auto-recovery, (b) a relaunch wouldn’t fix whatever caused the crash, and (c) the gateway-recovery itself is observably broken (no DNS, no exit-node) if it does go down.

### 11.17 Host-Side Traffic Leaking Past the Exit Node (with Remote Clients Routing Correctly)

**Symptom.** ‘toggle.sh‘ reports success. Remote tailnet clients (e.g. an S24 phone) correctly egress through Cloudflare WARP and see a Cloudflare IP on ‘whatismyip.com‘. But on the **HOST Mac running the gateway**, ‘whatismyip.com‘ reports the underlying physical ISP’s ASN (e.g. ‘campus_isp‘ for a university campus Wi-Fi). The gateway container itself is healthy; the leak is specifically on the host.

**Root causes.** Three distinct mechanisms can each produce this exact symptom on the host alone, while leaving remote clients unaffected. They are mutually rankable so the diagnostic picks the active one deterministically.

* **(A) SOCKS5 fallback lane active.** ‘toggle.sh‘’s three Phase-B pre-flight checks (‘tailscale ping‘ AdGuard via ‘dig +tcp‘  WARP internet, toggle.sh ~lines 750-820) sometimes flake during the first minute. When ANY of the three fails, the script sets ‘SKIP_EXIT_NODE=true‘ (toggle.sh:849) and instead activates the SOCKS5 fallback path (toggle.sh:894). The SOCKS5 fallback hijacks DNS to ‘127.0.0.1‘ (so AdGuard filtering still works) but **modern browsers on macOS do NOT honor the system SOCKS5 proxy**  they ignore ‘networksetup -setsocksfirewallproxy‘. Result: DNS gets ad-filtered but actual HTTP/S traffic exits direct through the campus ISP. This branch is invisible from a remote tailnet client because the client’s tunnel is unaffected.

* **(B) IPv6 leak over campus Wi-Fi.** The Tailscale exit node and WARP tunnel are both IPv4-only (gluetun + ‘post-rules.txt‘ drop all IPv6 forwarded traffic; devref 10.6). But many university and enterprise APs are dual-stack  the host happily accepts AAAA DNS responses and sends IPv6 traffic straight out ‘en0‘. macOS happily encrypts that traffic through Tailscale ONLY if Tailscale has IPv6 advertised; with an IPv4-only exit node, IPv6 traffic skips Tailscale entirely. This branch is invisible from a phone because iOS/Android Tailscale apps actively sinkhole IPv6 in their VPN profile, but the macOS host’s ‘tailscaled‘ does not.

* **(C) Route-table freeze and ‘--accept-routes‘ reset after ‘tailscaled‘ wake/toggle.** Running ‘tailscale up --reset‘ clears the exit-node preference and reverts ‘--accept-routes‘ back to its default (‘false‘). Without explicitly passing ‘--accept-routes=true‘, ‘tailscaled‘ will not attempt to install the default route through the ‘utun*‘ tunnel interface even if the exit node preference is reconciled. Additionally, macOS occasionally freezes and fails to apply route-table changes (devref 10.26). In both cases, ‘netstat -rn‘ shows the default route still pointing to the physical Wi-Fi gateway (e.g. ‘172.17.0.1‘) instead of the Tailscale ‘utun*‘ interface, causing host traffic to exit direct while remote clients route correctly.

**Why remote clients don’t see this.** Each remote phone/laptop uses its OWN Tailscale tunnel to reach the container’s ‘100.x.x.x:443‘ over the mesh  they’re end-to-end encrypted regardless of how the host Mac itself is configured. Only the HOST’s own egress sockets (HTTP requests from the host’s browser, curl, etc.) are affected.

**Resolution.** A single script: ‘scripts/diagnose-host-leak.sh‘. It runs the 7 checks, classifies into one of A / B / C / OK, prints a verdict + a ready-to-run fix command on stdout, AND writes the full report to ‘host-leak-diagnostic-<UTC>.txt‘ so the user can paste the file back instead of scrolling-and-copying from the terminal. With ‘--fix‘, the matched remediation is applied and the two checks that could change (warp + ipv6) are re-verified. With ‘--watch‘ (or ‘--watch 30‘), it runs the full baseline diagnostic then enters a continuous loop checking warp/IPv6/default-route every N seconds, alerting only on state changes  logs to ‘host-leak-watch.log‘.

‘‘‘bash
# Diagnose only:
bash scripts/diagnose-host-leak.sh

# Diagnose + apply matched fix + re-verify:
bash scripts/diagnose-host-leak.sh --fix
‘‘‘

**What it prints.** A 7-section report (‘tailscale status‘, SOCKS5 state, ‘cdn-cgi/trace‘ warp=, IPv6 leak probe, host default route, last toggle.sh output.log hints, resolved gateway Tailscale IP), followed by a classification block (‘Scenario: A | B | C | OK‘) with the exact command(s) the user needs to paste into their terminal  they don’t have to figure out which ‘tailscale up‘ flags match their environment.

```

```
**What it does NOT do.** It does not touch `toggle.sh` or `toggle.sh`'s pre-flight logic, does not modify `.env` (except `--fix` mode appends `GATEWAY_BYPASS_PING=true` when fixing Scenario A), and does not restart Docker. It is a read-only diagnostic with an opt-in remediation flag. Safe to run on a healthy gateway  the worst case is a 30-line report that says "OK".

**Why cdn-cgi/trace over whatismyip.com for the verdict.** `whatismyip.com`'s ISP database routinely mislabels campus IPv6 ranges as residential ISPs (that's why the local ISP's ASN appears even when you're routing through Cloudflare). `cdn-cgi/trace` is hosted by Cloudflare ITSELF and reports `warp=on|off` plus the exact edge colo serving the request. It is the only public endpoint that can truthfully confirm whether the WARP tunnel is in use.

**Common false alarms.**

* **`warp=on` but whatismyip.com still shows the local ISP.** `whatismyip.com`'s ISP-detection database is unreliable on academic networks. Trust `cdn-cgi/trace`'s `warp=on` over `whatismyip.com`'s ISP string. Use `https://api.ipify.org` (returns just the IP, no ISP DB lookup) as a third confirmatory check.
* **`tailscale status` shows the host online but exit node preference is missing.** `tailscale up --reset --exit-node=` (empty value) clears any stale preference; `tailscale up --reset --exit-node=<100.x.x.x>` re-establishes it. The diagnostic prints the exact command with the resolved TS_IP filled in.
* **Both IPv6 leak AND route-freeze flags set simultaneously.** Scenario B outranks Scenario C  fixing IPv6 makes IPv4 the only viable egress, after which the route-freeze usually self-resolves within ~30s (or one more `toggle.sh` cycle).

#### 10.30.1 Host Leak Probe  Continuous Sub-Second Egress Monitor

**What it is.** `scripts/host-leak-probe.sh` is a lightweight background daemon (~1.4 MB RSS, ~01% CPU) that polls `https://www.cloudflare.com/cdn-cgi/trace` every 300ms directly from the host via `curl` not via `docker compose exec`. This is a fundamentally different vantage point from the WARP Watcher (10.32): the WARP Watcher asks the WARP *container* whether it sees `warp=on`; the Host Leak Probe asks what a real browser request leaving the host's physical NIC looks like. The two blind spots it covers that the WARP Watcher cannot:

1. **Host-side flash-leaks**  a Cloudflare anycast edge re-route, a browser reusing a stale pooled connection across a tunnel state change, or a split-second routing gap during a Gluetun healthcheck restart (see 10.18  acknowledged 14s window).
2. **Host egress while container is healthy**  the container can report `warp=on` while the host's own traffic exits direct (the three-scenario leak class documented in 10.30).

**Lifecycle.** Started by `toggle.sh`'s START path (`start_host_leak_probe`) immediately after `start_warp_watcher`. Controlled by:

| Signal path | What happens |
|---|---|
| `toggle.sh` STOP | `stop_host_leak_probe()`  SIGTERM + PID file cleanup |
| `toggle.sh` `cleanup_handler` (error/INT/TERM/HUP) | Same |
| `recover.sh` (default, non `--post-wake`) | 6d block  kill by PID file |
| `recover.sh --post-wake` | Left running  the gateway is still live |

PID file: `/tmp/nullexit-host-leak-probe.pid`. Toggle with `HOST_LEAK_PROBE=false` in `.env` to disable at next gateway start.

**Output format.** All events are written to `output.log` (repo root) with UTC timestamps. Only state *changes* are logged  the probe is silent during normal healthy operation.

```
[09:10:26] LEAK warp=off ip=45.23.1.1 prev_warp=on prev_ip=104.28.246.50
[09:10:26] ROTATE warp=on ip=104.28.248.11 prev_ip=104.28.246.50
[09:10:26] HOST-PROBE failed/timeout (count=1)
~~~

curl transport errors (`TLS handshake failed`, `Connection refused`, etc.) are also appended to `output.log` via `2>>output.log` nothing is silenced to `/dev/null`.

Grep cheatsheet.

```bash
grep LEAK output.log            # real warp=off events from the host
grep ROTATE output.log          # Cloudflare anycast edge rotations (harmless)
grep 'HOST-PROBE' output.log    # curl failures / timeouts
~~~

**Resource budget.** ~1.4 MB RSS, ~0% CPU at rest, ~3 HTTPS requests/second to Cloudflare. Safe to leave running for hours. Back off the polling interval (`bash scripts/host-leak-probe.sh 1.0`) if you observe probe-failure pile-ups indicating Cloudflare rate-limiting.

**Detection vs prevention.** This script detects leaks; it does not prevent them. A sub-300ms flash-leak can still slip between two polls undetected. If `grep LEAK output.log` shows confirmed leak events, the next step is a kill-switch (prevention)  an `iptables` rule on the host's `en0` that drops non-Tailscale, non-local egress whenever the gateway is active. That is a separate engineering task not yet implemented.
```
```

~~~~
  * `ProcessType: Background`  hint to App Nap not to suspend us.
  * `StandardOutPath/StandardErrorPath: /tmp/nullexit-watcher.{out,err}.log`  separates launchd-level diagnostics from the script's own `/tmp/nullexit-watcher.log` (which `exec >>` redirects).

#### Install (one-time)

The plist lives in the repo at **`launchd/com.nullexit.wake-recovery.plist`** for version control. The installed (live) copy must land in `~/Library/LaunchAgents/` so launchd picks it up on the user session.

```bash
# Copy the plist to the user-launchd location (run from repo root)
cp ./launchd/com.nullexit.wake-recovery.plist \
   ~/Library/LaunchAgents/

# Substitute __NULLEXIT_HOME__ with your local install absolute path.
# The plist uses WorkingDirectory + a relative program arg, so this
# single substitution is the only place it references your machine.
sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|" \
    ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist

# Validate the XML
plutil -lint ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist

# Load + persist this user session + every future login
launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist

# Confirm it actually started
launchctl list | grep nullexit
```

To **stop** the watcher (without uninstalling):

```bash
launchctl bootout gui/$UID/com.nullexit.wake-recovery
# or:
launchctl unload -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
```

To **uninstall** completely:

```bash
launchctl bootout gui/$UID/com.nullexit.wake-recovery 2>/dev/null || true
rm ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
```

#### Why this fix was preferred over alternatives

* **Polling with `StartCalendarInterval`** would have given ~30-60s detection latency per wake. The unified-log stream is sub-second.
* **Sleepwatcher** would have covered wake events but not network changes. We'd still need a separate `scutil` listener, and introducing a third-party dependency for what amounts to ~150 lines of shell is unjustified.
* **Forcing `caffeinate -l`** doesn't exist and the closest equivalent (`caffeinate -s`) only works on AC power. The whole point of the gateway is to stay usable on battery while traveling  lid close must NOT silently kill the gateway.
* **A kernel extension** is impossible (no entitlement) and out of scope.

#### Reproduction recipe

Test both events in isolation to confirm the fix actually closes the gap.

* **(A) Lid-only repro**: with the gateway up, run in a terminal: `pmset displaysleepnow && sleep 30 && pmset wake` (without physically closing the lid). Watch the wake wake-fires the watcher  post-wake routine runs  login should still work in <5s after wake. Compare to baseline pre-fix: pre-fix the gateway would be dead for 30-90s.
* **(B) Roam-only repro**: with the gateway up: `networksetup -setairportpower off && sleep 30 && networksetup -setairportpower on` (or simply walk out of Wi-Fi range and back). The captive-portal WILL repave DHCP DNS for a few seconds; the watcher fires on the second `State:/Network/Global/IPv4` change (after the Wi-Fi re-associates) and the post-wake routine re-hijacks DNS within 2-3s.

#### Operative files

* `recover.sh`  added arg parsing; wrapped destructive sections behind `POST_WAKE`; added re-hijack DNS / refresh exit-node / force-recreate-if-unhealthy path.
* `toggle.sh`  added `write_gateway_active_marker` / `clear_gateway_active_marker` called at the END of START and TOP of STOP / cleanup_handler.
* `scripts/watcher.sh`  new long-running daemon.
* `launchd/com.nullexit.wake-recovery.plist`  launchd LaunchAgent descriptor.
* `~/Library/LaunchAgents/com.nullexit.wake-recovery.plist`  the installed copy (one-time copy).

~~~~

```
#### Honest assessment (read before testing)

This fix has two asymmetric halves:

* **Network / roam / captive-portal path  RELIABLE.** `scutil n.watch` on `State:/Network/Global/IPv{4,6}` catches every Wi-Fi roam, captive-portal DHCP-rebind, hotspot handoff, and VPN change with zero missed events. To validate, drop Wi-Fi for 30 s and re-add it; `/tmp/nullexit-watcher.log` will record a `NET:` trigger and `recover.sh --post-wake` will run within ~10 s of the network coming back.
* **Lid close / wake path  BEST-EFFORT on macOS Sonoma+.** Apple *suspends* LaunchAgents process-state during forced sleep and *resumes* them on wake; it does NOT re-launch them every wake. As a result: (a) `log stream` is a live subscription  events emitted during the agent's suppressed window are DROPPED, not buffered, so the wake event that prompted the resume is invisible to `log stream` on resume; (b) the "fire on startup" guard runs once on `launchctl load -w` and after a `KeepAlive` crash-recovery, NOT on every successive wake; (c) `subsystem == "com.apple.powermanagement"` will catch FUTURE emissions (display dim/restore, manual sleep/wake) but not the lid-closeopen wake directly.

In practice the lid-wake gap is short, because `toggle.sh`'s existing DNS Watcher already polls every 5 s and re-hijacks DNS (so DNS recovers within 5 s of any roam or wake), `tailscaled` self-heals via DERP fallback within 3060 s, and `gluetun` reconnects via its healthcheck retry within 30 s. The user-visible pain on lid open is ~30 s of wonky DNS, not a permanent outage.

**Test the ROAM path FIRST.** If ROAM works in your testing, the lid-wake gap is acceptable. If lid-wake handling is critical, the two clean follow-ups are:
1. **Sleepwatcher** (one canonical install): `brew install sleepwatcher`; drop `recover.sh --post-wake` into `~/.sleep` and `~/.wake` so Sleepwatcher invokes it directly on every wake without the launchd propagation problem.
2. **Move the watcher to a LaunchDaemon** (root) and use `IOPMSchedulePowerEvent` via a tiny C wrapper  not portable to shell.

A truly reliable shell-only solution to "wake events from inside a LaunchAgent" does not exist on macOS Sonoma+.

#### Empirical lessons from deployment

Two findings came up while deploying this fix end-to-end; recording so the next operator doesn't lose an evening to each.

1. **Bash function-call-before-definition gotcha.** While introducing the gateway-active marker helpers, the defensive `clear_gateway_active_marker` call was inserted at the very top of `toggle.sh` (right after `set -e`), but the function definition lived near `PID_FILE="/tmp/nullexit-caffeinate.pid"` ~90 lines down. Bash parses top-to-bottom and resolves names at first invocation, so the call site exited with code 127 (`clear_gateway_active_marker: command not found`). The gateway stayed unreachable from the START path even though `bash -n` passed (the parser doesn't catch order-of-resolution errors). Rule: define functions BEFORE any block that calls them. Verify with `grep -n '^name()\s*{'` followed by `grep -n '^name\s*$'`  the latter must appear on a line number AFTER the former.

2. **`networksetup -setairportpower off+on` is not a faithful roam reproduction.** Synthetic Wi-Fi radio-cycling (`sudo networksetup -setairportpower Wi-Fi off` for 30 s, then back on) is expected to produce a `NET:` trigger in `scutil n.watch` but produced ZERO during testing  because `setairportpower` powers the radio at the driver level while leaving the network SERVICE in the `"up"` state from scutil's perspective, so no `n.state State:/Network/Global/IPv4` event is generated. A real roam (macOS briefly loses the AP and re-associates into a different one) DOES fire the event because the link actually changes.
   Better reproduction recipes in priority order:
   * Walk between two real SSIDs via `networksetup -switchtolocation` (or actual physical station movement)  guaranteed to fire the scutil event.
   * Force a DHCP rebind on the same SSID with `sudo ipconfig set en0 DHCP`  triggers `State:/Network/Global/IPv4` to update.
   * Trust the existing 30-second DNS Watcher inside `toggle.sh` for the DNS path: it re-hijacks DNS automatically. The post-wake watcher adds clear value mostly for tailscale exit-node re-assertion and warp gluetun force-recreate, both of which self-heal within ~30 s anyway.

### 11.18 Infinite Egress Routing Loops & Subnet Takeover Paradoxes

**Problem:** Two distinct network loops can break host egress connectivity entirely when the exit node is active:

1. **The Recursive Host-VM Tunnel Loop:**
   - **Mechanism:** The host Mac's default route points to the Tailscale exit node (`utun*`). The exit node container sends its encrypted tunnel packets (destined for the WARP endpoint `162.159.192.1`) back to the host via the VM NAT. Without a static route exception on the host Mac, the host Mac routes those packets right back into the exit node (`utun*`), creating an infinite loop that blackouts all host network egress.
   - **ARP-Scoping Pitfall:** Simply binding the bypass route to the interface (`route add -host 162.159.192.1 -interface en0`) fails when the default route is deleted or redirected to `utun*`. Since `162.159.192.1` is not local to the physical link, macOS fails to resolve it via ARP, dropping all tunnel packets.
```

~~~
   - **Fix:** Dynamically resolve the active physical gateway IP (e.g. `172.17.0.1`) on startup, and add the static host routes for the WARP endpoints via the gateway IP directly (`route add -host 162.159.192.1 <gateway_ip>`). Additionally, because standalone `tailscaled` on macOS does not automatically modify the system default route via the CLI, we manually redirect the default gateway to `utun*` using `setup_exit_node_routing`.

2. **The Docker Compose Subnet Takeover Collision:**
   - **Mechanism:** To avoid collisions with the host's campus network, Colima's default bridge (`docker.bip`) is moved (e.g. to `10.200.0.1/24`). However, because `172.17.0.0/16` is now free inside the VM, Docker Compose's IPAM dynamically takes it over for the project's custom network (`nullexit_default`). Because the containers receive IPs on `172.17.0.0/16`, the host Mac cannot send local peer discovery packets (e.g., Tailscale `magicsock` packets on `172.17.0.2:41641`) directly. The host routing table sends them out `en0` to the physical Wi-Fi, returning `host is down` or `no route to host`.
   - **Fix:** Explicitly configure a custom default network subnet `10.200.1.0/24` in `docker-compose.yml` to prevent Docker Compose from ever reclaiming the conflicting `172.17/172.18` ranges.

> See also 10.23 (The Hotspot Paradox) for the related infinite loop that occurs when the physical upstream router is also a Tailscale exit-node client.

### 11.19 In-Flight WARP Tunnel Liveness Monitor & Statistical Auto-Shutdown

**Why.** nullexit's core promise is that your IP always appears as Cloudflare. If the WARP tunnel silently dies  due to a UDP NAT timeout, a Cloudflare edge rotation, a Colima VM clock skew causing TLS cert rejection, or any of a dozen other failure modes  the host silently falls back to egressing through the physical ISP. The user sees the gateway as "up" (containers running, Tailscale connected, DNS hijacked) but their real IP is exposed. This is the exact class of failure that Ingo Blechschmidt documented in his chilling 2024 Linux kernel post-mortem: for over two years, LUKS disk encryption on suspend was silently broken because a refactored kernel syscall returned success while doing nothing ([source](https://mathstodon.xyz/@iblech/116769502749142438)). The lesson: **a security mechanism that fails silently is worse than no mechanism at all**  it creates a false sense of safety that prevents the user from taking corrective action.

**Design principle.** The WARP Watcher (`start_warp_watcher()` in `toggle.sh`) is a background daemon that polls `cdn-cgi/trace` every 30 seconds and applies a **statistically calibrated threshold** before triggering any safety response. This threshold distinguishes transient network blips (which self-heal within seconds) from genuine outages (which require intervention). A single `warp=off` reading is meaningless  Docker might be slow, the network might be congested, a container healthcheck might be restarting. But 6 consecutive off readings (30 continuous seconds of downtime) has vanishingly low probability of being a false positive: even at an unrealistically high 10% false-positive rate per poll, P(6 consecutive) = 0.1  0.0001%. In practice, the per-poll false-positive rate is far lower than 10%, making the real probability astronomically small.

**What it does.**

1. **Always logs.** Every state transition (onoff, offon) is timestamped to `output.log`. The user can `grep WARP output.log` to audit the tunnel's entire lifetime. This is the "never fail silently" mandate.

2. **Tracks consecutive failures.** A counter increments on each `off` reading and resets to 0 on any `on` reading. This ensures the watcher never fires on intermittent flapping.

3. **Auto-shuts down at threshold.** When the counter reaches `WARP_FAIL_THRESHOLD` (default 6, configurable in `.env`), the watcher declares a statistically significant outage and runs `recover.sh`  the nuclear recovery script that disconnects Tailscale, stops Docker containers, resets DNS to `1.1.1.1`, flushes routes, and power-cycles Wi-Fi. This instantly reverts the host to normal (un-encrypted) internet, guaranteeing no traffic leaks through a dead tunnel while the user thinks they're protected. A trigger file at `/tmp/nullexit-warp-shutdown-triggered` prevents double-firing.

4. **Exits cleanly.** After shutdown, the watcher exits (the gateway is dead, there's nothing left to monitor). `stop_warp_watcher()` is also called by `toggle.sh`'s STOP path and `cleanup_handler` to terminate the daemon cleanly during normal teardown.

**Configuration.** Set `WARP_FAIL_THRESHOLD=3` in `.env` for a more aggressive 15s timeout, or `WARP_FAIL_THRESHOLD=12` for a conservative 60s window. The variable is parsed from `.env` using the same `grep` pattern as `GATEWAY_MSS` and `GATEWAY_BYPASS_PING`.

**Log sample.** After a real outage (or a forced test via `docker compose pause warp`):

```
[2026-07-03T21:15:17Z] WARP DOWN  cdn-cgi/trace reports warp=off
[2026-07-03T21:15:47Z] WARP SHUTDOWN  6 consecutive failures (threshold=6). Running recover.sh to kill the gateway and restore normal internet. Your IP is NO LONGER Cloudflare.
[2026-07-03T21:15:52Z] WARP SHUTDOWN  recover.sh completed, watcher exiting. Your IP is now your real ISP IP.
```

Or, if WARP self-recovers before the threshold:

```
[2026-07-03T21:15:17Z] WARP DOWN  cdn-cgi/trace reports warp=off
~~~

```
[2026-07-03T21:15:32Z] WARP RECOVERED  warp=on after 3 consecutive off readings (threshold was 6)
‘‘‘

**Relationship to other monitors.** The WARP Watcher is the **innermost safety layer**  it runs as a child of ‘toggle.sh‘ and only while the gateway is active. It complements (does not replace) the ‘scripts/watcher.sh‘ daemon (10.29, which handles sleep/wake and Wi-Fi roam) and ‘scripts/diagnose-host-leak.sh‘ (10.30, which is a manual diagnostic tool). The WARP Watcher is the only component that actively verifies end-to-end tunnel liveness and takes autonomous action when it fails.

---

## 12. Recent Updates

Reflects the current branch’s state. Each entry one line + commit hash; read the commit message for detail.

- **July 10, 2026  ‘fix(race): suppress WARP Watcher during post-wake force-recreate‘**  Fixed a race condition where switching Wi-Fi caused the host IP to leak as the raw ISP IP (‘132.x‘) on every network roam. ‘recover.sh --post-wake‘ force-recreating the warp container (~35s) caused the background WARP Watcher to accumulate 6 consecutive ‘warp=off‘ readings and fire nuclear ‘recover.sh‘, killing a gateway that was healing itself. Fixed by adding a ‘/tmp/nullexit-warp-inhibit.marker‘: ‘recover.sh‘ writes it at startup in ‘--post-wake‘ mode (with ‘trap ... EXIT‘ for guaranteed cleanup); the WARP Watcher loop in ‘toggle.sh‘ checks it first and resets ‘consec_off=0‘ while it exists. See 25 for full post-mortem.

- **July 6, 2026  ‘fix: resolve edge-case vulnerabilities and system state leaks‘**  Addressed a suite of subtle but critical edge cases identified in security review:
  1. **DNS Watcher Interface Independence:** Fixed ‘start_dns_watcher‘ in ‘toggle.sh‘ to dynamically detect the active interface via ‘get_active_service()‘ rather than hardcoding a ‘grep‘ for "Wi-Fi", ensuring roaming on Ethernet/Thunderbolt stays protected.
  2. **Robust Lock Verification:** Upgraded the ‘toggle.sh‘ lock-file logic to use ‘ps -p‘ verification, preventing permanent lockouts if a crashed script’s PID is recycled.
  3. **Fail-Closed Crypto:** Changed the ‘crypto.sh‘ integrity check in ‘toggle.sh‘/‘recover.sh‘ from an executable bit check (‘[ -x ]‘) to a pure file-existence check (‘[ -f ]‘), guaranteeing execution via ‘bash scripts/crypto.sh‘ even if git strips the ‘+x‘ bit.
  4. **Strict ‘iptables‘ Pinning:** Upgraded ‘post-rules.txt‘ FORWARD-chain rules to explicitly pin traffic between ‘tailscale0‘ and ‘tun0‘ in both directions, mathematically preventing Tailscale traffic from escaping through ‘eth0‘ during WARP tunnel drops.
  5. **Docker Local Network Isolation:** Modified ‘docker-compose.yml‘ to explicitly bind the exposed WARP container ports (‘1080‘, ‘5354‘) to ‘127.0.0.1‘. This prevents other devices on the local bridged Wi-Fi/LAN from accessing the SOCKS5 proxy or DNS resolver if Colima provisions a LAN IP.
  6. **Routing Verification & Sudoers:** Added a ‘netstat -nr‘ check (handling macOS ‘0/1‘ syntax) to verify the default route (‘0.0.0.0/1‘) actually overrides to Tailscale, and updated the ‘README.md‘ and ‘setup.sh‘ sudoers templates to ensure ‘/usr/bin/python3‘ is permitted so the fallback DNS proxy doesn’t silently fail.

- **July 4, 2026  ‘cc789c5‘ (fix: resolve concurrency, setup variables, CI workflow, and untrack review file)**  Added concurrency protection on ‘toggle.sh‘ using ‘/tmp/nullexit-toggle.lock‘ to prevent parallel runs. Initialized ‘TS_AUTHKEY‘ defensively in ‘setup-common.sh‘ to prevent unbound variable crashes under ‘set -u‘. Added Go compilation (‘go vet‘ and ‘go build‘) steps to GitHub Actions CI workflow. Ported AdGuard Home container to cap logs. Fixed macOS BSD ‘grep‘ compatibility in ‘setup-common.sh‘ using ‘grep -oE‘, resolved space stripping in ‘common.sh‘, fixed quoting/PID file/routing text in ‘recover-linux.sh‘, and removed redundant bypass routes block in ‘recover.sh‘.

- **July 2, 2026  ‘8c5fae1‘ + ‘181e1e1‘ (feat(routing): resolve recursive host-VM tunnel loop & Docker subnet collision)**  Two infinite routing loops fixed. (1) Host-VM tunnel loop: WARP endpoint bypass routes now injected via physical gateway IP (not interface scoping) to avoid ARP failures under default-route redirection; ‘setup_exit_node_routing‘ added to manually force the host default route to ‘utun*‘ since standalone ‘tailscaled‘ CLI on macOS does not update the routing table automatically. (2) Docker Compose subnet takeover: ‘docker-compose.yml‘ now locks the default project network to ‘10.200.1.0/24‘ to prevent IPAM reclaiming ‘172.17/172.18‘ after ‘docker.bip‘ is moved. ‘--accept-routes=true‘ added explicitly to all ‘tailscale up --reset‘ calls. Documented in 10.31.

- **July 4, 2026  ‘fix: post-wake routing loop & destructive recovery‘ (networking: routing)**  Resolved a critical fatal bug where waking the Mac from sleep or roaming Wi-Fi permanently broke the internet and required a full system reboot to fix.
  1. **The Routing Loop Bug:** ‘common.sh‘’s ‘add_warp_bypass_routes‘ was previously refactored to dynamically look up the default route without accepting explicit interface arguments. During ‘post_wake‘, Tailscale is already active, so ‘route get default‘ resolved to the Tailscale ‘utun‘ interface. This caused Cloudflare WARP traffic to be routed back *into* Tailscale, creating an infinite recursive routing loop that instantly killed the internet on every wake. Fixed by making ‘add_warp_bypass_routes‘ accept explicit target arguments and updating ‘recover.sh‘ to pass the correct physical interface/gateway.
  2. **The Destructive Recovery Bug:** When users ran ‘recover.sh‘ to fix the broken internet, the script blindly ran ‘sudo route -n flush‘. On macOS, this doesn’t just clear VPN routesit destroys essential system routes including loopback (‘127.0.0.1‘) and multicast (‘224.0.0.0‘). Bouncing Wi-Fi does not automatically restore these internal routes, meaning macOS network services (‘configd‘, ‘mDNSResponder‘) remained permanently wedged until a reboot. Fixed by replacing ‘route -n flush‘ with targeted deletions of specific gateway routes (‘0.0.0.0/1‘, ‘128.0.0.0/1‘) and WARP bypass routes.

```

```
- **July 2, 2026  `feat: secure execution` (security: script lockdown & auth)**  Fixed a major privilege escalation vulnerability and added native macOS authentication prompts to `Toggle Gateway.app` and `Recover Gateway.app`. Previous setups granted `/usr/bin/python3` blanket `NOPASSWD` sudo access, allowing malware to bypass macOS security. We restricted `NOPASSWD` exclusively to `/usr/bin/python3 /Users/<USER>/Developer/nullexit/scripts/dns-proxy.py`. Additionally, `scripts/dns-proxy.py` was locked down with `chown root:wheel` and `chmod 755` so attackers cannot overwrite the script and exploit the restricted sudo rule. `toggle.sh` was also updated to capture `warp` container logs on failure before tearing down the environment.
- **July 3, 2026  `fix: resolve numerous system regressions`**  Addressed several breakages introduced in recent commits:
  1. Fixed a truncated line syntax error in `/etc/sudoers.d/nullexit` that broke `NOPASSWD` fallback commands.
  2. Fixed a `[Errno 13] Permission denied` error in `sync-rules.py` cache writes by using a temporary file and atomic `os.replace()` to bypass inherited mode locks (0444/000). **(Later reverted  see below.)**
  3. Fixed a quoting bug in `recover.sh` where `ts_args=""` passed literal escaped quotes.
  4. Repaired ANSI color outputs in `diagnose-host-leak.sh` and `fix-docker-bridge-collision.sh` using `%b` instead of `%s` and `-e` flags.
  5. Fixed `toggle-linux.sh` calling a macOS-only `networksetup` command instead of `resolvectl dns`.
  6. Corrected the AdGuard REST API refresh POST in `sync-rules.py` to correctly target port `3000`.
  7. Restored the missing `START_GATEWAY=true` variable in `toggle.sh` and purged a stale port `80` mapping from `docker-compose.yml`.
  8. Verified `output.log` is untracked and safely in `.gitignore`.
- **July 3, 2026  `unlock-files.sh` + revert atomic writes in `sync-rules.py`**  The `.tmp` + `os.replace()` atomic-write pattern introduced in the commit above was a workaround for stale restrictive file permissions (mode `0444`/`000`) left on disk by old `chmod` calls that had already been removed from the codebase. Rather than keep the workaround, we created `scripts/unlock-files.sh`  a one-shot script that permanently fixes the stale permissions by replacing each locked inode with a fresh writable one via atomic rename (`cp` + `mv`), operating on the directory entry instead of the file contents. No `chmod` used. All 5 `.tmp` + `os.replace()` sites in `sync-rules.py` were reverted back to direct writes. `scripts/unlock-files.sh` covers: `.env` (mode `000`), `adguard/work/userfilters/compiled_rules.txt`, `ip_blocklist.ipset`, `cache/*.txt`, `cache/ip/*.txt`, and `adguard/work/data/filters/*.txt` (mode `0444`). Run once after setup or pulling an old repo; `toggle.sh` and `sync-rules.py` then work without permission errors. Documented in 10.28 + 3.
- **July 1, 2026  `c5b92c0` (refactor: personal-leak cleanup)**  eliminated every residual personal-username leak across source + config files. The launchd `com.nullexit.wake-recovery.plist` now uses `WorkingDirectory` + a relative program arg with the `__NULLEXIT_HOME__` sentinel; the install recipe in 10.29 gains a single `sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|"` step right after the `cp`. 8 `devref.md` prose sites genericized to `~/.colima/...`, `~/Library/LaunchAgents/...`, `$USER` for sample output, `<USER>` for the sudoers example.
- **July 1, 2026  `a5e2a5a` (refactor: script-relative paths)**  replaced 6 hardcoded `/Users/<user>/<anywhere>/nullexit/...` source-code sites (former per-user install-location literals) with script-relative resolution. `recover.sh` injects `SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"` near the top and uses it for the post-wake warn (l.321) and the broken-for-60s hint echo (l.446). `scripts/watcher.sh` uses the same pattern; `RECOVER="$SCRIPT_DIR/../recover.sh"` resolves without launching-path dependency.
- **June 28, 2026  `f8f8e4e` (M1: scripts/ move)**  8 internal-collaborator scripts (routing-fix.sh, socks5-proxy.py, dns-proxy.py, sync-rules.py, logger.py, toggle-linux.sh, recover-linux.sh, setup-linux.sh) consolidated into a `scripts/` subfolder. The 4 user-facing / orchestrator / config files (`toggle.sh`, `recover.sh`, `setup.sh`, `docker-compose.yml`) deliberately stayed at repo root.
- **June 28, 2026  `0875ef3` (ci: glob)**  CI `py_compile` line switched from `python3 -m py_compile *.py scripts/*.py` to `python3 -m py_compile scripts/*.py` after M1 left root with zero `.py` files (the old form was lazy-empty on bash `nullglob=off` runners).
- **June 28, 2026  `25b37a1` (docs: scripts/ prefix)**  `devref.md` + `README.md` updated to use the `scripts/` prefix for every reference of the 8 moved files. ~22 sites in devref + ~9 sites in README = ~31 patches.

### End-to-end verification (commit `c5b92c0` cyclone)
Manual START  STOP cycle ran clean on macOS after the path relativization + personal-leak cleanup landed:
- `bash toggle.sh` START: exit 0, ~135s (cold Colima boot); marker present (`2026-07-02T03:41:41Z`), all 5 containers healthy, host DNS hijacked to `100.100.21.8` (no fallback), exit node selected, Cloudflare trace through WARP succeeds.
- `bash toggle.sh` STOP: exit 0, ~12s; marker cleared, all nullexit containers gone, host DNS restored to `1.1.1.1`, Tailscale disconnected.

---

## 13. Privacy Architecture, Threat Model & Upgrades

### Layered Defense
The gateway stacks four layers targeting different attack surfaces:

- **Layer 1  ISP-Level Surveillance (WARP):** Your ISP sees only an encrypted WireGuard tunnel to a Cloudflare IP. In the US, ISPs can legally sell your browsing metadata and are subject to secret NSLs (National Security Letters). WARP removes their visibility entirely.
- **Layer 2  DNS Tracking (AdGuard Home):** DNS is the phone book of the internet. By default, queries go to your ISP's resolver, giving them a complete log of your traffic. AdGuard intercepts all DNS from mesh devices, blocks trackers, and resolves queries through the WARP tunnel.
- **Layer 3  Device Identity & IP Exposure (Tailscale):** On untrusted networks (coffee shops, public Wi-Fi), your device's IP is exposed. Tailscale creates an encrypted WireGuard mesh between your devices, routing all traffic to your gateway exit node.
```

```
- **Layer 4  Kernel-Level IP Blocking (ipset/iptables):** Sophisticated malware bypasses DNS entirely with hardcoded IPs. The ‘rule-compiler‘ fetches ~16,700 threat-intelligence IPs/CIDRs (Spamhaus, Feodo Tracker, Emerging Threats, CINS) and enforces them as ‘DROP‘ rules in the kernel ‘FORWARD‘ chain  blocking both outbound C2 connections and inbound attack traffic.

### The Documented Threat: Why This Matters
This architecture is calibrated against documented mass-surveillance programs revealed in the Snowden disclosures:
- **PRISM:** Bulk collection of communication content from major tech providers.
- **UPSTREAM:** Tapping physical backbone fiber, collecting metadata and content in bulk.
- **XKeyscore:** Retroactive search of unencrypted traffic.
- **MUSCULAR:** Infiltration of unencrypted internal datacenter interconnect links.

Passive bulk collection is not paranoia  it is a documented reality. By double-encrypting and routing traffic, nullexit prevents your ISP from harvesting your metadata.

### Trust Assumptions & Shifting
Every component shifts trust rather than eliminating it:
1. **Physical Device Integrity:** You trust your physical devices are not compromised.
2. **Cloudflare Integrity:** You trust Cloudflare not to correlate your WARP tunnel with exit traffic.
3. **Tailscale Integrity:** You trust Tailscale's coordination server not to MITM WireGuard keys.

The goal: no single provider has a complete picture. Your ISP sees an encrypted tunnel. WARP sees traffic with no account identity. Tailscale sees node topology but not content.

### OPSEC: Tor-over-WARP Architecture

When extreme anonymity is required, integrating the Tor network into ‘nullexit‘ provides a powerful Tor-over-VPN topology:
* **The University/ISP** sees only an encrypted Cloudflare WireGuard tunnel, rendering them completely blind to the fact that you are using Tor (effectively bypassing Enterprise firewall Tor blocks).
* **Cloudflare** sees an encrypted TCP stream heading to a Tor Guard Node. They cannot see your DNS lookups, the destination website, or the content of your traffic.
* **The Tor Exit Node** sees your traffic, but not your real IP (only the Cloudflare IP).

#### The "Transparent Proxy" Trap (What NOT to do)
It is tempting to build a system-wide "Transparent Tor Proxy" that forces all host traffic (e.g., standard Chrome/Safari browsers, background iCloud syncs) through Tor automatically. **This is a massive OPSEC trap and destroys anonymity.**
Modern operating systems and browsers will instantly leak your identity through background syncing (e.g., Apple Mail, Google Drive) and browser fingerprinting (canvas fingerprints, WebRTC, screen resolution). The Tor network protects your *IP*, but if your payload contains identifying cookies or unique fingerprints, the Tor Exit Node (which could be run by malicious actors) will instantly tie your session to your real identity.

#### The nullexit Implementation: Isolated Procedural Proxy
To safely integrate Tor without triggering the transparent proxy trap, ‘nullexit‘ implements an **Isolated Tor SOCKS5 Proxy** with **Procedurally Generated Ports**:
1. **Isolated Container:** A lightweight ‘tor‘ Docker container runs securely inside the ‘warp‘ network namespace. It does not hijack system traffic.
2. **Opt-In Usage:** You must consciously configure a highly-hardened browser (like the official Tor Browser or a dedicated Firefox profile) to use the proxy. This prevents accidental background sync leaks.
3. **Procedurally Generated Ports:** Rather than hardcoding the standard proxy ports (like ‘127.0.0.1:9050‘ or ‘1080‘), ‘toggle.sh‘ randomizes all internal proxy ports on every boot into the ephemeral range (10000-65000) and silently writes them to ‘/tmp/nullexit-ports.env‘. This completely defeats static fingerprinting by malware.
4. **Kernel-Level Collision Avoidance:** To prevent the randomizer from picking a port already in use by a root daemon or Apple service (which would crash the gateway), the script directly queries the macOS kernel socket table (‘netstat -an -p tcp‘) to verify the port is absolutely free before assigning it.
5. **The Honey-Port Tripwire:** While procedural ports defeat static fingerprinting, advanced malware might attempt a full sequential port scan of ‘localhost‘ to find the hidden proxy. To counter this, ‘toggle.sh‘ spawns a "Honey-Port" (a fake random port with a netcat listener attached). If any local application attempts to connect to the Honey-Port, it instantly logs a critical security alert to ‘output.log‘ and fires a macOS desktop notification, serving as an early-warning Intrusion Detection System (IDS) for local malware.

#### Hardening: Tor Bridges and obfs4 Pluggable Transports
For users operating under severe Deep Packet Inspection (DPI), ‘nullexit‘ supports configuring Tor to connect exclusively through private bridges using the ‘obfs4‘ pluggable transport.

When ‘TOR_USE_BRIDGES=true‘ is set:
- **What it does:** It hides the entry IP from being matched against the public Tor consensus list, and disguises the traffic's protocol fingerprint from the WARP exit provider (Cloudflare).
- **What it does NOT do:** It does *not* add any extra layers of anonymity beyond what Tor already provides. It is purely a censorship-circumvention and obfuscation mechanism.
- **Limitations:** ‘obfs4‘ is highly effective against passive DPI, but it is not guaranteed to defeat active-probing-resistant detection by a highly sophisticated state-level adversary.

```

```
> **Architecture Rule:** The ‘nullexit‘ gateway deliberately does NOT bundle, hardcode, or automatically fetch bridge lines. Bridge lines are highly sensitive and expire. Users must obtain their own bridge lines from ‘https://bridges.torproject.org‘ and manually populate the ‘tor-bridges.txt‘ file. If bridges are enabled but the file is missing or empty, the Tor container will intentionally crash and fail to start rather than silently falling back to public guard relays.

**The Bootstrapping Paradox (Out-of-Band Key Exchange)**
Automating the bridge acquisition process (e.g., scripting the gateway to log into the Telegram ‘@GetBridgesBot‘ via MTProto API) introduces catastrophic OPSEC flaws:
1. **Identity Leaks:** Baking a Telegram session into the container explicitly links the "anonymous" gateway to a real-world physical phone number.
2. **The Chicken & Egg Deadlock:** In heavily censored regimes, Telegram itself is usually blocked. If the container requires Telegram to fetch bridges to bypass the firewall, it will fail because it cannot bypass the firewall to reach Telegram.
3. **Bridge Exhaustion & CAPTCHAs:** Automated scraping behaves identically to state-sponsored scrapers, triggering Tor’s anti-bot CAPTCHAs which leads to silent proxy failures and exhausts the limited public bridge pool.

Instead, users should rely on a less secure layer to manually reach Telegram, obtain the bridges, and populate ‘tor-bridges.txt‘. This can be done via the standard ‘nullexit‘ WARP tunnel, a mobile phone on cellular data, Mullvad VPN, or a secure remote VPS. *(Note: We plan on fully integrating Mullvad and VPS architectures directly into ‘nullexit‘ in the future, but have not had the time to build it yet).* This intentionally "air-gaps" the cryptographic key exchange from the proxy infrastructure, leaving zero API tokens and zero network traces for an adversary to exploit.

### OPSEC: Python Runtime Airgap (Isolated Mode)
‘nullexit‘ explicitly relies on the host’s native system Python 3 runtime for critical components like ‘scripts/dns-proxy.py‘. Because this script runs with elevated privileges (‘sudo -n‘) to bind to the privileged UDP/53 socket, modifying or polluting this Python environment is strictly forbidden.

**The Zero-Dependency Rule & Isolated Mode (‘-I‘)**
You must **NEVER** install third-party ‘pip‘ packages (e.g., ‘requests‘, ‘pyyaml‘) into the system Python environment that ‘nullexit‘ uses.
- All Python scripts in this repository must be written using *only* the Python Standard Library (‘socket‘, ‘urllib‘, ‘json‘, ‘ssl‘).
- Third-party packages introduce a massive supply-chain attack vector. A compromised ‘pip‘ package installed globally could allow local malware to hook into the Python runtime and exploit the ‘sudo‘ privilege of the DNS proxy to gain full root access.
- To enforce strict immunity against user-installed malicious packages, ‘toggle.sh‘ explicitly invokes the proxy using **Python’s Isolated Mode** (‘python3 -I‘). This flag forces the interpreter to completely ignore ‘PYTHONPATH‘, ‘PYTHONHOME‘, and the user’s ‘site-packages‘ directory, executing exclusively from the read-only, Apple-signed system framework.

### Recommended Upgrades

| Layer | Default | Upgraded |
|---|---|---|
| VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, proven no-logs, anonymous payment) |
| DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH (Cloudflare-blind) |
| Coordination | Tailscale (US, OAuth) | Headscale (self-hosted, no third-party) |
| Auth | OAuth / persistent keys | Ephemeral auth keys (no identity persistence) |
| Content | WireGuard asymmetric | WireGuard + PSKs (coordination server blind) |

#### Mullvad (Replacing WARP)
Swedish-based. Police raided their offices in 2023 and left empty-handed  no logs exist to seize. Accounts are random numbers. Payment by cash or Monero. Replace ‘VPN_SERVICE_PROVIDER=custom‘ with ‘VPN_SERVICE_PROVIDER=mullvad‘ in ‘docker-compose.yml‘.

#### Mullvad DoH Upstreams
Point AdGuard’s upstream resolvers to ‘https://adblock.dns.mullvad.net/dns-query‘. Cloudflare WARP only sees encrypted HTTPS to Mullvad’s IPs  completely blind to your DNS queries.

#### Headscale
Self-hosted Tailscale coordination server. Eliminates Tailscale the company, keeping all node topology under your control.

#### Ephemeral Auth Keys
Generate in the Tailscale admin panel. Used once to authenticate; node disappears from the admin panel after disconnect. Breaks persistent identity linkage.

#### Pre-Shared Keys (PSKs)
Add a WireGuard PSK between peer nodes. Adds a symmetric encryption layer that Tailscale’s coordination server has no knowledge of, blinding it from reading transit content.


### Structural Limitations
- **Traffic Analysis / Metadata:** Passive fiber tapping (UPSTREAM) can observe timestamps, data volumes, and IP ranges without reading content. Defeating traffic analysis requires mixing networks (Tor) or continuous dummy packet padding  both heavily degrade performance.
- **Targeted State-Level Adversaries:** Consumer-grade VPNs are not a complete shield against a nation-state actively targeting you. This stack is designed to defeat bulk passive surveillance and
```

```
corporate dragnet collection.

---

## 14. Per-Device Access Control

Because every mesh device routes internet-bound traffic through the ‘warp‘ container’s ‘FORWARD‘ chain, they all appear with their stable ‘100.x.x.x‘ Tailscale IP as the ‘src‘ address. This gives two powerful enforcement surfaces:

### IP & Port Level (‘scripts/routing-fix.sh‘)
Write raw ‘iptables‘ rules targeting specific devices. The 30-second idempotent loop auto-survives Gluetun reconnects:

‘‘‘bash
# Block a specific device from a target subnet
iptables -I FORWARD -s 100.x.x.x -d 203.0.113.0/24 -j DROP

# Restrict a device to only HTTP/HTTPS
iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 3000 -j DROP
iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 443 -j DROP

# Block a device from internet egress entirely (mesh only)
iptables -I FORWARD -s 100.x.x.x -j DROP

# Time-based (e.g., cut off 10 PM  7 AM)
iptables -I FORWARD -s 100.x.x.x -m time --timestart 22:00 --timestop 07:00 -j DROP
‘‘‘

### DNS Level (AdGuard REST API)
AdGuard Home supports per-client rules keyed by Tailscale IP:

‘‘‘bash
curl -X POST http://localhost:80/control/clients/add \
  -H "Content-Type: application/json" \
  -d ’{
    "name": "kids-ipad",
    "ids": ["100.x.x.x"],
    "blocked_services": ["youtube", "tiktok", "instagram"],
    "safesearch_enabled": true
  }’
‘‘‘

---

## 15. Packaging nullexit as a macOS Application (.dmg)

nullexit can be packaged into a native ‘.app‘ bundle, but this introduces nested virtualization requirements.

nullexit uses **Colima** (Apple Virtualization Framework ‘vz‘) to run a Linux VM hosting Docker. Installing the ‘.app‘ inside a virtualized macOS environment (Parallels, cloud Mac) means running a Linux VM inside a macOS VM  requiring hardware-level **nested virtualization**.

### Hardware Support
- **Supported:** Apple Silicon M3/M4 (macOS Sequoia 15+), Intel Macs with VMX passthrough.
- **Unsupported:** M1, M2, A18 Pro. Colima crashes silently; the terminal (‘setup.sh‘) surfaces crash logs.

### Build Steps
‘‘‘bash
# 1. Verify nested virtualization support
sysctl -a | grep hv_nested_virt_supported  # expect: 1

# 2. Compile the AppleScript launcher
osacompile -o "Nullexit.app" "Toggle Gateway.applescript"

# 3. Embed scripts into app resources
mkdir -p "Nullexit.app/Contents/Resources/scripts"
cp toggle.sh setup.sh recover.sh docker-compose.yml "Nullexit.app/Contents/Resources/"

# 4. Package into DMG
hdiutil create -volname "Nullexit Installer" -srcfolder "./Nullexit.app" -ov -format UDZO Nullexit.dmg

# 5. Bypass Gatekeeper (unsigned app)
xattr -cr /Applications/Nullexit.app
‘‘‘

Without a paid Apple Developer certificate ($99/yr), users see an "App is damaged" Gatekeeper warning and need to run step 5.

```

```markdown
---

## 16. Moonlight/Sunshine + Tailscale Race Condition

When streaming from a remote Windows host running Sunshine over a Tailscale mesh (e.g., while the client is on a cellular hotspot), a race condition can occur on boot:

1. Both Tailscale and Sunshine are set to start automatically on boot.
2. Tailscale takes a few seconds to fully initialize its ‘100.x.x.x‘ virtual adapter and establish a connection.
3. **Sunshine starts too fast.** It launches before Tailscale is fully ready. Since Sunshine only binds to network interfaces that are active at the exact moment of startup, it misses the Tailscale adapter entirely.
4. Moonlight clients (even when configured with the correct Tailscale IP) will fail to connect because Sunshine isn’t listening on that interface.

**The "Magic Toggle" Symptom:**
If the user manually enables or disables Wi-Fi on the host PC, it triggers a global Windows network change event. Sunshine listens for these events, re-enumerates all network adapters, and says, "Oh, there’s a Tailscale adapter here!" and finally binds to it. Suddenly, Moonlight can connect.

**The Permanent Fix:**
On the Windows host, open ‘services.msc‘, locate the **Sunshine Service**, and change its Startup type from **Automatic** to **Automatic (Delayed Start)**. This forces Windows to wait a minute or two after booting before launching Sunshine, ensuring Tailscale’s virtual adapter is fully initialized first.

---

## 17. Future Work

- **Direct P2P on VM Hosts:** Non-Linux hosts run Docker inside a VM, making true UDP hole-punching impossible. The only working solution is a native Linux host (Raspberry Pi, Intel NUC) where Docker runs without VM hypervisor translation.
- **Native Linux Deployment:** Benchmark on a Raspberry Pi  native container footprint is ~75MB without macOS hypervisor overhead.
- **Mesh-Wide Filesystem (SFTP over Tailscale):** Any mesh device passively exposes its filesystem over SFTP. Android via Termux + OpenSSH + wakelock; iOS is client-only due to background process limits.
- **Post-Quantum Cryptography (PQC):** Tailscale uses Curve25519, theoretically vulnerable to "harvest now, decrypt later" quantum attacks. A future iteration could replace the Tailscale container with raw WireGuard + [Rosenpass](https://rosenpass.eu/) to negotiate post-quantum PSKs.

## 18. Geo-IP Blocking

To block traffic to and from specific countries, the system dynamically downloads country-specific IP ranges from [ipdeny.com](http://www.ipdeny.com/ipblocks/data/countries/) and drops them via iptables ‘FORWARD‘ rules.

To add a new country to the blocklist:
1. Find the 2-letter ISO country code (e.g., ‘cn‘ for China, ‘ru‘ for Russia, ‘il‘ for Israel, ‘kp‘ for North Korea).
2. Open your ‘.env‘ file.
3. Update the ‘BLOCKED_COUNTRIES‘ variable: e.g., ‘BLOCKED_COUNTRIES="kp il cn ru"‘.
4. Run ‘toggle.sh‘ to restart the gateway and apply the new environment variables to the routing-fix container.

> [!NOTE]
> **Limitations of Geo-IP Blocking:** IP-based blocking is highly effective at blocking servers, direct apps, and raw infrastructure physically located and registered in a specific country (e.g., ‘www.gov.il‘). However, it will **not** block high-profile websites (e.g., ‘jpost.com‘) that route their traffic through global Content Delivery Networks (CDNs) like Google Cloud, Cloudflare, or Fastly. Because the CDN’s IP addresses are registered in the US or globally, the network traffic physically never routes to the blocked country, allowing it to bypass the Geo-IP firewall.

## 19. Incident Post-Mortems

### Incident: The Overnight Silent IP Leak (July 2026)

**The Problem:** The host leaked traffic over its raw, unencrypted ‘eth0‘ IP continuously for roughly 8 hours, completely bypassing the VPN. The ‘toggle.sh‘ state and the container liveness checks all reported the gateway as healthy and active.

**The Investigation:**
1. **Sleep/Wake Checks:** We initially suspected macOS sleep cycles broke the routing table. A check of ‘pmset -g log‘ confirmed the laptop literally never slept (0 sleep events recorded since boot), ruling out all sleep/wake code paths.
2. **Keepalive Expiry:** We discovered ‘docker-compose.yml‘ was missing ‘WIREGUARD_PERSISTENT_KEEPALIVE_INTERVAL‘. Without a keepalive, standard router NAT tables (which typically garbage collect UDP after 30 to 120 seconds of silence) quietly expired the WireGuard port mapping overnight.
```

```
3. **Container State Masking:** Because WireGuard is stateless, the ‘warp‘ container didn’t immediately crash. It stayed "Up", which meant Docker’s healthchecks and our ‘toggle.sh‘ liveness checks never noticed the tunnel inside it was totally dead.

**The Solution:**
1. Added ‘WIREGUARD_PERSISTENT_KEEPALIVE_INTERVAL=25s‘ (the WireGuard standard to beat standard 30s NAT timeouts) to keep the UDP tunnel permanently pinned open through the router.
2. Injected a **Fail-Closed Kill-Switch** into ‘routing-fix.sh‘: A 30-second loop now actively verifies tunnel health by running ‘curl‘ over ‘tun0‘ to Cloudflare’s trace endpoint. If it fails 3 consecutive times, it immediately rewrites the default route to ‘blackhole‘, severing all traffic instead of letting it leak through ‘eth0‘.
3. Added a listener in ‘watcher.sh‘ that detects this fail-closed event and instantly pushes a native macOS UI notification to the desktop, alerting the user to manually intervene.

> [!NOTE]
> **Why dual failure systems?**
> The system now relies on two independent failure handlers that cover each other’s blind spots:
> 1. **Automated Self-Healing (WARP Watcher + ‘recover.sh‘):** Triggered when the ‘warp‘ container logs ‘warp=off‘. This happens when the server actively kicks the client. Because the container *knows* it is disconnected, it triggers ‘recover.sh‘ to safely reboot Docker and reconnect.
> 2. **Fail-Closed Kill-Switch (‘routing-fix.sh‘ + ‘watcher.sh‘):** Triggered when a silent network failure occurs (e.g., NAT timeouts). The container incorrectly believes it is connected (‘warp=on‘), so auto-recovery never fires. Instead of auto-recovering (which risks falling back to raw Wi-Fi mid-process while the user isn’t looking), this kill-switch actively blackholes the internet and forces a manual intervention via a desktop notification.

## 20. Design Decisions: Exit Node IP Rotation

A common question for privacy architectures is whether the system should aggressively rotate its public exit IP (e.g., every 5 minutes, like a Tor circuit or a proxy rotator).

‘nullexit‘ uses Cloudflare WARP via Gluetun. WARP provides **anonymity in a crowd** rather than aggressive rotation. Thousands of users share the same Cloudflare Edge IP simultaneously, making your traffic indistinguishable from the herd. The IP may occasionally rotate on its own (which ‘host-leak-probe.sh‘ logs as a ‘ROTATE‘ event), but it remains generally stable.

**Why aggressive IP rotation was intentionally excluded:**
1. **Broken TCP Connections:** Every rotation instantly drops all long-lived TCP sessions (SSH, large downloads, WebSockets, gaming, Zoom).
2. **CAPTCHA & 2FA Hell:** Modern web security (e.g., Cloudflare, Akamai) aggressively flags IP-hopping as bot behavior. Aggressive rotation guarantees the user will be bombarded with "Verify you are human" CAPTCHAs and "New Login Detected" 2FA emails constantly.
3. **WireGuard Architecture:** WireGuard is stateless and does not natively support IP hopping on the fly. To rotate an IP, the VPN client must actively drop the tunnel, request a new endpoint, and re-handshake. This introduces latency gaps where the kill-switch would repeatedly trigger and blackhole the user’s internet.

**Verdict:** For a daily-driver secure gateway, shared-IP crowding (WARP) is superior to aggressive rotation. As an added benefit, Cloudflare WARP IPs are actually highly static (tied to the persistent WireGuard public key generated for your device identity). Because your IP stays static and is part of Cloudflare’s own trusted network, it builds a high "trust score," virtually eliminating CAPTCHA loops while still hiding you within the massive crowd of other users in that data center. All mesh devices routing through your gateway will reliably share this exact same trusted IP.

*(Note: The only exception where aggressive IP rotation is genuinely required is for specialized offensive security taskslike high-volume web scraping or dark web researchwhere avoiding IP bans or active tracking overrides the need for TCP stability.)*

## 21. Cloudflare WARP Privacy & Logging

Because ‘nullexit‘ uses Cloudflare WARP (via Gluetun) as its upstream exit node, the system inherits Cloudflare’s consumer privacy policy.

**What Cloudflare DOES NOT log (Strict No-Logs Policy):**
* **Browsing History:** They do not log the domains or URLs you visit.
* **Traffic Destinations:** They do not log the IP addresses of the servers you connect to.
* **Content:** They do not inspect or log the payload/data of your traffic.
* **Data Brokering:** They explicitly guarantee they will never sell or rent your data to advertisers.

**What Cloudflare DOES log (Minimal Telemetry):**
* **Data Transfer Volume:** Total bandwidth used (required to enforce 10GB/mo quotas if you are not using a paid WARP+ key).
* **Performance Metrics:** Anonymized connection speeds and latency to optimize their network routing.
* **Aggregate Volume:** Total traffic volumes to popular destinations in aggregate (e.g., "datacenter X sent 50TB to Netflix"), stripped of all user IDs.
* **Device ID (Public Key):** They store the persistent WireGuard public key generated by your container to authorize the connection.

**Privacy Verdict:**
```

~~~markdown
Cloudflare WARP provides excellent **historical privacy**. Because they do not log your destinations, they cannot comply with historical subpoenas asking what websites you visited yesterday. However, it is **not absolute anonymity**. They still know your real ISP IP address while you are actively connected. For daily-driver privacy against ISPs, hackers, and corporate trackers, it is top-tier. For nation-state evasion, use Tor.

---

## 22. Battery & Power Measurement Quirks

When trying to optimize this framework (or any daemon) for battery life, developers often look for a way to programmatically measure the exact Wattage consumed by a specific process (e.g., "How many Watts is `toggle.sh` using?").

**This is a hardware impossibility on macOS and Linux.**

### 22.1 Why Activity Monitor is Lying to You
Your machine's logic board only has physical power sensors for the *entire* CPU package, the GPU, and the RAM. It knows the CPU is pulling 5W total, but it has no physical hardware capability to know whether Docker is using 3W and Chrome is using 2W.

The "Energy Impact" score you see in macOS Activity Monitor is a **synthetic, heuristic score** calculated by `powerd`. It penalizes apps for waking up the CPU (idle wakes), doing disk I/O, or keeping the screen awake. It is a relative score, not actual Wattage.

### 22.2 The Proper A/B Testing Methodology
If you want to truly know how many Watt-hours or mAh your background daemons (`routing-fix.sh`, `host-leak-probe.sh`, etc.) are costing you, you must perform a hardware-level A/B drain test:

1. Unplug the laptop, run the gateway, and note your exact battery mAh using:
   ```bash
   ioreg -l | grep "AppleRawCurrentCapacity"
   ```
2. Leave the laptop idle for exactly 1 hour.
3. Run the command again to see exactly how many mAh were drained.
4. Turn the gateway off and repeat the test for another hour.

The delta in mAh drained between the two tests is the true, exact cost of running the software. When optimizing polling loops (e.g., changing `sleep 5` to `sleep 30`), this is the only reliable way to measure the actual battery life returned to the user.

---

## 23. Architectural Limitations: Host Lockdown Mode

A common privacy feature request is a "Host Lockdown Mode" (Mode 3): A mode where the Docker exit node remains perfectly functional for remote devices, but the host machine itself is completely blocked from accessing the internet (to prevent even encrypted host traffic from leaking metadata).

**Why this is impossible on macOS:**
On Linux (e.g., a Raspberry Pi), Docker runs natively. The host's traffic traverses the `OUTPUT` iptables chain, while Docker traffic traverses the `FORWARD` chain. We could easily drop all `OUTPUT` traffic to kill the host's internet, and Docker would continue routing traffic perfectly.

However, Docker on macOS runs inside a Linux Virtual Machine (via `qemu` or Apple `Virtualization.framework`). This VM runs as a standard macOS user-space application. If you configure the macOS firewall (`pf`) to drop all host outbound traffic, it will indiscriminately drop the VM application's traffic as well, instantly killing the Cloudflare WARP tunnel and the exit node.

Writing complex macOS `pf` rules to "allow the VM app, allow Cloudflare WARP IPs, allow Tailscale DERP IPs, but block everything else" is highly fragile and heavily discouraged. Since the current `toggle.sh` default mode already fully encrypts the host's traffic via the WARP tunnel, the host is already protected from ISP leaks.

### 23.1 July 4, 2026: Multi-stage Docker Builds for Go Ports & Teardown Hang Fix
**Symptom:** The Python implementations of `logger.py` and `sync-rules.py` were slow and required a heavy python runtime inside Alpine containers. Also, `routing-fix` container teardown was hanging for 30s during `toggle.sh` shutdown.
**Root Cause:**
- Python is slower than Go and installing it dynamically via `apk add` added overhead and complexity to the containers.
- Docker containers ignore `SIGTERM` if the init process doesn't handle it, causing a full 30s timeout on shutdown.
**Resolution:**
- Ported `logger` and `sync-rules` to Go using Goroutines for massive concurrent speedups.
- Implemented **Multi-stage Docker builds** (using `golang:1.22-alpine` as builder) to compile the static binaries inside Docker. The final containers are raw Alpine with zero dependencies.
- Added `--build` to `docker compose up -d` in `toggle.sh` to ensure updates are consistently applied on boot.
- Added `stop_grace_period: 1s` to `routing-fix` in `docker-compose.yml` which eliminates the 30-second teardown hang instantly.
~~~

```
- Implemented a cryptographic integrity checker (`scripts/crypto.sh`) using HMAC-SHA256 and `NULLEXIT_SEED` in `.env` to prevent tampering of bash scripts.

### 23.2 July 4, 2026: Consolidation of Core Bash Scripts (Refactoring)
**Symptom:** Over 200 lines of bash logic were directly duplicated across `toggle.sh` and `recover.sh` (e.g. `restart_tailscaled_daemon`, `stop_sleep_prevention`, WARP bypass routes). This caused drift when one script was updated without the other.
**Root Cause:**
- Historical separation of responsibilities allowed `recover.sh` to grow complex alongside `toggle.sh`.
**Resolution:**
- Massively refactored both scripts by extracting all duplicated networking logic, PID lifecycle management, and environmental configuration down to `scripts/common.sh`.
- Specifically, the `stop_sleep_prevention`, `stop_dns_watcher`, `stop_host_leak_probe`, and `stop_warp_watcher` functions were collapsed into a single `stop_pidfile_daemon()` abstraction.
- Proxy disable loops were abstracted to `disable_all_proxies()`.
- The `162.159.192.1/.193.1` WARP edge IPs are now dynamically resolved from `.env` instead of hardcoded.

---

### 23.3 July 4, 2026: The "Network Unreachable" Host Leak & Post-Wake Container Destructions
**Symptom:**
1. Running `docker compose config` with dummy keys corrupted the user's `.env` file by appending invalid WireGuard keys. This caused the `warp` container to become unhealthy on boot, which triggered a full teardown of the gateway by `toggle.sh`.
2. After fixing `.env`, `toggle.sh` successfully booted the gateway, but the host's internet started bypassing the tunnel entirely (a massive host leak). The Cloudflare trace returned `warp=off` and exposed the host's physical ISP IP.
3. Running `recover.sh --post-wake` would violently force-recreate all gateway containers and drop the host's internet connection.

**Root Cause:**
- **Bug 1 (The Host Leak):** In `toggle.sh`, the logic to find the Tailscale `utun` interface used `ifconfig | grep -B4 "inet 100."`. Due to the 4 lines of before-context, this regex was accidentally capturing the interface *above* the actual Tailscale interface (e.g. grabbing `utun4` instead of `utun5`). Because `utun4` had no IP address, the subsequent `sudo route add -net 0.0.0.0/1 -interface utun4` silently failed with "Network is unreachable", leaving the host's default route exposed.
- **Bug 2 (The Post-Wake Destructions):** The health check in `recover.sh --post-wake` relied on `docker compose exec -T warp curl -sf https://www.cloudflare.com/cdn-cgi/trace`. However, the newer `qmcgaw/gluetun` Alpine-based Docker image no longer ships with `curl` pre-installed. The health check failed with `executable file not found in $PATH`, tricking `recover.sh` into thinking the tunnel was completely dead and needed to be forcefully recreated via `docker compose up -d --force-recreate`.
- **Bug 3 (Restart Flags):** Previous refactors incorrectly permitted `toggle.sh restart` instead of strictly enforcing `toggle.sh --restart`.

**Resolution:**
- Cleaned the `.env` file of duplicate dummy entries.
- Replaced the brittle `grep -B4` interface parsing in `toggle.sh` with a strict AWK parser: `awk '/^[a-z0-9]+:/{iface=$1} /inet 100\./{print iface; exit}' | tr -d ':'`. This mathematically isolates the exact interface possessing the Tailscale `100.x.x.x` IP address, preventing the `0.0.0.0/1` route from silently failing.
- Changed the health check in `recover.sh` to use `wget -qO- --timeout=3` which is natively installed in the Gluetun image. This stopped the unnecessary destructive recreations of the containers during post-wake events.
- Reverted the `restart` alias in `toggle.sh` to strictly require `--restart`.

### 23.4 July 4-5, 2026: Tailscale Data-Plane Loop (The DERP Relay Deadlock)
**Symptom:**
After bypassing the Tailscale control plane (`192.200.0.0/16`) to fix the "offline" mesh status, the Mac appeared online. However, external peer devices (like the user's phone on cellular) could not successfully ping or SSH into the Mac. `tailscale netcheck` reported `Nearest DERP: unknown (no response to latency probes)`.

**Root Cause:**
While bypassing the control plane kept the daemon connected to the coordination servers, Tailscale's actual peer-to-peer data plane relies on STUN (for NAT traversal) and DERP relays. These relays span ~80 dynamically allocated IP addresses across multiple cloud providers. Because we were forcing `0.0.0.0/1` through the `utun*` exit node, the Mac's own outbound connection attempts to DERP relays were being routed back into the tunnel. To prevent an infinite loop, Tailscale's daemon dropped its own packets when it saw them returning on the TUN interface. This effectively left the daemon deaf and blind to peer connections.

**Resolution:**
- Modified `add_warp_bypass_routes` in `scripts/common.sh` to execute a live API fetch (`curl -s https://login.tailscale.com/derpmap/default`) during startup.
- The script parses out all active DERP relay IPv4 addresses (~80 IPs) and adds a physical host bypass route for every single one of them.
- These IPs are written to a temporary file (`/tmp/nullexit-derp-ips.txt`) so they can be cleanly un-routed by `remove_warp_bypass_routes` when the gateway shuts down. Peer connectivity (ping, SSH, SFTP) was fully restored.

### 23.5 July 5, 2026: Hardcoded WARP Endpoints in docker-compose
```

```
**Symptom:**
The bash scripts allowed overriding the default Cloudflare WARP IP endpoints via `.env` variables (`WARP_ENDPOINT_1` and `WARP_ENDPOINT_2`) to establish bypass routes. However, `docker-compose.yml` statically hardcoded `162.159.192.1` for the `warp` container's `WIREGUARD_ENDPOINT_IP`.

**Root Cause & Risk:**
If a user set a custom endpoint in `.env`, the script would successfully bypass the custom IP on the host level. However, Gluetun would still attempt to connect to the hardcoded `162.159.192.1`. Since `162.159.192.1` was no longer explicitly bypassed, its packets would be sucked into the `utun*` exit node, causing an infinite WireGuard routing loop that instantly breaks the gateway.

**Resolution:**
Modified `docker-compose.yml` to dynamically read the environment variable via `${WARP_ENDPOINT_1:-162.159.192.1}`, ensuring both the host routing scripts and the container use the exact same endpoint.

### 23.6 July 6, 2026: Packet Filter (pfctl) Defaulting Local LAN Rules to TCP-Only
**Symptom:**
Devices on the exact same local Wi-Fi network as the gateway were experiencing ~500ms latency to the gateway instead of the expected ~80ms.

**Root Cause:**
When writing the `pf.conf` rules to whitelist local LAN traffic (`pass out quick on $ext_if to { 192.168.0.0/16, ... }`), the rule omitted an explicit protocol. The macOS `pfctl` compiler implicitly applies state tracking to all `pass` rules. However, because state tracking (`keep state`) natively evaluates TCP sessions, `pfctl` automatically appended the `flags S/SA` modifier to the compiled rule in the kernel. This silently restricted the whitelist to **TCP only**, causing all local UDP traffic to be dropped by the default-deny kill-switch. Since Tailscale relies on UDP port 41641 for direct P2P connections, the local devices were unable to hole-punch and were forced to fall back to routing traffic through a remote Tailscale DERP relay, massively inflating latency.

**Resolution:**
Split the single implicit rule into explicit `proto tcp`, `proto udp`, and `proto icmp` rules in `scripts/pf.conf` to guarantee the macOS compiler allows all three core transport protocols on the local subnet without mutating the rule into a TCP-only lock.
### 23.7 July 8, 2026: Network Watcher Event Mismatch, Sudo-in-Background Crash, and Loop-Prevention
**Symptom:**
1. Switching Wi-Fi or waking the Mac from sleep did not trigger a post-wake recovery (`recover.sh --post-wake`) automatically.
2. The network connection would eventually get completely sinkholed/blocked. If the user tried to restart or wait, it did not self-heal.
3. During the recovery attempt, checking the public IP on the host would return the unencrypted campus/ISP IP (starting with `132.`) instead of the encrypted tunnel IP.

**Root Cause:**
- **Bug 1 (Network Watcher Mismatch):** In `scripts/watcher.sh`, the network change listener watched `State:/Network/Global/IPv4` changes via `scutil n.watch` but matched them against `*n.state*|*SCEventUpdate*`. On macOS, the actual output is `changedKey [0] = State:/Network/Global/IPv4`. Because the pattern did not match, all network switches were silently ignored.
- **Bug 2 (Sudo Caching Background Crash):** When the background WARP Watcher eventually detected the broken tunnel (after 6 failures/30s), it triggered the default nuclear `recover.sh` (with `POST_WAKE=false`). Because there was no active TTY in the background context, the `sudo -v` caching check aborted instantly with a terminal required error. Due to `set -e`, `recover.sh` died immediately before resetting DNS, disabling the packet-filter killswitch, or stopping containers, leaving the host network completely sinkholed.
- **Bug 3 (Post-Wake Infinite Loop):** Once the watcher patterns were fixed, `recover.sh --post-wake` triggered successfully on network changes. However, because the script deleted the default routing entries at the start, spent ~15 seconds rebuilding 88 DERP relay bypass routes, and re-added the default routes at the end, this execution time exceeded the watcher's 10-second debounce threshold. Furthermore, the watcher recorded the *start* timestamp in the debounce file. Consequently, the route changes made by the script itself triggered the watcher again immediately after completion, trapping the system in an infinite loop of recovery runs. During this loop, the VPN routes were constantly deleted, resulting in the host leaking traffic through the real ISP interface (an IP starting with `132.`) for 15s out of every cycle.

**Resolution:**
- **Fixed Watcher Matching:** Updated `scripts/watcher.sh` to match `*changedKey*` and `*State:/Network/Global/IPv*` to correctly catch network changes.
- **Bypassed Background Sudo:** Added `--auto`/`--non-interactive` flags to `recover.sh` and added a TTY check `[ -t 0 ]` to skip interactive `sudo -v` authentication when running headlessly in the background. Updated the WARP Watcher call in `toggle.sh` to pass `--auto`.
- **Prevented Infinite Loops:** Modified `scripts/watcher.sh` to write the **completion** timestamp of `recover.sh` to `DEBOUNCE_FILE` (instead of only the start timestamp). This ensures all buffered network config events generated during route reconstruction are evaluated against the end-of-run timestamp and successfully debounced, breaking the infinite loop.
- **Centralized & Timestamped Logs:** Redirected `watcher.sh` stdout/stderr from `/tmp/nullexit-watcher.log` to the repository's main `output.log` and updated `scripts/common.sh`'s `step`, `ok`, `warn`, `fail`, and `die` helpers to prefix logs with a local `[YYYY-MM-DD HH:MM:SS]` timestamp.

### 23.8 July 11, 2026: Tor Container Hardening & obfs4proxy Restoration
**Symptom:**
```

```
1. The Tor container entered a fatal crash loop upon boot, throwing ‘exec: "obfs4proxy": executable file not found in $PATH‘ when ‘TOR_USE_BRIDGES=true‘.
2. When the user pasted Tor bridge lines containing comments (‘#‘) or blank lines into ‘tor-bridges.txt‘, the container failed to validate the file properly and crashed.

**Root Cause:**
- **Bug 1 (Missing Pluggable Transports):** The Tor Dockerfile was built on ‘debian:bullseye-slim‘, which had silently dropped or failed to resolve the ‘obfs4proxy‘ package in its latest apt repositories.
- **Bug 2 (Fragile Validation):** The ‘entrypoint.sh‘ bridge validation check merely checked ‘[ -s tor-bridges.txt ]‘ (file size > 0). It did not verify if the file actually contained valid, uncommented bridge lines. Passing empty lines or comments tricked the validation script into appending invalid ‘Bridge‘ directives to the ‘torrc‘, causing the Tor daemon to instantly exit.

**Resolution:**
- **Upgraded Base Image:** Shifted the Tor Dockerfile base image to ‘debian:bookworm-slim‘, restoring access to the ‘obfs4proxy‘ pluggable transport package.
- **Robust Bridge Validation:** Replaced the fragile ‘[ -s ]‘ check with a strict ‘grep -vE ’^\s*(#|$)’‘ command that strips comments and empty lines. If no valid lines remain, the container safely falls back to standard public guard relays instead of crashing, ensuring Tor remains available even with a misconfigured bridge file.
- **Image Pinning:** Explicitly pinned ‘image: nullexit-tor:v1.0.0‘ in ‘docker-compose.yml‘ to prevent arbitrary local builds from drifting to ‘:latest‘.

### 23.9 July 12, 2026: Tor ControlPort Dynamic Password Authentication & User Config
**Symptom:**
Querying the Tor Control Port (e.g., via the ‘/sweep‘ script or ‘check_circuits.py‘) returned empty responses or connection errors. Logs showed that Tor automatically closed its ControlPort:
‘You have a ControlPort set to accept unauthenticated connections from a non-local address. ... That’s so bad that I’m closing your ControlPort for you.‘

**Root Cause:**
Docker port mapping (especially under Colima on macOS) requires container sockets to bind to ‘0.0.0.0‘ to be reachable from the host. However, when Tor’s ControlPort is bound to ‘0.0.0.0‘ (non-local) with no authentication, Tor closes it by default as a safety precaution. Sourcing SOCKS5/ControlPort to ‘127.0.0.1‘ inside the container was not an option as it broke the Docker bridge routing path.

**Resolution:**
- **Dynamic Password Authentication**: Updated the Tor container’s ‘entrypoint.sh‘ to read ‘TOR_PASSWORD‘ and dynamically generate its HMAC hash via ‘tor --hash-password‘, adding ‘HashedControlPassword <hash>‘ to ‘torrc‘. This secures the ControlPort on ‘0.0.0.0‘ so Tor keeps it open.
- **Unified Credentials in ‘.env‘**: Exposed ‘ADGUARD_USER=admin‘ and ‘ADGUARD_PASSWORD=nullexit‘ in ‘.env‘ to serve as the unified gateway credentials. Sourced ‘TOR_PASSWORD‘ directly from ‘ADGUARD_PASSWORD‘ in ‘docker-compose.yml‘.
- **Client Authentication**: Updated ‘scripts/check_circuits.py‘ to fetch ‘TOR_PASSWORD‘ (defaulting to ‘ADGUARD_PASSWORD‘ or ‘nullexit‘) and authenticate with the ControlPort using ‘AUTHENTICATE "<password>"‘ to regain access.

---

## 24. Censorship-Resistant Transport & Egress Compartmentalization

### 24.1 Why this is needed
WARP can be blocked from deployment countries with strict internet controls. The most likely cause isn’t that "encrypted traffic is blocked" in general  it’s that WireGuard has a recognizable handshake signature, and ‘WIREGUARD_ENDPOINT_IP=162.159.192.1‘ on ‘WIREGUARD_ENDPOINT_PORT=2408‘ is a fixed, easily-enumerable target (Cloudflare’s known WARP range). That combination is exactly what IP/ASN-based and DPI-based blocking is good at catching.

### 24.2 Important correction: Gluetun’s ‘SHADOWSOCKS=on‘ is the wrong direction
Gluetun’s built-in Shadowsocks option runs a Shadowsocks **server** inside the already-connected VPN tunnel, so other LAN devices can reach out through Gluetun’s *existing* connection via the SS protocol. It is not a way to make Gluetun itself connect **outbound** through a Shadowsocks server Gluetun only speaks OpenVPN or WireGuard as its actual transport. There is no ‘VPN_TYPE=shadowsocks‘. Confirmed against Gluetun’s own maintainer/community discussion: people have asked for a "Shadowsocks-client" mode specifically to chain through Gluetun, and the standing answer is "run a separate Shadowsocks client container."

### 24.3 Diagnose before building anything
The right fix depends entirely on what’s actually being blocked:
- Point Gluetun at a **different provider/IP/ASN** (any other Gluetun-supported WireGuard/OpenVPN provider, or a self-hosted WireGuard server) and test. If that gets through, the block is Cloudflare/WARP-specific, not a general WireGuard block  **no new component needed**, just swap ‘VPN_SERVICE_PROVIDER‘ / the endpoint.
- If WireGuard fails regardless of endpoint, the block is protocol-level (DPI fingerprinting the handshake)  proceed to obfuscation.

### 24.4 Path A: Swap WARP for a different Gluetun-native transport
Simplest fix, only applies if 24.3 shows the block is Cloudflare-specific. Change ‘VPN_SERVICE_PROVIDER‘ / ‘VPN_TYPE‘ / endpoint in ‘docker-compose.yml‘ and in the now-centralized ‘WARP_ENDPOINT_*‘ values in ‘.env‘. Nothing else in the stack needs to change.

```

```
### 24.5 Path B: Add an obfuscation hop
Needed if WireGuard itself is being fingerprinted/blocked. Two ways to slot it in:

**B1  Wrap (recommended):** Run a Shadowsocks (or obfs4/v2ray) client as a new sidecar container. Point Gluetun’s ‘WIREGUARD_ENDPOINT_IP‘ at ‘127.0.0.1:<local-port>‘ instead of Cloudflare directly; the sidecar relays that traffic to a Shadowsocks server you run yourself abroad. Keeps Gluetun’s kill-switch, healthchecks, and the entire rest of the stack (Tailscale, AdGuard, ipset, ‘routing-fix.sh‘) untouched, since none of them know the transport changed underneath ‘warp‘.

**B2  Replace:** Swap the ‘warp‘ service entirely for a Shadowsocks-client + ‘tun2socks‘ container that owns the network namespace instead of Gluetun. More invasive  loses Gluetun’s built-in kill-switch/healthcheck, which would need to be rebuilt by hand (see Section 24.9 for the pluggable architecture to support this natively).

**Recommendation: B1.** Smaller surface area, preserves the self-healing architecture already built and documented.

### 24.6 Fingerprinting caveat
Plain Shadowsocks can still be caught by active-probing DPI in more aggressive censorship environments. If plain SS gets blocked too, the next step up is an obfuscation plugin (‘v2ray-plugin‘, ‘Cloak‘) or a newer protocol designed against active probing (VLESS+Reality, Hysteria2). Test plain SS first don’t over-build before confirming it’s needed.

### 24.7 AmneziaWG (The High-Speed Alternative)
If you want to maintain the bare-metal speeds of WireGuard while bypassing DPI (e.g. in Egypt or Russia), the best alternative to Shadowsocks/V2Ray is AmneziaWG. AmneziaWG is a fork of WireGuard that pads the handshake packets with randomized "junk" data, entirely destroying the 148-byte DPI signature that firewalls look for.
- **Pros:** Because it runs entirely as UDP without TCP-over-TCP overhead, it completely avoids "TCP meltdown" latency spikes associated with Shadowsocks/V2Ray wrappers.
- **Cons:** You cannot use Cloudflare WARP. Furthermore, it requires completely ripping Gluetun out of the ‘nullexit‘ stack and building a custom Docker container, meaning you lose Gluetun’s built-in kill-switches and health-check logic.

### 24.8 Infrastructure Costs & Privacy
To use either Shadowsocks (Path B) or AmneziaWG, you cannot use the free Cloudflare WARP infrastructure, because Cloudflare servers only speak standard WireGuard. The software for both custom protocols is 100% free and open-source, but you must host the remote server infrastructure yourself.
- **Free Tier (Oracle Cloud):** You can host your remote server on Oracle Cloud’s "Always Free" tier for $0. However, this routes your highly sensitive, censorship-evading traffic directly through Oraclea massive corporate data broker. If privacy is the core objective, handing all your metadata to Oracle defeats the purpose.
- **Paid Tier (Hetzner / Mullvad):** The recommended path is to rent a standard ˜$4/month Virtual Private Server (VPS) in a free country (e.g., Hetzner in Germany, subject to strict EU GDPR privacy laws) and self-host the server. Alternatively, Mullvad VPN (5/month) provides native v2ray/Shadowsocks bridges built into their network, allowing you to bypass censorship with a strict zero-logs policy. It is highly recommended to pay with untrackable payment methods to maintain complete anonymity, which these providers typically support without requiring local residency.

### 24.9 The Future: Egress Compartmentalization (Pluggable Architecture)
To natively support invasive replacement paths like **Path B2** or **AmneziaWG** (Section 24.7) without breaking the core ‘nullexit‘ routing and monitoring logic, the architecture must be refactored into a modular, "pluggable" design.

1. **The Egress Plugin System:** Currently, WARP-specific logic (like ‘cdn-cgi/trace‘ health checks and hardcoded interface names) is scattered across ‘toggle.sh‘, ‘scripts/routing-fix.sh‘, and ‘scripts/host-leak-probe.sh‘. This should be abstracted into a ‘scripts/plugins/‘ directory, where each egress method (‘warp.sh‘, ‘v2ray.sh‘) implements standardized functions: ‘get_egress_interface()‘, ‘check_health()‘, and ‘get_bypass_ips()‘.
2. **Dynamic Docker Compose:** Based on an ‘EGRESS_TYPE‘ variable in ‘.env‘, dynamic compose file generation would spin up only the required egress containers (e.g., skipping the ‘warp‘ container when ‘EGRESS_TYPE=v2ray‘ is set).
3. **Agnostic Routing & Watchers:** ‘routing-fix.sh‘ would dynamically read the ‘EGRESS_INTERFACE‘ from the active plugin to apply ‘iptables‘ rules, and the background watchers in ‘toggle.sh‘ would become an agnostic ‘Egress Watcher‘ that invokes ‘check_health()‘ periodically.

This compartmentalization transforms ‘nullexit‘ into a universal, censorship-resistant gateway where the entire egress layer can be swapped by changing a single ‘.env‘ variable and running ‘./toggle.sh --restart‘.

---

## 25. Incident Post-Mortem: WARP Watcher Race vs Post-Wake (July 10, 2026)

### Symptom
After switching Wi-Fi networks, the host IP appeared as the raw ISP IP (‘132.x.x.x‘) instead of a Cloudflare/WARP IP. The gateway appeared to complete post-wake recovery successfully in the logs, but nuclear shutdown fired immediately after and killed everything.

### Root Cause
A fatal race condition between two concurrent processes:
```

````

1. **Wi-Fi roam**  ‘watcher.sh‘ fires ‘recover.sh --post-wake‘
2. **Post-wake** finds the ‘warp‘ container missing/dead  calls ‘docker compose up -d --force-recreate‘ (~35 seconds)
3. **In parallel**, the WARP Watcher (running as a child of ‘toggle.sh‘) polls the warp container every 5s during failures
4. While the container is being recreated it returns ‘warp=off‘ for those 35 seconds  7 consecutive failures  **WARP SHUTDOWN fires**, calling nuclear ‘recover.sh‘
5. Nuclear ‘recover.sh‘ tears down Tailscale, resets DNS to 1.1.1.1, stops all containers  **gateway dead, IP exposed**
6. The post-wake ‘docker compose up‘ finishes at ~35s mark, container healthy  but the gateway is already gone

The evidence in ‘output.log‘:
‘‘‘
[2026-07-10T06:07:27Z] NET: ...  bash recover.sh --post-wake
[2026-07-10T06:07:47Z] WARP DOWN  cdn-cgi/trace reports warp=off    watcher starts counting
...
add net 0.0.0.0: gateway utun5    post-wake successfully re-routes at 02:08:22
...
[2026-07-10T06:09:04Z] WARP SHUTDOWN  6 consecutive failures         watcher fires nuclear
‘‘‘

### Fix (July 10, 2026)
Added a **WARP Watcher inhibit marker** (‘/tmp/nullexit-warp-inhibit.marker‘):

- ‘recover.sh --post-wake‘ writes the marker at startup **before** any container operations
- The WARP Watcher loop checks for the marker at the top of every iteration; if present it resets ‘consec_off=0‘ and sleeps 5s (skips the poll entirely)
- ‘recover.sh‘ removes the marker at the very end (on both success and failure via ‘trap cleanup_inhibit EXIT‘)
- This guarantees the watcher never counts failures during the intentional container-down window of a post-wake force-recreate

The marker file path is hardcoded to ‘/tmp/nullexit-warp-inhibit.marker‘ in both ‘toggle.sh‘ and ‘recover.sh‘.

## 26. Incident Post-Mortem: Startup Pre-Flight Check Tailscale Race (July 10, 2026)

### Symptom
Immediately after startup via ‘toggle.sh‘, the pre-flight connectivity check ‘[1/3] Gateway reachable via Tailscale‘ returned ‘FAIL‘, forcing ‘toggle.sh‘ to skip the full exit-node activation and fall back to SOCKS5/local DNS proxy mode.

### Root Cause
A timing race condition during startup. In ‘toggle.sh‘, the host joins the mesh using ‘tailscale up --reset‘ (Phase A) right before performing the pre-flight check. Because ‘tailscale up‘ resets and reconfigures the host’s ‘tailscaled‘ daemon, the local daemon must fetch the latest netmap asynchronously from the coordination servers.
This network map propagation and route establishment takes a few seconds. Running ‘tailscale ping‘ immediately after ‘tailscale up‘ returns (with no delay) causes the check to fail because the local daemon has not yet discovered the gateway node ‘100.100.21.8‘.

### Fix (July 10, 2026)
Upgraded pre-flight Check 1 in both ‘toggle.sh‘ and ‘scripts/toggle-linux.sh‘ to use a 5-attempt retry loop with a 1-second delay between checks. This allows up to 5 seconds for local tailnet routing paths to settle, ensuring the gateway is correctly reached and a pong is received before deciding whether to enable exit-node routing.

## 27. Incident Post-Mortem: Concurrency Collision between Nuclear Teardown and Post-Wake (July 10, 2026)

### Symptom
When the background WARP Watcher triggers a nuclear shutdown (due to a real or simulated tunnel failure), the gateway is completely torn down. However, the network configuration changes made during the teardown trigger a ‘State:/Network/Global/IPv4‘ network change event. The LaunchAgent-based network watcher (‘watcher.sh‘) intercepts this event and concurrently spawns ‘recover.sh --post-wake‘ to heal/restore the gateway. This causes ‘docker compose down‘ (from nuclear teardown) and ‘docker compose up‘ (from post-wake) to run in parallel, resulting in container errors like ‘dependency failed to start: container warp exited (0)‘ and the gateway becoming wedged.

### Root Cause
A lack of coordination/locking between the lifecycle control scripts. While ‘toggle.sh‘ used ‘/tmp/nullexit-toggle.lock‘ to prevent concurrent ‘toggle.sh‘ runs, ‘recover.sh‘ (both nuclear and ‘--post-wake‘ modes) did not utilize or check any lock files.

### Fix (July 10, 2026)
Implemented a shared lifecycle lock file (‘/tmp/nullexit-toggle.lock‘) across both scripts:
- **‘recover.sh‘ (all modes)**: Checks ‘/tmp/nullexit-toggle.lock‘ at startup. If the lock is held by another running lifecycle script (‘toggle.sh‘ or ‘recover.sh‘):
  - In ‘--post-wake‘ mode, it logs a skip message to ‘output.log‘ and exits cleanly (‘exit 0‘). This safely prevents the auto-recovery agent from recreating containers during a teardown.
````

```
  - In nuclear recovery mode, it exits with an error.
- **‘recover.sh‘ Lock Cleanup**: Cleans up the lock file upon completion using ‘trap cleanup_recover EXIT‘ (which checks if the lock file belongs to the current PID).
- **‘toggle.sh‘ update**: The lock check in ‘toggle.sh‘ was updated to look for both ‘toggle.sh‘ and ‘recover.sh‘ in the running processes to enforce proper exclusion.

## 28. Incident Post-Mortem: Infinite Auto-Recovery Loop after WARP Watcher Nuclear Shutdown (July 10, 2026)

### Symptom
When the background WARP Watcher triggered a nuclear shutdown (due to a tunnel outage), it successfully executed ‘recover.sh‘ (nuclear). However, immediately after the teardown completed, the system started spawning ‘recover.sh --post-wake‘ and rebuilding the containers again. This created an infinite loop of tearing down and auto-restarting the gateway, keeping the host's internet route permanently wedged.

### Root Cause
An active-state marker file leak. The LaunchAgent-based network watcher (‘watcher.sh‘) only fires ‘recover.sh --post-wake‘ when ‘/tmp/nullexit-gateway-active.marker‘ is present on disk. While ‘toggle.sh‘ (STOP path) correctly deleted this marker, the nuclear ‘recover.sh‘ script did not. When the WARP Watcher ran ‘recover.sh‘ (nuclear), the containers were stopped but the active-state marker was left on disk. The network changes from the teardown then triggered ‘watcher.sh‘, which saw the marker, believed the gateway was still supposed to be active, and triggered ‘--post-wake‘ to start it back up.

### Fix (July 10, 2026)
Modified the start of ‘recover.sh‘ to explicitly delete ‘/tmp/nullexit-gateway-active.marker‘ if ‘POST_WAKE‘ is ‘false‘ (nuclear mode). This ensures that any subsequent network configuration changes made during the teardown are correctly ignored by ‘watcher.sh‘ because the marker file is gone.

## 29. Incident Post-Mortem: Pre-Flight Check False Negatives due to VPN Firewall STUN Blocks (July 10, 2026)

### Symptom
Immediately after a cold boot or full restart of the gateway, the pre-flight connectivity check ‘[1/3] Gateway reachable via Tailscale‘ failed, causing ‘toggle.sh‘ to skip the full exit-node configuration and fall back to SOCKS5/local DNS proxy mode. However, manually running ‘tailscale ping 100.100.21.8‘ immediately after startup succeeded in 1ms.

### Root Cause
An asynchronous Tailscale handshake delay caused by firewall restrictions. Inside the container network namespace, the ‘warp‘ container (gluetun) implements a strict firewall that blocks all outbound UDP traffic to the public internet except to the designated WireGuard endpoint. Because of this, the ‘tailscale‘ container's STUN requests to public STUN servers are blocked, causing the local daemon to report ‘UDP is blocked, trying HTTPS‘.
Without UDP STUN capability, Tailscale is forced to fall back to a slower DERP-assisted handshake (relayed via HTTPS/TCP). This negotiation and direct-path punch-through takes roughly 30 to 35 seconds to establish on cold boots (after a full ‘tailscale up --reset‘). Since the pre-flight check only waited 15 seconds (15 attempts with 1-second sleeps), the check timed out and aborted before the path completed.

### Fix (July 10, 2026)
Increased the pre-flight ping retry limit from 15 to 45 attempts (45 seconds) in both ‘toggle.sh‘ and ‘scripts/toggle-linux.sh‘. This provides sufficient time for the DERP-assisted handshake to complete on cold boots, while still passing immediately (in 1-2 seconds) on warm starts or once the path is established.

## 30. Incident Post-Mortem: Docker Image Build DNS Failures on Immediate Restart (July 10, 2026)

### Symptom
When executing ‘toggle.sh --restart‘, the script successfully stops the gateway but then immediately fails during startup during ‘docker compose build‘ with DNS resolution errors:
‘failed to do request: Head "https://registry-1.docker.io/...": dial tcp: lookup registry-1.docker.io on 192.168.5.1:53: no such host‘.

### Root Cause
A race condition between physical link negotiation and virtual machine startup. The STOP path of the restart command invokes ‘cleanup_network_state‘ which power-cycles the host's physical Wi-Fi interface (‘en0‘) to flush stale routing states. Immediately after, ‘toggle.sh‘ starts the gateway boot sequence. Because ‘toggle.sh‘ did not verify the status of the physical interface, it proceeded to boot Colima and build Docker containers while ‘en0‘ was still negotiating a DHCP lease. Consequently, the host (and by extension, the VM bridge) had no active internet gateway/DNS path, causing Docker's registry check to fail.

### Fix (July 10, 2026)
1. Created a unified ‘wait_for_dhcp_settle‘ helper function in ‘scripts/common.sh‘ that polls ‘ipconfig getpacket‘ and ‘ifconfig‘ until a valid IP and router are assigned. To handle typical macOS Wi-Fi reassociation and DHCP negotiation latency (which commonly takes 15-20 seconds after a power-cycle), this loop was configured to poll up to 60 times (30 seconds total).
2. Injected a call to ‘wait_for_dhcp_settle‘ at the beginning of the ‘START‘ action in ‘toggle.sh‘ (right before checking Colima status) to block container builds until the host's physical network is
```

```
online.
3. Refactored ‘recover.sh‘ to reuse the unified ‘wait_for_dhcp_settle‘ function.

## 31. Incident Post-Mortem: PF Kill-Switch Bricks Host Internet in SOCKS5 Fallback Mode (July 10, 2026)

### Symptom
When the pre-flight checks fail (e.g. because host Tailscale was unauthenticated/logged out), the script falls back to SOCKS5 proxy mode but the host immediately loses all internet connectivity and tailscaled reports ‘You are logged out... connect: no route to host‘. Even running ‘sudo tailscale up‘ to log in fails because the connection to the control plane times out.

### Root Cause
An architectural mismatch. The macOS Packet Filter (PF) Kill-Switch works at the IP level: it blocks all outbound traffic on the physical interface (‘en0‘) except to explicitly whitelisted VPN endpoints, expecting all other traffic to go through the virtual VPN interface (‘utun*‘).
In SOCKS5 fallback mode, the exit-node (‘utun*‘) is NOT enabled; instead, only specific applications (like browsers) use the localhost SOCKS5 proxy, while all other host traffic (including the ‘tailscaled‘ daemon's UDP packets) continues to go through ‘en0‘. Because the script still enabled the PF Kill-Switch during SOCKS5 fallback, it blocked all non-SOCKS5 traffic on ‘en0‘, completely bricking the host's internet and locking the Tailscale daemon out of the control plane.

### Fix (July 10, 2026)
Modified ‘toggle.sh‘ to remove the ‘enable_killswitch‘ call from the SOCKS5 fallback path. The firewall kill-switch is now strictly reserved for exit-node VPN mode, ensuring SOCKS5 fallback leaves the host's direct internet route open for system daemons to authenticate.

## 32. Incident Post-Mortem: Host Tailscale Logouts due to Aggressive reset Flags (July 10, 2026)

### Symptom
Every time the user runs ‘toggle.sh‘ or ‘recover.sh‘ which disconnects/reconnects the host client, the host client randomly gets logged out, forcing them to re-run ‘sudo tailscale up‘ to authenticate.

### Root Cause
An overly aggressive configuration reset. The host-side ‘tailscale up‘ calls (used in ‘disconnect_tailscale_host‘ and the startup/enable exit node paths) all passed the ‘--reset‘ flag. On macOS, ‘--reset‘ resets the entire configuration state and authentication token cache of the daemon. Since the script does not pass a ‘TS_AUTHKEY‘ to the host's commands (which are meant for the user's private interactive client), the ‘--reset‘ flag effectively logged the user out of their tailnet, requiring interactive re-login.

### Fix (July 10, 2026)
Removed the ‘--reset‘ flag from all host-side ‘tailscale up‘ invocations in ‘toggle.sh‘ and ‘scripts/common.sh‘. This ensures that exit-node state transitions (enabling/disabling) are handled safely while preserving the host client's authenticated session.

## 33. Incident Post-Mortem: Discarded Docker Exec Errors in logs (July 10, 2026)

### Symptom
When the ‘warp‘ container is stopped, dead, or failing to connect to its endpoints, the ‘toggle.sh‘ and ‘recover.sh‘ connectivity health checks fail, but no details are printed to ‘output.log‘. The logs only show generic ‘FAIL‘ outputs, making it hard to see if the failure was a network timeout, Docker daemon error, or a missing binary.

### Root Cause
Overly broad error silencing. The ‘docker compose exec‘ commands in the health checks (Check 3 in ‘toggle.sh‘ and the traffic check in ‘recover.sh‘) both redirected stderr to ‘/dev/null‘ (‘&>/dev/null‘ and ‘2>/dev/null‘ respectively), completely throwing away any diagnostic error messages produced by Docker or ‘wget‘.

### Fix (July 10, 2026)
Redirected stderr of the ‘docker compose exec‘ check commands to ‘output.log‘ (‘2>> output.log‘). This ensures that any command execution failures or connection timeout details are logged.

## 34. Architectural Design: How nullexit Mimics Commercial VPNs during Roaming

The ‘nullexit‘ roaming/network recovery subsystem replicates the exact engineering mechanics used by commercial, premium VPN clients (like Mullvad or NordVPN) to transition across Wi-Fi networks safely and seamlessly.

### The 4-Step Transition Cycle
When your Mac disconnects from one Wi-Fi and associates with another, ‘nullexit‘ coordinates the following events:

1. **Firewall Lock (Kill-Switch)**: The macOS Packet Filter (PF) anchor ‘com.apple/nullexit‘ remains locked on the physical interface (‘en0‘), blocking all direct outbound IPv4 and IPv6 traffic. This guarantees that your real ISP IP or unencrypted DNS requests **never leak** onto the new network while the connection is rebuilding.
2. **System Event Interception**: A launchd LaunchAgent (‘watcher.sh‘) listens to the macOS System Configuration framework for the ‘State:/Network/Global/IPv4‘ key changes, spawning ‘recover.sh --post-wake‘ in under 500ms when a change is detected.
```

```
3. **Bypass Routing**: The script dynamically resolves the new physical network’s IP gateway (e.g. ‘192.168.137.1‘), cleans up stale bypass routes from the previous network, and adds static bypass routes pointing to Cloudflare’s WARP endpoints and the Tailscale control plane directly via the new gateway. This allows the VPN client to negotiate its handshake outside of the tunnel.
4. **Hole-Punching / NAT Re-negotiation**: The script runs a target check on the ‘warp‘ container’s tunnel. If the WireGuard connection is wedged by the network switch, it force-recreates the container to establish a fresh NAT binding to Cloudflare, restoring connection in ~15 seconds.

## 35. Incident Post-Mortem: macOS Tailscale Flag Requirement Abort (July 10, 2026)

### Symptom
When the ‘toggle.sh‘ stop trap or the roaming watcher triggered, the gateway failed to shut down or disconnect properly. The ‘output.log‘ showed the error: ‘Error: changing settings via ’tailscale up’ requires mentioning all non-default flags. To proceed, either re-run your command with --reset...‘

### Root Cause
In a previous attempt to stop host-side logouts (Section 32), we removed the ‘--reset‘ flag from all ‘tailscale up‘ invocations. However, if the Tailscale daemon on macOS is already running with non-default flags (like ‘--ssh‘ or ‘--accept-routes‘), invoking ‘tailscale up‘ with only a subset of flags (like ‘--exit-node=‘) triggers a strict safety check in the Tailscale CLI that aborts the command completely. Because the command aborted, exit-node routing remained stuck.

### Fix (July 10, 2026)
Migrated all specific preference changes in ‘toggle.sh‘ and ‘scripts/common.sh‘ from ‘tailscale up‘ to a two-step sequence:
1. ‘tailscale up‘ (with no flags)  to safely ensure the interface is brought up using the current authenticated state.
2. ‘tailscale set --exit-node=...‘  to modify the specific target preference cleanly without triggering the missing flags error or requiring ‘--reset‘.

## 36. Incident Post-Mortem: macOS SNAT Endpoint Poisoning & The P2P Blackhole (July 10, 2026)

### Symptom
When a Tailscale client (like an S24) connected to the exact same Wi-Fi network as the macOS gateway, it completely lost internet connectivity. However, if the S24 connected to a Windows PC’s hotspot on the same network, the tunnel worked flawlessly.

### Root Cause
Claude’s critique correctly dismantled the port collision theory: the gateway daemon wasn’t failing to bind, and macOS wasn’t load-balancing ports. The real culprit was **macOS SNAT Source Port Mangling poisoning Tailscale’s path discovery**.

Here is the exact step-by-step of the "infinite loop" blackhole:
1. **The P2P Handshake:** The S24 and the Mac are on the same Wi-Fi (‘192.168.1.x‘). The S24 discovers the Mac’s local IP and sends a UDP hole-punch packet to ‘192.168.1.5:41641‘. Colima successfully forwards this to the gateway container.
2. **The Bypass Rule:** The gateway replies. Because ‘routing-fix.sh‘ forces all Tailscale UDP traffic out ‘eth0‘ to bypass WARP, the reply goes to the macOS host.
3. **macOS SNAT Mangling:** macOS routes the reply out ‘en0‘ to the S24. Because the packet crosses from the Colima VM bridge to the Wi-Fi interface, macOS applies Source NAT (SNAT). Critically, macOS **randomizes the source port** (e.g., changes it from ‘41641‘ to ‘58291‘).
4. **Endpoint Poisoning:** The S24 receives the authenticated Tailscale packet from ‘192.168.1.5:58291‘. Tailscale’s ‘magicsock‘ is designed to handle NAT dynamically, so it assumes the Mac has roamed to port ‘58291‘! The S24 updates its endpoint and starts sending all its encrypted internet traffic to ‘192.168.1.5:58291‘.
5. **The Blackhole:** macOS has no port forward for ‘58291‘. It drops 100% of the S24’s outbound data packets. The connection is completely dead.

### Why did the Hotspot work?
When the S24 connected to the PC hotspot, it was behind the PC’s NAT. The S24 sent the packet, PC NATted it, and the Mac replied (mangled to ‘58291‘). When the reply hit the PC, the PC’s strict NAT dropped it because the source port (‘58291‘) didn’t match the destination port it originally sent to (‘41641‘). Because the mangled packet was dropped, the S24 never poisoned its endpoint! It safely gave up on P2P, fell back to the 100% reliable TCP DERP relay, and the internet worked perfectly.

### Fix & Resolution (July 10, 2026)

**Phase 1  Port disambiguation:** Changed the gateway container’s Tailscale listen port from the default ‘41641‘ to ‘41642‘ across ‘docker-compose.yml‘, ‘post-rules.txt‘, and ‘routing-fix.sh‘. This prevents any bind conflict with the macOS host’s native Tailscale daemon.

**Phase 2  RFC1918 DROP rules:** Updated ‘routing-fix.sh‘’s ‘add_tailscale_p2p_bypass()‘ to explicitly drop all outgoing Tailscale UDP packets destined for private subnets (‘192.168.0.0/16‘, ‘10.0.0.0/8‘, ‘172.16.0.0/12‘) when ‘TAILSCALE_ALLOW_LAN_P2P=false‘. This surgically kills the local P2P hole-punch attempt *before* it can trigger the macOS gvproxy SNAT mangling. The S24 receives a clean failure and immediately locks onto the public DERP relay with 0% packet loss.

**Phase 3  Automatic network detection:** Because ‘TAILSCALE_ALLOW_LAN_P2P=false‘ breaks P2P on trusted hotspots that genuinely don’t have AP isolation, a ‘.env‘ toggle and a full auto-detection system were implemented:

```

```
- **‘.env‘ toggle:** ‘TAILSCALE_ALLOW_LAN_P2P=true/false‘  static fallback, used only if auto-detection cannot run.
- **‘watcher.sh‘ auto-detection (‘detect_lan_p2p_mode‘):** On every network change, ‘watcher.sh‘ runs two checks:
  1. **WPA2-Enterprise detection:** ‘airport -I‘ is used to read the current Wi-Fi ‘link auth‘ field. If the auth type does not contain ‘-psk‘ (e.g., it is plain ‘wpa2‘ = 802.1x), the network is classified as enterprise and P2P is forced off.
  2. **AP Isolation probe:** For WPA2-Personal networks, a short ‘ping‘ is sent to the default gateway. If the gateway responds (it always should on a real home/hotspot network), P2P is allowed. If not, AP isolation is inferred and P2P is forced off.
  - The result (‘true‘ or ‘false‘) is written to ‘.lan_p2p_detected‘ in the repo root.
- **‘routing-fix.sh‘ dynamic reading:** ‘add_tailscale_p2p_bypass()‘ re-reads ‘.lan_p2p_detected‘ on every 30-second loop iteration and atomically adds or removes the RFC1918 DROP rules. **No container restart is needed when switching networks.**

### Known Unknowns (Pending Verification)
- The gvproxy SNAT source port mangling theory is mechanistically sound and backed by observed behavior (gvproxy’s UDP proxy changelog has documented edge cases in this exact code path), but has not yet been confirmed via ‘tcpdump -i en0 udp portrange 40000-60000‘ while triggering a P2P handshake from the phone. A packet capture cross-referenced with ‘tailscale ping‘ endpoint reports on the phone side would confirm the theory definitively.
- Claude’s recommendation: run ‘tcpdump -i en0 udp port 41642 or portrange 40000-60000‘ on the Mac while triggering the handshake from the phone to see whether the reply leaves with a different source port than ‘41642‘.
- **Client-Side VPN Cycling (Roaming Recovery):** An observation (July 10, 2026) suggests that when a hung DNS probe state is triggered by roaming between Access Points (APs) on a WPA2-Enterprise network, simply toggling the VPN connection (Tailscale) off and on locally on the client phone immediately resolves the issue. You do not need to cycle the phone’s physical Wi-Fi. This reinforces the theory that roaming on Enterprise networks leaves a stale/poisoned P2P endpoint in the phone’s Tailscale magicsock state, which gets instantly flushed upon a local VPN restart. (Status: Possible but not formally verified).

## 37. TODO

* **Verify Wi-Fi Roaming:** Investigate and test whether switching Wi-Fi networks now successfully and smoothly recovers the gateway end-to-end without any tailscale flag errors or dead tunnels. (Pending user verification).
* **Fix Diagnostic Script Check 5/8:** ~~Investigate why ‘diagnose-host-leak.sh‘ check 5/8 (‘Host default route‘) sometimes reports "default route goes via physical Wi-Fi  Tailscale route assertion failed" on macOS even though the ‘warp=on‘ egress checks confirm that traffic is successfully tunneling.~~ **Closed**  fixed by switching check 5 from ‘netstat -rn | grep default‘ to ‘route -n get 1.1.1.1‘. macOS Tailscale uses interface-scoped routes and does not replace the DHCP default route entry. See 36 Known Unknowns.
* **Expose Tor Control Port (9051):** ~~Consider mapping the Tor Control port to localhost and adding ‘nyx‘ (the Tor terminal status monitor) to allow users to inspect their entry, middle, and exit node IPs in real-time.~~ **Closed**  mapped the Tor Control Port to a procedurally generated, localhost-bound port ‘TOR_CONTROL_PORT‘, configured ‘ControlPort‘ and disabled cookie authentication in ‘docker/tor/entrypoint.sh‘, and integrated it into the ‘/sweep‘ verification protocol.
* **Host-Only Tor Mode (Pluggable Egress):** Implement an ‘EGRESS_TYPE=tor_host_only‘ mode for users in heavily censored (DPI-restricted) environments. This mode would skip booting ‘warp‘ and ‘tailscale‘, boot ‘adguardhome‘ and ‘tor‘ (using Telegram ‘obfs4‘ bridges), set AdGuard’s upstream to Tor’s ‘DNSPort‘, and use the ‘pf‘ kill-switch to force all host Mac traffic strictly through the Tor network. This provides a 100% free, DPI-resistant evasion path without requiring an external VPS.
  * **The Technical Realities (What You Need to Watch Out For):**
    * **The UDP Problem:** Tor only transports TCP traffic. It does not support UDP (aside from DNS requests passed directly to its DNSPort). macOS is incredibly chatty over UDP (FaceTime, mDNS, QUIC protocols). Your pf rules defined in ‘scripts/pf.conf‘ must be configured to aggressively and silently DROP all host UDP traffic (except for local DNS queries to AdGuard). If you don’t drop it cleanly, macOS services might hang indefinitely while waiting for timeouts.
    * **The "Daily Driver" Friction:** Routing a whole host OS through Tor is brutal on performance. Background daemons (iCloud sync, macOS software updates, Spotlight indexing) will blindly attempt to download gigabytes of data over Tor, which could saturate the circuits or time out completely.
    * **Bridge Rot:** State firewalls actively hunt and block obfs4 bridges. When a user’s bridges burn out, they will lose all internet access due to the pf kill-switch. You will need to ensure the user has a frictionless way to update the ‘tor-bridges.txt‘ file and restart the Tor container without exposing their IP.
    * **Exit Node Discrimination:** Because all traffic will exit from known Tor nodes, the user will face an onslaught of Cloudflare CAPTCHAs, and many banking or streaming services will flat-out reject the connections.

## 38. Observation: AdGuard Returns Two Different Sinkhole IPs (Unverified)

> **Status: Observed but not formally verified. Needs a deeper audit of the rule-compiler sources to confirm.**

### Observation (July 10, 2026)
When querying AdGuard Home for blocked ad domains, some domains resolve to ‘0.0.0.0‘ and others resolve to ‘127.0.0.1‘. Both are blocked, both cause connection failure, but the different responses were unexpected.

‘‘‘
```

~~~
doubleclick.net               0.0.0.0    (blocked)
ads.google.com                127.0.0.1  (blocked)
pagead2.googlesyndication.com  0.0.0.0  (blocked)
scorecardresearch.com         127.0.0.1  (blocked)
```

### Likely Explanation (Unverified)
There are three historical schools of thought on the "correct" DNS sinkhole response, and AdGuard faithfully preserves whichever the source blocklist used:

| Sinkhole IP | Convention | Origin |
|---|---|---|
| `0.0.0.0` | AdBlock-style lists (`\|\|domain^`) | Modern DNS-level blocking; AdGuard's native format. Fails fast  OS doesn't attempt a TCP connection. Works for both IPv4 and IPv6 without a separate rule. |
| `127.0.0.1` | Hosts-file-style lists (`127.0.0.1 domain`) | 90s-era `/etc/hosts` tradition. Steven Black's hosts list (500k+ domains) still uses this. |
| `NXDOMAIN` | Purist / Pi-hole default | Returns "domain doesn't exist." Semantically most accurate but requires browsers to handle NXDOMAIN gracefully. |

The dual-response behavior in this project is because `rule-compiler` pulls from both AdBlock-format sources (Hagezi, uBlock origin lists) and hosts-format sources (Spamhaus DROP, Steven Black, Feodo Tracker), and AdGuard returns whatever sinkhole IP the matching rule specifies.

### To Verify
Run `docker compose exec tailscale cat /etc/hosts` to confirm no hosts-file rules are being injected at the container level, and check which specific AdGuard rule matched each domain via the AdGuard query log at `http://100.100.21.8:3000/#logs`.

## 39. Observation: AGY CLI Appears to Generate Native macOS Notification Pop-ups (Unverified)

> **Status: Observed but source unverified. macOS security logs ruled out system-level origin.**

### Observation (July 10, 2026)
While running a DNS blocklist verification via the Antigravity CLI (`agy`), a command was proposed that included `malware.wicar.org` (a known-safe DNS blocklist test domain) in a `dig` loop. Shortly after, a macOS-style notification popup appeared describing a "malicious" or "blocked" event. The AGY CLI terminal session then closed.

### Investigation
- **macOS XProtect / Gatekeeper logs:** Empty. Zero events in the 30-minute window around the incident.
- **macOS Endpoint Security / System Policy logs:** Empty.
- **Broad `log show` search for "malicious", "malware", "blocked":** Empty.
- **Conclusion:** macOS itself did not generate the notification. No system security subsystem fired.

### Likely Explanation (Unverified)
The notification appears to have originated from **AGY CLI's own internal safety system**. AGY CLI is a native macOS application (not just a terminal process) and has access to the macOS Notification Center API (`UNUserNotificationCenter`). When its guardrails detected a domain containing the word "malware" (`malware.wicar.org`) in a proposed shell command, it likely:
1. Blocked the command from executing
2. Posted a native macOS-style notification via the Notification Center

This is surprising behavior for a CLI tool  most terminal programs don't send GUI notifications  but AGY CLI appears to bridge both worlds: it runs in a terminal but is packaged as a native app with full system API access. The notification would be indistinguishable from a system security alert at a glance.

### Notes
- `malware.wicar.org` is the DNS/AV equivalent of the EICAR test file  a deliberately benign domain that triggers blocklist/AV systems to verify they work. It was included in the test set to confirm the AdGuard blocklist was catching it. AGY's safety system is not aware of this distinction.
- For future DNS blocklist testing, use only ad/tracking domains (e.g., `doubleclick.net`, `adnxs.com`) to avoid triggering AGY's guardrails.

## 40. Apple Silently Removed the `airport` Binary in macOS Sonoma 14.4 (March 2024)

### What happened
`watcher.sh`'s `detect_lan_p2p_mode()` was written to use the `airport` CLI tool at:
```
/System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/airport
```
This path returned `exit 127: no such file or directory`. The function silently fell back to `reason="no-wifi"` even while the Mac was actively connected to WPA2 Enterprise Wi-Fi.

### Why Apple removed it
`airport` was never a real public binary  it lived inside a **private framework** and was always technically unsupported. Apple deprecated it quietly years ago and finally removed it in **macOS Sonoma 14.4 (March 2024)**. The removal is part of a broader privacy clampdown on Wi-Fi metadata:
- Starting in **macOS 14.5**, BSSIDs and other Wi-Fi association details are **redacted** (`<redacted>`) in most tools, including the official replacement `wdutil`
~~~

~~~
- Apple's official recommendation is to use the **CoreWLAN framework** (Swift/Objective-C) with proper entitlements  useless for a bash script
- The official CLI replacement `wdutil` requires `sudo` for most useful output  also useless for a background LaunchAgent

### Fix
Replaced `airport -I | awk '/link auth/'` with:
```bash
system_profiler SPAirPortDataType | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF; exit}'
```
This returns human-readable strings like `WPA2 Enterprise`, `WPA2 Personal`, or `None` for the currently associated network  without `sudo`, and without any removed private binary.

**Confirmed working output on this machine:**
```
P2P detect: security='Enterprise'  allow=false (reason: 802.1x-enterprise)
```

###  Risk: `system_profiler` may not survive forever either
`system_profiler SPAirPortDataType` still works as of macOS Sequoia (15.x) without sudo and without redaction. However, given Apple's trajectory on Wi-Fi privacy, this could be locked down in a future release. If `security` comes back empty on a future macOS version while the Mac is actively on Wi-Fi, the function will silently fall back to `allow=false` (safe default), but the detection will be broken again. The long-term fix would be a small Swift helper using CoreWLAN that we compile once and bundle in the repo.

## 41. Fixed: Watcher.sh Suppressed Exit Code Bug

### Observation
The `scripts/watcher.sh` script is a long-running daemon that invokes `recover.sh --post-wake` when it detects system wake or network roam events. Previously, it contained the following bug:
```bash
  bash "$RECOVER" --post-wake
  local exit_code=$?
  ...
  return $exit_code || true
```
The `|| true` mistakenly swallowed the actual `$exit_code` returned by `recover.sh`, causing `run_recover()` to always return `0` (success), masking any underlying failures in the wake recovery process.

### Fix
The `|| true` statement was removed from the return line:
```bash
  return $exit_code
```
Since `watcher.sh` explicitly omits `set -e` (to prevent pipe breakages from killing the daemon), removing `|| true` safely allows the underlying exit code to propagate without risking daemon termination. The watcher daemon was then reloaded (`launchctl kickstart -k`) to pick up the script changes in memory.
## 42. Network Readiness Race Condition on `--restart` (Fixed)

**Problem:** When running `./toggle.sh --restart`, the script tears down the gateway and immediately begins the startup sequence. On systems with Wi-Fi (or any wireless interface), the OS takes a moment to re-associate with the network after the teardown's DNS/routing changes. If the startup sequence ran before the OS had a default route, `get_active_service` would return nothing, routing bypass targets would be wrong, and DNS/WARP setup would silently fail  resulting in a partially configured gateway that self-heals only once the `watcher.sh` re-runs.

**Root cause:** The original code had no gate before the first network-dependent call (`ACTIVE_SERVICE=$(get_active_service)` at the top of the start branch). This was a pure timing race.

**First fix (Wi-Fi specific):** A polling loop on `ipconfig getifaddr en0` was added to wait for an IPv4 address on `en0` before proceeding. This worked but was brittle  it only handled Wi-Fi and would fail for Ethernet, USB-C adapters, or iPhone USB tethering.

**Generalized fix:** Replaced the `en0` check with `route get default | grep 'gateway:'`. This detects a valid default gateway on *any* interface, covering all connection types. The loop polls every 2 seconds with a 60-second hard timeout (then proceeds with a warning rather than hanging forever). The output prints the detected gateway IP so failures are easy to diagnose in logs.

```bash
# In toggle.sh, before ACTIVE_SERVICE=$(get_active_service)
while true; do
  if route get default 2>/dev/null | grep -q 'gateway:'; then
    _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
    echo "  Network ready (default gateway: $_gw)."
    break
  fi
  ...
done
~~~

````
‘‘‘

**Why not ping?** Pinging an IP (e.g. ‘1.1.1.1‘) during restart is unreliable because DNS may still be hijacked from the previous session, and the exit node routing may not yet be cleared, causing the ping to loop back through the tunnel. ‘route get default‘ is a pure local kernel query  no packets sent.

## 43. Low Gateway Throughput (DERP Relay & MTU Fragmentation) Fixed

**Problem:** The host Mac’s internet throughput through the gateway dropped significantly (e.g., from 34 Mbps raw to 2.1Mbps via gateway). This was caused by a combination of two bottlenecks:

**1. Tailscale DERP Relay Bottleneck:**
Because the host Mac and the Docker container operate behind the same public IP, standard hole-punching for Tailscale P2P failed, forcing traffic through Tailscale’s NYC DERP relay. Normally, ‘routing-fix.sh‘ explicitly drops container Tailscale UDP packets destined for local subnets (when ‘TAILSCALE_ALLOW_LAN_P2P=false‘ in ‘.env‘ or auto-detected). We nuke these P2P connections on purpose to prevent the severe macOS ‘gvproxy‘ SNAT endpoint poisoning issue we faced earlier (see ‘devref.md 36‘). However, this strict policy unintentionally blocked direct communication between the container and the Mac host itself via the Docker bridge network.

**Fix:** Tailscale’s control plane registers the Mac host using its physical IP (e.g., ‘172.17.52.223‘), not the Docker bridge IP. If this physical IP falls within the dropped local subnets (like ‘172.16.0.0/12‘), the P2P handshake is dropped. To solve this natively, ‘scripts/common.sh‘ now features a ‘write_host_ips()‘ function that actively grabs all physical IPv4 addresses on the Mac and writes them to ‘.host_ips‘. ‘routing-fix.sh‘ dynamically reads this file every 30 seconds and injects priority ‘RETURN‘ rules for those exact physical IPs. This guarantees direct container-to-host P2P over the local bridge (bypassing DERP) while continuing to block external LAN devices (like phones) to prevent the 36 SNAT poisoning bug.

**2. MTU Double-Encapsulation Fragmentation:**
The host’s Tailscale interface (‘utun5‘) defaulted to an MTU of 1280. The WARP tunnel inside the container also uses an MTU of 1280. When a full 1280-byte Tailscale packet from the host entered the container and was encapsulated by WARP (adding ˜60 bytes of overhead), the resulting packet exceeded 1280 bytes, leading to severe fragmentation and performance degradation.

**Fix:** In ‘toggle.sh‘’s ‘setup_exit_node_routing‘ function, the host’s Tailscale ‘utun‘ interface MTU is explicitly lowered to ‘1200‘ (configurable via ‘HOST_MTU‘ in ‘.env‘). This ensures that host-originated packets can comfortably fit inside the container’s WARP tunnel encapsulation without fragmenting.

## 44. Architectural Limitation: Chrome Remote Desktop Performance (UDP NAT)

### Observation (July 11, 2026)
When the user connects to their host Mac using Chrome Remote Desktop (CRD) while ‘nullexit‘ is active, the connection suffers from massive latency, lag, and degraded video quality. AdGuard Home’s ‘blocked.log‘ shows zero blocked DNS requests for Google, WebRTC, or ‘chromoting‘ endpoints, indicating this is not a DNS sinkhole issue.

### Root Cause
This is a fundamental limitation of tunneling all host traffic through a strict commercial NAT (Cloudflare WARP).
1. Chrome Remote Desktop attempts to use **STUN (UDP hole-punching)** to establish a lightning-fast, direct peer-to-peer WebRTC connection between the client device and the host Mac.
2. Because all non-Tailscale traffic is forcefully routed through the ‘warp‘ container, the STUN packets exit the network from a Cloudflare IP.
3. Cloudflare’s enterprise NAT infrastructure strictly drops unsolicited inbound UDP packets. The UDP hole-punch fails.
4. Because a direct P2P connection cannot be established, CRD falls back to **Relay Mode** (TURN), bouncing all video data back and forth through Google’s cloud servers instead of sending it directly over the local or mesh network. This relaying introduces massive latency.

### Workaround / Fix
This lag is the accepted "tax" for maintaining absolute cryptographic anonymity; the only way to restore CRD speed would be to leak its UDP traffic outside the tunnel (violating zero-trust).
To achieve low-latency remote access, users should bypass CRD entirely and use macOS’s built-in **Screen Sharing (VNC)** directly over the Tailscale IP. Tailscale’s sophisticated custom DERP/WireGuard architecture successfully UDP hole-punches through strict NATs, providing a direct, encrypted, high-speed connection without relying on external cloud relays.

## 45. Architectural Decision: Dynamic IP Fetch vs. MagicDNS

### Observation
The ‘nullexit‘ gateway uses a dynamically fetched IP address (cached in ‘.gateway_ip‘) to configure host routing and the ‘pf‘ kill-switch, rather than utilizing Tailscale’s user-friendly MagicDNS hostname (e.g., ‘nullexit-gateway‘).

### Why MagicDNS is Not Used
1. **The "Chicken and Egg" Bootstrapping Problem:**
   macOS’s Packet Filter (‘pf‘) and low-level routing commands (‘route add‘) operate strictly at Layer 3 and require raw IP addresses. If ‘toggle.sh‘ or ‘recover.sh‘ attempted to use ‘nullexit-gateway‘,
````

~~~
the host Mac would need to perform a DNS lookup to resolve it. However, the very first step of the gateway's initialization is to completely hijack the host's DNS and point it *at the gateway itself*. The Mac cannot resolve the gateway's MagicDNS name if it needs the gateway's IP to know where to send the DNS query!
2. **Dynamic Self-Healing (Avoiding Stale Cache):**
   To solve the bootstrapping problem without causing bugs if the node is deleted and re-authenticated on the Tailscale Admin Console, `toggle.sh` explicitly runs `docker compose exec ... tailscale ip -4` during boot to fetch the absolute freshest IP dynamically. It saves this to `.gateway_ip` so that fast-roaming scripts like `recover.sh` can instantly retrieve the routing target without incurring the latency of querying Docker.

## 46. Tor Container Kill-Switch Inheritance & Fail-Closed Egress Verification

### Context
Unlike the host's SOCKS5 fallback path (whose kill-switch interaction is documented in 31), the `tor` container's connection status and fail-closed behavior under VPN tunnel drops is not governed by macOS `pf` rules. Instead, it relies on network namespace inheritance inside Docker Compose.

### Mechanics
1. **Shared Network Namespace:** The `tor` service is configured with `network_mode: service:warp` in `docker-compose.yml`. This shares the network namespace of the `warp` (Gluetun) container.
2. **Inherited Egress Rules:** All socket communication within the namespace is bound to the same networking stack. Gluetun manages this stack by redirecting traffic through `tun0` (the WireGuard/WARP tunnel) and applying `iptables` rules that explicitly drop outbound traffic originating on `eth0` (the physical/bridge interface), except for permitted local subnets or the VPN server's endpoint.
3. **Tunnel Fail-Closed:** If the WARP connection drops or the `tun0` interface is brought down, the routing table entries for `tun0` become invalid or disappear. Because the `iptables` rules in the shared namespace continue to drop any outgoing internet traffic attempting to exit via `eth0`, the `tor` container cannot leak raw traffic to the host's underlying networks. Egress fails closed.

### Verification / Manual Test Case
To manually verify that the Tor container fails closed and does not leak traffic when the tunnel dies:

1. **Verify active path:** Ensure the gateway is active (`toggle.sh start`) and fetch the Tor port (`$TOR_SOCKS_PORT` in `/tmp/nullexit-ports.env`). Verify Tor traffic routes correctly:
   ```bash
   curl --socks5-hostname 127.0.0.1:$TOR_SOCKS_PORT https://check.torproject.org/api/ip
   ```
   *(This should output a Tor exit node IP).*

2. **Simulate tunnel drop:** Bring down the virtual interface inside the shared network namespace:
   ```bash
   docker compose exec warp ip link set dev tun0 down
   ```

3. **Re-run the egress check:**
   ```bash
   curl --socks5-hostname 127.0.0.1:$TOR_SOCKS_PORT https://check.torproject.org/api/ip
   ```
   * **Expected Result:** The request must hang and eventually time out, or immediately fail with a proxy error (e.g., `curl: (97) SOCKS5: connection failed` or `curl: (7) Failed to connect`).
   * **Leaked Egress Check:** Check the host's Wi-Fi router or firewall logs. No outbound packets from the Tor container should route directly over the bridge network to the internet.

## 47. Architectural Decision: Transparent .onion Routing via RFC 2544 Subnet (198.18.0.0/15)

### Context
To enable seamless dark web browsing across the entire mesh, the gateway requires a mechanism to dynamically map `.onion` domains to IP addresses that the host operating system and network layers can route.

### IP Address Conflict & Solution
1. **Initial Conflict:** Initially, the Tor virtual address network was configured to map `.onion` sites within the `10.192.0.0/10` range. However, the Colima Docker daemon dynamically allocated `10.200.1.0/24` to the containers. Because this Docker subnet mathematically falls within the `10.192.0.0/10` block, a routing loop occurred: all host packets destined for the Docker containers on ports like `9050` or `9051` were intercepted by the transparent proxy NAT rules and dropped, causing proxy failures.
2. **Tailscale Private IP Exclusion:** Shifting the subnet to another private range like `10.123.0.0/16` resolved the port conflict, but led to connection timeouts. On macOS, Tailscale's Exit Node routing actively excludes RFC 1918 private subnets (e.g. `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) to maintain accessibility to local LAN resources (like printers/routers). Thus, packets targeting `10.123.x.x` were dropped locally by Tailscale before reaching the gateway tunnel.
3. **The Benchmarking Subnet Solution:** To bypass these private IP exclusions without manual static routing tables, Tor's virtual network was changed to **`198.18.0.0/15`** (reserved by RFC 2544 for benchmarking). Since this is not a private RFC 1918 range, Tailscale exit-node routing automatically encapsulates and routes all traffic destined for `198.18.0.0/15` through the tunnel to the gateway.
4. **Transparent NAT Redirection:** The `routing-fix` sidecar applies NAT rules in the shared network namespace:
   ```bash
~~~

~~~~
    iptables -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040
    ```
    This intercepts all incoming TCP traffic targeting the mapped `.onion` range and transparently proxies it through Tor's transparent port (`9040`).

### Exit Node Exclusions
The gateway supports the ability to exclude specific countries from being used as exit nodes (e.g. to avoid certain jurisdictions or nodes with poor network latency). This is configured globally via the `TOR_EXCLUDE_EXIT_NODES` environment variable in `.env`, which gets dynamically appended to Tor's `ExcludeExitNodes` and strictly enforced with `StrictNodes 1`.
~~~~

5 diagrams.md

~~~~
# nullexit  Flow Diagrams & Architecture

---

## 1. System Architecture  What's Running & Where

```mermaid
graph TB
    subgraph INTERNET["  Internet / Cloudflare Edge"]
        CF["Cloudflare WARP\n(Exit Point)\nwarp=on"]
    end

    subgraph TAILNET[" Tailscale Mesh (100.x.x.x)"]
        PHONE[" Phone / Laptop\n(Tailnet Client)"]
    end

    subgraph MACHOST[" macOS Host"]
        direction TB
        PFFIREWALL["macOS pf Firewall\n(Kill-Switch & TCP MSS Clamping)"]
        TSDAEMON["tailscaled\n(brew service)\nhost Tailscale"]
        HOSTDNS["Host DNS\n Gateway IP\n(hijacked by toggle.sh)"]
        HOSTSOCKS["Host SOCKS5\n127.0.0.1:1080\n(fallback only)"]
        HOSTTOR["Host Tor SOCKS5\n127.0.0.1:$TOR_SOCKS_PORT\n(randomized)"]
        HOSTTORCTRL["Host Tor Control\n127.0.0.1:$TOR_CONTROL_PORT\n(randomized)"]

        subgraph MONITORS[" Background Daemons (toggle.sh children)"]
            WARPWATCH["WARP Watcher\n(every 30s, via docker exec)\n output.log"]
            LEAKPROBE["Host Leak Probe\n(every 300ms, via curl from HOST)\n output.log"]
            DNSWATCHER["DNS Watcher\n(re-hijacks if drift detected)\n output.log"]
            CAFFEINATE["caffeinate -i\n(sleep prevention)"]
        end

        subgraph COLIMAVM["  Colima VM (Linux, 600MB RAM)"]
            subgraph NETNS["warp container network namespace (shared by ALL containers)"]
                GLUETUN["warp / Gluetun\nWireGuard  tun0\n(owns the namespace)"]
                TS["tailscale container\nAdvertises exit node\ntailscale0 interface"]
                SOCKS["tailscale\nSOCKS5\nport 1080"]
                ADGUARD["adguardhome\nDNS sinkhole\nport 5335"]
                ROUTINGFIX["routing-fix sidecar\nGeo-IP Blocking & Go Logger\n(writes blocked.log)"]
                TOR["tor container\nSOCKS5: port 9050\nControl: port 9051\nTransparent Proxy: port 9040\nDNSPort: port 5353"]
            end
        end

        subgraph LAUNCHD[" launchd (always-on)"]
            WATCHER["scripts/watcher.sh\n(sleep/wake + Wi-Fi roam\n+ LAN P2P auto-detection)"]
        end
    end

    subgraph RECOVERY[" Recovery"]
        RECOVER["recover.sh\n(nuclear reset)"]
        RECOVERPOST["recover.sh --post-wake\n(light refresh)"]
    end

    %% Traffic flow
    PHONE -->|"WireGuard / Tailscale mesh"| TSDAEMON
    TSDAEMON -->|"exit-node route utun*"| TS
    TS -->|"FORWARD  tun0 (MASQUERADE)"| GLUETUN
    GLUETUN -->|"WireGuard UDP (macOS Host Egress)"| PFFIREWALL
    PFFIREWALL -->|"TCP MSS Clamped / Kill-Switch check"| CF
    TS -->|"198.18.0.0/15 PREROUTING redirect"| TOR
    TOR -->|"internal path / exit nodes via tun0"| GLUETUN

~~~~

~~~
    %% DNS
    HOSTDNS -->|"UDP:53  TCP:5354"| ADGUARD
    ADGUARD -->|"DNS upstream via tun0"| CF
    ADGUARD -->|"onion queries  localhost:5353"| TOR

    %% Monitoring
    WARPWATCH -->|"docker exec warp wget cdn-cgi/trace"| GLUETUN
    LEAKPROBE -->|"curl cdn-cgi/trace (HOST NIC)"| CF
    DNSWATCHER -->|"networksetup -getdnsservers"| HOSTDNS

    %% Logging
    ROUTINGFIX -->|"writes"| BLOCKEDLOG["blocked.log\n(Host-side)"]
    HOSTTOR -->|"port mapping"| TOR
    HOSTTORCTRL -->|"port mapping"| TOR

    %% Recovery triggers
    WARPWATCH -->|"6 consecutive warp=off"| RECOVER
    WATCHER -->|"sleep/wake / roam"| RECOVERPOST
    RECOVERPOST -->|"if gateway unhealthy"| RECOVER
    WATCHER -->|"writes .lan_p2p_detected"| ROUTINGFIX

    style MONITORS fill:#1a1a2e,color:#e0e0ff
    style COLIMAVM fill:#0d2137,color:#e0e0ff
    style NETNS fill:#0a1628,color:#e0e0ff
    style RECOVERY fill:#2d0a0a,color:#ffe0e0
    style INTERNET fill:#0a2d1a,color:#e0ffe0
    style TAILNET fill:#1a2d0a,color:#e8ffe0
    style LAUNCHD fill:#2d1a2d,color:#ffe0ff
```

---

## 2. toggle.sh  Full START Flow

```mermaid
flowchart TD
    START(["./toggle.sh"])
    CHECK{"is_gateway_active?\n(containers running OR\nDNS  1.1.1.1)"}

    START --> CHECK
    CHECK -->|"YES  already ON"| STOPFLOW[" See STOP Flow"]
    CHECK -->|"NO  start it"| S1

    S1["1. Reset DNS  1.1.1.1\n(prevent deadlocks)"]
    S2["2. tailscale down\n(prevent exit-node routing during boot)"]
    S3["3. Check Colima VM\n(colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged)"]
    S4["4. Configure VM swap\n(400MB, prevent OOM)"]
    S5["5. Clean corrupted AdGuard config\n(prevent container crash loop)"]
    S6["6. docker compose up -d --build\n(compile Go threat rules & boot containers)"]
    S7["7. Poll tailscale status\n(wait for 'offers exit node'\nup to 60s)"]
    TSREADY{"container\nTailscale ready?"}
    ABORT([" ABORT\n cleanup_handler"])
    S8["8. Resolve gateway Tailscale IP\n(.gateway_ip or docker exec)"]
    S9["9. Verify tailscaled running\n(auto-start if needed)"]
    S10["10. tailscale up --reset\n--exit-node=\n--accept-routes=true\n--ssh=true --accept-dns=false"]

    PF1{"Pre-flight 1/3\ntailscale ping gateway"}
    PF2{"Pre-flight 2/3\ndig +tcp AdGuard DNS"}
    PF3{"Pre-flight 3/3\nWARP internet check"}
    ALLPASS{"All 3 pass?"}

    EXITPATH[" EXIT NODE PATH\n tailscale up --exit-node=\n force_dns_to_gateway (3 retry)\n SOCKS5 disabled"]
    SOCKSPATH[" SOCKS5 FALLBACK PATH\n macOS SOCKS5  127.0.0.1:1080\n DNS proxy  127.0.0.1:53\n No exit node (SKIP_EXIT_NODE=true)"]

    DAEMONS["Start background daemons\n caffeinate -i (sleep prevention)\n DNS Watcher\n WARP Watcher\n Host Leak Probe\n(all write  output.log)"]

    RULECOMPILE["Rule compiler status\n(log rule/IP counts)"]
    WRITEMARKER["write_gateway_active_marker\n(/tmp/nullexit-gateway-active.marker)"]
    DONE([" Gateway UP"])

    S1 --> S2 --> S3 --> S4 --> S5 --> S6 --> S7
    S7 --> TSREADY
    TSREADY -->|"NO (40 NoState)"| ABORT
    TSREADY -->|"YES"| S8
    S8 --> S9 --> S10
    S10 --> PF1 --> PF2 --> PF3
    PF1 & PF2 & PF3 --> ALLPASS
~~~

~~~
    ALLPASS -->|"YES"| EXITPATH
    ALLPASS -->|"NO"| SOCKSPATH
    EXITPATH --> DAEMONS
    SOCKSPATH --> DAEMONS
    DAEMONS --> RULECOMPILE --> WRITEMARKER --> DONE

    style ABORT fill:#5c1010,color:#fff
    style EXITPATH fill:#0a3d1a,color:#c0ffc0
    style SOCKSPATH fill:#3d2d00,color:#ffe0a0
    style DAEMONS fill:#0a1f3d,color:#c0d8ff
    style DONE fill:#0a3d1a,color:#fff
```

---

## 3. toggle.sh  STOP Flow

```mermaid
flowchart TD
    STOP(["./toggle.sh\n(gateway already ON)"])

    ST1["1. tailscale down\n(disconnect from mesh)"]
    ST2["2. docker compose down -t 5\n(stop all containers)"]
    ST3["3. Reset DNS  1.1.1.1\n(networksetup on all services)"]
    ST4["4. stop_local_dns_proxy\n(kill Python UDP proxy)"]
    ST5["5. stop_pidfile_daemon\n(kill caffeinate, rm PID)"]
    ST6["6. stop_pidfile_daemon\n(kill DNS Watcher, rm PID)"]
    ST7["7. stop_pidfile_daemon\n(kill WARP Watcher, rm PID)"]
    ST8["8. stop_pidfile_daemon\n(kill Leak Probe, rm PID)"]
    ST9["9. clear_gateway_active_marker\n(rm /tmp/nullexit-gateway-active.marker)"]
    ST10["10. cleanup_network_state\n disable_all_proxies\n Flush DNS cache (dscacheutil)\n Flush stale routes\n Power-cycle Wi-Fi (offon)"]

    DONE([" Gateway DOWN\nInternet restored"])

    STOP --> ST1 --> ST2 --> ST3 --> ST4
    ST4 --> ST5 --> ST6 --> ST7 --> ST8 --> ST9 --> ST10 --> DONE

    style DONE fill:#0a3d1a,color:#fff
```

---

## 4. Monitoring Layer  The 4 Background Daemons

```mermaid
graph LR
    subgraph DAEMONS["Background Daemons (all start with gateway, stop on teardown)"]
        direction TB

        subgraph D1[" Sleep Prevention"]
            CAFF["caffeinate -i\nPID: /tmp/nullexit-caffeinate.pid\n\nKeeps Mac awake while\ngateway is active"]
        end

        subgraph D2[" DNS Watcher"]
            DNSW["Polls networksetup every ~30s\nPID: /tmp/nullexit-dns-watcher.pid\n\nIf DNS drifts from gateway IP\n re-hijacks automatically\n output.log"]
        end

        subgraph D3[" WARP Watcher"]
            WARPW["Polls every 30s via:\ndocker compose exec warp wget cdn-cgi/trace\nPID: /tmp/nullexit-warp-watcher.pid\n\nCounts consecutive warp=off\n WARP_FAIL_THRESHOLD (default 6=30s)\n runs recover.sh (nuclear)\n output.log"]
        end

        subgraph D4[" Host Leak Probe"]
            LEAKP["Polls every 300ms via:\ncurl cdn-cgi/trace (HOST NIC directly)\nPID: /tmp/nullexit-host-leak-probe.pid\n\nLogs ONLY on state change:\n  LEAK / ROTATE / HOST-PROBE failed\nAll errors output.log\n(HOST_LEAK_PROBE=true in .env)"]
        end
    end

    subgraph BLIND["What each monitor can/can't see"]
        B1["WARP Watcher sees:\n Container WARP tunnel state\n Host NIC traffic\n Flash leaks < 5s"]
        B2["Host Leak Probe sees:\n Host NIC egress (browser path)\n Sub-second transitions\n Container internals"]
    end

    D3 -.->|"covers"| B1
    D4 -.->|"covers"| B2
~~~

~~~

    style D1 fill:#1a2d0a
    style D2 fill:#0a1a2d
    style D3 fill:#2d0a2d
    style D4 fill:#2d1a00
    style BLIND fill:#1a1a1a,color:#aaa
```

---

## 5. Traffic Flow  Data Path (Normal Operation)

```mermaid
sequenceDiagram
    participant PHONE as  Phone<br/>(Tailnet client)
    participant TS_HOST as tailscaled<br/>(macOS host)
    participant TS_CONTAINER as tailscale<br/>(container)
    participant GLUETUN as Gluetun/warp<br/>(container)
    participant CF as  Cloudflare<br/>WARP Edge
    participant INTERNET as  Internet

    Note over PHONE,INTERNET: Normal EXIT NODE path (all 3 pre-flights passed)

    PHONE->>TS_HOST: WireGuard packet<br/>(encrypted, UDP 41641)
    TS_HOST->>TS_CONTAINER: route via utun*  tailscale0
    TS_CONTAINER->>GLUETUN: FORWARD chain<br/>(iptables MASQUERADE on tun0)
    GLUETUN->>TS_HOST: WireGuard tunnel (tun0) egresses macOS Host
    Note over TS_HOST: macOS pf Firewall applies<br/>Kill-Switch & TCP MSS Clamping
    TS_HOST->>CF: re-encrypted UDP packet (MSS 1160)
    CF->>INTERNET: Plain HTTPS from<br/>Cloudflare IP

    Note over PHONE,INTERNET: DNS Resolution path
    PHONE->>TS_HOST: DNS query
    TS_HOST->>TS_CONTAINER: UDP:53  TCP:5354  AdGuard :5335
    TS_CONTAINER->>GLUETUN: AdGuard upstream DNS via tun0
    GLUETUN->>CF: Encrypted DNS query
    CF-->>GLUETUN: DNS response
    GLUETUN-->>TS_CONTAINER: response
    TS_CONTAINER-->>TS_HOST: response
    TS_HOST-->>PHONE: DNS answer
```

---

6. recover.sh Decision Tree

```mermaid
flowchart TD
    INVOKE(["recover.sh called"])
    MODE{"--post-wake\nflag?"}

    subgraph POSTWAKE[" --post-wake (light refresh, gateway stays UP)"]
        PW1["Re-hijack DNS  gateway IP"]
        PW2["Flush DNS cache"]
        PW3["Refresh tailscale exit-node"]
        PW4["Force-recreate warp container\n(if Gluetun healthcheck failed)"]
        PW5["Leave: containers, Wi-Fi,\ncaffeinate, DNS watcher,\nHost Leak Probe  untouched"]
    end

    subgraph NUCLEAR[" Default (nuclear  gateway torn down)"]
        N0["Sudo keep-alive loop (background)"]
        N1["1. tailscale down\n(disconnect from mesh)"]
        N2["2. Reset DNS  1.1.1.1\n(ALL network services)"]
        N3["3. disable_all_proxies\n(all interfaces)"]
        N4["4. Flush DNS cache\n(dscacheutil -flushcache)"]
        N5["5. Flush stale routes\n(WARP endpoint routes)"]
        N6a["6a. docker compose down -t 30\n(stop all containers)"]
        N6b["6b. stop_pidfile_daemon\n(kill caffeinate)"]
        N6c["6c. stop_pidfile_daemon\n(kill DNS Watcher)"]
        N6d["6d. stop_pidfile_daemon\n(kill Leak Probe)"]
        N7["7. Power-cycle Wi-Fi\n(networksetup offsleep 2on)"]
        N8["8. Verify internet working\n(curl ifconfig.io)"]
    end

    DONE_PW([" Gateway refreshed\n(still running)"])
    DONE_NUC([" Internet restored\n(gateway fully stopped)"])

    INVOKE --> MODE
    MODE -->|"yes"| PW1
    PW1 --> PW2 --> PW3 --> PW4 --> PW5 --> DONE_PW
    MODE -->|"no"| N0
~~~

~~~~
    N0 --> N1 --> N2 --> N3 --> N4 --> N5
    N5 --> N6a --> N6b --> N6c --> N6d --> N7 --> N8 --> DONE_NUC

    style POSTWAKE fill:#0a2d1a,color:#c0ffc0
    style NUCLEAR fill:#2d0505,color:#ffc0c0
    style DONE_PW fill:#0a3d1a,color:#fff
    style DONE_NUC fill:#3d1a00,color:#ffe0c0
‘‘‘

---

## 7. Failure Paths & Self-Healing

‘‘‘mermaid
flowchart LR
    subgraph EVENTS["Failure / Event"]
        E1["WARP tunnel drops\n(warp=off)"]
        E2["Mac wakes from sleep\nor Wi-Fi roams"]
        E3["DNS drifts from\ngateway IP"]
        E4["toggle.sh crashes /\nCtrl-C / SIGTERM"]
        E5["Host-side flash leak\ndetected (HOST NIC)"]
        E6["Silent NAT timeout\n(ping stops working)"]
    end

    subgraph DETECTORS["Detector"]
        D1["WARP Watcher\n(30s poll, docker exec)"]
        D2["scripts/watcher.sh\n(launchd, always on)"]
        D3["DNS Watcher\n(30s poll, networksetup)"]
        D4["cleanup_handler\n(ERR/INT/TERM/HUP trap)"]
        D5["Host Leak Probe\n(300ms poll, HOST curl)"]
        D6["routing-fix.sh\n(30s curl over tun0)"]
    end

    subgraph ACTIONS["Response"]
        A1[" threshold consecutive off\n recover.sh (nuclear)"]
        A2["recover.sh --post-wake\n(gentle refresh)"]
        A3["Re-hijack DNS\n(networksetup setdnsservers)"]
        A4["Stop all daemons\nReset DNS  1.1.1.1\nTear down Tailscale"]
        A5["Log LEAK to output.log"]
        A6["Blackhole internet traffic\n+ Drop TUNNEL_FAILED_CLOSED.marker"]
        A7["Push macOS Desktop Notification\n(via watcher.sh listener)"]
    end

    E1 --> D1 --> A1
    E2 --> D2 --> A2
    E3 --> D3 --> A3
    E4 --> D4 --> A4
    E5 --> D5 --> A5
    E6 --> D6 --> A6
    A6 -.->|"marker detected"| D2
    D2 --> A7

    style EVENTS fill:#2d1010,color:#ffcccc
    style DETECTORS fill:#0a1a2d,color:#cce0ff
    style ACTIONS fill:#0a2d0a,color:#ccffcc
‘‘‘

---

## 8. output.log  Who Writes What

All events converge into a single ‘output.log‘ at the repo root.

| Writer | Event Types | Format |
|--------|-------------|--------|
| ‘toggle.sh‘ | All startup/shutdown steps, errors, pre-flight results | Plain text with timestamps |
| ‘recover.sh‘ | Every recovery step (nuclear or post-wake) | Plain text |
| **WARP Watcher** | ‘WARP DOWN‘, ‘WARP RECOVERED‘, ‘WARP SHUTDOWN‘ | ‘[UTC timestamp]‘ prefix |
| **Host Leak Probe** | ‘LEAK‘, ‘ROTATE‘, ‘HOST-PROBE failed/timeout‘ | ‘[HH:MM:SS.mmm]‘ prefix |
| **DNS Watcher** | DNS re-hijack events | Appended inline |
| ‘docker compose‘ | Container logs on failure (last 100 lines of warp) | Dumped on ERR path |
| ‘routing-fix.sh‘ | (via ‘nullexit-logger‘ inside container) | Structured |

**Grep cheatsheet:**
‘‘‘bash
grep LEAK output.log           # Host-side warp=off events (real leaks)
grep ROTATE output.log         # Cloudflare edge IP rotations (harmless)
grep ’HOST-PROBE’ output.log   # curl failures from probe
grep ’WARP DOWN’ output.log    # Container-side tunnel drops
grep ’WARP SHUTDOWN’ output.log # Auto-recovery triggered
grep ’WARP RECOVERED’ output.log # Self-healed before threshold
~~~~

~~~
```
~~~

6 toggle.sh

```bash
#!/bin/bash
# toggle.sh  nullexit gateway toggle script for macOS
# Automatically handles Docker containers, Colima, DNS hijacking, and Tailscale exit node routing.

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

# Enforce Cryptographic Script Integrity
if [ -f "scripts/crypto.sh" ]; then
  if ! bash scripts/crypto.sh --verify; then
    exit 1
  fi
fi

# Define log file
LOG_FILE="$PWD/output.log"

# Rotate log file if it exceeds 50MB (52428800 bytes)
if [ -f "$LOG_FILE" ]; then
  log_size=$(stat -f%z "$LOG_FILE" 2>/dev/null || stat -c%s "$LOG_FILE" 2>/dev/null || echo 0)
  if [ "$log_size" -gt 52428800 ]; then
    mv "$LOG_FILE" "${LOG_FILE}.old" 2>/dev/null || rm -f "$LOG_FILE"
    echo "[$(date ’+%Y-%m-%d %H:%M:%S’)] Log file exceeded 50MB; rotated to output.log.old" > "$LOG_FILE"
  fi
fi

# --- MAIN EXECUTION LOGGING ---
echo -e "\n========================================================" >> "$LOG_FILE"
echo "[$(date ’+%Y-%m-%d %H:%M:%S’)] toggle.sh invoked" >> "$LOG_FILE"
echo "========================================================" >> "$LOG_FILE"

# (Previously this script chmod 600’d .env at start to "unlock" it for docker compose,
# and chmod 000’d it on exit. That pattern broke docker compose on macOS/Colima when
# the umask produced 0600  the script removed its own ability to read .env before it could even proceed.)

# --- PROCEDURAL PORT GENERATION ---
# To prevent static port fingerprinting by malware, we randomize exposed ports
# on every boot and persist them to a hidden environment file.
PORTS_FILE="/tmp/nullexit-ports.env"
if [[ "$1" == "" || "$1" == "--restart" ]]; then

  # Collision-avoidance function
  GENERATED_PORTS=""
  get_free_port() {
    local port
    while true; do
      port=$(jot -r 1 10000 65000)
      # 1. Prevent self-collision within this loop
      if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
        continue
      fi
      # 2. Prevent collision with ANY process on the Mac (root daemons, terminal apps, etc)
      # We use netstat instead of lsof because netstat reads the kernel socket table
      # directly and doesn’t require sudo to see ports bound by other users/root.
      if ! netstat -an -p tcp | awk ’{print $4}’ | grep -q "\.$port$"; then
        echo "$port"
        return
      fi
    done
  }

  export TOR_SOCKS_PORT=$(get_free_port)
  GENERATED_PORTS="$GENERATED_PORTS $TOR_SOCKS_PORT"

  export TOR_CONTROL_PORT=$(get_free_port)
  GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"

  export SOCKS_PROXY_PORT=$(get_free_port)
  GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"

  export DNS_PROXY_PORT=$(get_free_port)
```

```

  echo "export TOR_SOCKS_PORT=$TOR_SOCKS_PORT" > "$PORTS_FILE"
  echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
  echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
  echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
  ADGUARD_PASS=$(grep -E "^ADGUARD_PASSWORD=" .env 2>/dev/null | cut -d’=’ -f2- | tr -d "\"’")
  echo "export TOR_PASSWORD=${ADGUARD_PASS:-nullexit}" >> "$PORTS_FILE"
elif [ -f "$PORTS_FILE" ]; then
  # On STOP, source the existing ports so docker-compose down matches
  source "$PORTS_FILE"
else
  # Fallbacks if file is lost
  export TOR_SOCKS_PORT=9050
  export TOR_CONTROL_PORT=9051
  export SOCKS_PROXY_PORT=1080
  export DNS_PROXY_PORT=5354
fi

# Start execution timer
TOGGLE_START_TIME=$SECONDS

if [[ "$1" == "--restart" ]]; then
  echo "Executing Gateway Restart Sequence..."
  # We must source common.sh here temporarily to check is_gateway_active before we proceed
  source "$SCRIPT_DIR/scripts/common.sh"
  if is_gateway_active; then
    echo "Gateway is currently running. Stopping it first..."
    bash "$0"
    echo "Gateway stopped. Now starting it..."
  else
    echo "Gateway is already stopped. Starting it..."
  fi
  bash "$0"
  exit 0
fi

# Global flag to track if the script completed successfully
SUCCESS_RUN=false

# Global variable to track the currently running background command PID
CURRENT_BG_PID=""

# Source common bash functions
source "$SCRIPT_DIR/scripts/common.sh"

# Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
LOCK_FILE="/tmp/nullexit-toggle.lock"
if [ -f "$LOCK_FILE" ]; then
  LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
  if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
    if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
      die "Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running."
    fi
  fi
fi
echo "$$" > "$LOCK_FILE"

# Helper function to run a GUI command as the logged-in console user
# (Prevents permission failures if the script is run with sudo)
run_gui_cmd() {
  local console_user
  console_user=$(stat -f ’%Su’ /dev/console 2>> output.log || echo "$SUDO_USER")
  if [ -z "$console_user" ] || [ "$console_user" = "root" ]; then
    console_user=$(logname 2>> output.log || echo "$USER")
  fi

  if [ -n "$console_user" ] && [ "$console_user" != "root" ] && [ "$EUID" -eq 0 ]; then
    sudo -u "$console_user" "$@"
  else
    "$@"
  fi
}

# Active-state marker: written at end of START path so external watchers
# (see launchd/com.nullexit.wake-recovery.plist + scripts/watcher.sh +
# devref 10.29) know the gateway is currently up. They only fire
# recover.sh --post-wake when this file is present. Cleared at the top
# of the STOP path and inside cleanup_handler on any error/signal path
# so a half-broken gateway never re-triggers post-wake refreshes.
write_gateway_active_marker() {
  date -u +%FT%TZ > /tmp/nullexit-gateway-active.marker
```

```
  # Set the watcher debounce timestamp to now to prevent a redundant post-startup recovery run.
  date +%s > /tmp/nullexit-watcher.last-recovery 2>/dev/null || true
}

clear_gateway_active_marker() {
  rm -f /tmp/nullexit-gateway-active.marker
  rm -f "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/TUNNEL_FAILED_CLOSED.marker"
}

# Defensive: clear any stale marker from a prior crashed/aborted run. If a
# previous toggle.sh wrote the marker but was OOM-killed in the middle of the
# START block (before the END-of-START write)  or if the user rebooted mid
# session  this prevents the watcher from firing post-wake against a
# non-running gateway on every wake event. Placed AFTER the function
# definitions so bash resolves the call at parse time.
clear_gateway_active_marker

PID_FILE="$PID_CAFFEINATE"

# Start macOS caffeinate to prevent sleep while gateway is running
start_sleep_prevention() {
  if ! command -v caffeinate >/dev/null 2>&1; then
    echo "  [Warning] caffeinate tool not found. Sleep prevention unavailable."
    return 1
  fi

  # Stop any existing sleep prevention process first to avoid duplicates
  stop_pidfile_daemon "/tmp/nullexit-caffeinate.pid" "system sleep prevention"

  echo "  Enabling system sleep prevention (caffeinate)..."
  # Run caffeinate wrapped in a bash trap to prevent idle system sleep.
  # Critically, this traps macOS shutdown signals (SIGTERM) and automatically
  # flushes hijacked DNS back to normal right before the Mac powers off.
  # We do not use recover.sh here because it is a heavy recovery script (managing
  # containers, restarting tailscaled, etc.) which takes too long and gets
  # terminated by macOS before it can reset the DNS. Instead, we surgically and
  # instantly restore normal DNS and proxy settings here.
  nohup bash -c "
    trap '
      echo \"[Shutdown Trap] Reverting network settings...\" >> \"$PWD/output.log\" 2>&1
      kill \$! 2>/dev/null || true
      sudo -n networksetup -setdnsservers \"$ACTIVE_SERVICE\" 1.1.1.1 >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setdnsservers \"$EN0_SERVICE\" 1.1.1.1 >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setsearchdomains \"$ACTIVE_SERVICE\" \"Empty\" >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setsearchdomains \"$EN0_SERVICE\" \"Empty\" >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setsocksfirewallproxystate \"$ACTIVE_SERVICE\" off >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setsocksfirewallproxystate \"$EN0_SERVICE\" off >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setwebproxystate \"$ACTIVE_SERVICE\" off >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setwebproxystate \"$EN0_SERVICE\" off >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setsecurewebproxystate \"$ACTIVE_SERVICE\" off >> \"$PWD/output.log\" 2>&1 || true
      sudo -n networksetup -setsecurewebproxystate \"$EN0_SERVICE\" off >> \"$PWD/output.log\" 2>&1 || true
      if command -v tailscale >/dev/null 2>&1; then
        tailscale up >> \"$PWD/output.log\" 2>&1 || true
        tailscale set --accept-dns=false --exit-node= >> \"$PWD/output.log\" 2>&1 || true
        TS_DOWN_ARGS=\"\"
        if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS=\"--accept-risk=lose-ssh\"; fi
        tailscale down \$TS_DOWN_ARGS >> \"$PWD/output.log\" 2>&1 &
      fi
      sleep 1
      echo \"[Shutdown Trap] Cleanup complete.\" >> \"$PWD/output.log\" 2>&1
      exit 0
    ' SIGTERM SIGINT SIGHUP
    caffeinate -i &
    wait \$!
  " >> output.log 2>&1 &
  local caffe_pid=$!
  echo "$caffe_pid" > "$PID_FILE"
  echo "  Sleep prevention active (PID $caffe_pid). Your Mac won't sleep while the gateway is running."
}



DNS_WATCHER_PID_FILE="$PID_DNS_WATCHER"
WARP_WATCHER_PID_FILE="/tmp/nullexit-warp-watcher.pid"

```

```
start_dns_watcher() {
  local target_ip=$1
  stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
  echo "  Starting background DNS Watcher for seamless Wi-Fi roaming..."
  nohup bash -c "
    source \"$SCRIPT_DIR/scripts/common.sh\"
    trap 'exit 0' SIGTERM SIGINT SIGHUP
    while true; do
      ACTIVE_IF=\$(get_active_service)
      if [ -n \"\$ACTIVE_IF\" ]; then
        CURRENT_DNS=\$(networksetup -getdnsservers \"\$ACTIVE_IF\" 2>/dev/null)
        if [ \"\$CURRENT_DNS\" != \"$target_ip\" ]; then
          networksetup -setdnsservers \"\$ACTIVE_IF\" \"$target_ip\" >/dev/null 2>&1
        fi
      fi
      sleep 30
    done
  " >> output.log 2>&1 &
  echo $! > "$DNS_WATCHER_PID_FILE"
}



# Background WARP tunnel liveness monitor. Polls cdn-cgi/trace every 30s
# while healthy, but accelerates to polling every 5s if a failure is detected.
# Always logs state transitions to output.log. When warp=off persists for
# WARP_FAIL_THRESHOLD consecutive polls (default 6 = 30s of downtime), the watcher
# triggers a nuclear recovery: runs recover.sh to tear down the entire
# gateway  disconnecting Tailscale, stopping containers, resetting DNS,
# and power-cycling Wi-Fi. Note: This minimizes exposure window, but
# only the pf kill switch guarantees absolutely zero leakage on drop.
#
# The threshold (default 6) is statistically chosen: even at an
# unrealistically high 10% false-positive rate per poll, P(6 consecutive
# false positives) = 0.1^6  0.0001%. Real-world false-positive rates
# are far lower, making 30s of continuous downtime overwhelmingly likely
# to be a genuine outage.
#
# Configurable via .env:  WARP_FAIL_THRESHOLD=3  (15s, more aggressive)
start_warp_watcher() {
  stop_warp_watcher
  # Parse threshold from .env (same pattern as GATEWAY_MSS, GATEWAY_BYPASS_PING, etc.)
  local threshold
  threshold=$(read_env_var WARP_FAIL_THRESHOLD)
  if [ -z "$threshold" ]; then
    threshold="${WARP_FAIL_THRESHOLD:-6}"
  fi
  local trigger_file="/tmp/nullexit-warp-shutdown-triggered"
  rm -f "$trigger_file"
  echo "  Starting background WARP Watcher (polling every 30s [accelerating to 5s on failure], shutdown
     after ${threshold} consecutive failures)..."
  nohup bash -c "
    trap 'exit 0' SIGTERM SIGINT SIGHUP
    last_state='on'
    consec_off=0
    threshold='$threshold'
    trigger_file='$trigger_file'
    out_log='$PWD/output.log'
    recover_bin='$PWD/recover.sh'
    inhibit_file='/tmp/nullexit-warp-inhibit.marker'
    while true; do
      #  Inhibit check
      # recover.sh --post-wake writes this marker while force-recreating the
      # warp container. During that window, the container is intentionally
      # down  don't count it as a failure or we'll fire nuclear recover.sh
      # and kill a gateway that was in the process of healing itself.
      if [ -f \"\$inhibit_file\" ]; then
        consec_off=0
        sleep 5
        continue
      fi

      state='off'
      if docker compose exec -T warp wget -qO- --timeout=5 \
          https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null \
          | grep -q 'warp=on'; then
        state='on'
      fi

      #  State-transition logging
      if [ \"\$state\" != \"\$last_state\" ]; then
```

```
        if [ \"\$state\" = ’off’ ]; then
          echo \"[\$(date -u +%FT%TZ)] WARP DOWN  cdn-cgi/trace reports warp=off\" >> \"\$out_log\"
        fi
        last_state=\"\$state\"
      fi

      #  Consecutive-failure tracking + auto-shutdown
      if [ \"\$state\" = ’off’ ]; then
        consec_off=\$((consec_off + 1))
        if [ \"\$consec_off\" -ge \"\$threshold\" ] && [ ! -f \"\$trigger_file\" ]; then
          touch \"\$trigger_file\"
          echo \"[\$(date -u +%FT%TZ)] WARP SHUTDOWN  \$consec_off consecutive failures (threshold=\$threshold). Running recover.sh to kill the gateway and restore normal internet. Your IP is NO LONGER Cloudflare.\" >> \"\$out_log\"
          # Nuclear recovery: tear down the entire gateway.
          # recover.sh resets DNS  1.1.1.1, disconnects Tailscale,
          # stops Docker containers, flushes routes, power-cycles Wi-Fi.
          bash \"\$recover_bin\" --auto >> \"\$out_log\" 2>&1 || true
          echo \"[\$(date -u +%FT%TZ)] WARP SHUTDOWN  recover.sh completed, watcher exiting. Your IP is now your real ISP IP.\" >> \"\$out_log\"
          exit 0
        fi
      else
        if [ \"\$consec_off\" -gt 0 ]; then
          echo \"[\$(date -u +%FT%TZ)] WARP RECOVERED  warp=on after \$consec_off consecutive off readings (threshold was \$threshold)\" >> \"\$out_log\"
        fi
        consec_off=0
      fi

      if [ \"\$state\" = \"off\" ]; then
        sleep 5
      else
        sleep 30
      fi
    done
  " >> output.log 2>&1 &
  echo $! > "$WARP_WATCHER_PID_FILE"
}

stop_warp_watcher() {
  if [ -f "$WARP_WATCHER_PID_FILE" ]; then
    local wp
    wp=$(cat "$WARP_WATCHER_PID_FILE")
    if [ -n "$wp" ] && kill -0 "$wp" 2>/dev/null; then
      echo "  Stopping background WARP Watcher (PID $wp)..."
      kill "$wp" 2>/dev/null || true
    fi
    rm -f "$WARP_WATCHER_PID_FILE"
  fi
}





# Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
cleanup_handler() {
  local exit_code=$?
  local trigger_type="$1"
  local line_no="$2"

  if [ "$SUCCESS_RUN" = "false" ]; then
    echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
    if [ -n "$ACTIVE_SERVICE" ]; then
      reset_dns
    fi

    if [ -n "$CURRENT_BG_PID" ]; then
      echo "Terminating active command (PID $CURRENT_BG_PID)..."
      kill -15 "$CURRENT_BG_PID" 2>> output.log || true
    fi

    # Disconnect host Tailscale and close the GUI application on failure
    disconnect_tailscale_host

    # Stop local DNS proxy if running
    stop_local_dns_proxy

    # Stop sleep prevention
```

```
    stop_pidfile_daemon "/tmp/nullexit-caffeinate.pid" "system sleep prevention"
    stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
    stop_warp_watcher
    clear_gateway_active_marker

    # Capture warp logs on failure for debugging before teardown
    if [ "$trigger_type" = "ERR" ]; then
      echo -e "\n--- WARP FAILURE LOGS ---" >> output.log
      docker compose logs warp --tail=100 >> output.log 2>&1 || true
      echo "--------------------------" >> output.log
    fi

    # Best-effort container teardown on failure
    docker compose down --remove-orphans -t 30 2>> output.log || true

    # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
    if [ -n "$ACTIVE_SERVICE" ]; then
      cleanup_network_state
    fi


    if [ "$trigger_type" = "ERR" ]; then
      echo -e "\n============================================="
      echo "ERROR: Script failed at line $line_no with exit code $exit_code."
      echo "============================================="
      echo "Troubleshooting steps:"
      echo "1. Run 'colima status' to verify the VM is active."
      echo "2. Run 'docker compose ps' to check container health."
      echo "3. Run 'docker compose logs' to view application logs."
      echo "4. Run 'tailscale status' to check host Tailscale status."
      echo "============================================="
    fi
    rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
  fi
}

trap 'cleanup_handler ERR $LINENO' ERR
trap 'cleanup_handler INT' INT
trap 'cleanup_handler TERM' TERM
trap 'cleanup_handler HUP' HUP

#  NOPASSWD sudo
# The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
# needed (networksetup, dnsmasq, dscacheutil, killall, pkill, socat).
# All privileged calls below use `sudo -n` which will run without prompting.
# No credential caching loop is needed.

# Add Homebrew and standard paths
setup_standard_path

# Check for local DNS proxy tools (used when tailnet data plane is unavailable)
# Uses a Python DNS proxy (handles TCP wire format correctly).
# Python3 is built into macOS  no external dependencies.
DNS_PROXY_BIN=""
if command -v python3 >> output.log 2>&1; then
  DNS_PROXY_BIN="python3 -I"
elif command -v python >> output.log 2>&1; then
  DNS_PROXY_BIN="python -I"
fi

# Path to the DNS proxy script (sibling of this script)
DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/dns-proxy.py"

# Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
# (`brew install tailscale` + `brew services start tailscale`)  NOT the
# Tailscale.app GUI  so there is no .app bundle or Network Extension sandbox
# to launch, no menu-bar click to perform, and no scutil Network Service to
# start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
# if you ran the install with sudo) and the control socket is always available.
TS_BIN=""
if command -v tailscale >> output.log 2>&1; then
  TS_BIN="tailscale"
fi

# Baseline: disable Tailscale DNS management at the daemon level.
# `tailscale set` writes a persistent preference. However, `tailscale up --reset`
# (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
# re-enable `--accept-dns=true` and nullify this `set`.
# Therefore, EVERY `tailscale up --reset` call in this file MUST also pass
# `--accept-dns=false` explicitly. See steps 7 and 9 for the fix.
#
```

```
# This initial ‘set‘ is still useful as a baseline for non-reset operations
# (e.g. if the user runs ‘tailscale up‘ manually without --reset) and covers
# the brief window between this line and the first ‘--reset‘ call.
#
# Runs once at script start while tailscaled is still connected (if it was
# left running from a previous toggle), so the pref takes effect before any
# disconnection or reconnection logic.
if [ -n "$TS_BIN" ]; then
  $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
fi

# Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
# *entire* teardown  no scutil tunnel stop, no Tailscale.app GUI quit, no pkill.
# tailscaled keeps running (as a LaunchAgent / LaunchDaemon), which is intentional:
# the next ‘tailscale up‘ then reconnects in seconds rather than waiting for a
# slow .app launch + Network Extension activation. The ‘tailscale down‘ call also
# retains the user’s auth state, so we don’t force a browser re-login every cycle.
#
# Defined early (before the if/else branches and referenced by cleanup_handler’s
# ERR trap) so that any failure before the main logic can still tear down Tailscale.


#  Wait for Network Readiness
# On --restart, the gateway tears down while the host network interface may
# still be reconnecting (Wi-Fi, Ethernet, USB tethering, etc.). If we proceed
# too early, get_active_service returns nothing, routing targets the wrong
# interface, and DNS/WARP setup silently fails.
#
# We use ‘route get default‘ to detect a valid default gateway rather than
# checking a specific interface (e.g. en0)  this generalizes across all
# connection types: Wi-Fi, Ethernet, USB-C adapters, iPhone tethering, etc.
# The moment the OS has any routable default gateway, we proceed.
_net_wait_secs=0
_net_timeout=60
echo "Waiting for network connectivity before starting..."
while true; do
  if route get default 2>/dev/null | grep -q ’gateway:’; then
    _gw=$(route get default 2>/dev/null | awk ’/gateway:/{print $2}’)
    echo "  Network ready (default gateway: $_gw)."
    break
  fi
  if [ "$_net_wait_secs" -ge "$_net_timeout" ]; then
    echo "  [Warning] No default gateway after ${_net_timeout}s  proceeding anyway."
    break
  fi
  sleep 2
  _net_wait_secs=$((_net_wait_secs + 2))
done

ACTIVE_SERVICE=$(get_active_service)

# Resolve the service name for en0 (usually "Wi-Fi")  macOS scutil DNS resolver
# is commonly scoped to en0, so per-service DNS changes on other interfaces are
# ignored unless en0’s service is also updated.
EN0_SERVICE=$(get_en0_service)

# Helper to add host-side bypass routes for the WARP WireGuard endpoints.
# This prevents an infinite recursive routing loop (tunnel loop) where
# the container’s WireGuard packets exit the VM, reach the host, match the
# default route (the exit node), and get routed back into the tunnel.
# We route via the gateway IP (not interface) to ensure packets are routed
# properly even when the host’s default route changes to the Tailscale interface.

# Helper to manually override the host’s default route to point through the
# Tailscale utun* interface. Standalone tailscaled on macOS (Homebrew version)
# lacks the platform-specific integration to override the default gateway
# automatically when --exit-node is used.
setup_exit_node_routing() {
  local ts_iface
  ts_iface=$(ifconfig | awk ’/^[a-z0-9]+:/{iface=$1} /inet 100\./{print iface; exit}’ | tr -d ’:’)

  if [ -n "$ts_iface" ]; then
    echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
    local host_mtu
    host_mtu=$(read_env_var HOST_MTU)
    host_mtu=${host_mtu:-1200}

    # Lower the host Tailscale MTU (defaults to 1200) to prevent fragmentation when passing
    # through the container’s WARP tunnel (which also has an MTU of 1280).
    sudo -n ifconfig "$ts_iface" mtu "$host_mtu" >> output.log 2>&1 || true

```

```
    # DO NOT delete the physical default route! It crashes Colima.
    # Use the /1 trick to mathematically override the default route.
    sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
    sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
    sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
    sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true

    if netstat -nr | grep -E -q "^(0\.0\.0\.0/1|0/1)[[:space:]]+.*$ts_iface"; then
      echo "  [] Default route successfully overridden via $ts_iface."
    else
      echo "  [!] Failed to verify exit node routing on $ts_iface."
    fi
  else
    echo "[Warning] Could not detect Tailscale utun interface for host routing."
  fi
}

  # Helper to restore host DNS to a clean state (used on script start, on failure,
  # and after a successful STOP). Without this, a successful toggle-off leaves the
  # host's resolver pinned to a now-dead gateway IP  every lookup stalls for macOS's
  # DNS timeout (~5s) before falling through to whatever next server is configured.
  # With it, we always leave the host in `1.1.1.1 / no search domain`.
  #
  # We set DNS on BOTH the active service AND the en0 service because macOS's scutil DNS
  # resolver is commonly scoped to en0 (Wi-Fi). A per-service change on e.g.
  # "USB 10/100 LAN" is ignored by the system resolver unless the en0 service is also updated.
  reset_dns() {
    if [ -n "$ACTIVE_SERVICE" ]; then
      for dns_svc in "$ACTIVE_SERVICE" "$EN0_SERVICE"; do
        networksetup -setsearchdomains "$dns_svc" "Empty" 2>> output.log || true
        networksetup -setdnsservers "$dns_svc" 1.1.1.1 2>> output.log || true
      done
    fi
  }

# Helper to nuke leftover network state after teardown (proxy settings, routing
# table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
# problem where stale state kills internet after the gateway stops.
cleanup_network_state() {
  echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."

  # 1. Disable any leftover proxy settings (needs root on many macOS versions)
  for svc in "$ACTIVE_SERVICE" "$EN0_SERVICE"; do
    disable_all_proxies "$svc"
  done
  echo "  Proxies disabled."

  # 2. Flush DNS cache
  if command -v dscacheutil >> output.log 2>&1; then
    sudo -n dscacheutil -flushcache 2>> output.log || true
  fi
  sudo -n killall -HUP mDNSResponder 2>> output.log || true
  echo "  DNS cache flushed."

  # 3. Flush stale routing table entries
  if command -v route >> output.log 2>&1; then
    sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
    sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
  fi
  remove_warp_bypass_routes
  disable_killswitch
  echo "  Routing table and firewall flushed."

  # 4. Power-cycle Wi-Fi to clear any lingering interface state
  WIFI_PORT=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2}')
  if [ -n "$WIFI_PORT" ]; then
    sudo -n networksetup -setairportpower "$WIFI_PORT" off 2>> output.log || true
    sleep 2
    sudo -n networksetup -setairportpower "$WIFI_PORT" on 2>> output.log || true
    echo "  Wi-Fi power-cycled (interface $WIFI_PORT)."
    sleep 3
  else
    echo "  Wi-Fi interface not detected; bouncing en0..."
    sudo -n ifconfig en0 down 2>> output.log || true
    sudo -n ifconfig en0 up 2>> output.log || true
  fi

  # Force restart sharing services to prevent AirDrop / AirPlay discovery freezes
  # after network interface/DNS changes.
  echo "  Resetting macOS sharing services (AirDrop/AirPlay)..."
```

```
  reset_sharing_services

  echo "Network state cleanup complete."
}

#  SOCKS5 Proxy (WARP Tunnel)
# When Tailscale’s exit node can’t route through WARP (due to userspace
# forwarding), we use a SOCKS5 proxy running inside the warp container’s
# network namespace. Connections created by this proxy go through the
# kernel routing table (table 200 -> tun0 -> WARP), unlike Tailscale’s
# userspace exit node which bypasses it.
#
# The SOCKS5 proxy is exposed on localhost:${SOCKS_PROXY_PORT} via Docker port mapping.
# ‘networksetup -setsocksfirewallproxy‘ on macOS routes system TCP traffic
# through the proxy, which then goes through WARP.
#
# IMPORTANT: CLI tools like curl do NOT respect macOS system proxy settings.
# They must use --socks5-hostname or the ALL_PROXY env variable explicitly.

SOCKS_PROXY_PORT=${SOCKS_PROXY_PORT}

enable_socks_proxy() {
  local svc="$1"
  if [ -z "$svc" ]; then
    svc="$ACTIVE_SERVICE"
  fi

  echo -n "  Enabling system-wide SOCKS5 proxy on $svc (localhost:$SOCKS_PROXY_PORT)... "
  sudo -n networksetup -setsocksfirewallproxy "$svc" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.log || true
  sudo -n networksetup -setsocksfirewallproxystate "$svc" on 2>> output.log || true

  # Also set on en0 service for macOS resolver consistency
  if [ "$svc" != "$EN0_SERVICE" ]; then
    sudo -n networksetup -setsocksfirewallproxy "$EN0_SERVICE" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.log || true
    sudo -n networksetup -setsocksfirewallproxystate "$EN0_SERVICE" on 2>> output.log || true
  fi

  # IMPORTANT: Exclude local LAN and mDNS traffic from the proxy so P2P works natively
  # without source-IP masking from the Colima VM bridge.
  sudo -n networksetup -setproxybypassdomains "$svc" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8 172.16.0.0/12 "*.local" 2>> output.log || true
  if [ "$svc" != "$EN0_SERVICE" ]; then
    sudo -n networksetup -setproxybypassdomains "$EN0_SERVICE" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8 172.16.0.0/12 "*.local" 2>> output.log || true
  fi

  echo "done."
  echo "  All TCP traffic now routed through gateway -> WARP tunnel -> internet."
  echo "  (CLI tools: export ALL_PROXY=socks5://127.0.0.1:$SOCKS_PROXY_PORT)"
}

disable_socks_proxy() {
  for svc in "$ACTIVE_SERVICE" "$EN0_SERVICE"; do
    sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
  done
  echo "  SOCKS5 proxy disabled."
}

#  Local DNS Proxy (Python)
# When the tailnet data plane cannot establish, we use a Python DNS proxy.
# It listens on UDP:53, receives DNS queries, forwards them over TCP to
# AdGuard at 127.0.0.1:${DNS_PROXY_PORT} (Docker port mapping), and returns the response.
# The Python script handles the DNS-over-TCP wire format (2-byte length prefix)
# correctly  unlike socat which just forwards raw bytes.
#
# Python3 is built into macOS  no external dependencies.
DNS_PROXY_PID=""

# Kill any leftover DNS proxy process
stop_local_dns_proxy() {
  if [ -n "$DNS_PROXY_PID" ]; then
    kill "$DNS_PROXY_PID" 2>/dev/null || true
    wait "$DNS_PROXY_PID" 2>/dev/null || true
    DNS_PROXY_PID=""
  fi
  # Clean up any stale Python DNS proxy processes
  sudo -n pkill -f "dns-proxy.py" 2>/dev/null || true
}

# Start local DNS proxy (Python)
```

```
start_local_dns_proxy() {
  if [ -z "$DNS_PROXY_BIN" ]; then
    echo "  Python not found. Python3 is built into macOS  this shouldn’t happen."
    return 1
  fi

  if [ ! -f "$DNS_PROXY_SCRIPT" ]; then
    echo "  DNS proxy script not found at $DNS_PROXY_SCRIPT"
    return 1
  fi

  # Kill any leftover process first
  stop_local_dns_proxy

  echo -n "  Starting local DNS proxy via Python (UDP:53  TCP:${DNS_PROXY_PORT})... "
  sudo -n bash -c "TARGET_PORT=$DNS_PROXY_PORT \"$DNS_PROXY_BIN\" \"$DNS_PROXY_SCRIPT\"" &
  DNS_PROXY_PID=$!
  disown "$DNS_PROXY_PID" 2>/dev/null || true

  # Give it a moment to bind
  sleep 0.5

  if kill -0 "$DNS_PROXY_PID" 2>/dev/null; then
    echo "started (PID $DNS_PROXY_PID)."

    # Hijack host DNS to localhost
    echo -n "  Hijacking host DNS to 127.0.0.1 for ad-blocking... "
    networksetup -setsearchdomains "$ACTIVE_SERVICE" "Empty" 2>/dev/null || true
    networksetup -setsearchdomains "$EN0_SERVICE" "Empty" 2>/dev/null || true
    if ! networksetup -setdnsservers "$ACTIVE_SERVICE" 127.0.0.1; then
      echo "FAILED (networksetup error  check permissions)."
      stop_local_dns_proxy
      return 1
    fi
    networksetup -setdnsservers "$EN0_SERVICE" 127.0.0.1 2>/dev/null || true
    sudo -n dscacheutil -flushcache 2>/dev/null || true
    sudo -n killall -HUP mDNSResponder 2>/dev/null || true

    # Verify DNS actually works through the proxy
    echo -n "  Verifying DNS resolution... "
    if dig @127.0.0.1 google.com +short +timeout=5 &>/dev/null; then
      echo "ok."
      return 0
    else
      echo "FAILED (DNS queries not reaching AdGuard)."
      echo "  Restoring DNS to 1.1.1.1..."
      reset_dns
      stop_local_dns_proxy
      return 1
    fi
  else
    echo "FAILED (could not bind port 53  is it in use?)."
    DNS_PROXY_PID=""
    return 1
  fi
}

# Helper to FORCIBLY hijack host DNS to a single server (the gateway’s AdGuard IP)
# and VERIFY via `networksetup -getdnsservers` that the only name server is exactly
# $1. Returns 0 iff the live state matches; non-zero if it can’t be confirmed.
#
# This deliberately does NOT append a 1.1.1.1 fallback: macOS’s resolver queries
# the list in order and falls back to the next entry on timeout, so on a
# misbehaving AdGuard (filter compile crash, OOM kill, etc.) the resolver still
# leaks to 1.1.1.1 and silently bypasses ad-blocking. With no fallback, a broken
# gateway manifests as a *visible* DNS outage that prompts the user to
# investigate  which is the correct failure mode.
force_dns_to_gateway() {
  local target_ip="$1"
  if [ -z "$target_ip" ] || [ -z "$ACTIVE_SERVICE" ]; then
    echo "  [force_dns] ERROR: target_ip or ACTIVE_SERVICE is empty" >&2
    return 1
  fi

  local attempt
  for attempt in 1 2 3; do
    echo -n "  [force_dns] Attempt $attempt/3: setting DNS to $target_ip on \"$ACTIVE_SERVICE\" + \"$EN0_SERVICE\"... "

    # Apply to BOTH the active service AND the en0 service  the scutil DNS resolver
    # is scoped to en0 (Wi-Fi) and ignores per-service changes that don’t include it.
```

```
    # Fail fast on ACTIVE_SERVICE; best-effort on EN0_SERVICE (it may not exist).
    networksetup -setsearchdomains "$ACTIVE_SERVICE" "ts.net" || true
    if ! networksetup -setdnsservers "$ACTIVE_SERVICE" "$target_ip"; then
      echo "FAILED (networksetup error  check permissions)"
      sleep 1
      continue
    fi
    networksetup -setsearchdomains "$EN0_SERVICE" "ts.net" || true
    networksetup -setdnsservers "$EN0_SERVICE" "$target_ip" || true

    local entries
    entries=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log \
              | awk '/^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/ {print}')
    if echo "$entries" | grep -qFx "$target_ip"; then
      echo "VERIFIED"
      return 0
    fi

    local current
    current=$(echo "$entries" | tr '\n' ' ' | sed 's/ $//')
    echo "NOT YET (current DNS: [${current:-none}])"

    if [ "$attempt" = "3" ]; then sleep 2; else sleep 1; fi
  done

  echo "  [force_dns] FAILED after 3 attempts"
  return 1
}

# Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
# during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
# `ts.net` search domain from a previous `tailscale up --accept-dns=true` run, otherwise macOS
# would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.


echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
reset_dns




echo "Checking Gateway Status..."
HIJACK_HOST=$(read_env_var GATEWAY_HIJACK_HOST | tr '[:upper:]' '[:lower:]')

if is_gateway_active; then
  echo -e "\n============================================="
  echo "Gateway is RUNNING. Stopping it now..."
  echo -e "=============================================\n"

  # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
  if [[ "$HIJACK_HOST" != "false" ]]; then
    disconnect_tailscale_host
    echo ""
  fi

  echo "Stopping Docker containers..."
  docker compose down --remove-orphans -t 30

  # Only stop Colima if the user explicitly opted in (false by default) to prevent breaking their other Docker dev projects
  STOP_COLIMA=$(read_env_var STOP_COLIMA_ON_EXIT | tr '[:upper:]' '[:lower:]')
  if [[ "$STOP_COLIMA" == "true" ]]; then
    echo -e "\nStopping Colima VM to free up host RAM and battery..."
    run_with_timeout 30 colima stop >> output.log 2>&1 || echo "Warning: Failed to stop Colima gracefully."
  else
    echo -e "\nLeaving Colima running. (Set STOP_COLIMA_ON_EXIT=true in .env to change this)"
  fi

  if [[ "$HIJACK_HOST" != "false" ]]; then
    # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
    # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
    # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
    echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)... "
    reset_dns

    # 3. Stop local DNS proxy if running
    stop_local_dns_proxy

    # Stop background daemons
```

```
    stop_pidfile_daemon "/tmp/nullexit-caffeinate.pid" "system sleep prevention"
    stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
    stop_warp_watcher
    clear_gateway_active_marker

    # 4. Nuke leftover network state so internet actually works after teardown
    cleanup_network_state
  fi

  ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
  echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
else
  START_GATEWAY=true
  echo "[$(date '+%Y-%m-%d %H:%M:%S')] Action: STARTUP GATEWAY" >> "$LOG_FILE"
  echo -e "\n============================================="
  echo "Gateway is STOPPED. Starting it now..."
  echo -e "=============================================\n"

  # 1. Prevent host exit-node deadlock during VM / Container startup
  if [[ "$HIJACK_HOST" != "false" ]]; then
    disconnect_tailscale_host
  fi

  # 2. Wait for physical DHCP lease to settle (crucial for restarts/wake-up/roaming)
  wait_for_dhcp_settle

  # 3. Boot Colima VM if it is not already running
  echo -e "\nChecking Colima VM status..."
  if ! run_with_timeout 15 colima status >> output.log 2>&1; then
    colima_network_mode="bridged"
    security_type=""
    if [[ "$OSTYPE" == "darwin"* ]]; then
      security_type=$(system_profiler SPAirPortDataType 2>/dev/null | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF; exit}')
      if echo "$security_type" | grep -qi "Enterprise"; then
        echo -e "\n  [!] WPA2-Enterprise Wi-Fi detected! Bridged mode will cause MAC-spoofing deauthentication."
        echo "      Falling back to '--network-mode shared' for Colima."
        colima_network_mode="shared"
      fi
    fi
    echo "Colima is not running. Starting Colima (600MB RAM allocation, vz VM, network address, $colima_network_mode)..."
    run_with_timeout 120 colima start --memory 0.6 --vm-type vz --network-address --network-mode "$colima_network_mode"
    echo "$colima_network_mode" > /tmp/nullexit_colima_mode.txt
  else
    echo "Colima is already running."
  fi

  # 3b. Auto-detect and fix Docker Subnet Collisions (e.g. if the user travels to a 172.17.x.x Wi-Fi)
  if [ -f "scripts/fix-docker-bridge-collision.sh" ]; then
    if bash scripts/fix-docker-bridge-collision.sh --dry | grep -q "collision detected"; then
      echo -e "\n[!] WARNING: Docker Subnet Collision Detected with your current Wi-Fi!"
      echo -e "    Auto-fixing Colima config to prevent internet jitter..."
      bash scripts/fix-docker-bridge-collision.sh --skip-toggle || true
    fi
  fi

  # 3c. Configure swap file inside the VM to prevent OOM on low-memory limits
  # We also set vm.swappiness=10 so the Linux kernel strictly prefers physical RAM
  # and avoids unnecessarily wearing out the SSD with proactive background swapping.
  if ! run_with_timeout 15 colima ssh -- grep -q 'swapfile' /proc/swaps >> output.log 2>&1; then
    echo "Configuring 400MB swap file inside the VM to prevent OOM..."
    run_with_timeout 30 colima ssh -- sudo sh -c "if [ ! -f /swapfile ]; then dd if=/dev/zero of=/swapfile bs=1M count=400 status=none && mkswap /swapfile; fi && swapon /swapfile && sysctl vm.swappiness=10" >> output.log 2>&1 || echo "Warning: Failed to enable swap file inside the VM."
  fi

  # 4. Clean up corrupted AdGuardHome configurations
  if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
    rm -f "adguard/conf/AdGuardHome.yaml"
    echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
  fi

  # 4b. Auto-heal stale Docker socket from hard reboots
  if ! run_with_timeout 15 docker ps >/dev/null 2>&1; then
    echo "Docker daemon is unresponsive (likely a stale socket from a crash). Auto-healing Colima..."
    run_with_timeout 120 colima restart >> output.log 2>&1
  fi

```

```
# 4c. Inject TCP MSS clamp from .env into post-rules.txt before starting
if [ -f .env ] && grep -q "^GATEWAY_MSS=" .env; then
  GATEWAY_MSS=$(read_env_var GATEWAY_MSS)
  if [[ "$GATEWAY_MSS" =~ ^[0-9]+$ ]]; then
    # macOS sed syntax
    sed -i '' "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null || \
    # Linux sed fallback
    sed -i "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null
  fi
fi

# 5. Start compose services
write_host_ips

# Procedurally generated ports have already been exported at the top of the script.
echo "  [Security] Procedurally randomized proxy ports generated to evade fingerprinting."

# --- HONEY-PORT TRIPWIRE ---
# Generate a random trap port. If any malware does a sequential port scan on localhost,
# it will inevitably hit this port. When it does, we instantly fire a macOS alert.
HONEY_PORT=$(get_free_port)
(
  if nc -l localhost "$HONEY_PORT" >/dev/null 2>&1; then
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown application just pinged the local Honey-Port ($HONEY_PORT)!" >> "$LOG_FILE"
    osascript -e 'display notification "An unknown application is aggressively scanning your local ports. Malware port-scan suspected." with title "nullexit OPSEC Alert" sound name "Basso"' 2>/dev/null || true
  fi
) &
disown %1 2>/dev/null || true
echo "  [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."

echo -e "\nStarting Docker containers..."
docker compose up -d --build

# Log the output of the rule compiler for debugging before removing it
docker compose logs rule-compiler >> output.log 2>&1
# Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
docker compose rm -s -f rule-compiler >> output.log 2>&1

RULE_COUNT=$(grep "! Total Block Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -F': ' '{print $2}' || echo "0")
NATIVE_COUNT=$(grep "! Native AdGuard Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -F': ' '{print $2}' || echo "0")

if [ "$RULE_COUNT" != "0" ] && [ "$RULE_COUNT" != "" ]; then
  if [ "$NATIVE_COUNT" != "0" ] && [ "$NATIVE_COUNT" != "" ]; then
    TOTAL_COUNT=$((RULE_COUNT + NATIVE_COUNT))
    # Format with commas for readability (e.g. 441,578)
    if command -v printf &> /dev/null; then
      FORMATTED_RULE=$(printf "%'d" "$RULE_COUNT")
      FORMATTED_TOTAL=$(printf "%'d" "$TOTAL_COUNT")
      echo "  DNS: Compiled $FORMATTED_RULE unique custom rules (Total active in AdGuard: ~$FORMATTED_TOTAL rules)."
    else
      echo "  DNS: Compiled $RULE_COUNT unique custom rules (Total active in AdGuard: ~$TOTAL_COUNT rules)."
    fi
  else
    echo "  DNS: Compiled and loaded $RULE_COUNT optimized rules."
  fi
fi

# Report IP blocklist compilation results
IP_COUNT=$(grep "^# Entries:" adguard/work/userfilters/ip_blocklist.ipset 2>/dev/null | awk -F': ' '{print $2}' || echo "0")
if [ "$IP_COUNT" != "0" ] && [ "$IP_COUNT" != "" ]; then
  if command -v printf &> /dev/null; then
    FORMATTED_IP=$(printf "%'d" "$IP_COUNT")
    echo "  IP:  Loaded $FORMATTED_IP threat intelligence IPs/CIDRs into kernel firewall."
  else
    echo "  IP:  Loaded $IP_COUNT threat intelligence IPs/CIDRs into kernel firewall."
  fi
fi

# 6. Wait for the gateway container's Tailscale connection to be ready
BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')

echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
```

```
  echo "  (This can take 30-60s  Tailscale goes through: NoState  Starting  Running  Online)"
  connected=false
  consecutive_failures=0
  for i in {1..60}; do
    # Run tailscale status and capture its output so the user can see what’s happening.
    # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
    ts_output=$(run_with_timeout 5 docker compose exec -T tailscale tailscale status 2>&1 || true)

    if echo "$ts_output" | grep -q "offers exit node"; then
      connected=true
      break
    fi

    # Track consecutive failures to detect a stuck state.
    # If we see 40+ consecutive ’NoState’ responses, give up early
    # the daemon is alive but can’t connect to the coordination server.
    if echo "$ts_output" | grep -q "NoState"; then
      consecutive_failures=$((consecutive_failures + 1))
      if [ "$consecutive_failures" -ge 40 ]; then
        echo ""
        echo "  [attempt $i/60] Stuck in ’NoState’ for $consecutive_failures checks."
        echo "    Tailscale daemon is running but can’t reach the coordination server."
        break
      fi
    else
      consecutive_failures=0
    fi

    # Print a condensed status every 10 seconds so the user can see progress
    if [ $((i % 10)) -eq 0 ]; then
      ts_short=$(echo "$ts_output" | head -1 | tr -d ’\r\n’)
      echo "  [attempt $i/60] $ts_short"
    elif [ $((i % 5)) -eq 0 ]; then
      echo -n "[$i]"
    else
      echo -n "."
    fi
    sleep 1
  done
  echo ""

  if [ "$connected" = "true" ]; then
    echo "Gateway container is online on the Tailscale mesh."
  elif [ "$BYPASS_PING" = "true" ]; then
    echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
  else
    echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
    echo "  The container was seen in the following states during the wait:" >&2
    echo "$ts_output" | sed ’s/ˆ/    /’ >&2
    echo "" >&2
    echo "  This could mean:" >&2
    echo "    1. Tailscale auth key has expired  check TS_AUTHKEY in .env" >&2
    echo "    2. Network issue inside the container  check: docker compose logs tailscale" >&2
    echo "    3. gluetun VPN tunnel not connecting  check: docker compose logs warp | tail -20" >&2
    echo "" >&2
    echo "  Diagnose manually:" >&2
    echo "    docker compose exec -T tailscale tailscale status" >&2
    echo "    docker compose exec -T tailscale tailscale netcheck" >&2
    cleanup_handler ERR $LINENO
    exit 1
  fi

  # 6b. Resolve the gateway’s Tailscale IP.
  # Dynamically query the container for the IP (handles IP changes after re-auth)
  # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
  #
  # CRITICAL: ‘docker compose exec -T‘ on macOS/Colima injects carriage
  # returns (\r) into captured output. If $TS_IP ends with \r, then
  # ‘networksetup -setdnsservers‘ silently rejects the malformed IP and
  # DNS stays at 1.1.1.1  the primary bug this project was hitting.
  # ‘tr -d ’\r’‘ strips the CR; ‘awk ’NR==1{print $1; exit}’‘ takes only
  # the first field of the first line.
  TS_IP=""
  if [ "$connected" = "true" ]; then
    TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d ’\r’ | awk ’NR==1{print $1; exit}’ || true)
    if [ -n "$TS_IP" ]; then
      echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
      echo "$TS_IP" > .gateway_ip
    else
```

```
      echo "    [!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2
      TS_IP=$(read_adguard_ip || true)
    fi
  else
    TS_IP=$(read_adguard_ip || true)
  fi

  # 7. Connect host Mac to Tailscale mesh.
  # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
  # system LaunchDaemon via 'brew services start tailscale'), the previous five-
  # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
  # no menu bar item to click, and no Accessibility permission required.
  #
  # CRITICAL: If tailscaled is not running, `tailscale up` will silently fail.
  # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
  # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
  # are impossible without the host on the mesh).
  #
  # We connect in two phases:
  #   Phase A  Join mesh WITHOUT exit node (so pre-flight checks can run)
  #   Phase B  Set exit node only after pre-flight checks confirm the gateway works
  HOST_ON_MESH=false
  SKIP_EXIT_NODE=false
  if [ "$HIJACK_HOST" = "false" ]; then
    echo -e "\n[Info] HIJACK_HOST is false (Headless Mode). Host networking will remain untouched."
    SKIP_EXIT_NODE=true
  elif [ -n "$TS_BIN" ]; then
    echo -n "Verifying tailscaled is reachable"
    daemon_ready=false
    status_exit=0
    for i in {1..10}; do
      # Test if the daemon responds to status check.
      # If status returns non-zero, check if it was a normal "stopped/disconnected" state
      # (exit code 1 is normal for disconnected). If the command exited quickly (not killed by timeout),
      # and the error is just "Tailscale is stopped", then the daemon is alive and ready.
      # But if it timed out (exit code 143) or is completely unresponsive, it's wedged.
      status_exit=0
      if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
        daemon_ready=true
        break
      else
        status_exit=$?
        if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
          daemon_ready=true
          break
        fi
      fi
      echo -n "."
      sleep 1
    done
    echo ""

    # If the daemon was unresponsive or socket check failed, attempt auto-restart
    if [ "$daemon_ready" != "true" ]; then
      if [ "$status_exit" -eq 143 ]; then
        warn "tailscaled daemon is wedged/unresponsive. Restarting it..."
      else
        warn "tailscaled daemon is not running. Attempting to start it..."
      fi
      restart_tailscaled_daemon

      # Re-verify
      if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
        daemon_ready=true
      fi
    fi

    if [ "$daemon_ready" = "true" ]; then
      echo "tailscaled is responsive (running as a system service)."

      #  Phase A: Join mesh without exit node
      echo "Connecting host to Tailscale mesh (no exit node yet)..."
      if $TS_BIN up; then
        $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node= || true
        HOST_ON_MESH=true
        echo "Host is on Tailscale mesh."
      else
        warn "tailscale up failed even though the daemon is running."
        echo "  This usually means the host hasn't authenticated with your tailnet yet."
        echo "  Run this command in a terminal to authenticate:"
        echo ""
```

```
        echo "      sudo tailscale up"
        echo ""
        echo "  A browser will open. Log in to your Tailscale account, then re-run toggle.sh."
      fi
    else
      fail "tailscaled could NOT be started automatically."
      echo "  Run this in a terminal to start and authenticate Tailscale:"
      echo ""
      echo "      sudo brew services start tailscale"
      echo "      sudo tailscale up"
      echo ""
      echo "  Then re-run toggle.sh."
    fi
  else
    echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
  fi

  #  Phase B: Pre-flight checks + enable exit node only if safe
  if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
    echo -e "\n"
    echo "Pre-flight connectivity check"
    echo ""

    # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
    # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
    # ("direct connection not established") even though the pong was received.
    # We check for ’pong’ in the output (stderr discarded) to accept relayed connections.
    echo -n "  [1/3] Gateway reachable via Tailscale... "
    ping_ok=false
    # The host tailscaled needs a few seconds to propagate the new network map and
    # establish a connection route after ’tailscale up --reset’. We retry up to 45 times.
    for attempt in {1..45}; do
      if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
        ping_ok=true
        break
      fi
      sleep 1
    done
    if [ "$ping_ok" = "true" ]; then
      echo "PASS"
    else
      echo "FAIL"
      SKIP_EXIT_NODE=true
    fi

    # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
    # Colima’s SSH tunnel only forwards TCP (not UDP), so we use +tcp.
    echo -n "  [2/3] AdGuard DNS via localhost... "
    if command -v dig &>/dev/null; then
      if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 &>/dev/null; then
        echo "PASS"
      else
        echo "FAIL"
        SKIP_EXIT_NODE=true
      fi
    elif command -v nslookup &>/dev/null; then
      if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 &>/dev/null; then
        echo "PASS"
      else
        echo "FAIL"
        SKIP_EXIT_NODE=true
      fi
    else
      echo "SKIP (dig/nslookup not available)"
    fi

    # Check 3: Does the WARP container have external internet?
    echo -n "  [3/3] WARP container internet... "
    tmp_err=$(mktemp)
    if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace >/dev/null 2>"$tmp_err"; then
      echo "PASS"
    else
      echo "FAIL"
      if [ -s "$tmp_err" ]; then
        err_msg=$(cat "$tmp_err" | tr -d ’\r’)
        echo "[$(date ’+%Y-%m-%d %H:%M:%S’)] [Check 3] WARP container check failed: $err_msg" >> output.log
      fi
      SKIP_EXIT_NODE=true
    fi
```

```
    rm -f "$tmp_err"

    # Act based on check results
    if [ "$SKIP_EXIT_NODE" != "true" ]; then
      echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
      $TS_BIN up || true
      if $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node="$TS_IP" --exit-node-allow-lan-access=true; then
        echo "Exit node enabled."
        add_warp_bypass_routes
        setup_exit_node_routing
        enable_killswitch
      else
        echo "[Warning] Failed to set exit node."
        SKIP_EXIT_NODE=true
      fi
    fi

    if [ "$SKIP_EXIT_NODE" = "true" ]; then
      warn "Pre-flight checks failed  EXIT NODE + DNS HIJACK SKIPPED."
      echo "  Host stays on Tailscale mesh but your internet routes through your normal connection."
      echo "  Troubleshoot with:"
      echo "    docker compose logs warp | tail -20"
      echo "    docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
      echo ""
      echo "  Falling back to SOCKS5 proxy for traffic routing..."
    fi
  elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
    echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
    SKIP_EXIT_NODE=true
  fi

  # 8. Apply DNS Hijacking.
  # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
  # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
  # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
  if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
    echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
    echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
    if force_dns_to_gateway "$TS_IP"; then
      echo "DNS hijack verified: single server = $TS_IP."
    else
      echo "[Warning] DNS hijack could not be verified."
      echo "  If DNS is broken, run manually:"
      echo "    networksetup -setdnsservers \"$ACTIVE_SERVICE\" $TS_IP"
      echo "    networksetup -setsearchdomains \"$ACTIVE_SERVICE\" ts.net"
    fi
  elif [ -n "$TS_IP" ] && [ "$HIJACK_HOST" != "false" ]; then
    echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
    echo "  Trying local DNS proxy for ad-blocking..."
    if start_local_dns_proxy; then
      echo "  Ad-blocking active via local DNS proxy through AdGuard."
    else
      echo "  Local DNS proxy unavailable. DNS stays at 1.1.1.1."
      echo "  AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
    fi

    # Enable SOCKS5 proxy for traffic routing through WARP
    # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
    echo -e "\n  Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
    echo "  (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
    enable_socks_proxy "$ACTIVE_SERVICE"

    # Verify the SOCKS5 proxy works through WARP
    echo -n "  Verifying SOCKS5 proxy routes through WARP... "
    if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null | grep -q "warp=on"; then
      echo "ok (warp=on)."
      echo "  All TCP traffic now encrypted through Cloudflare WARP tunnel."
    else
      echo "FAILED (proxy not routing through WARP)."
      echo "  Disabling SOCKS5 proxy  internet stays on direct connection."
      disable_socks_proxy
      echo "  SOCKS proxy available at localhost:$SOCKS_PROXY_PORT for manual configuration."
    fi
  else
    echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
  fi

  # 9. Verify host connectivity
```

```
  if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
    if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
      echo "Host is online on the Tailscale mesh."
    else
      echo "[Warning] Host is not responding on Tailscale."
    fi
  fi

  ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
  echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."

  # Prevent system from going to sleep while gateway is running
  start_sleep_prevention

  if [[ "$HIJACK_HOST" != "false" ]]; then
    if [ -n "$TS_IP" ] && [ "$TS_IP" != "1.1.1.1" ]; then
      start_dns_watcher "$TS_IP"
    fi

    # Start WARP liveness monitor (logs to output.log on warp flip)
    start_warp_watcher
  fi

  # Reset sharing services after network configuration to prevent AirDrop freezes
  echo "Resetting macOS sharing services (AirDrop/AirPlay)..."
  reset_sharing_services

  # Tell external watchers (post-wake / network-change) the gateway is up.
  write_gateway_active_marker

  #  Local Network Surveillance Check
  echo -e "\n"
  echo "Local Network Surveillance Check"
  echo ""
  # Count populated ARP cache entries to detect network scanning or high congestion
  # Using ’-an’ avoids hanging on DNS reverse resolution when the network state is in transition.
  ARP_COUNT=$(arp -an 2>/dev/null | grep -iv ’incomplete’ | wc -l | tr -d ’ ’)
  if [ "$ARP_COUNT" -gt 15 ]; then
    warn "High ARP activity detected ($ARP_COUNT devices in cache)."
    warn "This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)."
    echo -e "  [] Your traffic remains fully encrypted and invisible.\n"
  else
    echo -e "  [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
  fi
fi

SUCCESS_RUN=true

# Final DNS state summary
if [ -n "$ACTIVE_SERVICE" ]; then
  FINAL_DNS=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log || true)
  echo -e "\n"
  echo -e "DNS STATE: $FINAL_DNS"
  echo -e ""
fi

if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
  if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
    echo -e "\n[Warning] One or more blocklist URLs failed to download (404/Offline)."
    echo "  The gateway gracefully fell back to yesterday’s cached blocklist."
    echo "  (Run ’docker logs rule-compiler’ later to investigate which link died.)"
  fi
fi

rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
if [ -t 0 ]; then
  read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
  if [[ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]]; then
    echo "Reversing gateway state..."
    exec bash "$0"
  fi
fi

echo -e "\nYou can close this terminal window now."
```

7 recover.sh

```
#!/bin/bash
# recover.sh  nullexit KILL-SWITCH recovery script for macOS
# Fixes internet when toggle.sh leaves you stranded.
#
# This is a NUCLEAR option: it undoes EVERYTHING toggle.sh does by:
#   1. Disconnecting host Tailscale from the mesh (exit-node off)
#   2. Resetting DNS to default on ALL network services
#   3. Disabling rogue proxy settings (SOCKS, web, secure web)
#   4. Flushing macOS DNS cache
#   5. Flushing stale routing table entries
#   6. Stopping gateway Docker containers
#   7. Power-cycling Wi-Fi
#   8. Verifying internet is working
#
# Usage:  double-click Recover-Gateway.applescript
#         OR  ./recover.sh
#         OR  ./recover.sh --post-wake    (lighter-touch refresh, keeps gateway live)

set -e

#  Resolve script directory (used for path-relative refs)
# recover.sh lives at the repo root; SCRIPT_DIR lets us launch toggle.sh
# (and reference any other repo-root file) from the same directory no
# matter where the user invoked recover.sh from.
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cd "$SCRIPT_DIR" || exit 1

# Enforce Cryptographic Script Integrity
if [ -f "scripts/crypto.sh" ]; then
  if ! bash scripts/crypto.sh --verify; then
    exit 1
  fi
fi

#  Argument parsing
# --post-wake: lightweight refresh after sleep/wake or Wi-Fi roam.
#              Keeps the gateway live while HUPing DNS caches, refreshing the
#              Tailscale exit-node leak, re-hijacking host DNS, and force-
#              recreating the warp container if its gluetun healthcheck failed.
#              Does NOT tear down Docker, Tailscale, sleep prevention, or Wi-Fi.
#              Called from scripts/watcher.sh on every wake / network-change event
#              while /tmp/nullexit-gateway-active.marker is present (i.e. the
#              gateway is currently up). See devref.md 10.29 for the why.
POST_WAKE=false
AUTO_MODE=false
for arg in "$@"; do
  if [ "$arg" = "--post-wake" ]; then
    POST_WAKE=true
  elif [ "$arg" = "--auto" ] || [ "$arg" = "--non-interactive" ]; then
    AUTO_MODE=true
  fi
done

rm -f "$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker"

# Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
LOCK_FILE="/tmp/nullexit-toggle.lock"
if [ -f "$LOCK_FILE" ]; then
  LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
  if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
    if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
      if [ "$POST_WAKE" = "true" ]; then
        echo "[$(date -u +%FT%TZ)] POST-WAKE skipped: another lifecycle script (PID $LOCK_PID) is running." >> output.log
        exit 0
      else
        echo "Error: Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running." >&2
        exit 1
      fi
    fi
  fi
fi
echo "$$" > "$LOCK_FILE"

# Clear the active-state marker in nuclear recovery mode since the gateway is being
# completely torn down. This prevents scripts/watcher.sh from triggering post-wake
# recoveries in response to network changes caused by the teardown itself.
if [ "$POST_WAKE" = "false" ]; then
  rm -f "/tmp/nullexit-gateway-active.marker"
fi
```

```

# WARP Watcher inhibit marker: written immediately in --post-wake mode so the
# background WARP Watcher (in toggle.sh) skips failure counting while we are
# intentionally tearing down / recreating the warp container. Without this, the
# watcher accumulates 6 consecutive warp=off readings during force-recreate (~35s)
# and fires nuclear recover.sh, killing a gateway that would have been healthy.
WARP_INHIBIT_FILE="/tmp/nullexit-warp-inhibit.marker"
if [ "$POST_WAKE" = "true" ]; then
  touch "$WARP_INHIBIT_FILE"
  echo "[$(date -u +%FT%TZ)] POST-WAKE started  WARP Watcher inhibited to prevent spurious shutdown
    during warp recreate." >> output.log
fi

# Ensure the inhibit marker and lock file are always removed when the script exits (success or error)
cleanup_recover() {
  rm -f "$WARP_INHIBIT_FILE"
  if [ -f "$LOCK_FILE" ] && [ "$(cat "$LOCK_FILE" 2>/dev/null)" = "$$" ]; then
    rm -f "$LOCK_FILE"
  fi
}
trap cleanup_recover EXIT

# Source common formatting and helper functions
source "$SCRIPT_DIR/scripts/common.sh"

# Always add Homebrew path
setup_standard_path

echo ""
if [ "$POST_WAKE" = "true" ]; then
  echo -e "${YELLOW}${BOLD}${NC}"
  echo -e "${YELLOW}${BOLD}  Nullexit  POST-WAKE / POST-ROAM LIGHT ${NC}"
  echo -e "${YELLOW}${BOLD}  (keeps the gateway live, refreshes)    ${NC}"
  echo -e "${YELLOW}${BOLD}${NC}"
  echo ""
  echo "Re-hijacking DNS, refreshing Tailscale exit-node, force-recreating"
  echo "warp if unhealthy, and resetting sharing services. The gateway stays"
  echo "up. docker compose / Wi-Fi / caffeinate are NOT touched."
  echo ""
else
  echo -e "${RED}${BOLD}${NC}"
  echo -e "${RED}${BOLD}     Nullexit Gateway  RECOVERY MODE     ${NC}"
  echo -e "${RED}${BOLD}${NC}"
  echo ""
  echo "This script will tear down the gateway and restore normal internet."
  echo ""
fi

#  Cache sudo credentials upfront (default mode ONLY)
# Skip in --post-wake: the LaunchAgent-launched watcher has no TTY, so ‘sudo -v‘
# would block forever waiting for a password. All post-wake commands use ‘sudo -n‘
# which is non-blocking and relies on /etc/sudoers.d/nullexit NOPASSWD entries.
SUDO_KEEPER_PID=""
if [ "$POST_WAKE" = "false" ] && [ "$AUTO_MODE" = "false" ] && [ -t 0 ]; then
  echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
  sudo -v
  (
    while true; do
      sudo -n true 2>/dev/null
      sleep 60
    done
  ) &
  SUDO_KEEPER_PID=$!
  trap ’kill $SUDO_KEEPER_PID 2>/dev/null’ EXIT
fi

#  Resolve physical default interface and gateway upfront
PHYSICAL_IFACE=$(networksetup -listallhardwareports 2>> output.log | awk ’/Hardware Port: (Wi-Fi|Ethernet
    )/{getline; print $2; exit}’)
[ -z "$PHYSICAL_IFACE" ] && PHYSICAL_IFACE="en0"

if [ "$POST_WAKE" = "true" ]; then
  wait_for_dhcp_settle
else
  # Quick resolution in non-post-wake mode (non-blocking)
  PHYSICAL_GW=$(ipconfig getpacket "$PHYSICAL_IFACE" 2>> output.log | awk -F’[{}]’ ’/router /{print $2}’)
fi

#  1. Disconnect host Tailscale (default) / Refresh exit-node (post-wake)

if [ "$POST_WAKE" = "true" ]; then
```

```
  step "Refreshing host Tailscale exit-node routing (post-wake)"
  if command -v tailscale >> output.log 2>&1; then
    # Re-issue the SAME exit-node up command toggle.sh START uses. This refreshes
    # the DERP relay mapping and re-asserts the exit-node preference without
    # dropping the host’s mesh connection (which ‘tailscale down‘ would).
    TS_IP=""
    for src in .gateway_ip; do
      [ -f "$src" ] || continue
      TS_IP=$(cat "$src" 2>> output.log | tr -d ’\r’ | awk ’NR==1{print $1; exit}’ || true)
      [ -n "$TS_IP" ] && break
    done

    if [ -n "$TS_IP" ]; then
      step "Re-establishing exit node $TS_IP via ’tailscale set --exit-node’"
      # ‘tailscale set‘ only mutates the named preference (exit-node here).
      # Crucially it does NOT call --reset, which would re-apply ALL flags and
      # trigger a sub-second route-table re-evaluation cycle each post-wake.
      # On already-connected nodes, the lighter ‘set‘ keeps the existing tunnel
      # alive while only changing the exit-node preference.
      if run_with_timeout 15 tailscale set --exit-node="$TS_IP" \
                             --exit-node-allow-lan-access=true \
                             >> output.log 2>&1; then
        ok "Exit-node $TS_IP re-asserted via ’tailscale set’"
      else
        warn "tailscale set --exit-node=$TS_IP didn’t respond; falling back to mesh-only"
        run_with_timeout 10 tailscale set --exit-node= \
                           >> output.log 2>&1 || true
      fi
    else
      warn ".gateway_ip missing  cannot re-assert exit node"
      run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= \
                         >> output.log 2>&1 || true
    fi
  else
    warn "tailscale CLI not found"
  fi
else
  step "Disconnecting host Tailscale from mesh"
  disconnect_tailscale_host "tailscale"
fi

#  2. Reset DNS to default on ALL network services (default) / Re-hijack gateway DNS (post-wake)
if [ "$POST_WAKE" = "true" ]; then
  step "Re-hijacking host DNS to gateway IP (post-wake / post-roam)"
  TS_IP=""
  for src in .gateway_ip; do
    [ -f "$src" ] || continue
    TS_IP=$(cat "$src" 2>> output.log | tr -d ’\r’ | awk ’NR==1{print $1; exit}’ || true)
    [ -n "$TS_IP" ] && break
  done

  if [ -n "$TS_IP" ]; then
    # Detect the active network service (using helper in common.sh)
    ACTIVE_SVC=$(get_active_service)
    EN0_SVC=$(get_en0_service)

    networksetup -setsearchdomains "$ACTIVE_SVC" "ts.net" 2>> output.log || true
    networksetup -setdnsservers "$ACTIVE_SVC" "$TS_IP" 2>> output.log || true
    networksetup -setsearchdomains "$EN0_SVC" "ts.net" 2>> output.log || true
    networksetup -setdnsservers "$EN0_SVC" "$TS_IP" 2>> output.log || true

    HITS=$(networksetup -getdnsservers "$ACTIVE_SVC" 2>> output.log | grep -Fx "$TS_IP" || true)
    if [ -n "$HITS" ]; then
      ok "DNS re-hijacked to $TS_IP on services: $ACTIVE_SVC, $EN0_SVC"
    else
      warn "networksetup accepted but DNS read-back didn’t include $TS_IP"
    fi
  else
    warn ".gateway_ip missing  cannot re-hijack DNS (user must run toggle.sh START)"
  fi
else
  step "Resetting DNS to default on all network services (empty = DHCP)"

  # Get ALL network services (handles spaces in names, skips disabled ones)
  SERVICES=$(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$" || true)

  if [ -z "$SERVICES" ]; then
    # Fallback to common names
    SERVICES="Wi-Fi Ethernet Thunderbolt Ethernet USB 10/100 LAN"
  fi
```

```

  RESET_COUNT=0
  while IFS= read -r service; do
    # Strip leading asterisk (disabled services)
    service_clean=$(echo "$service" | sed 's/^\*//')
    [ -z "$service_clean" ] && continue

    # Check if this service has a hardware port (skip VPN/service-only entries)
    if networksetup -listallhardwareports 2>> output.log | grep -A1 "Port: $service_clean$" | grep -q "Device:" || [ "$service_clean" = "Wi-Fi" ]; then
      (
        networksetup -setsearchdomains "$service_clean" "Empty" 2>> output.log
        # Set to "empty" so macOS uses DHCP-assigned DNS (not hardcoded 1.1.1.1)
        sudo networksetup -setdnsservers "$service_clean" "empty" 2>> output.log
      ) && RESET_COUNT=$((RESET_COUNT + 1)) || true
    fi
  done <<< "$SERVICES"

  ok "DNS reset on $RESET_COUNT service(s)"
fi

#  3. Disable rogue proxy settings (default only  gateway uses proxy in non-exit-node path)
if [ "$POST_WAKE" = "false" ]; then
  step "Disabling proxy settings (SOCKS, web, secure web)"
  for svc in $(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$" || echo "Wi-Fi"); do
    svc_clean=$(echo "$svc" | sed 's/^\*//')
    [ -z "$svc_clean" ] && continue
    disable_all_proxies "$svc_clean"
  done
  ok "Proxy settings cleared"
else
  step "Proxy settings: leaving untouched (post-wake keeps gateway SOCKS wiring alive)"
fi

#  4. Flush macOS DNS cache (always  safe and useful in both modes)
step "Flushing DNS cache"
if command -v dscacheutil >> output.log 2>&1; then
  flush_dns_cache
  ok "DNS cache flushed (mDNSResponder HUP)"
else
  warn "dscacheutil not found"
fi

#  5. Clear stale routing table (always  safe in both modes)
step "Flushing stale routing table entries"
# Resolve physical default gateway IP before flushing
# We cannot trust `route get default` here because Tailscale may have already hijacked
# the default route to utunX. We must read the actual DHCP router assignment from the hardware.
# PHYSICAL_IFACE and PHYSICAL_GW resolved upfront

# Instead of full route -n flush (which destroys macOS loopback/multicast and wedges the OS until reboot),
# we cleanly delete the Tailscale overrides and Cloudflare WARP bypass routes in ALL modes.
# This ensures tailscaled isn't blocked from reaching its control plane when waking from sleep.
sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
remove_warp_bypass_routes
ok "Routing overrides cleared"

if [ "$POST_WAKE" = "false" ]; then
  disable_killswitch
  ok "Routing table flush completed via targeted deletes and firewall disabled"
else
  ok "Routing overrides cleared (post-wake mode to preserve Colima bridge100)"
fi

# In post-wake mode we don't bounce Wi-Fi, so we MUST manually restore the physical default route
if [ "$POST_WAKE" = "true" ] && [ -n "$PHYSICAL_GW" ]; then
  # Ensure physical gateway is set just in case it was dropped
  sudo -n route add default "$PHYSICAL_GW" >> output.log 2>&1 || true
fi

# CONDITIONAL BYPASS ROUTE RESTORATION:
# If this was a post-wake recovery and the exit node is active, the route flush
# just wiped the static bypass routes for the WARP endpoints. We must re-add
# them immediately to prevent a routing loop when Tailscale starts sending traffic.
if [ "$POST_WAKE" = "true" ]; then
  if [ -z "${ACTIVE_IF:-}" ]; then
    ACTIVE_IF=$(route get default 2>> output.log | awk '/interface:/{print $2; exit}')
    if [ -z "$ACTIVE_IF" ] || [[ "$ACTIVE_IF" =~ ^utun ]] || [[ "$ACTIVE_IF" =~ ^tun ]]; then
```

```
      for i in en0 en1 en2 en3; do
        if ifconfig "$i" 2>> output.log | grep -q 'status: active'; then
          ACTIVE_IF="$i"; break
        fi
      done
    fi
    [ -z "$ACTIVE_IF" ] && ACTIVE_IF="en0"
  fi

  echo "Re-adding host bypass routes for Cloudflare WARP endpoints..."
  remove_warp_bypass_routes
  if [ -n "${PHYSICAL_GW:-}" ]; then
    add_warp_bypass_routes "$PHYSICAL_GW"
  else
    add_warp_bypass_routes "$ACTIVE_IF"
  fi

  # Also re-apply the default route pointing to the Tailscale interface
  ts_iface=$(ifconfig 2>> output.log | grep -B4 "inet 100." | grep -E '^[a-z0-9]+' | cut -d: -f1 | head -n 1)
  if [ -n "$ts_iface" ]; then
    echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
    # DO NOT delete the physical default route! It crashes Colima.
    # Use the /1 trick to mathematically override the default route.
    sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
    sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
    sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
    sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
    enable_killswitch
  fi
fi

#  6. Stop gateway Docker containers (default) / Force-recreate warp if unhealthy (post-wake)
if [ "$POST_WAKE" = "true" ]; then
  #  6a. Check for Roaming Network Mismatch (Bridged vs Enterprise Wi-Fi)
  if [ -f "/tmp/nullexit_colima_mode.txt" ] && [ -f "$SCRIPT_DIR/../.lan_p2p_detected" ]; then
    current_colima_mode=$(cat /tmp/nullexit_colima_mode.txt)
    p2p_allowed=$(cat "$SCRIPT_DIR/../.lan_p2p_detected")
    required_colima_mode="bridged"
    [ "$p2p_allowed" == "false" ] && required_colima_mode="shared"

    if [ "$current_colima_mode" != "$required_colima_mode" ]; then
      warn "Network roaming mismatch detected!"
      warn "Colima is running in '$current_colima_mode' mode but new network requires '$required_colima_mode'."
      warn "Executing full gateway restart to protect against MAC-spoofing deauthentication..."
      echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Roam] Mismatch ($current_colima_mode vs $required_colima_mode). Triggering toggle.sh --restart." >> output.log
      nohup bash "$SCRIPT_DIR/../toggle.sh" --restart >/dev/null 2>&1 &
      exit 0
    fi
  fi

  step "Checking Colima VM state"
  colima_status=$(colima status 2>&1)
  if echo "$colima_status" | grep -qi "running"; then
    ok "Colima VM is running"
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima status check passed." >> output.log
  else
    warn "Colima VM is NOT running or unresponsive"
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima is unhealthy or dead! Output: $colima_status" >> output.log
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Attempting to restart Colima..." >> output.log
    colima restart >> output.log 2>&1 || warn "Failed to restart Colima"
  fi

  step "Checking warp container health (gluetun UDP tunnel)"
  echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Inspecting warp container state..." >> output.log
  # The warp container is the most failure-prone link in the chain at wake/roam:
  # its UDP NAT binding to Cloudflare's edge (162.159.192.1:2408) silently dies
  # during a Wi-Fi roam. We verify the tunnel is actually passing traffic via curl.
  WARP_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
  echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container status: $WARP_STATE" >> output.log

  if [ "$WARP_STATE" = "missing" ]; then
    warn "warp container is missing  leaving gateway containers alone (full relaunch requires \"$SCRIPT_DIR/toggle.sh\")"
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container missing from Docker engine. Requires full toggle.sh launch." >> output.log
  else
    tmp_err=$(mktemp)
```

```
    if docker compose exec -T warp wget -qO- --timeout=3 https://www.cloudflare.com/cdn-cgi/trace 2>"$tmp_err" | grep -q "warp=on"; then
      ok "warp container is healthy and actively tunneling traffic (no recreate needed)"
      echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container passed live traffic healthcheck (Cloudflare trace successful)." >> output.log
    else
      warn "warp container tunnel is dead (Docker status: $WARP_STATE)  force-recreating all gateway containers to restore network namespace"
      echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container is DEAD or STUCK. Forcing recreation of gateway stack..." >> output.log
      if [ -s "$tmp_err" ]; then
        err_msg=$(cat "$tmp_err" | tr -d '\r')
        echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Healthcheck failure details: $err_msg" >> output.log
      fi
      docker compose up -d --force-recreate >> output.log 2>&1 || warn "force-recreate failed"

      NEW_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
      ok "warp container health after recreate: $NEW_STATE"
      echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container health after recreation: $NEW_STATE" >> output.log
    fi
    rm -f "$tmp_err"
  fi
else
  step "Stopping gateway Docker containers"
  if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
    if [ -f "docker-compose.yml" ]; then
      docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
    else
      warn "docker-compose.yml not found in current directory"
    fi
  else
    warn "Docker not available  skipping"
  fi
fi

#  6b. Stopping sleep prevention (caffeinate)  default only
if [ "$POST_WAKE" = "false" ]; then
  step "Stopping sleep prevention"
  stop_pidfile_daemon "$PID_CAFFEINATE" "system sleep prevention"
else
  step "Sleep prevention: leaving untouched (caffeinate already re-acquired by parent shell)"
fi

#  6c. Stopping DNS Watcher  default only
if [ "$POST_WAKE" = "false" ]; then
  step "Stopping DNS Watcher"
  stop_pidfile_daemon "$PID_DNS_WATCHER" "background DNS Watcher"
else
  step "DNS Watcher: leaving untouched (it keeps re-hijacking DNS on every roam)"
fi

#  7. Power-cycle Wi-Fi  default only
if [ "$POST_WAKE" = "false" ]; then
  step "Power-cycling Wi-Fi"
  bounce_wifi_interfaces
else
  step "Wi-Fi: leaving untouched (post-wake keeps the current Wi-Fi association)"
fi

#  8. Verify (default checks direct internet; post-wake verifies the gateway)
if [ "$POST_WAKE" = "true" ]; then
  step "Verifying gateway is healthy after post-wake refresh"

  # Wait a moment for tailscale + warp healthcheck to settle
  sleep 3

  GATEWAY_OK=false
  if command -v dig >> output.log 2>&1; then
    # Source procedural ports if available
    if [ -f "/tmp/nullexit-ports.env" ]; then
      source "/tmp/nullexit-ports.env"
    fi
    DNS_PORT=${DNS_PROXY_PORT:-5354}

    # AdGuard is exposed at localhost:${DNS_PORT}. If this resolves, the warp tunnel
    # is functioning properly because AdGuard routes upstream to warp.
    if dig +tcp @127.0.0.1 -p "$DNS_PORT" google.com +short +timeout=5 >> output.log 2>&1; then
```

```
      ok "Gateway DNS (AdGuard via localhost:${DNS_PORT}) works"
      GATEWAY_OK=true
    else
      warn "Gateway DNS not responding yet"
    fi
  fi

  # Direct host  gateway check via Tailscale ping (relayed pong counts as success)
  if command -v tailscale >> output.log 2>&1; then
    TS_IP=""
    [ -f .gateway_ip ] && TS_IP=$(read_adguard_ip || true)
    if [ -n "$TS_IP" ] && tailscale ping --until-direct=false -c 1 --timeout 4s "$TS_IP" 2>> output.log | grep -q pong; then
      ok "Gateway $TS_IP reachable via Tailscale"
      GATEWAY_OK=true
    else
      warn "Tailscale ping to gateway did not pong  exit-node may still be re-establishing"
    fi
  fi

  if [ "$GATEWAY_OK" = "true" ]; then
    echo -e "  ${GREEN}${BOLD} Gateway is healthy after post-wake refresh.${NC}"
  else
    echo -e "  ${YELLOW}${BOLD} Gateway recovery is in-progress.${NC}"
    echo "    The route may need another 30s before queries succeed."
    echo "    If it stays broken for >60s, run: bash \"$SCRIPT_DIR/toggle.sh\""
  fi
else
  step "Verifying internet connectivity"

  # Wait a moment for the network to settle
  sleep 2

  INTERNET_OK=false

  # Try DNS resolution first
  if host -W 3 google.com 1.1.1.1 >> output.log 2>&1; then
    ok "DNS resolution works (google.com via 1.1.1.1)"
    INTERNET_OK=true
  elif nslookup google.com 1.1.1.1 >> output.log 2>&1; then
    ok "DNS resolution works (nslookup google.com via 1.1.1.1)"
    INTERNET_OK=true
  else
    warn "DNS resolution check failed  will still try ping/curl"
  fi

  # Try actual HTTP connectivity
  if command -v curl >> output.log 2>&1; then
    if curl -sf --max-time 5 https://1.1.1.1 >> output.log 2>&1; then
      ok "Internet reachable via HTTP"
      INTERNET_OK=true
    elif curl -sf --max-time 5 http://1.1.1.1 >> output.log 2>&1; then
      ok "Internet reachable via HTTP (plain)"
      INTERNET_OK=true
    else
      warn "HTTP check failed"
    fi
  elif command -v ping >> output.log 2>&1; then
    if ping -c 1 -W 3 1.1.1.1 >> output.log 2>&1; then
      ok "Internet reachable via ping"
      INTERNET_OK=true
    else
      warn "Ping check failed"
    fi
  fi
fi

#  Summary
echo ""
echo -e "${BOLD}${NC}"
if [ "$POST_WAKE" = "true" ]; then
  echo -e "${BOLD}POST-WAKE SUMMARY${NC}"
else
  echo -e "${BOLD}RECOVERY SUMMARY${NC}"
fi
echo -e "${BOLD}${NC}"

# Show final DNS state
FINAL_DNS=$(networksetup -getdnsservers "Wi-Fi" 2>/dev/null || echo "N/A")
echo "  DNS (Wi-Fi):  $FINAL_DNS"

```

```
FINAL_DNS2=$(networksetup -getdnsservers "Ethernet" 2>/dev/null || true)
if [[ -z "$FINAL_DNS2" || "$FINAL_DNS2" == *"not a recognized"* || "$FINAL_DNS2" == *"Error"* ]]; then
  FINAL_DNS2="N/A (Not configured)"
fi
echo "  DNS (Ethernet): $FINAL_DNS2"

echo ""

if [ "$POST_WAKE" = "false" ]; then
  if [ "$INTERNET_OK" = "true" ]; then
    echo -e "  ${GREEN}${BOLD} Internet is working.${NC}"
  else
    echo -e "  ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
    echo "    This might mean:"
    echo "    - Your Wi-Fi is disconnected from the router"
    echo "    - Tailscale is still interfering (try restarting tailscaled:"
    echo "        brew services restart tailscale)"
    echo "    - You need to re-connect to Wi-Fi in the menu bar"
    echo "    - Try toggling Wi-Fi off/on in the menu bar"
    echo ""
    echo "    Or try manually:"
    echo "      networksetup -setdnsservers Wi-Fi empty"
    echo "      tailscale down"
    echo "      sudo route delete -net 0.0.0.0/1"
    echo "      sudo route delete -net 128.0.0.0/1"
    echo "      brew services restart tailscale"
  fi
fi

# Reset sharing services to prevent AirDrop freezes after interface changes
# This is the SAME step in BOTH modes (post-wake inherits the previous fix from 10.27).
echo -e "  Resetting macOS sharing services (AirDrop/AirPlay)..."
reset_sharing_services

echo ""
if [ "$POST_WAKE" = "true" ]; then
  echo -e "${YELLOW}${BOLD}Post-wake refresh complete.${NC}"
  # Remove the WARP Watcher inhibit marker now that the gateway is fully healthy.
  # The watcher will resume normal liveness monitoring on its next 5s poll.
  rm -f "$WARP_INHIBIT_FILE"
  echo "[$(date -u +%FT%TZ)] POST-WAKE complete  WARP Watcher inhibit released." >> output.log
else
  echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
fi
echo ""
```

8 setup.sh

```
#!/bin/bash
# setup.sh  nullexit one-shot setup script (macOS)
# Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
set -euo pipefail

# Source common formatting and helper functions
source "$(dirname "$0")/scripts/common.sh"

# Run common installation logic
source "$(dirname "$0")/scripts/setup-common.sh"

#  Compile macOS Shortcuts
echo -e "\n${BOLD}Compiling macOS Desktop Shortcuts...${NC}"
if command -v osacompile &> /dev/null; then
    osacompile -o "Toggle Gateway.app" "Toggle-Gateway.applescript" >/dev/null 2>&1
    osacompile -o "Recover Gateway.app" "Recover-Gateway.applescript" >/dev/null 2>&1
    echo "   Created 'Toggle Gateway.app' and 'Recover Gateway.app'"
fi

#  Done
echo ""
echo -e "${GREEN}${BOLD}${NC}"
echo -e "${GREEN}${BOLD}            Setup complete!                   ${NC}"
echo -e "${GREEN}${BOLD}${NC}"
echo ""
echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
echo "   https://login.tailscale.com/admin/machines"
echo "   Find your gateway  '...'  'Edit route settings'  enable Exit Node"
```

```
echo ""

echo -e "${YELLOW}${BOLD}Sudoers / Background Execution Requirements:${NC}"
echo "  To allow silent, non-interactive execution of the firewall and network routes (especially during sleep/wake recovery), you must configure passwordless sudo."
echo "  Run this exact command in your terminal:"
echo "    echo \"\$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /usr/bin/dscacheutil, /usr/bin/killall, /usr/bin/pkill, /bin/kill, /sbin/route, /sbin/ifconfig, /usr/bin/true, /opt/homebrew/bin/brew, /usr/local/bin/brew, /usr/bin/python3 -I \$PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3 -I \$PWD/scripts/dns-proxy.py, /usr/local/bin/python3 -I \$PWD/scripts/dns-proxy.py\" | sudo tee /etc/sudoers.d/nullexit"
echo ""

if [[ -n "${TS_IP:-}" ]]; then
    echo -e "${BOLD}AdGuard Home dashboard:${NC}  http://${TS_IP}:3000  (username: admin)"
    echo ""
fi

echo -e "${BOLD}To toggle the gateway on/off:${NC}"
echo "  macOS:   double-click the ’Toggle Gateway’ app icon!"
echo "  Windows: double-click Toggle-Gateway.bat"
echo ""

if [[ "$OSTYPE" == "darwin"* ]]; then
    # LuLu localhost filtering check
    if [ -d "/Applications/LuLu.app" ] || systemextensionsctl list 2>/dev/null | grep -q "com.objective-see.lulu"; then
        echo -e "${YELLOW}${BOLD}LuLu Firewall Detected:${NC}"
        echo "  LuLu’s localhost (loopback) filtering is OFF by default. To properly protect the"
        echo "  local nullexit proxy ports from malicious local processes, you must manually enable"
        echo "  ’Allow Loopback’ (or block it selectively) in LuLu’s Preferences -> Rules."
        echo ""
    fi

    echo -e "${YELLOW}${BOLD}Note on macOS Tailscale Sandboxing:${NC}"
    echo "  The Tailscale GUI app (tailscale-app) installed on macOS is sandboxed."
    echo "  While you cannot run a Tailscale SSH server on this Mac host, you can"
    echo "  still SSH outbound to other devices on the mesh using their MagicDNS"
    echo "  addresses directly (e.g. ssh user@device.tailnet-name.ts.net)."
    echo ""
fi
```

9 scripts/toggle-linux.sh

```
#!/bin/bash
# toggle-linux.sh  nullexit gateway toggle script for Linux
# Automatically handles Docker containers, Colima, DNS hijacking, and Tailscale exit node routing.

set -e

# --- PROCEDURAL PORT GENERATION ---
# To prevent static port fingerprinting by malware, we randomize exposed ports
# on every boot and persist them to a hidden environment file.
PORTS_FILE="/tmp/nullexit-ports.env"
if [[ "$1" == "" || "$1" == "--restart" || "$1" == "restart" ]]; then

  # Collision-avoidance function
  GENERATED_PORTS=""
  get_free_port() {
    local port
    while true; do
      # Linux uses shuf or $RANDOM instead of jot
      port=$(shuf -i 10000-65000 -n 1 2>/dev/null || echo $((10000 + RANDOM % 55000)))
      # 1. Prevent self-collision within this loop
      if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
        continue
      fi
      # 2. Prevent collision with ANY process on the machine (root daemons, terminal apps, etc)
      # We use ss to read the kernel socket table directly.
      if ! ss -lnt | awk ’{print $4}’ | grep -q ":$port$"; then
        echo "$port"
        return
      fi
    done
  }

```

```
  export TOR_SOCKS_PORT=$(get_free_port)
  GENERATED_PORTS="$GENERATED_PORTS $TOR_SOCKS_PORT"

  export TOR_CONTROL_PORT=$(get_free_port)
  GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"

  export SOCKS_PROXY_PORT=$(get_free_port)
  GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"

  export DNS_PROXY_PORT=$(get_free_port)

  echo "export TOR_SOCKS_PORT=$TOR_SOCKS_PORT" > "$PORTS_FILE"
  echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
  echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
  echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
elif [ -f "$PORTS_FILE" ]; then
  # On STOP, source the existing ports so docker-compose down matches
  source "$PORTS_FILE"
else
  # Fallbacks if file is lost
  export TOR_SOCKS_PORT=9050
  export TOR_CONTROL_PORT=9051
  export SOCKS_PROXY_PORT=1080
  export DNS_PROXY_PORT=5354
fi

# Start execution timer
TOGGLE_START_TIME=$SECONDS

if [[ "$1" == "restart" ]] || [[ "$1" == "--restart" ]]; then
  echo "Executing Gateway Restart Sequence..."
  # We must source common.sh here temporarily to check is_gateway_active before we proceed
  source "$(dirname "${BASH_SOURCE[0]}")/common.sh"
  if is_gateway_active; then
    echo "Gateway is currently running. Stopping it first..."
    bash "$0"
    echo "Gateway stopped. Now starting it..."
  else
    echo "Gateway is already stopped. Starting it..."
  fi
  bash "$0"
  exit 0
fi

# Global flag to track if the script completed successfully
SUCCESS_RUN=false

# Global variable to track the currently running background command PID
CURRENT_BG_PID=""

# Source common bash functions
source "$(dirname "${BASH_SOURCE[0]}")/common.sh"


PID_FILE="/tmp/nullexit-caffeinate.pid"

# Start macOS caffeinate to prevent sleep while gateway is running
start_sleep_prevention() {
  if ! command -v systemd-inhibit >/dev/null 2>&1; then
    echo "  [Warning] systemd-inhibit tool not found. Sleep prevention unavailable."
    return 1
  fi

  # Stop any existing sleep prevention process first to avoid duplicates
  stop_sleep_prevention

  echo "  Enabling system sleep prevention (systemd-inhibit)..."
  # Run systemd-inhibit wrapped in a bash trap to prevent idle system sleep.
  # Critically, this traps shutdown signals (SIGTERM) and automatically
  # flushes hijacked DNS back to normal right before the PC powers off.
  # We do not use recover-linux.sh here because it is a heavy recovery script
  # which takes too long and gets terminated before it can reset the DNS. Instead,
  # we surgically and instantly restore normal DNS settings here.
  nohup bash -c "
    trap ’
      echo \"[Shutdown Trap] Reverting network settings...\" >> \"$PWD/output.log\" 2>&1
      kill \$! 2>/dev/null || true
      sudo -n resolvectl domain \"$ACTIVE_SERVICE\" \"\" >> \"$PWD/output.log\" 2>&1 || true
      sudo -n resolvectl revert \"$ACTIVE_SERVICE\" >> \"$PWD/output.log\" 2>&1 || true
      resolvectl domain \"$ACTIVE_SERVICE\" \"\" >> \"$PWD/output.log\" 2>&1 || true
      resolvectl revert \"$ACTIVE_SERVICE\" >> \"$PWD/output.log\" 2>&1 || true
```

```
      if command -v tailscale >/dev/null 2>&1; then
        tailscale up --accept-dns=false --exit-node= >> \"$PWD/output.log\" 2>&1 &
        TS_DOWN_ARGS=\"\"
        if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS=\"--accept-risk=lose-ssh\"; fi
        tailscale down \$TS_DOWN_ARGS >> \"$PWD/output.log\" 2>&1 &
      fi
      sleep 1
      echo \"[Shutdown Trap] Cleanup complete.\" >> \"$PWD/output.log\" 2>&1
      exit 0
    ’ SIGTERM SIGINT SIGHUP
    systemd-inhibit --what=sleep --who=nullexit --why=\"gateway running\" --mode=block sleep infinity &
    wait \$!
  " >> output.log 2>&1 &
  local caffe_pid=$!
  echo "$caffe_pid" > "$PID_FILE"
  echo "  Sleep prevention active (PID $caffe_pid). Your PC won’t sleep while the gateway is running."
}

# Stop sleep prevention when gateway is stopped
stop_sleep_prevention() {
  if [ -f "$PID_FILE" ]; then
    local caffe_pid
    caffe_pid=$(cat "$PID_FILE")
    if [ -n "$caffe_pid" ] && kill -0 "$caffe_pid" 2>/dev/null && ps -p "$caffe_pid" -o command= 2>/dev/null | grep -q -E "systemd-inhibit|recover-linux.sh"; then
      echo "  Stopping system sleep prevention (systemd-inhibit wrapper PID $caffe_pid)..."
      kill "$caffe_pid" 2>/dev/null || true
    fi
    rm -f "$PID_FILE"
  fi
}

DNS_WATCHER_PID_FILE="/tmp/nullexit-dns-watcher.pid"

start_dns_watcher() {
  local target_ip=$1
  stop_dns_watcher
  echo "  Starting background DNS Watcher for seamless Wi-Fi roaming..."
  nohup bash -c "
    trap ’exit 0’ SIGTERM SIGINT SIGHUP
    while true; do
      if [ -n \"\$ACTIVE_SERVICE\" ]; then
        CURRENT_DNS=\$(resolvectl dns \"\$ACTIVE_SERVICE\" 2>/dev/null | grep -oE ’[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+’ | head -1)
        if [ \"\$CURRENT_DNS\" != \"$target_ip\" ]; then
          resolvectl dns \"\$ACTIVE_SERVICE\" \"$target_ip\" >/dev/null 2>&1 || true
        fi
      fi
      sleep 30
    done
  " >> output.log 2>&1 &
  echo $! > "$DNS_WATCHER_PID_FILE"
}

stop_dns_watcher() {
  if [ -f "$DNS_WATCHER_PID_FILE" ]; then
    local watcher_pid
    watcher_pid=$(cat "$DNS_WATCHER_PID_FILE")
    if [ -n "$watcher_pid" ] && kill -0 "$watcher_pid" 2>/dev/null; then
      echo "  Stopping background DNS Watcher (PID $watcher_pid)..."
      kill -9 "$watcher_pid" 2>/dev/null || true
    fi
    rm -f "$DNS_WATCHER_PID_FILE"
  fi
}

# Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
cleanup_handler() {
  local exit_code=$?
  local trigger_type="$1"
  local line_no="$2"

  if [ "$SUCCESS_RUN" = "false" ]; then
    echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
    if [ -n "$ACTIVE_SERVICE" ]; then
      reset_dns
    fi

    if [ -n "$CURRENT_BG_PID" ]; then
      echo "Terminating active command (PID $CURRENT_BG_PID)..."
```

```
      kill -15 "$CURRENT_BG_PID" 2>> output.log || true
    fi

    # Disconnect host Tailscale and close the GUI application on failure
    disconnect_tailscale_host

    # Stop local DNS proxy if running
    stop_local_dns_proxy

    # Stop sleep prevention
    stop_sleep_prevention
    stop_dns_watcher

    # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
    if [ -n "$ACTIVE_SERVICE" ]; then
      cleanup_network_state
    fi


    if [ "$trigger_type" = "ERR" ]; then
      echo -e "\n=============================================="
      echo "ERROR: Script failed at line $line_no with exit code $exit_code."
      echo "=============================================="
      echo "Troubleshooting steps:"
      echo "1. Run ’colima status’ to verify the VM is active."
      echo "2. Run ’docker compose ps’ to check container health."
      echo "3. Run ’docker compose logs’ to view application logs."
      echo "4. Run ’tailscale status’ to check host Tailscale status."
      echo "=============================================="
    fi
  fi
}

trap ’cleanup_handler ERR $LINENO’ ERR
trap ’cleanup_handler INT’ INT
trap ’cleanup_handler TERM’ TERM
trap ’cleanup_handler HUP’ HUP

#  NOPASSWD sudo
# The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
# needed (networksetup, dnsmasq, dscacheutil, killall, pkill, socat).
# All privileged calls below use ‘sudo -n‘ which will run without prompting.
# No credential caching loop is needed.

# Add Homebrew and standard paths
export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"

# Check for local DNS proxy tools (used when tailnet data plane is unavailable)
# Uses a Python DNS proxy (handles TCP wire format correctly).
# Python3 is built into macOS  no external dependencies.
DNS_PROXY_BIN=""
if command -v python3 >/dev/null 2>&1; then
  DNS_PROXY_BIN="python3 -I"
elif command -v python >/dev/null 2>&1; then
  DNS_PROXY_BIN="python -I"
fi

# Path to the DNS proxy script (sibling of this script)
DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/dns-proxy.py"

# Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
# (‘brew install tailscale‘ + ‘systemctl start tailscaled‘)  NOT the
# Tailscale.app GUI  so there is no .app bundle or Network Extension sandbox
# to launch, no menu-bar click to perform, and no scutil Network Service to
# start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
# if you ran the install with sudo) and the control socket is always available.
TS_BIN=""
if command -v tailscale >> output.log 2>&1; then
  TS_BIN="tailscale"
fi

# Baseline: disable Tailscale DNS management at the daemon level.
# ‘tailscale set‘ writes a persistent preference. However, ‘tailscale up --reset‘
# (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
# re-enable ‘--accept-dns=true‘ and nullify this ‘set‘.
# Therefore, EVERY ‘tailscale up --reset‘ call in this file MUST also pass
# ‘--accept-dns=false‘ explicitly. See steps 7 and 9 for the fix.
#
# This initial ‘set‘ is still useful as a baseline for non-reset operations
# (e.g. if the user runs ‘tailscale up‘ manually without --reset) and covers
# the brief window between this line and the first ‘--reset‘ call.
```

```
#
# Runs once at script start while tailscaled is still connected (if it was
# left running from a previous toggle), so the pref takes effect before any
# disconnection or reconnection logic.
if [ -n "$TS_BIN" ]; then
  $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
fi

# Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
# *entire* teardown  no need to mess with system network managers.
# tailscaled keeps running (as a systemd service), which is intentional:
# the next ‘tailscale up‘ then reconnects in seconds rather than waiting for a
# slow daemon launch. The ‘tailscale down‘ call also retains the user’s
# auth state, so we don’t force a browser re-login every cycle.
#
# Defined early (before the if/else branches and referenced by cleanup_handler’s
# ERR trap) so that any failure before the main logic can still tear down Tailscale.
restart_tailscaled_daemon() {
  echo "  [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
  echo "  [Tailscale Recovery] Attempting to restart tailscaled service via systemctl..."
  if run_with_timeout 15 sudo -n systemctl restart tailscaled >> output.log 2>&1; then
    echo "  [Tailscale Recovery] Successfully restarted tailscaled service."
  else
    echo "  [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
  fi
  sleep 3
}
disconnect_tailscale_host() {
  if [ -n "$TS_BIN" ]; then
    echo "Disconnecting host Tailscale from mesh (tailscaled stays running as system service)..."
    local ts_args=""
    if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
    if ! run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1; then
      restart_tailscaled_daemon
      run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1 || true
    fi
  fi
}

# Function to get the active network service name (e.g., "Wi-Fi" or "USB 10/100 LAN")
get_active_service() {
  # Get the interface used for default internet routing
  local iface
  iface=$(ip route get 1.1.1.1 2>/dev/null | awk ’{print $5; exit}’)
  if [ -z "$iface" ]; then
    echo ""
    return 1
  fi
  echo "$iface"
}

ACTIVE_SERVICE=$(get_active_service)

# Resolve the service name for en0 (usually "Wi-Fi")  macOS scutil DNS resolver
# is commonly scoped to en0, so per-service DNS changes on other interfaces are
# ignored unless en0’s service is also updated.
# Helper to restore host DNS to a clean state (used on script start, on failure,
# and after a successful STOP). Without this, a successful toggle-off leaves the
# host’s resolver pinned to a now-dead gateway IP  every lookup stalls for macOS’s
# DNS timeout (~5s) before falling through to whatever next server is configured.
# With it, we always leave the host in ‘1.1.1.1 / no search domain‘.
reset_dns() {
  if [ -n "$ACTIVE_SERVICE" ]; then
    resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
    resolvectl revert "$ACTIVE_SERVICE" 2>/dev/null || true
  fi
}

# Helper to nuke leftover network state after teardown (proxy settings, routing
# table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
# problem where stale state kills internet after the gateway stops.
cleanup_network_state() {
  echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."

  # 1. Disable any leftover proxy settings (needs root on many macOS versions)
  # (Skipped for Linux since we don’t set global SOCKS proxy)
  echo "  Proxies disabled."

  # 2. Flush DNS cache
  if command -v resolvectl >> output.log 2>&1; then
```

```
    sudo -n resolvectl flush-caches 2>> output.log || true
  elif command -v systemd-resolve >> output.log 2>&1; then
    sudo -n systemd-resolve --flush-caches 2>> output.log || true
  fi
  echo "  DNS cache flushed."

  # 3. Flush stale routing table entries
  if command -v ip >> output.log 2>&1; then
    sudo -n ip route flush cache >> output.log 2>&1 || true
  fi
  echo "  Routing table flushed."

  # 4. Power-cycle Wi-Fi to clear any lingering interface state
  echo "  Restarting network interface..."
  sudo -n ip link set dev "$ACTIVE_SERVICE" down 2>> output.log || true
  sleep 1
  sudo -n ip link set dev "$ACTIVE_SERVICE" up 2>> output.log || true
  echo "  Interface power-cycled ($ACTIVE_SERVICE)."
  sleep 3

  echo "Network state cleanup complete."
}

#  SOCKS5 Proxy (WARP Tunnel)
# When Tailscale's exit node can't route through WARP (due to userspace
# forwarding), we use a SOCKS5 proxy running inside the warp container's
# network namespace. Connections created by this proxy go through the
# kernel routing table (table 200 -> tun0 -> WARP), unlike Tailscale's
# userspace exit node which bypasses it.
#
# The SOCKS5 proxy is exposed on localhost:${SOCKS_PROXY_PORT} via Docker port mapping.
# We route system TCP traffic through the proxy, which then goes through WARP.
#
# IMPORTANT: CLI tools like curl do NOT respect macOS system proxy settings.
# They must use --socks5-hostname or the ALL_PROXY env variable explicitly.

enable_socks_proxy() {
  local svc="$1"
  echo "  (Linux global SOCKS proxy configuration is desktop-environment specific, skipping.)"
  echo "  SOCKS5 proxy is active and listening on 127.0.0.1:$SOCKS_PROXY_PORT"
}

disable_socks_proxy() {
  local svc="$1"
  echo "  (Linux global SOCKS proxy configuration skipped.)"
}

#  Local DNS Proxy (Python)
# When the tailnet data plane cannot establish, we use a Python DNS proxy.
# It listens on UDP:53, receives DNS queries, forwards them over TCP to
# AdGuard at 127.0.0.1:${DNS_PROXY_PORT} (Docker port mapping), and returns the response.
# The Python script handles the DNS-over-TCP wire format (2-byte length prefix)
# correctly  unlike socat which just forwards raw bytes.
#
# Python3 is built into macOS  no external dependencies.
DNS_PROXY_PID=""

# Kill any leftover DNS proxy process
stop_local_dns_proxy() {
  if [ -n "$DNS_PROXY_PID" ]; then
    kill "$DNS_PROXY_PID" 2>/dev/null || true
    wait "$DNS_PROXY_PID" 2>/dev/null || true
    DNS_PROXY_PID=""
  fi
  # Clean up any stale Python DNS proxy processes
  sudo -n pkill -f "dns-proxy.py" 2>/dev/null || true
}

# Start local DNS proxy (Python)
start_local_dns_proxy() {
  if [ -z "$DNS_PROXY_BIN" ]; then
    echo "  Python not found. Python3 is built into macOS  this shouldn't happen."
    return 1
  fi

  if [ ! -f "$DNS_PROXY_SCRIPT" ]; then
    echo "  DNS proxy script not found at $DNS_PROXY_SCRIPT"
    return 1
  fi

  # Kill any leftover process first
```

```
  stop_local_dns_proxy

  echo -n "  Starting local DNS proxy via Python (UDP:53  TCP:${DNS_PROXY_PORT})... "
  sudo -n bash -c "TARGET_PORT=$DNS_PROXY_PORT \"$DNS_PROXY_BIN\" \"$DNS_PROXY_SCRIPT\"" &
  DNS_PROXY_PID=$!
  disown "$DNS_PROXY_PID" 2>/dev/null || true

  # Give it a moment to bind
  sleep 0.5

  if kill -0 "$DNS_PROXY_PID" 2>/dev/null; then
    echo "started (PID $DNS_PROXY_PID)."

    # Hijack host DNS to localhost
    echo -n "  Hijacking host DNS to 127.0.0.1 for ad-blocking... "
    resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
    resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
    if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" 127.0.0.1; then
      echo "FAILED (resolvectl error  check permissions)."
      stop_local_dns_proxy
      return 1
    fi
    resolvectl dns "$ACTIVE_SERVICE" 127.0.0.1 2>/dev/null || true
    sudo -n resolvectl flush-caches 2>/dev/null || true

    # Verify DNS actually works through the proxy
    echo -n "  Verifying DNS resolution... "
    if dig @127.0.0.1 google.com +short +timeout=5 &>/dev/null; then
      echo "ok."
      return 0
    else
      echo "FAILED (DNS queries not reaching AdGuard)."
      echo "  Restoring DNS to 1.1.1.1..."
      reset_dns
      stop_local_dns_proxy
      return 1
    fi
  else
    echo "FAILED (could not bind port 53  is it in use?)."
    DNS_PROXY_PID=""
    return 1
  fi
}

# Helper to FORCIBLY hijack host DNS to a single server (the gateway's AdGuard IP)
# and VERIFY via ‘networksetup -getdnsservers‘ that the only name server is exactly
# $1. Returns 0 iff the live state matches; non-zero if it can't be confirmed.
#
# This deliberately does NOT append a 1.1.1.1 fallback: macOS's resolver queries
# the list in order and falls back to the next entry on timeout, so on a
# misbehaving AdGuard (filter compile crash, OOM kill, etc.) the resolver still
# leaks to 1.1.1.1 and silently bypasses ad-blocking. With no fallback, a broken
# gateway manifests as a *visible* DNS outage that prompts the user to
# investigate  which is the correct failure mode.
force_dns_to_gateway() {
  local target_ip="$1"

  for attempt in {1..3}; do
    echo -n "  [force_dns] Attempt $attempt/3: setting DNS to $target_ip on $ACTIVE_SERVICE... "
    sudo -n resolvectl domain "$ACTIVE_SERVICE" "~ts.net" 2>/dev/null || true
    if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" "$target_ip"; then
      echo "FAILED (resolvectl error)"
    else
      # Verify
      entries=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE ’[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+’)
      if [ "$entries" == "$target_ip" ]; then
        echo "VERIFIED"
        return 0
      else
        echo "MISMATCH (Resolver refused lock)"
      fi
    fi
    sleep 2
  done
  return 1
}

# Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
# during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
# ‘ts.net‘ search domain from a previous ‘tailscale up --accept-dns=true‘ run, otherwise macOS
```

```
# would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
# Capture current DNS before resetting so we can check if it was hijacked
INITIAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE ’[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+’ | head -1 || true)

echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
reset_dns


  #  Local Network Surveillance Check
  echo -e "\n"
  echo "Local Network Surveillance Check"
  echo ""
  # Count populated ARP cache entries to detect network scanning or high congestion
  if command -v ip >/dev/null 2>&1; then
    ARP_COUNT=$(ip neigh | grep -iv ’incomplete’ | wc -l | tr -d ’ ’)
  else
    # Using ’-an’ avoids hanging on DNS reverse resolution when the network state is in transition.
    ARP_COUNT=$(arp -an 2>/dev/null | grep -iv ’incomplete’ | wc -l | tr -d ’ ’)
  fi
  if [ "$ARP_COUNT" -gt 15 ]; then
    echo -e "  \033[1;33m[!] WARNING: High ARP activity detected ($ARP_COUNT devices in cache).\033[0m"
    echo -e "  \033[1;33m[!] This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan).\033[0m"
    echo -e "  [] Your traffic remains fully encrypted and invisible.\n"
  else
    echo -e "  [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
  fi

echo "Checking Gateway Status..."
if is_gateway_active; then
  echo -e "\n============================================="
  echo "Gateway is RUNNING. Stopping it now..."
  echo -e "=============================================\n"

  # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
  disconnect_tailscale_host
  echo ""

  echo "Stopping Docker containers..."
  docker compose down -t 5

  echo -e "\nDocker daemon handles container teardown automatically."

  # The host’s DNS was hijacked to the gateway IP during ENABLE; now that the
  # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
  # don’t stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
  echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)... "
  reset_dns

  # 3. Stop local DNS proxy if running
  stop_local_dns_proxy

  # Stop sleep prevention
  stop_sleep_prevention
  stop_dns_watcher

  # 4. Nuke leftover network state so internet actually works after teardown
  cleanup_network_state

  ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
  echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
else
  echo -e "\n============================================="
  echo "Gateway is STOPPED. Starting it now..."
  echo -e "=============================================\n"

  # 1. Prevent host exit-node deadlock during VM / Container startup
  disconnect_tailscale_host

  echo -e "\nUsing native Linux Docker..."

  # 4. Clean up corrupted AdGuardHome configurations
  if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
    rm -f "adguard/conf/AdGuardHome.yaml"
    echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
  fi



  # 5. Start compose services
```

```
write_host_ips
# Procedurally generated ports have already been exported at the top of the script.
echo "  [Security] Procedurally randomized proxy ports generated to evade fingerprinting."

# --- HONEY-PORT TRIPWIRE ---
# Generate a random trap port. If any malware does a sequential port scan on localhost,
# it will inevitably hit this port. When it does, we instantly fire a desktop alert.
HONEY_PORT=$(get_free_port)
(
  if nc -l -p "$HONEY_PORT" 127.0.0.1 >/dev/null 2>&1 || nc -l localhost "$HONEY_PORT" >/dev/null 2>&1; then
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown application just pinged the local Honey-Port ($HONEY_PORT)!" >> "$PWD/output.log"
    if command -v notify-send >/dev/null 2>&1; then
      notify-send -u critical "nullexit OPSEC Alert" "An unknown application is aggressively scanning your local ports. Malware port-scan suspected."
    fi
  fi
) &
disown %1 2>/dev/null || true
echo "  [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."

echo -e "\nStarting Docker containers..."
docker compose up -d

# Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
docker compose rm -s -f rule-compiler >> output.log 2>&1

# 6. Wait for the gateway container's Tailscale connection to be ready
BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')

echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
echo "  (This can take 30-60s  Tailscale goes through: NoState  Starting  Running  Online)"
connected=false
consecutive_failures=0
for i in {1..60}; do
  # Run tailscale status and capture its output so the user can see what's happening.
  # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
  ts_output=$(run_with_timeout 5 docker compose exec -T tailscale tailscale status 2>&1 || true)

  if echo "$ts_output" | grep -q "offers exit node"; then
    connected=true
    break
  fi

  # Track consecutive failures to detect a stuck state.
  # If we see 40+ consecutive 'NoState' responses, give up early
  # the daemon is alive but can't connect to the coordination server.
  if echo "$ts_output" | grep -q "NoState"; then
    consecutive_failures=$((consecutive_failures + 1))
    if [ "$consecutive_failures" -ge 40 ]; then
      echo ""
      echo "  [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
      echo "    Tailscale daemon is running but can't reach the coordination server."
      break
    fi
  else
    consecutive_failures=0
  fi

  # Print a condensed status every 10 seconds so the user can see progress
  if [ $((i % 10)) -eq 0 ]; then
    ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
    echo "  [attempt $i/60] $ts_short"
  elif [ $((i % 5)) -eq 0 ]; then
    echo -n "[$i]"
  else
    echo -n "."
  fi
  sleep 1
done
echo ""

if [ "$connected" = "true" ]; then
  echo "Gateway container is online on the Tailscale mesh."
elif [ "$BYPASS_PING" = "true" ]; then
  echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
else
  echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
```

```
    echo "  The container was seen in the following states during the wait:" >&2
    echo "$ts_output" | sed ’s/^/    /’ >&2
    echo "" >&2
    echo "  This could mean:" >&2
    echo "    1. Tailscale auth key has expired  check TS_AUTHKEY in .env" >&2
    echo "    2. Network issue inside the container  check: docker compose logs tailscale" >&2
    echo "    3. gluetun VPN tunnel not connecting  check: docker compose logs warp | tail -20" >&2
    echo "" >&2
    echo "  Diagnose manually:" >&2
    echo "    docker compose exec -T tailscale tailscale status" >&2
    echo "    docker compose exec -T tailscale tailscale netcheck" >&2
    cleanup_handler ERR $LINENO
    exit 1
  fi

  # 6b. Resolve the gateway’s Tailscale IP.
  # Dynamically query the container for the IP (handles IP changes after re-auth)
  # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
  #
  # CRITICAL: ‘docker compose exec -T‘ on macOS/Colima injects carriage
  # returns (\r) into captured output. If $TS_IP ends with \r, then
  # ‘networksetup -setdnsservers‘ silently rejects the malformed IP and
  # DNS stays at 1.1.1.1  the primary bug this project was hitting.
  # ‘tr -d ’\r’‘ strips the CR; ‘awk ’NR==1{print $1; exit}’‘ takes only
  # the first field of the first line.
  TS_IP=""
  if [ "$connected" = "true" ]; then
    TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d ’\r’ | awk ’NR==1{print $1; exit}’ || true)
    if [ -n "$TS_IP" ]; then
      echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
      echo "$TS_IP" > .gateway_ip
    else
      echo "    [!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2
      TS_IP=$(read_adguard_ip || true)
    fi
  else
    TS_IP=$(read_adguard_ip || true)
  fi

  # 7. Connect host Mac to Tailscale mesh.
  # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
  # system LaunchDaemon via ’systemctl start tailscaled’), the previous five-
  # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
  # no menu bar item to click, and no Accessibility permission required.
  #
  # CRITICAL: If tailscaled is not running, ‘tailscale up‘ will silently fail.
  # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
  # if it can’t be started (DNS hijack to a 100.x.x.x IP and exit-node routing
  # are impossible without the host on the mesh).
  #
  # We connect in two phases:
  #   Phase A  Join mesh WITHOUT exit node (so pre-flight checks can run)
  #   Phase B  Set exit node only after pre-flight checks confirm the gateway works
  HOST_ON_MESH=false
  SKIP_EXIT_NODE=false
  if [ -n "$TS_BIN" ]; then
    echo -n "Verifying tailscaled is reachable"
    daemon_ready=false
    for i in {1..10}; do
      # ‘tailscale status‘ returns exit code 1 when the daemon is alive but
      # disconnected ("Tailscale is stopped."). We check the daemon socket
      # directly as a fallback  if the socket exists, tailscaled is running
      # even if the host isn’t connected to the mesh yet.
      if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
        daemon_ready=true
        break
      fi
      echo -n "."
      sleep 1
    done
    echo ""

    if [ "$daemon_ready" != "true" ]; then
      echo -e "\033[0;33m[!] tailscaled daemon is not running. Attempting to start it...\033[0m"

      # Try system-wide daemon (with timeout  sudo may prompt for password)
      if [ "$daemon_ready" != "true" ]; then
        if run_with_timeout 10 sudo systemctl start tailscaled 2>> output.log; then
          echo "  Started tailscaled as system LaunchDaemon."
          for i in {1..15}; do
```

```
            if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
              daemon_ready=true
              break
            fi
            sleep 1
          done
        fi
      fi
    fi

    if [ "$daemon_ready" = "true" ]; then
      echo "tailscaled is responsive (running as a system service)."

      #  Phase A: Join mesh without exit node
      echo "Connecting host to Tailscale mesh (no exit node yet)..."
      if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node=; then
        HOST_ON_MESH=true
        echo "Host is on Tailscale mesh."
      else
        echo -e "\033[0;33m[!] tailscale up failed even though the daemon is running.\033[0m"
        echo "  This usually means the host hasn’t authenticated with your tailnet yet."
        echo "  Run this command in a terminal to authenticate:"
        echo ""
        echo "      sudo tailscale up"
        echo ""
        echo "  A browser will open. Log in to your Tailscale account, then re-run toggle-linux.sh."
      fi
    else
      echo -e "\033[0;31m[!] tailscaled could NOT be started automatically.\033[0m"
      echo "  Run this in a terminal to start and authenticate Tailscale:"
      echo ""
      echo "      sudo systemctl start tailscaled"
      echo "      sudo tailscale up"
      echo ""
      echo "  Then re-run toggle-linux.sh."
    fi
  else
    echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
  fi

  #  Phase B: Pre-flight checks + enable exit node only if safe
  if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
    echo -e "\n"
    echo "Pre-flight connectivity check"
    echo ""

    # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
    # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
    # ("direct connection not established") even though the pong was received.
    # We check for ’pong’ in the output (stderr discarded) to accept relayed connections.
    echo -n "  [1/3] Gateway reachable via Tailscale... "
    ping_ok=false
    # The host tailscaled needs a few seconds to propagate the new network map and
    # establish a connection route after ’tailscale up --reset’. We retry up to 45 times.
    for attempt in {1..45}; do
      if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
        ping_ok=true
        break
      fi
      sleep 1
    done
    if [ "$ping_ok" = "true" ]; then
      echo "PASS"
    else
      echo "FAIL"
      SKIP_EXIT_NODE=true
    fi

    # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
    echo -n "  [2/3] AdGuard DNS via localhost... "
    if command -v dig >/dev/null 2>&1; then
      if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 >/dev/null 2>&1; then
        echo "PASS"
      else
        echo "FAIL"
        SKIP_EXIT_NODE=true
      fi
    elif command -v nslookup >/dev/null 2>&1; then
      if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 >/dev/null 2>&1; then
        echo "PASS"
      else
```

```bash
        echo "FAIL"
        SKIP_EXIT_NODE=true
      fi
    else
      echo "SKIP (dig/nslookup not available)"
    fi

    # Check 3: Does the WARP container have external internet?
    echo -n "  [3/3] WARP container internet... "
    if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace &>/dev/null; then
      echo "PASS"
    else
      echo "FAIL"
      SKIP_EXIT_NODE=true
    fi

    # Act based on check results
    if [ "$SKIP_EXIT_NODE" != "true" ]; then
      echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
      if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node="$TS_IP" --exit-node-allow-lan-access=true; then
        echo "Exit node enabled."
      else
        echo "[Warning] Failed to set exit node."
        SKIP_EXIT_NODE=true
      fi
    fi

    if [ "$SKIP_EXIT_NODE" = "true" ]; then
      echo -e "\n\033[0;33m[!] Pre-flight checks failed  EXIT NODE + DNS HIJACK SKIPPED.\033[0m"
      echo "  Host stays on Tailscale mesh but your internet routes through your normal connection."
      echo "  Troubleshoot with:"
      echo "    docker compose logs warp | tail -20"
      echo "    docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
      echo ""
      echo "  Falling back to SOCKS5 proxy for traffic routing..."
    fi
  elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
    echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
    SKIP_EXIT_NODE=true
  fi

  # 8. Apply DNS Hijacking.
  # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
  # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
  # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
  if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
    echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
    echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
    if force_dns_to_gateway "$TS_IP"; then
      echo "DNS hijack verified: single server = $TS_IP."
      start_dns_watcher "$TS_IP"
    else
      echo "[Warning] DNS hijack could not be verified."
      echo "  If DNS is broken, run manually:"
      echo "    networksetup -setdnsservers \"$ACTIVE_SERVICE\" $TS_IP"
      echo "    networksetup -setsearchdomains \"$ACTIVE_SERVICE\" ts.net"
    fi
  elif [ -n "$TS_IP" ]; then
    echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
    echo "  Trying local DNS proxy for ad-blocking..."
    if start_local_dns_proxy; then
      echo "  Ad-blocking active via local DNS proxy through AdGuard."
    else
      echo "  Local DNS proxy unavailable. DNS stays at 1.1.1.1."
      echo "  AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
    fi

    # Enable SOCKS5 proxy for traffic routing through WARP
    # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
    echo -e "\n  Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
    echo "  (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
    enable_socks_proxy "$ACTIVE_SERVICE"

    # Verify the SOCKS5 proxy works through WARP
    echo -n "  Verifying SOCKS5 proxy routes through WARP... "
    if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null | grep -q "warp=on"; then
      echo "ok (warp=on)."
```

```
      echo "  All TCP traffic now encrypted through Cloudflare WARP tunnel."
    else
      echo "FAILED (proxy not routing through WARP)."
      echo "  Disabling SOCKS5 proxy  internet stays on direct connection."
      disable_socks_proxy
      echo "  SOCKS proxy available at localhost:$SOCKS_PROXY_PORT for manual configuration."
    fi
  else
    echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
  fi

  # 9. Verify host connectivity
  if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
    if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
      echo "Host is online on the Tailscale mesh."
    else
      echo "[Warning] Host is not responding on Tailscale."
    fi
  fi

  ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
  echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."

  # Prevent system from going to sleep while gateway is running
  start_sleep_prevention
fi

SUCCESS_RUN=true

# Final DNS state summary
if [ -n "$ACTIVE_SERVICE" ]; then
  FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' |
     tr '
' ' ' || true)
  echo -e "\n"
  echo -e "DNS STATE: $FINAL_DNS"
  echo -e ""
fi

if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
  if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
    echo -e "\n[Warning] One or more blocklist URLs failed to download (404/Offline)."
    echo "  The gateway gracefully fell back to yesterday's cached blocklist."
    echo "  (Run 'docker logs rule-compiler' later to investigate which link died.)"
  fi
fi

echo -e "\n"
read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
if [[ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]]; then
  echo "Reversing gateway state..."
  exec bash "$0"
fi

echo -e "\nYou can close this terminal window now."
```

10 scripts/recover-linux.sh

```
#!/bin/bash
# recover.sh  nullexit KILL-SWITCH recovery script for Linux
# Fixes internet when toggle.sh leaves you stranded.
#
# This is a NUCLEAR option: it undoes EVERYTHING toggle.sh does by:
#   1. Disconnecting host Tailscale from the mesh (exit-node off)
#   2. Resetting DNS to default on ALL network services
#   3. Disabling rogue proxy settings (SOCKS, web, secure web)
#   4. Flushing macOS DNS cache
#   5. Flushing stale routing table entries
#   6. Stopping gateway Docker containers
#   7. Power-cycling Wi-Fi
#   8. Verifying internet is working
#
# Usage:  double-click Recover-Gateway.applescript
#         OR  ./recover.sh

set -e
```

```bash

# Source common formatting and helper functions
source "$(dirname "$0")/common.sh"

# Always add Homebrew path
export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"

echo ""
echo -e "${RED}${BOLD}${NC}"
echo -e "${RED}${BOLD}     Nullexit Gateway  RECOVERY MODE     ${NC}"
echo -e "${RED}${BOLD}${NC}"
echo ""
echo "This script will tear down the gateway and restore normal internet."
echo ""

#  Cache sudo credentials upfront
# Prevents the script from hanging halfway through when it needs to run
# privileged commands (route flush, Wi-Fi power cycle, etc.).
echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
sudo -v
(
  while true; do
    sudo -n true 2>/dev/null
    sleep 60
  done
) &
SUDO_KEEPER_PID=$!
trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT

#  1. Disconnect host Tailscale
step "Disconnecting host Tailscale from mesh"
if command -v tailscale >> output.log 2>&1; then
  # Check if actually connected first (with timeout  tailscaled could be wedged)
  if run_with_timeout 5 tailscale status >> output.log 2>&1; then
    # Also explicitly reset any exit-node so it doesn't linger.
    if run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= >> output.log 2>&1; then
      ok "Exit-node preference cleared"
    else
      warn "tailscale up --reset didn't respond (tailscaled may be wedged)"
    fi
    ts_args=""
    if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
    if run_with_timeout 10 tailscale down $ts_args >> output.log 2>&1; then
      ok "Tailscale disconnected"
    else
      warn "tailscale down didn't respond"
    fi
  else
    warn "tailscaled not reachable  skipping (timeout after 5s)"
  fi
else
  warn "tailscale CLI not found  skipping"
fi

#  2. Reset DNS to default on ALL network services
step "Resetting DNS to default on active interface"

ACTIVE_SERVICE=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
if [ -n "$ACTIVE_SERVICE" ]; then
  sudo resolvectl revert "$ACTIVE_SERVICE" 2>> output.log || true
  ok "DNS reset on $ACTIVE_SERVICE"
else
  warn "Could not determine active network interface"
fi

#  3. Disable rogue proxy settings
step "Disabling proxy settings (SOCKS, web, secure web)"
echo "  (Linux global SOCKS proxy configuration skipped.)"
ok "Proxy settings cleared"


#  4. Flush macOS DNS cache
step "Flushing DNS cache"
if command -v resolvectl >> output.log 2>&1; then
  sudo resolvectl flush-caches 2>> output.log || true
  ok "DNS cache flushed"
else
  warn "resolvectl not found"
fi

```

```
#  5. Clear stale routing table
step "Flushing stale routing table entries"
sudo ip route flush cache >> output.log 2>&1 || true
ok "Routing table flushed"

#  6. Stop gateway Docker containers
step "Stopping gateway Docker containers"
if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
  if [ -f "docker-compose.yml" ]; then
    docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
  else
    warn "docker-compose.yml not found in current directory"
  fi
else
  warn "Docker not available  skipping"
fi

#  6b. Stopping sleep prevention (systemd-inhibit)
step "Stopping sleep prevention"
PID_FILE="/tmp/nullexit-caffeinate.pid"
if [ -f "$PID_FILE" ]; then
  INHIBIT_PID=$(cat "$PID_FILE")
  if [ -n "$INHIBIT_PID" ] && kill -0 "$INHIBIT_PID" 2>/dev/null; then
    kill "$INHIBIT_PID" 2>/dev/null || true
    ok "Sleep prevention stopped (PID $INHIBIT_PID)"
  else
    warn "Sleep prevention process not running, cleaning up stale PID file"
  fi
  rm -f "$PID_FILE"
else
  ok "Sleep prevention was not active"
fi

#  7. Power-cycle Wi-Fi
step "Power-cycling Wi-Fi"
if command -v nmcli &>/dev/null; then
  sudo nmcli radio wifi off 2>> output.log || true
  sleep 2
  sudo nmcli radio wifi on 2>> output.log || true
  ok "Wi-Fi bounced via nmcli"
  sleep 3
else
  warn "nmcli not found. Falling back to ip link..."
  WIFI_IFACE=$(ip link | grep -E ’^[0-9]+: (wl[a-zA-Z0-9]+|wlan[0-9]+):’ | awk -F’: ’ ’{print $2}’)
  if [ -n "$WIFI_IFACE" ]; then
    sudo ip link set "$WIFI_IFACE" down 2>> output.log || true
    sleep 2
    sudo ip link set "$WIFI_IFACE" up 2>> output.log || true
    ok "Wi-Fi bounced via ip link (interface $WIFI_IFACE)"
    sleep 3
  else
    warn "No Wi-Fi interface detected."
  fi
fi

#  8. Verify internet connectivity
step "Verifying internet connectivity"

# Wait a moment for the network to settle
sleep 2

INTERNET_OK=false

# Try DNS resolution first
if host -W 3 google.com 1.1.1.1 >> output.log 2>&1; then
  ok "DNS resolution works (google.com via 1.1.1.1)"
  INTERNET_OK=true
elif nslookup google.com 1.1.1.1 >> output.log 2>&1; then
  ok "DNS resolution works (nslookup google.com via 1.1.1.1)"
  INTERNET_OK=true
else
  warn "DNS resolution check failed  will still try ping/curl"
fi

# Try actual HTTP connectivity
if command -v curl >> output.log 2>&1; then
  if curl -sf --max-time 5 https://1.1.1.1 >> output.log 2>&1; then
    ok "Internet reachable via HTTP"
    INTERNET_OK=true
  elif curl -sf --max-time 5 http://1.1.1.1 >> output.log 2>&1; then
```

```bash
    ok "Internet reachable via HTTP (plain)"
    INTERNET_OK=true
  else
    warn "HTTP check failed"
  fi
elif command -v ping >> output.log 2>&1; then
  if ping -c 1 -W 3 1.1.1.1 >> output.log 2>&1; then
    ok "Internet reachable via ping"
    INTERNET_OK=true
  else
    warn "Ping check failed"
  fi
fi

#  Summary
echo ""
echo -e "${BOLD}${NC}"
echo -e "${BOLD}RECOVERY SUMMARY${NC}"
echo -e "${BOLD}${NC}"

# Show final DNS state
FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tr '
' ' ' || echo "N/A")
echo "  DNS ($ACTIVE_SERVICE):  $FINAL_DNS"

echo ""

if [ "$INTERNET_OK" = "true" ]; then
  echo -e "  ${GREEN}${BOLD} Internet is working.${NC}"
else
  echo -e "  ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
  echo "    This might mean:"
  echo "    - Your Wi-Fi is disconnected from the router"
  echo "    - Tailscale is still interfering (try restarting tailscaled:"
  echo "        systemctl restart tailscaled)"
  echo "    - You need to re-connect to Wi-Fi in the menu bar"
  echo "    - Try toggling Wi-Fi off/on in the menu bar"
  echo ""
  echo "    Or try manually:"
  echo "      resolvectl revert $ACTIVE_SERVICE"
  extra_args=""
  if is_kill_switch_enabled; then extra_args="--accept-risk=lose-ssh"; fi
  echo "      tailscale down $extra_args"
  echo "      sudo ip route flush cache"
  echo "      systemctl restart tailscaled"
fi

echo ""
echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
echo ""
```

11 scripts/setup-linux.sh

```bash
#!/bin/bash
# scripts/setup-linux.sh  nullexit one-shot setup script (Linux)
# Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
set -euo pipefail

# Source common formatting and helper functions
source "$(dirname "$0")/common.sh"

# Run common installation logic
source "$(dirname "$0")/setup-common.sh"

#  Compile Linux Desktop Shortcuts
echo -e "\n${BOLD}Creating Linux Desktop Shortcuts...${NC}"
DIR="$(pwd)"
cat <<EOF > "Toggle Gateway.desktop"
[Desktop Entry]
Version=1.0
Name=Toggle Gateway
Comment=Start or Stop Nullexit Gateway
Exec=bash -c "cd '$DIR' && ./scripts/toggle-linux.sh; read -p 'Press Enter to close...' dummy"
Icon=utilities-terminal
Terminal=true
```

```
Type=Application
Categories=Network;Security;
EOF

cat <<EOF > "Recover Gateway.desktop"
[Desktop Entry]
Version=1.0
Name=Recover Gateway
Comment=Emergency DNS and Network Recovery
Exec=bash -c "cd '$DIR' && ./scripts/recover-linux.sh; read -p 'Press Enter to close...' dummy"
Icon=utilities-terminal
Terminal=true
Type=Application
Categories=Network;Security;
EOF
echo "   Created 'Toggle Gateway.desktop' and 'Recover Gateway.desktop'"

#  Done
echo ""
echo -e "${GREEN}${BOLD}${NC}"
echo -e "${GREEN}${BOLD}            Setup complete!                   ${NC}"
echo -e "${GREEN}${BOLD}${NC}"
echo ""
echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
echo "   https://login.tailscale.com/admin/machines"
echo "   Find your gateway  '...'  'Edit route settings'  enable Exit Node"
echo ""

if [[ -n "${TS_IP:-}" ]]; then
    echo -e "${BOLD}AdGuard Home dashboard:${NC}  http://${TS_IP}:3000  (username: admin)"
    echo ""
fi

echo -e "${BOLD}To toggle the gateway on/off:${NC}"
echo "  Linux: double-click the 'Toggle Gateway' desktop icon!"
echo "  (or run ./scripts/toggle-linux.sh in terminal)"
echo ""
```

12 docker-compose.yml

```
services:
  warp:
    image: qmcgaw/gluetun:v3.41.1
    container_name: warp
    hostname: nullexit-gateway
    cap_add:
      - NET_ADMIN
    devices:
      - /dev/net/tun:/dev/net/tun
    ports:
      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/udp"    # AdGuard DNS (5335 is used by SSH on host, use procedural port instead)
      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/tcp"    # AdGuard DNS (TCP)
      - "41642:41642/udp"  # Tailscale WireGuard (direct connection to host)
      - "127.0.0.1:${SOCKS_PROXY_PORT:-1080}:1080/tcp"    # SOCKS5 proxy (routed through WARP tunnel)
      - "127.0.0.1:${TOR_SOCKS_PORT:-9050}:9050/tcp"  # Tor SOCKS5 proxy (procedurally generated)
      - "127.0.0.1:${TOR_CONTROL_PORT:-9051}:9051/tcp"  # Tor Control port (procedurally generated)
    environment:
      - TZ=America/New_York
      - VPN_SERVICE_PROVIDER=custom
      - VPN_TYPE=wireguard
      - WIREGUARD_PRIVATE_KEY=${WIREGUARD_PRIVATE_KEY}
      - WIREGUARD_PUBLIC_KEY=${WIREGUARD_PUBLIC_KEY}
      - WIREGUARD_ADDRESSES=${WIREGUARD_ADDRESSES}
      - WIREGUARD_ENDPOINT_IP=${WARP_ENDPOINT_1:-162.159.192.1}
      - WIREGUARD_ENDPOINT_PORT=2408
      # Set to 25s (WireGuard default) to beat standard 30s hardware NAT/firewall UDP timeouts
      - WIREGUARD_PERSISTENT_KEEPALIVE_INTERVAL=25s
      # MTU Fix (Critical): Prevent packet fragmentation from double-encryption
      - WIREGUARD_MTU=1280
      # Give WARP more time to establish the connection before internal restart (default is 6s which is too short)
      - HEALTH_VPN_DURATION_INITIAL=30s
      # Use plaintext DNS resolvers instead of DNS-over-TLS (DoT) to prevent startup failures on networks where port 853 is blocked
      - DNS_UPSTREAM_RESOLVER_TYPE=plain
```

```
      - DNS_UPSTREAM_PLAIN_ADDRESSES=1.1.1.1:53,8.8.8.8:53
      # Allow local network traffic to bypass WARP so Tailscale can establish fast, direct peer-to-peer connections (no DERP relays) on the LAN
      - FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8
      # Built-in DNS blocking for ads, malicious domains, and tracking
      # NOTE: This is the massive "Hagezi-style" built-in blocklist.
      # You can turn these to 'on' for maximum security if you have spare RAM
      # or are running this on a cloud exit node with adequate autonomous resources (requires approx 1-2 GB of RAM).
      # Disabled intentionally: AdGuard Home provides the DNS filtering layer instead.
      # If AdGuard fails, there is no fallback filtering, but the healthcheck
      # dependency chain ensures Tailscale won't come up if AdGuard is down.
      - BLOCK_MALICIOUS=off
      - BLOCK_SURVEILLANCE=off
      - BLOCK_ADS=off
    dns:
      - 1.1.1.1
    sysctls:
      - net.ipv4.ip_forward=1
    volumes:
      - ./post-rules.txt:/iptables/post-rules.txt:ro
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "/gluetun-entrypoint healthcheck"]
      interval: 2s
      timeout: 5s
      retries: 15
      start_period: 60s
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

  tailscale:
    image: tailscale/tailscale:v1.98.4
    container_name: tailscale
    network_mode: service:warp
    depends_on:
      warp:
        condition: service_healthy
      adguardhome:
        condition: service_healthy
    cap_add:
      - NET_ADMIN
      - NET_RAW
      - SYS_MODULE
    sysctls:
      - net.ipv4.ip_forward=1
      - net.ipv6.conf.all.forwarding=1
    environment:
      - TZ=America/New_York
      - TS_EXTRA_ARGS=--advertise-exit-node
      - PORT=41642
      - TS_USERSPACE=false
      - TS_AUTHKEY=${TS_AUTHKEY}
      - TS_STATE_DIR=/var/lib/tailscale
      # (Note: For headless/cloud deployments, you can bypass this footgun by
      # using Tailscale Ephemeral Keys which auto-clean themselves).
      - TS_AUTH_ONCE=true
      # MTU Optimization: Force Tailscale packets to be smaller than WARP's 1280 MTU
      # to prevent WireGuard-in-WireGuard fragmentation, completely eliminating bufferbloat
      - TS_DEBUG_MTU=1200
      # Enable Tailscale's built-in SOCKS5 proxy to route out of the WARP tunnel
      - TS_SOCKS5_SERVER=:1080
    devices:
      - /dev/net/tun:/dev/net/tun
    volumes:
      - tailscale_state:/var/lib/tailscale
    restart: unless-stopped
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

  routing-fix:
    image: nullexit-routing-fix:v1.0.0
    build:
      context: ./scripts
      dockerfile: Dockerfile.routing-fix
```

```
    container_name: routing-fix
    network_mode: service:warp
    # NOTE: Deliberately NOT sharing tailscale’s PID namespace  the shared
    # namespace caused routing-fix to be killed whenever tailscale restarted
    # (e.g. during gluetun health-check flaps), which dropped all routing
    # table changes and left traffic leaking through eth0 instead of tun0.
    cap_add:
      - NET_ADMIN
      - SYSLOG
    depends_on:
      warp:
        condition: service_healthy
      rule-compiler:
        condition: service_completed_successfully
    environment:
      - TZ=America/New_York
      - WARP_ENDPOINT=${WARP_ENDPOINT_1:-162.159.192.1}
      - BLOCKED_COUNTRIES=${BLOCKED_COUNTRIES}
      - TAILSCALE_ALLOW_LAN_P2P=${TAILSCALE_ALLOW_LAN_P2P:-false}
    volumes:
      - ./scripts/routing-fix.sh:/routing-fix.sh:ro
      # SECURITY: ./adguard/work/ is bind-mounted into this container.
      # Never write to it from the host or another container. The
      # single allowed writer is ‘rule-compiler‘ container  manual
      # edits corrupt the AdGuard cache + BoltDB session store.
      - ./adguard/work/userfilters:/userfilters:ro
      - ./adguard/work:/adguard_work:ro
      - .:/app
    restart: unless-stopped
    stop_grace_period: 1s
    logging:
      driver: "json-file"
      options:
        max-size: "1m"
        max-file: "1"

  adguardhome:
    # WARNING: AdGuard is pre-configured with hardcoded credentials (admin/nullexit).
    # This is fine for a local Mac running behind a NAT, but if you are deploying
    # this on a public cloud VPS with port 80/3000 exposed, change this immediately!
    image: adguard/adguardhome:v0.107.77
    container_name: adguardhome
    network_mode: service:warp
    depends_on:
      warp:
        condition: service_healthy
      rule-compiler:
        condition: service_completed_successfully
    environment:
      - TZ=America/New_York
    healthcheck:
      test: ["CMD-SHELL", "nc -z 127.0.0.1 5335 || exit 1"]
      interval: 2s
      timeout: 5s
      retries: 10
    volumes:
      # SECURITY: ./adguard/work/ is bind-mounted into this container.
      # Never write to it from the host or another container. The
      # single allowed writer is ‘rule-compiler‘ container  manual
      # edits corrupt the AdGuard cache + BoltDB session store.
      - ./adguard/work:/opt/adguardhome/work
      - ./adguard/conf:/opt/adguardhome/conf
    restart: unless-stopped
    stop_grace_period: 2s
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

  rule-compiler:
    image: nullexit-rule-compiler:v1.0.0
    build:
      context: ./scripts
      dockerfile: Dockerfile.rule-compiler
    container_name: rule-compiler
    volumes:
      - .:/app

  tor:
    build: ./docker/tor
```

```yaml
    image: nullexit-tor:v1.0.0
    container_name: tor
    network_mode: service:warp
    depends_on:
      warp:
        condition: service_healthy
    environment:
      - TZ=America/New_York
      - TOR_USE_BRIDGES=${TOR_USE_BRIDGES:-false}
      - TOR_BRIDGE_LINES_FILE=${TOR_BRIDGE_LINES_FILE:-./tor-bridges.txt}
      - TOR_PASSWORD=${ADGUARD_PASSWORD:-nullexit}
      - TOR_EXCLUDE_EXIT_NODES=${TOR_EXCLUDE_EXIT_NODES:-{us}}
    working_dir: /nullexit
    volumes:
      - ./:/nullexit:ro
      - ./output.log:/output.log:rw
    restart: unless-stopped
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

volumes:
  tailscale_state:

networks:
  default:
    ipam:
      config:
        - subnet: 10.200.1.0/24
```

13 scripts/common.sh

```bash
#!/bin/bash
# common.sh  shared bash utilities for nullexit scripts

#  Formatting
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
BOLD='\033[1m'
NC='\033[0m'

#  Constants & Environment
export MARKER_FILE="/tmp/nullexit-gateway-active.marker"
export PID_CAFFEINATE="/tmp/nullexit-caffeinate.pid"
export PID_DNS_WATCHER="/tmp/nullexit-dns-watcher.pid"
export DEFAULT_WARP_ENDPOINT_1="162.159.192.1"
export DEFAULT_WARP_ENDPOINT_2="162.159.193.1"

setup_standard_path() {
  export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
}

step() { echo -e "\n[$(date '+%Y-%m-%d %H:%M:%S')] ${BLUE}${BOLD} $*${NC}"; }
ok()   { echo -e "[$(date '+%Y-%m-%d %H:%M:%S')]   ${GREEN} $*${NC}"; }
warn() { echo -e "[$(date '+%Y-%m-%d %H:%M:%S')]   ${YELLOW} $*${NC}"; }
fail() { echo -e "[$(date '+%Y-%m-%d %H:%M:%S')]   ${RED} $*${NC}"; }
die()  { echo -e "\n[$(date '+%Y-%m-%d %H:%M:%S')]   ${RED} $*${NC}\n"; exit 1; }

#  Helper Functions

# Pure-bash timeout (no dependency on GNU coreutils' timeout)
# Runs a command with a safety cutoff so a wedged daemon can't hang the script.
run_with_timeout() {
  local timeout_sec="$1"
  shift
  if [ $# -eq 0 ]; then return 1; fi

  "$@" &
  local cmd_pid=$!
  CURRENT_BG_PID="$cmd_pid"

  (
```

```bash
    sleep "$timeout_sec"
    if kill -0 "$cmd_pid" 2>> output.log; then
      echo -e "\n[Timeout] Command '$*' exceeded $timeout_sec seconds. Terminating..." >&2
      kill -15 "$cmd_pid" 2>> output.log || true
      sleep 2
      if kill -0 "$cmd_pid" 2>> output.log; then
        kill -9 "$cmd_pid" 2>> output.log || true
      fi
    fi
  ) &
  local watcher_pid=$!

  # Disable set -e temporarily to capture exit status of wait
  set +e
  wait "$cmd_pid" 2>> output.log
  local exit_status=$?
  set -e

  # Kill the watcher since the command finished
  kill "$watcher_pid" 2>> output.log && wait "$watcher_pid" 2>> output.log || true

  CURRENT_BG_PID=""
  return "$exit_status"
}

# Check if the gateway is active (either containers are running, or host DNS is hijacked)
is_gateway_active() {
  # 1. Check if containers are running (suppress stderr to avoid errors if docker is down)
  if run_with_timeout 15 docker compose ps --status running 2>/dev/null | grep -q 'warp'; then
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (warp container is running)" >> "$LOG_FILE"
    return 0
  fi

  # 2. Check if host DNS was hijacked (not 1.1.1.1 and not default/empty)
  local current_dns=""
  local active_svc=""
  if [[ "$OSTYPE" == "darwin"* ]]; then
    active_svc=$(get_active_service)
    current_dns=$(networksetup -getdnsservers "$active_svc" 2>> output.log || true)
  else
    current_dns=$(resolvectl dns 2>/dev/null | awk '/Global|Link/ {for(i=4;i<=NF;i++) print $i}' | head -n1)
  fi

  if [[ -n "$current_dns" && "$current_dns" != "1.1.1.1" && ! "$current_dns" =~ "There aren't any DNS Servers" ]]; then
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (DNS was hijacked: $current_dns)" >> "$LOG_FILE"
    return 0
  fi

  # 3. Check if SOCKS proxy is enabled (macOS only)
  if [[ "$OSTYPE" == "darwin"* ]]; then
    local socks_proxy
    socks_proxy=$(networksetup -getsocksfirewallproxy "$active_svc" 2>> output.log || true)
    if echo "$socks_proxy" | grep -q "Enabled: Yes"; then
      echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (SOCKS proxy enabled)" >> "$LOG_FILE"
      return 0
    fi
  fi

  echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: FALSE (Containers down, DNS clean, SOCKS disabled)" >> "$LOG_FILE"
  return 1
}

# Reads an environment variable from a file (default: .env), strips quotes and whitespace
read_env_var() {
  local var_name="$1"
  local env_file="${2:-.env}"
  grep -E "^${var_name}=" "$env_file" 2>/dev/null | cut -d'=' -f2- | tr -d "\"\\'" | sed -e 's/^[[:space:]]*//' -e 's/[[:space:]]*$//' || echo ""
}

# Load KILL_SWITCH variable from .env (defaults to false if not found/empty)
export KILL_SWITCH
KILL_SWITCH=$(read_env_var "KILL_SWITCH" 2>/dev/null | tr '[:upper:]' '[:lower:]')
if [ -z "$KILL_SWITCH" ]; then
  KILL_SWITCH="false"
fi
```

```bash

# Function to get the active network service name (e.g., "Wi-Fi" or "USB 10/100 LAN")
get_active_service() {
  local iface
  iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')

  # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
  if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
    for i in en0 en1 en2 en3; do
      if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "inet "; then
        iface="$i"
        break
      fi
    done
  fi

  # Default fallback if still empty or not enX
  if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
    iface="en0"
  fi

  # Map interface (e.g., en0) to service name (e.g., Wi-Fi)
  local service
  service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B 1 "Device: $iface" | head -n 1 | sed -E 's/^\([0-9\*]+\) //')

  if [ -n "$service" ]; then
    echo "$service"
  else
    echo "Wi-Fi"
  fi
}

get_en0_service() {
  local service
  service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B1 "Device: en0" | head -1 | sed -E 's/^\([0-9\*]+\) //' || true)
  if [ -z "$service" ]; then
    echo "Wi-Fi"
  else
    echo "$service"
  fi
}

get_warp_endpoint_1() { read_env_var "WARP_ENDPOINT_1" ".env" | grep -Eo '^[0-9\.]+' || echo "$DEFAULT_WARP_ENDPOINT_1"; }
get_warp_endpoint_2() { read_env_var "WARP_ENDPOINT_2" ".env" | grep -Eo '^[0-9\.]+' || echo "$DEFAULT_WARP_ENDPOINT_2"; }

restart_tailscaled_daemon() {
  echo "  [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
  echo "  [Tailscale Recovery] Attempting to restart tailscaled service..."

  if run_with_timeout 15 brew services restart tailscale >> output.log 2>&1; then
    echo "  [Tailscale Recovery] Successfully restarted tailscaled (user service)."
  elif run_with_timeout 15 sudo -n brew services restart tailscale >> output.log 2>&1; then
    echo "  [Tailscale Recovery] Successfully restarted tailscaled (system service)."
  else
    echo "  [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
  fi
  sleep 3
}

disconnect_tailscale_host() {
  local ts_bin="${1:-tailscale}"
  if command -v "$ts_bin" >> output.log 2>&1; then
    echo "  Disconnecting host Tailscale from mesh..."

    # Check if actually connected first (with timeout  tailscaled could be wedged)
    local status_ok=false
    if run_with_timeout 5 "$ts_bin" status >> output.log 2>&1; then
      status_ok=true
    else
      local status_exit=$?
      # If it was a quick exit code 1 (disconnected), it is still reachable
      if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
        status_ok=true
      fi
```

```
    fi

    if [ "$status_ok" = "false" ]; then
      restart_tailscaled_daemon
    fi

    # Explicitly reset any exit-node so it doesn't linger.
    "$ts_bin" up >> output.log 2>&1 || true
    if run_with_timeout 10 "$ts_bin" set --ssh=true --accept-dns=false --exit-node= >> output.log 2>&1; then
      echo "  [] Exit-node preference cleared."
    else
      echo "  [!] tailscale up didn't respond (tailscaled may be wedged)"
    fi

    local ts_args=""
    if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
    if run_with_timeout 10 "$ts_bin" down $ts_args >> output.log 2>&1; then
      echo "  [] Tailscale disconnected."
    else
      echo "  [!] tailscale down didn't respond"
    fi
  else
    echo "  [!] tailscale CLI not found  skipping"
  fi
}

stop_pidfile_daemon() {
  local pid_file="$1"
  local label="$2"
  if [ -f "$pid_file" ]; then
    local pid
    pid=$(cat "$pid_file")
    if kill -0 "$pid" 2>/dev/null; then
      echo "  Stopping $label (PID $pid)..."
      sudo -n kill "$pid" 2>> output.log || kill "$pid" 2>> output.log || true
      sleep 0.5
      if kill -0 "$pid" 2>/dev/null; then
        sudo -n kill -9 "$pid" 2>> output.log || kill -9 "$pid" 2>> output.log || true
      fi
    fi
    rm -f "$pid_file"
  fi
}

disable_all_proxies() {
  local svc="$1"
  if [ -n "$svc" ]; then
    sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
    sudo -n networksetup -setwebproxystate "$svc" off 2>> output.log || true
    sudo -n networksetup -setsecurewebproxystate "$svc" off 2>> output.log || true
  fi
}

flush_dns_cache() {
  if [[ "$OSTYPE" == "darwin"* ]]; then
    sudo -n dscacheutil -flushcache 2>> output.log || true
    sudo -n killall -HUP mDNSResponder 2>> output.log || true
  else
    resolvectl flush-caches 2>> output.log || true
  fi
}

wait_for_dhcp_settle() {
  # Resolve physical interface (Wi-Fi or Ethernet)
  local physical_iface
  physical_iface=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet)/{getline; print $2; exit}')
  [ -z "$physical_iface" ] && physical_iface="en0"

  echo -n "Waiting for physical network interface ($physical_iface) DHCP lease to settle..."
  local settled=false
  for attempt in {1..60}; do
    PHYSICAL_GW=$(ipconfig getpacket "$physical_iface" 2>> output.log | awk -F'[{}]' '/router /{print $2}')
    local local_ip
    local_ip=$(ifconfig "$physical_iface" 2>/dev/null | awk '/inet /{print $2}')

    if [ -n "$PHYSICAL_GW" ] && [[ "$PHYSICAL_GW" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
       [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
       [[ "$local_ip" != "169.254."* ]]; then
```

```
      echo " settled (Interface IP: $local_ip, Router IP: $PHYSICAL_GW)."
      settled=true
      break
    fi

    if [ "$attempt" -gt 15 ] && [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && [[ "$local_ip" != "169.254."* ]]; then
      local fallback_gw
      fallback_gw=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
      if [ -n "$fallback_gw" ] && [[ "$fallback_gw" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
        PHYSICAL_GW="$fallback_gw"
      fi
      echo " static/settled detected (Interface IP: $local_ip, Gateway IP: ${PHYSICAL_GW:-unknown})."
      settled=true
      break
    fi
    echo -n "."
    sleep 0.5
  done
  if [ "$settled" = "false" ]; then
    echo " timed out or no active link. Proceeding anyway."
  fi
}

reset_sharing_services() {
  if [[ "$OSTYPE" == "darwin"* ]]; then
    sudo -n killall sharingd rapportd 2>> output.log || true
  fi
}

read_adguard_ip() {
  if [ -f ".gateway_ip" ]; then
    tr -d '\r' < ".gateway_ip" | awk 'NR==1{print $1;exit}'
  fi
}

is_kill_switch_enabled() {
  [ "$KILL_SWITCH" = "true" ]
}

add_warp_bypass_routes() {
  local target="${1:-}"
  local ep1
  ep1=$(get_warp_endpoint_1)
  local ep2
  ep2=$(get_warp_endpoint_2)

  if [[ "$OSTYPE" == "darwin"* ]]; then
    local routing_arg=""
    local msg_via=""
    if [ -n "$target" ]; then
      if [[ "$target" =~ ^en[0-9]+$ || "$target" == "bridge"* ]]; then
        routing_arg="-interface $target"
        msg_via="interface $target"
      else
        routing_arg="$target"
        msg_via="gateway $target"
      fi
    else
      local gateway_ip
      gateway_ip=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
      if [ -z "$gateway_ip" ]; then
        local iface
        iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
        if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
          iface="en0"
        fi
        routing_arg="-interface $iface"
        msg_via="interface $iface"
      else
        routing_arg="$gateway_ip"
        msg_via="gateway $gateway_ip"
      fi
    fi

    echo -e "\nAdding host bypass routes for Cloudflare WARP endpoints via $msg_via..."
    sudo -n route delete -host "$ep1" 2>/dev/null || true
    sudo -n route delete -host "$ep2" 2>/dev/null || true
    sudo -n route add -host "$ep1" $routing_arg >> output.log 2>&1 || true
    sudo -n route add -host "$ep2" $routing_arg >> output.log 2>&1 || true

```

```
    echo -e "Adding host bypass routes for Tailscale control plane via $msg_via..."
    sudo -n route delete -net 192.200.0.0/24 2>/dev/null || true
    sudo -n route add -net 192.200.0.0/24 $routing_arg >> output.log 2>&1 || true

    echo -e "Adding host bypass routes for Tailscale DERP relays via $msg_via..."
    local derp_ips
    derp_ips=$(curl -s --connect-timeout 5 https://login.tailscale.com/derpmap/default | grep -oE '"IPv4"[[:space:]]*:[[:space:]]*"[0-9.]+"' | cut -d'"' -f4 | grep -E '^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$' || true)
    if [ -n "$derp_ips" ]; then
      echo "$derp_ips" > /tmp/nullexit-derp-ips.txt
      for ip in $derp_ips; do
        sudo -n route delete -host "$ip" 2>/dev/null || true
        sudo -n route add -host "$ip" $routing_arg >> output.log 2>&1 || true
      done
    fi
  else
    local gateway_ip="${target}"
    if [ -z "$gateway_ip" ]; then
      gateway_ip=$(ip route show default 2>/dev/null | awk '/default/ {print $3}')
    fi
    if [ -n "$gateway_ip" ]; then
      sudo ip route add "$ep1" via "$gateway_ip" >> output.log 2>&1 || true
      sudo ip route add "$ep2" via "$gateway_ip" >> output.log 2>&1 || true
    fi
  fi
}

remove_warp_bypass_routes() {
  local ep1
  ep1=$(get_warp_endpoint_1)
  local ep2
  ep2=$(get_warp_endpoint_2)

  if [[ "$OSTYPE" == "darwin"* ]]; then
    echo -e "\nRemoving host bypass routes for Cloudflare WARP endpoints..."
    sudo -n route delete -host "$ep1" >> output.log 2>&1 || true
    sudo -n route delete -host "$ep2" >> output.log 2>&1 || true
    echo -e "Removing host bypass routes for Tailscale control plane..."
    sudo -n route delete -net 192.200.0.0/24 >> output.log 2>&1 || true
    if [ -f /tmp/nullexit-derp-ips.txt ]; then
      echo -e "Removing host bypass routes for Tailscale DERP relays..."
      while read -r ip; do
        sudo -n route delete -host "$ip" >> output.log 2>&1 || true
      done < /tmp/nullexit-derp-ips.txt
      rm -f /tmp/nullexit-derp-ips.txt
    fi
  else
    sudo ip route del "$ep1" >> output.log 2>&1 || true
    sudo ip route del "$ep2" >> output.log 2>&1 || true
  fi
}

bounce_wifi_interfaces() {
  if [[ "$OSTYPE" == "darwin"* ]]; then
    local wifi_port
    wifi_port=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2}')
    if [ -n "$wifi_port" ]; then
      sudo -n networksetup -setairportpower "$wifi_port" off 2>> output.log || true
      sleep 2
      sudo -n networksetup -setairportpower "$wifi_port" on 2>> output.log || true
      echo "Wi-Fi bounced (interface $wifi_port)"
      sleep 3
    else
      echo "Could not detect Wi-Fi interface. Bouncing fallback interfaces..."
      set +e
      for iface in en0 en1 en2 en3 en4 en5; do
        if ifconfig "$iface" >> output.log 2>&1; then
          sudo -n ifconfig "$iface" down 2>> output.log || true
          sudo -n ifconfig "$iface" up 2>> output.log || true
        fi
      done
      set -e
      echo "Fallback interfaces bounced"
    fi
  fi
}

get_active_interface() {
  local iface
```

```bash
  iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')

  # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
  if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
    for i in en0 en1 en2 en3; do
      if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "inet "; then
        iface="$i"
        break
      fi
    done
  fi

  # Default fallback if still empty or not enX
  if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
    iface="en0"
  fi

  echo "$iface"
}

enable_killswitch() {
  if [[ "$OSTYPE" == "darwin"* ]]; then
    if ! is_kill_switch_enabled; then
      return 0
    fi

    local ext_if
    ext_if=$(get_active_interface)
    local ep1
    ep1=$(get_warp_endpoint_1)
    local ep2
    ep2=$(get_warp_endpoint_2)
    local pf_conf_path
    pf_conf_path="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/pf.conf"

    echo -e "\nEnabling macOS Packet Filter (PF) Kill-Switch on $ext_if..."

    # Enable PF and capture the reference token
    local token_out
    token_out=$(sudo -n pfctl -E 2>> output.log || true)
    local token
    token=$(echo "$token_out" | awk '/Token/{print $NF}')
    if [ -n "$token" ]; then
      echo "$token" > /tmp/nullexit-pf-token.txt
      echo "  [] PF enabled with token $token."
    else
      echo "  [!] WARNING: Could not capture PF reference token (may already be enabled or sudo failed)."
    fi

    # Load the rules into the anchor
    if sudo -n pfctl -D "ext_if=$ext_if" -a com.apple/nullexit -f "$pf_conf_path" >> output.log 2>&1; then
      echo "  [] Ruleset loaded into anchor com.apple/nullexit."
    else
      echo "  [!] ERROR: Failed to load ruleset into anchor."
    fi

    # Populate tables in the anchor
    sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T flush >> output.log 2>&1 || true
    sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep1" >> output.log 2>&1 || true
    sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep2" >> output.log 2>&1 || true

    if [ -f /tmp/nullexit-derp-ips.txt ]; then
      local derp_ips
      derp_ips=$(tr '\n' ' ' < /tmp/nullexit-derp-ips.txt)
      if [ -n "$derp_ips" ]; then
        sudo -n pfctl -a com.apple/nullexit -t derp_relays -T flush >> output.log 2>&1 || true
        sudo -n pfctl -a com.apple/nullexit -t derp_relays -T add $derp_ips >> output.log 2>&1 || true
      fi
    fi
    echo "  [] Kill-switch tables populated."
  fi
}

disable_killswitch() {
  if [[ "$OSTYPE" == "darwin"* ]]; then
    if ! is_kill_switch_enabled; then
      return 0
    fi
```

```
    echo -e "\nDisabling macOS PF Kill-Switch..."

    # Flush rules from anchor com.apple/nullexit
    sudo -n pfctl -a com.apple/nullexit -F all >> output.log 2>&1 || true

    # Release the enable reference if token is present
    if [ -f /tmp/nullexit-pf-token.txt ]; then
      local token
      token=$(cat /tmp/nullexit-pf-token.txt)
      if [ -n "$token" ]; then
        sudo -n pfctl -X "$token" >> output.log 2>&1 || true
        echo "  [] Released PF enable reference token $token."
      fi
      rm -f /tmp/nullexit-pf-token.txt
    else
      echo "  [] Rules flushed from anchor com.apple/nullexit."
    fi
  fi
}


write_host_ips() {
  # Gather all active IPv4 addresses on the host and write to .host_ips
  # routing-fix.sh will read this file to explicitly whitelist them for direct Tailscale P2P
  local repo_dir
  repo_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

  if [[ "$OSTYPE" == "darwin"* ]]; then
    ifconfig | awk '/inet /{print $2}' | grep -v '127.0.0.1' > "$repo_dir/.host_ips" 2>/dev/null || true
  else
    ip -4 addr show 2>/dev/null | awk '/inet /{print $2}' | cut -d/ -f1 | grep -v '127.0.0.1' > "$repo_dir/.host_ips" 2>/dev/null || true
  fi
}
```

14 scripts/watcher.sh

```
#!/bin/bash
# scripts/watcher.sh  nullexit post-wake / post-roam recovery watcher
#
# Long-running daemon. Launched by launchd as a LaunchAgent at
# ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
# (label com.nullexit.wake-recovery).
#
# Two independent listeners run as backgrounded pipelines:
#
#   (1) WAKE listener  ‘log stream‘ live-follows the macOS unified log for
#       power-management events (lid closewake, manual wake, DarkWake from
#       clamshell). Each event triggers run_recover (with a global debounce so
#       a single wake that emits 2-3 log lines only causes ONE post-wake run).
#
#   (2) NETWORK listener  ‘scutil n.watch‘ on ‘State:/Network/Global/IPv4‘
#       fires on every Wi-Fi roam, ethernet swap, hotspot join, captive-portal
#       re-bind, VPN up/down, etc. Same debounce.
#
# Both listeners call run_recover, which:
#   - checks /tmp/nullexit-gateway-active.marker (only act when toggle.sh left
#     the gateway up)
#   - debounces (10s default) so multiple events don't pile up
#   - shells out to ‘bash <repo>/recover.sh --post-wake‘ directly (no GUI auth prompt needed due to NOPASSWD sudoers)
#
# See devref.md 10.29 for the full Apple power management primer and the
# rationale behind using ‘log stream‘ + ‘scutil n.watch‘ from a shell daemon.

# NOTE: errexit (‘set -e‘) is intentionally NOT enabled. This is a long-running
# daemon, not a discrete-step script, and errexit actively breaks the reconnect
# loops below: any failed pipe (log stream on rotation, scutil on state churn)
# would kill the outer subshell before ‘echo "reconnecting..."; sleep 5;‘ runs,
# AND the parent's final ‘wait‘ would propagate a non-zero child exit and kill
# the entire watcher. Every error path is already wrapped in ‘|| true‘ /
# ‘2>/dev/null‘ fallbacks.
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$SCRIPT_DIR/common.sh"
setup_standard_path

```

```
LOG="$SCRIPT_DIR/../output.log"
# Resolve our own directory; recover.sh lives one level up at the repo root.
# This makes the daemon install-agnostic  no need to template the path
# in launchd, no need to keep a deploy-specific hardcoded location in sync.
RECOVER="$SCRIPT_DIR/../recover.sh"
MARKER="$MARKER_FILE"
DEBOUNCE_FILE="/tmp/nullexit-watcher.last-recovery"
DEBOUNCE_SECONDS="${NULLEXIT_DEBOUNCE_SECONDS:-10}"

exec >> "$LOG" 2>&1

#  Single-instance lock
# Without this, repeated lid-closewake cycles under macOS Sonoma+ accumulate
# orphan watcher.sh processes: launchd suspends on sleep and SIGCONT-resumes
# on wake, and KeepAlive.Crashed=true caused relaunch on any non-zero exit;
# the listener grandchildren (log stream | while ...   and  (echo n.add ...;
# sleep 86400) | scutil | while ...) stay alive because pipe producers don’t
# reliably respond to plain SIGTERM if the consumer is in a half-closed state.
#
# We solve it with a PID-file lock (POSIX-portable; macOS does NOT ship ‘flock‘).
# The trap below removes the lock file on every exit so a stale lock never
# outlives a crashed watcher. On launch: read the existing PID, check it is
# alive AND is actually a ‘scripts/watcher.sh‘ process; if so, exit 0; else
# overwrite with our PID and proceed. The check-then-write window has a TOCTOU
# race but is microseconds wide and only matters for hand-launched duplicate
# test invocations; production is single-launch via launchctl load -w.
LOCK_FILE="/tmp/nullexit-watcher.lock"
LOCK_PID=""
if [ -e "$LOCK_FILE" ]; then
  LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
  if [ -n "$LOCK_PID" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
    # Verify the holder is actually a scripts/watcher.sh process (not some
    # unrelated process that got the recycled PID). The exact-launchd pattern
    # ‘bash <abs-path>/scripts/watcher.sh‘ is hard to accidentally match with
    # a ‘grep scripts/watcher.sh .‘ invocation because we anchor on ‘^bash‘
    # followed by whitespace and end-of-line on the script path.
    if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -qE "(^|[[:space:]])bash[[:space:]]+.*scripts/watcher\.sh$"; then
      echo "[$(date -u +%FT%TZ)] watcher.sh: another instance holds $LOCK_FILE (pid=$LOCK_PID), exiting"
      exit 0
    fi
  fi
fi
echo $$ > "$LOCK_FILE"

echo "[$(date -u +%FT%TZ)] watcher.sh start (pid=$$ ppid=$PPID user=$(id -un) lock=acquired)"

# Treat launchd-resume as a wake signal. Apple’s launchd SUSPENDS LaunchAgents
# during system sleep and restores them on wake; during that suspended gap,
# macOS’s ‘log stream‘ cannot catch the wake event that prompted the resume.
# The next thing that happens after wake IS our own startup, so we always
# fire one post-wake run unconditionally here. The marker check + debounce
# in run_recover() make this idempotent: if the gateway isn’t up or we just
# debounced, it no-ops. Without this, the FIRST wake-after-install goes
# unseen and the watcher appears broken until the SECOND wake.
#  Helper: run recover.sh --post-wake if the gateway is active
run_recover() {
  local why="$1"
  if [ ! -f "$MARKER" ]; then
    echo "[$(date -u +%FT%TZ)] $why  no $MARKER, skip"
    return 0
  fi
  local now last
  now=$(date +%s)
  last=$(cat "$DEBOUNCE_FILE" 2>/dev/null || echo 0)
  if [ "$((now - last))" -lt "$DEBOUNCE_SECONDS" ]; then
    echo "[$(date -u +%FT%TZ)] $why  debounced ($((now - last))s since last, threshold ${DEBOUNCE_SECONDS}s)"
    return 0
  fi
  echo "$now" > "$DEBOUNCE_FILE"
  echo "[$(date -u +%FT%TZ)] $why  bash $RECOVER --post-wake"
  bash "$RECOVER" --post-wake
  local exit_code=$?
  echo "$(date +%s)" > "$DEBOUNCE_FILE"
  echo "[$(date -u +%FT%TZ)] $why  exit=$exit_code (now=$(date +%s))"
  return $exit_code
}

if [ -f "$MARKER" ]; then
  run_recover "initial post-wake (launchd resume = wake signal)"
```

```
fi

#  LAN P2P Auto-Detection
# Detects whether the current network allows safe Tailscale LAN P2P by:
#   1. Checking WPA2-Enterprise (802.1x) via airport -I  always forces P2P=false
#      because enterprise networks almost universally have AP Client Isolation.
#   2. AP Isolation probe  sends a broadcast ping and checks if ANY LAN host responds.
#      If no LAN hosts respond on a non-empty network, AP Isolation is active.
# Writes 'true' or 'false' to .lan_p2p_detected in the repo root.
# routing-fix.sh reads this file dynamically every 30s loop iteration.
P2P_OVERRIDE_FILE="$SCRIPT_DIR/../.lan_p2p_detected"


detect_lan_p2p_mode() {
  local reason="unknown" allow="false"

  # Step 1: Detect Wi-Fi security type via system_profiler (airport binary removed in macOS 14.4+)
  # SPAirPortDataType lists all known networks; the first "Security:" line is the current association.
  local security
  security=$(system_profiler SPAirPortDataType 2>/dev/null \
    | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF; exit}')

  if [ -z "$security" ]; then
    # Not on Wi-Fi or system_profiler unavailable  fail safe
    reason="no-wifi" allow="false"
  elif echo "$security" | grep -qi "Enterprise"; then
    # WPA2/WPA3 Enterprise = 802.1x  always AP isolated
    reason="802.1x-enterprise" allow="false"
  elif echo "$security" | grep -qi "Personal\|None\|Open"; then
    # WPA2/WPA3 Personal or open  probe for AP isolation
    local gw
    gw=$(route -n get 1.1.1.1 2>/dev/null | awk '/gateway:/{print $2}')
    if [ -z "$gw" ]; then
      reason="no-default-route" allow="false"
    else
      local responses
      responses=$(ping -c 3 -t 1 -q "$gw" 2>/dev/null | awk '/packets transmitted/{print $4}')
      if [ "${responses:-0}" -gt 0 ]; then
        reason="wpa-personal-lan-reachable" allow="true"
      else
        reason="wpa-personal-ap-isolated" allow="false"
      fi
    fi
  else
    # Unknown security type  fail safe
    reason="unknown-security-type" allow="false"
  fi

  echo "[$(date -u +%FT%TZ)] P2P detect: security='${security:-none}'  allow=${allow} (reason: ${reason})"
  echo "$allow" > "$P2P_OVERRIDE_FILE"
  write_host_ips
}

# Run once on startup
detect_lan_p2p_mode

#  Listener 1: WAKE events via unified log
# Predicate picks up:
#   * "didWake"              regular kernel wake events (lid open, RTC alarm)
#   * "Wake from Sleep"      powerd-driven normal wake
#   * "DarkWake"             clamshell-display wake that didn't fully wake the
#                             Mac (relevant when lid opens during display sleep)
#   * "Waking from"          generic catch-all for variations in newer macOS
# `[c]` makes the match case-insensitive (the unified log uses mixed-case
# phrases). --info reduces noise by dropping debug/trace log levels.
# Subsystem-based predicate catches every powermanagement emission (willSleep,
# didWake, didDim, didUndim, DarkWake, etc.) regardless of message phrasing
# across kernel versions. Phrasing-based predicates ('didWake', 'Wake from Sleep')
# are fragile across macOS releases. We accept the extra log noise because the
# run_recover() debounce + marker check filter out anything that isn't a real
# gateway-relevant event.
(
  # Reconnect loop: macOS rotates the unified log buffer periodically; when
  # that happens, this `log stream` subscriber's pipe EOFs, the inner
  # `while read` consumer terminates, and without this wrapper the listener
  # would be silently dead for the rest of the watcher's lifetime.
  while :; do
    log stream --predicate \
      'subsystem == "com.apple.powermanagement"' \
       --style compact --info 2>/dev/null \
```

```
   | while IFS= read -r line; do
     [ -z "$line" ] && continue
     run_recover "WAKE: $(echo "$line" | head -c 100)"
    done
  echo "[$(date -u +%FT%TZ)] log stream subshell exited; reconnecting in 5s"
  sleep 5
 done
) &

#  Listener 2: NETWORK state changes via scutil
# ‘scutil n.watch‘ on a state key prints SCEventUpdate lines whenever the key
# changes. We watch Global/IPv4 because that’s the umbrella that catches link,
# address, route, and DNS changes for ALL interfaces in one namespace.
# The pipe is kept alive by an ‘sleep 86400‘ loop so scutil never sees EOF
# from us (which would silently terminate the watch).
(
 # Reconnect loop: scutil can exit noisily on certain state changes (rare,
 # but observed). Without the wrapper, the network listener’s pipe EOFs and
 # every subsequent roam goes unseen until launchd restarts us.
 while :; do
  (
   # Watch both IPv4 and IPv6 umbrella state  IPv6-only or dual-stack
   # networks never fire on the IPv4 key alone, and the user’s first roam
   # in a hotel / airport / on cellular hotspot is often IPv6-first under
   # NAT64.
   echo "n.add State:/Network/Global/IPv4"
   echo "n.add State:/Network/Global/IPv6"
   echo "n.watch"
   while true; do sleep 86400; done
  ) | script -q /dev/null scutil 2>/dev/null \
   | while IFS= read -r line; do
     case "$line" in
      *changedKey*|*State:/Network/Global/IPv*|*n.state*|*SCEventUpdate*)
       detect_lan_p2p_mode
       run_recover "NET: $(echo "$line" | head -c 100)"
       ;;
      *) ;;
     esac
    done
  echo "[$(date -u +%FT%TZ)] scutil pipe exited; reconnecting in 5s"
  sleep 5
 done
) &
#  Listener 3: TUNNEL_FAILED_CLOSED.marker Watcher
# Polls for the fail-closed marker dropped by the routing-fix container.
(
 last_notified=0
 while :; do
  if [ -f "$SCRIPT_DIR/../TUNNEL_FAILED_CLOSED.marker" ]; then
   now=$(date +%s)
   # Notify once every 5 minutes (300s) if it stays failed
   if [ "$((now - last_notified))" -ge 300 ]; then
    osascript -e ’display notification "VPN Tunnel Health Check Failed. Internet traffic is blackholed to prevent IP leak." with title "NullExit Alert"’ || true
    last_notified=$now
   fi
  else
   last_notified=0
  fi
  sleep 3
 done
) &

#  Lifetime
# ‘wait‘ blocks until both background pipelines exit (which they normally
# never do). launchctl ‘bootout‘/SIGTERM triggers the trap, which kills the
# whole process group so the sleep-forever in the scutil pipe also dies.
# We catch TERM/INT/HUP for explicit signals and EXIT for any normal-exit path
# (so listeners always get cleaned up even if ‘exit 0‘ runs somewhere).
# ‘rm -f $LOCK_FILE‘ first so a stale lock can never block the next launch.
# ‘pkill -TERM -P $$‘ then targets direct listener-children, then ‘kill 0‘
# takes care of any backgrounded group-mates not reachable via the parent.
trap ’echo "[$(date -u +%FT%TZ)] watcher.sh terminating (signal/exit=$?)"; rm -f "$LOCK_FILE" 2>/dev/null || true; pkill -TERM -P $$ 2>/dev/null || true; kill 0 2>/dev/null || true; exit 0’ TERM INT HUP EXIT
wait
```

15 scripts/crypto.sh

```bash
#!/usr/bin/env bash
# crypto.sh - Cryptographic Integrity Check

set -euo pipefail
cd "$(dirname "$0")/.."

if [ "$#" -ne 1 ] || { [ "$1" != "--sign" ] && [ "$1" != "--verify" ]; }; then
  echo "Usage: $0 --sign | --verify"
  exit 1
fi

if [ ! -f .env ]; then
  echo "Error: .env not found."
  exit 1
fi

SEED=$(grep "^NULLEXIT_SEED=" .env | cut -d ’=’ -f2)
if [ -z "$SEED" ]; then
  echo "Error: NULLEXIT_SEED not found in .env."
  exit 1
fi

if [ "$1" == "--sign" ]; then
  echo "Generating cryptographic signatures using seed..."
  rm -f .signatures
  for file in toggle.sh recover.sh scripts/common.sh scripts/routing-fix.sh scripts/toggle-linux.sh scripts/recover-linux.sh scripts/diagnose-host-leak.sh scripts/watcher.sh scripts/pf.conf post-rules.txt docker-compose.yml; do
    if [ -f "$file" ]; then
      hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk ’{print $NF}’)
      echo "$file:$hash" >> .signatures
      echo "  [Signed] $file"
    else
      echo "  [Skipped] $file not found"
    fi
  done
  echo "Signatures saved to .signatures"
  exit 0
fi

if [ "$1" == "--verify" ]; then
  if [ ! -f .signatures ]; then
    echo "Error: .signatures file missing. Run $0 --sign first."
    exit 1
  fi
  FAIL=0
  while IFS=: read -r file expected_hash; do
    if [ -z "$file" ]; then continue; fi
    if [ ! -f "$file" ]; then
      echo "[FAIL] $file is missing!"
      FAIL=1
      continue
    fi
    actual_hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk ’{print $NF}’)
    if [ "$actual_hash" != "$expected_hash" ]; then
      echo "[FAIL] $file has been modified! Hash mismatch."
      FAIL=1
    fi
  done < .signatures

  if [ "$FAIL" -eq 1 ]; then
    echo ""
    echo "CRITICAL: Script integrity verification failed!"
    echo "One or more core files have been modified without authorization."
    echo "To authorize these changes, run: ./scripts/crypto.sh --sign"
    echo ""
    exit 1
  fi
  exit 0
fi
```

16 .aiignore

```
# Environment variables and secrets
.env
tor-bridges.txt
```

17 .gitignore

```
# Environment variables and secrets
.env

# wgcf generated files containing keys and account info
wgcf-account.toml
wgcf-profile.conf

# Tailscale persistent state containing node identity
tailscale-state/

# Binaries
wgcf

# macOS App bundles
*.app/
*.app

# AdGuard Home persistent state
adguard/
.gateway_ip
logs.txt

#
stability_incident_report.md
output.log
.DS_Store
*.desktop
blocked.log
wtf.md
host-leak-diagnostic-*.txt
nullexit_review.md
nullexit_unified.tex
nullexit_unified.txt

nullexit_unified.synctex.gz
tor-bridges.txt

# Runtime state files
.host_ips
.lan_p2p_detected
.signatures
TUNNEL_FAILED_CLOSED.marker

# Python cache
__pycache__/
*.py[cod]
```

18 .host_ips

```
172.17.52.223
100.109.94.19
192.168.64.1
```

19 Recover-Gateway.applescript

```
tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
try
  do shell script "true" with administrator privileges
on error
  return
end try
```

```
tell application "Terminal"
  activate
  do script "cd " & quoted form of scriptDir & "; bash recover.sh"
end tell
```

20 Toggle-Gateway.applescript

```
tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
try
  do shell script "true" with administrator privileges
on error
  return
end try
tell application "Terminal"
  activate
  do script "cd " & quoted form of scriptDir & "; ./toggle.sh"
end tell
```

21 benchmarks/benchmark.sh

```
#!/usr/bin/env bash

# benchmark.sh
# Automates the full benchmarking suite (both Python download test and Lighthouse HTML test)
# Runs the suite with Gateway ON, toggles OFF, runs again, then toggles back ON.

set -e

# Change to project root so we can access toggle.sh reliably
cd "$(dirname "$0")/.."

# Ensure jq is installed
if ! command -v jq &> /dev/null; then
    echo "Error: ’jq’ is not installed. Please run ’brew install jq’"
    exit 1
fi

run_lighthouse() {
    local url=$1
    local file_prefix=$2
    echo "  [Lighthouse] Testing $url ..."

    # Run lighthouse headlessly, silencing standard output
    npx -y lighthouse "$url" --chrome-flags="--headless" --only-categories=performance --output=json --output-path="${file_prefix}.json" >/dev/null 2>&1

    # Parse the results
    local score=$(jq -r ’.categories.performance.score’ "${file_prefix}.json")
    local tti=$(jq -r ’.audits["interactive"].displayValue’ "${file_prefix}.json")
    local si=$(jq -r ’.audits["speed-index"].displayValue’ "${file_prefix}.json")

    # Convert score to percentage
    local score_pct=$(awk "BEGIN {print $score*100}")

    echo "    -> Performance Score: ${score_pct}%"
    echo "    -> Time to Interactive: $tti"
    echo "    -> Speed Index: $si"

    # Cleanup
    rm -f "${file_prefix}.json"
}

echo "============================================="
echo "   PHASE 1: Testing with Gateway ON"
echo "============================================="
# 1. Run raw Python download test
python3 benchmarks/test_load.py

# 2. Run Lighthouse HTML render test
echo ""
echo "Starting Lighthouse HTML Tests (this takes ˜30-60s per site)..."
run_lighthouse "https://www.cnet.com/" "on_cnet"
```

```
run_lighthouse "https://www.independent.co.uk/" "on_ind"


echo ""
echo "============================================="
echo "   PHASE 2: Toggling Gateway OFF"
echo "============================================="
./toggle.sh
echo "Waiting 10 seconds for macOS Wi-Fi to stabilize after DNS reset..."
sleep 10


echo ""
echo "============================================="
echo "   PHASE 3: Testing with Gateway OFF"
echo "============================================="
# 1. Run raw Python download test
python3 benchmarks/test_load.py

# 2. Run Lighthouse HTML render test
echo ""
echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
run_lighthouse "https://www.cnet.com/" "off_cnet"
run_lighthouse "https://www.independent.co.uk/" "off_ind"


echo ""
echo "============================================="
echo "   PHASE 4: Restoring Gateway ON"
echo "============================================="
./toggle.sh

echo ""
echo "============================================="
echo " BENCHMARKS COMPLETE!"
echo "You can scroll up to compare Phase 1 (AdGuard ON) vs Phase 3 (AdGuard OFF)."
echo "============================================="
```

22 benchmarks/test_load.py

```
import urllib.request
import urllib.error
import time
from html.parser import HTMLParser
import concurrent.futures

class ResourceParser(HTMLParser):
    def __init__(self):
        super().__init__()
        self.urls = set()

    def handle_starttag(self, tag, attrs):
        attrs = dict(attrs)
        url = None
        if tag in ['img', 'script']:
            url = attrs.get('src')
        elif tag == 'link' and attrs.get('rel') == 'stylesheet':
            url = attrs.get('href')

        if url and url.startswith('http'):
            self.urls.add(url)

def fetch_url(url):
    try:
        req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
        urllib.request.urlopen(req, timeout=3)
        return True
    except Exception:
        return False

def measure_url(target_url):
    print(f"\nTesting {target_url} Load...")
    start_total = time.time()

    # Fetch main HTML
    try:
```

```
        req = urllib.request.Request(target_url, headers={’User-Agent’: ’Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36’})
        response = urllib.request.urlopen(req, timeout=5)
        html = response.read().decode(’utf-8’, errors=’ignore’)
    except Exception as e:
        print(f"Failed to fetch {target_url}: {e}")
        return

    parser = ResourceParser()
    parser.feed(html)
    urls = list(parser.urls)
    print(f"Found {len(urls)} subresources to fetch (scripts, images, css)...")

    # Concurrently fetch subresources
    success_count = 0
    fail_count = 0
    max_threads = min(64, max(1, len(urls)))
    with concurrent.futures.ThreadPoolExecutor(max_workers=max_threads) as executor:
        results = list(executor.map(fetch_url, urls))

    success_count = sum(1 for r in results if r)
    fail_count = len(results) - success_count

    total_time = time.time() - start_total
    print(f"Time taken: {total_time:.2f} seconds")
    print(f"Successfully fetched: {success_count}, Failed/Blocked: {fail_count}")

urls_to_test = [
    "https://www.dailymail.co.uk/home/index.html",
    "https://www.cnet.com/",
    "https://www.independent.co.uk/"
]

for u in urls_to_test:
    measure_url(u)
```

23 black_list.txt

```
ad.doubleclick.net
adclick.g.doubleclick.net
adeventtracker.spotify.com
adfstat.yandex.ru
ads-api.tiktok.com
ads-api.twitter.com
ads-fa.spotify.com
ads-sg.tiktok.com
ads.tiktok.com
adserver.unityads.unity3d.com
adsfs.oppomobile.com
an.facebook.com$important
connect.facebook.net$important
pixel.facebook.com$important
analytics.query.yahoo.com
analytics.s3.amazonaws.com
analytics.spotify.com
analyticsengine.s3.amazonaws.com
appmetrica.yandex.ru
audio2.spotify.com
bdapi-ads.realmemobile.com
bdapi-in-ads.realmemobile.com
bounceexchange.com
bs.serving-sys.com
business-api.tiktok.com
ck.ads.oppomobile.com
crashdump.spotify.com
data.ads.oppomobile.com
data.mistat.india.xiaomi.com
data.mistat.rus.xiaomi.com
data.mistat.xiaomi.com
desktop.spotify.com
doubleclick.net
ds.metric.spotify.com
gemini.yahoo.com
googleads.g.doubleclick.net
googleadservices.com
googlesyndication.com
```

```
grs.hicloud.com
gtm.spotify.com
iot-eu-logser.realme.com
iot-logser.realme.com
log.byteoversea.com
log.fc.yahoo.com
log.spotify.com
m.doubleclick.net
mediavisor.doubleclick.net
metrics.spotify.com
metrika.yandex.ru
omaze.com
pagead2.googlesyndication.com
securepubads.g.doubleclick.net
static.doubleclick.net
stats.g.doubleclick.net
tracking.rus.miui.com
udcm.yahoo.com
v.jwpcdn.com
weblb-wg.gslb.spotify.com
webview.unityads.unity3d.com
diagassets.apple.com
iphonesubmissions.apple.com
radarsubmissions.apple.com
metrics.apple.com
xp.apple.com
heads-fa.spotify.com
heads4.spotify.com
pixel.spotify.com
segs.spotify.com
audio-fa.spotify.com
events.data.microsoft.com
datarouter-prod.ak.epicgames.com
nel.cloudflare.com
eu-mobile.events.data.microsoft.com
stocks-analytics-events.apple.com
```

24 cloud-init.yaml

```
#cloud-config
# Generic Cloud-Init Bootstrap Script for Remote Exit Nodes (Ubuntu/Debian)
# Paste this into the "User Data" or "Initialization Script" section when creating a VPS.

package_update: true
package_upgrade: true

packages:
  - apt-transport-https
  - ca-certificates
  - curl
  - gnupg
  - lsb-release
  - git
  - python3
  - python3-pip

runcmd:
  # 1. Install Docker natively
  - curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
  - echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
  - sudo apt-get update
  - sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

  # 2. Add default user (ubuntu) to docker group
  - usermod -aG docker ubuntu || true
  - usermod -aG docker debian || true

  # 3. Create the installation directory
  - mkdir -p /opt/nullexit
  - chown -R ubuntu:ubuntu /opt/nullexit || chown -R debian:debian /opt/nullexit

  # 4. Enable IPv4 and IPv6 forwarding for VPN routing
  - echo "net.ipv4.ip_forward=1" >> /etc/sysctl.d/99-tailscale.conf
```

```
  - echo "net.ipv6.conf.all.forwarding=1" >> /etc/sysctl.d/99-tailscale.conf
  - sysctl -p /etc/sysctl.d/99-tailscale.conf

final_message: "The remote exit node has been fully bootstrapped. SSH in, clone the repo to /opt/nullexit, add your .env, and run ./setup-linux.sh!"
```

25 docker/tor/Dockerfile

```
FROM debian:bookworm-slim

RUN apt-get update && \
    apt-get install -y tor obfs4proxy gosu bash && \
    rm -rf /var/lib/apt/lists/*

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

ENTRYPOINT ["/entrypoint.sh"]
```

26 docker/tor/entrypoint.sh

```
#!/bin/bash
set -e

TORRC="/etc/tor/torrc"
mkdir -p /etc/tor /var/lib/tor

# Base configuration (SOCKS proxy on 9050, Control port on 9051, strictly localhost/container boundaries)
cat <<EOF > $TORRC
SocksPort 0.0.0.0:9050
ControlPort 0.0.0.0:9051
CookieAuthentication 0
Log notice stdout
DataDirectory /var/lib/tor

# Transparent proxying & DNS resolution for .onion mapping
TransPort 0.0.0.0:9040
DNSPort 0.0.0.0:5353
AutomapHostsOnResolve 1
VirtualAddrNetworkIPv4 198.18.0.0/15
EOF

if [ -n "$TOR_PASSWORD" ]; then
    HASH=$(tor --hash-password "$TOR_PASSWORD" | tail -n 1)
    echo "HashedControlPassword $HASH" >> $TORRC
fi

if [ -n "$TOR_EXCLUDE_EXIT_NODES" ]; then
    echo "ExcludeExitNodes $TOR_EXCLUDE_EXIT_NODES" >> $TORRC
    echo "StrictNodes 1" >> $TORRC
fi

if [ "${TOR_USE_BRIDGES:-false}" = "true" ]; then
    BRIDGE_FILE="${TOR_BRIDGE_LINES_FILE:-/tor-bridges.txt}"

    if ! grep -vE "^[[:space:]]*#" "$BRIDGE_FILE" | grep -q '[^[:space:]]'; then
        MSG="[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL] [Tor Proxy] TOR_USE_BRIDGES is enabled, but no bridge lines found at $BRIDGE_FILE. Proxy startup ABORTED. Get bridges from https://bridges.torproject.org."
        echo "$MSG"
        if [ -w "/output.log" ]; then
            echo "$MSG" >> /output.log
        fi
        # Sleep indefinitely to prevent docker-compose 'unless-stopped' from triggering a crash loop
        exec sleep infinity
    fi

    echo "UseBridges 1" >> $TORRC
    echo "ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy" >> $TORRC

    while IFS= read -r line; do
        # Ignore comments and empty lines
```

```
        [[ -z "$line" || "$line" == \#* ]] && continue
        echo "Bridge $line" >> $TORRC
    done < "$BRIDGE_FILE"
fi

# Debian tor package uses ’debian-tor’ user
chown -R debian-tor:debian-tor /var/lib/tor /etc/tor

exec gosu debian-tor /usr/bin/tor -f $TORRC
```

27 launchd/com.nullexit.wake-recovery.plist

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.nullexit.wake-recovery</string>

    <!-- WorkingDirectory + relative program arg keeps this plist install-agnostic.
         The source-of-truth for the install path is __NULLEXIT_HOME__, which
         setup.sh or the user must substitute with their absolute nullexit
         install root at install time. Program arg is ‘scripts/watcher.sh‘
         RELATIVE to WorkingDirectory, so launchd’s evaluated cwd IS the
         install root  and BASH_SOURCE inside scripts/watcher.sh still
         resolves to the real <install>/scripts/watcher.sh, preserving the
         SCRIPT_DIR pattern. After install-time ‘sed -i ’’ ’s|__NULLEXIT_HOME__|<abs_path>|’‘,
         the plist is portable to any clone location. -->
    <key>WorkingDirectory</key>
    <string>__NULLEXIT_HOME__</string>

    <key>ProgramArguments</key>
    <array>
        <string>/bin/bash</string>
        <string>scripts/watcher.sh</string>
    </array>

    <!-- Start the daemon immediately on launchctl load (instead of waiting
         for the next user login). Combined with keepAlive below this means
         once installed: load once, runs forever across logouts/reboots. -->
    <key>RunAtLoad</key>
    <true/>

    <!-- Restart policy:
         SuccessfulExit=false   SIGTERM / clean exit does NOT relaunch (so
                                 ‘launchctl bootout‘ cleanly stops, not loops).
         Crashed=false          HARD crashes ALSO do NOT relaunch. We’ve seen
                                 that repeated sleep/wake cycles on macOS
                                 Sonoma+ accumulate orphan watcher.sh PIDs
                                 because SIGCONT/resume doesn’t reliably kill
                                 the prior instance. The watcher.sh script
                                 now enforces single-instance via a manual
                                 PID-file and process-identity check at the
                                 top, so a re-launch on crash is
                                 actively dangerous (it would skip the lock
                                 and exit; better to surface the crash in
                                 logs than to silently 0-process). -->
    <key>KeepAlive</key>
    <dict>
        <key>SuccessfulExit</key>
        <false/>
        <key>Crashed</key>
        <false/>
    </dict>

    <!-- Background hint to App Nap so it doesn’t get suspended when
         the user is inactive. We’re the entire reason this LaunchAgent
         needs to NOT nap. -->
    <key>ProcessType</key>
    <string>Background</string>

    <!-- launchd-level stdout/stderr (the bash launcher shell). The script
         itself writes to /tmp/nullexit-watcher.log via exec >>. These
         separate channels help us tell "launchd crashed" from "the bash
         wrapper piped to recover.sh errored". -->
    <key>StandardOutPath</key>
```

```
    <string>/tmp/nullexit-watcher.out.log</string>
    <key>StandardErrorPath</key>
    <string>/tmp/nullexit-watcher.err.log</string>
</dict>
</plist>
```

28 post-rules.txt

```
iptables -A FORWARD -i tailscale0 -o tun0 -j ACCEPT
iptables -A FORWARD -i tun0 -o tailscale0 -j ACCEPT
iptables -A INPUT -i tailscale0 -j ACCEPT
iptables -A INPUT -i eth0 -p udp --dport 41642 -j ACCEPT
iptables -t nat -A POSTROUTING -o tun0 -j MASQUERADE
# Clamp MSS to 1120. With Tailscale overhead on top of WARP (each WireGuard encapsulation adds ~60 bytes),
# the safe MSS for double-tunneled traffic is 1120 to prevent strict-MSS endpoints from stalling.
iptables -t mangle -A POSTROUTING -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1120

iptables -t nat -A PREROUTING -i tailscale0 -p udp --dport 53 -j REDIRECT --to-port 5335
iptables -t nat -A PREROUTING -i tailscale0 -p tcp --dport 53 -j REDIRECT --to-port 5335

# Also redirect DNS from Docker bridge interface so the host can reach AdGuard via localhost:53
iptables -t nat -A PREROUTING -i eth0 -p udp --dport 53 -j REDIRECT --to-port 5335
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 53 -j REDIRECT --to-port 5335

# Block IPv6 exit-node traffic to prevent leakage (Gluetun/WARP is IPv4-only)
ip6tables -A FORWARD -i tailscale0 -j DROP

# Block DNS-over-TLS (Port 853) to force Android Private DNS to fallback to Port 53
iptables -A FORWARD -i tailscale0 -p tcp --dport 853 -j REJECT --reject-with tcp-reset
iptables -A FORWARD -i tailscale0 -p udp --dport 853 -j REJECT

# Block QUIC (UDP 443) to force Facebook/Google to fallback to TCP.
# UDP bypasses TCP MSS clamping, causing fragmentation. Over weak Wi-Fi (far from gateway),
# dropping a single fragment destroys the entire packet, stalling image downloads.
iptables -A FORWARD -i tailscale0 -p udp --dport 443 -j REJECT
```

29 scripts/Dockerfile.routing-fix

```
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY logger/ ./logger/
RUN cd logger && go build -o logger main.go

FROM alpine:3.20
RUN apk add --no-cache iptables iproute2 curl ipset
COPY --from=builder /app/logger/logger /usr/local/bin/nullexit-logger
COPY routing-fix.sh /routing-fix.sh
CMD ["sh", "/routing-fix.sh"]
```

30 scripts/Dockerfile.rule-compiler

```
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY rule-compiler/ ./rule-compiler/
RUN cd rule-compiler && go build -o sync-rules main.go

FROM alpine:3.20
COPY --from=builder /app/rule-compiler/sync-rules /usr/local/bin/sync-rules
WORKDIR /app
CMD ["sync-rules"]
```

31 scripts/Toggle-Gateway.bat

```
<# :
@echo off
powershell -NoProfile -ExecutionPolicy Bypass -Command "Invoke-Expression ([System.IO.File]::ReadAllText('%~f0') -replace '^(?s).*?#\s*>\s*\r?\n','')"
exit /b
#>
# Toggle-Gateway.bat (Hybrid Batch-PowerShell Toggler)  nullexit Gateway Toggler for Windows
# Mirrors the sophisticated error handling, checks, and automation of toggle.sh

#  1. Administrator Check & Elevation
$isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
if (-not $isAdmin) {
    Write-Host "Requesting Administrator privileges..." -ForegroundColor Yellow
    Start-Process PowerShell -ArgumentList "-NoProfile -ExecutionPolicy Bypass -Command `"Invoke-Expression ([System.IO.File]::ReadAllText('$PSCommandPath') -replace '^(?s).*?#\s*>\s*\r?\n','')`"" -Verb RunAs
    Exit
}

# Set console output encoding to UTF8 for clean symbols
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
Clear-Host

Write-Host "" -ForegroundColor Green
Write-Host "           nullexit Windows Toggler           " -ForegroundColor Green
Write-Host "" -ForegroundColor Green
Write-Host ""

#  2. Helpers
function Reset-HostDNS {
    Write-Host "Restoring default DNS (DHCP/DHCP-assigned DNS)..." -ForegroundColor Cyan
    $activeAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
    foreach ($adapter in $activeAdapters) {
        Write-Host "  -> Resetting DNS on adapter: $($adapter.Name)" -ForegroundColor Gray
        Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ResetServerAddresses -ErrorAction SilentlyContinue
    }
}

function Hijack-HostDNS ($ip) {
    Write-Host "Hijacking DNS to route through AdGuard ($ip)..." -ForegroundColor Yellow
    $activeAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
    foreach ($adapter in $activeAdapters) {
        Write-Host "  -> Pointing DNS to $ip on adapter: $($adapter.Name)" -ForegroundColor Gray
        Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ServerAddresses $ip -ErrorAction SilentlyContinue
    }
}

function Get-HostTailscalePath {
    $paths = @(
        "$env:ProgramFiles\Tailscale\tailscale.exe",
        "$env:ProgramFiles (x86)\Tailscale\tailscale.exe",
        "tailscale.exe"
    )
    foreach ($path in $paths) {
        if (Test-Path $path) { return $path }
    }
    return $null
}

#  3. Detect State
# Prevent deadlocks by resetting DNS to default temporarily during checks
Reset-HostDNS

Write-Host "Checking Docker daemon status..." -ForegroundColor Cyan
& docker info >$null 2>&1
if ($LASTEXITCODE -ne 0) {
    Write-Host "Docker is not running. Starting Docker Desktop..." -ForegroundColor Yellow
    $dockerPath = "C:\Program Files\Docker\Docker\Docker Desktop.exe"
    if (Test-Path $dockerPath) {
        Start-Process $dockerPath
        Write-Host "Waiting for Docker daemon to become responsive..." -ForegroundColor Gray
        $timeout = 60
        while ($timeout -gt 0) {
            Start-Sleep -Seconds 2
```

```
            & docker info >$null 2>&1
            if ($LASTEXITCODE -eq 0) {
                Write-Host "Docker is ready!" -ForegroundColor Green
                break
            }
            $timeout -= 2
        }
        if ($timeout -le 0) {
            Write-Error "Docker failed to start in time. Aborting."
            Exit 1
        }
    } else {
        Write-Error "Docker Desktop executable not found at standard path. Please start Docker manually."
        Exit 1
    }
}

# Detect if gateway is already running
$runningWarp = docker compose ps --status running --format json | ConvertFrom-Json -ErrorAction SilentlyContinue | Where-Object { $_.Service -eq "warp" }
$isGatewayActive = ($null -ne $runningWarp)

#  4. Execution
if ($isGatewayActive) {
    #  STOP PATH
    Write-Host "Gateway is currently RUNNING. Disabling now..." -ForegroundColor Yellow
    Write-Host ""

    # Disconnect host Tailscale from exit node if installed
    $tsCli = Get-HostTailscalePath
    if ($null -ne $tsCli) {
        Write-Host "Disconnecting host Tailscale from exit node..." -ForegroundColor Cyan
        & $tsCli up --exit-node= --accept-dns=true >$null 2>&1
    }

    Write-Host "Stopping Docker containers..." -ForegroundColor Cyan
    docker compose down -t 5

    Reset-HostDNS
    Write-Host "Gateway has been successfully STOPPED." -ForegroundColor Green
} else {
    #  START PATH
    Write-Host "Gateway is currently STOPPED. Enabling now..." -ForegroundColor Yellow
    Write-Host ""

    # Clean up corrupted AdGuardHome configurations
    $confFile = "adguard\conf\AdGuardHome.yaml"
    if (Test-Path $confFile) {
        $fileSize = (Get-Item $confFile).Length
        if ($fileSize -eq 0) {
            Remove-Item $confFile -Force
            Write-Host "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop." -ForegroundColor Yellow
        }
    }

    Write-Host "Starting Docker containers..." -ForegroundColor Cyan
    docker compose up -d

    Write-Host "Waiting for container's Tailscale connection to offer Exit Node..." -ForegroundColor Cyan
    Write-Host "  (This can take up to 60 seconds...)" -ForegroundColor Gray

    $elapsed = 0
    $maxWait = 60
    $resolvedIP = $null

    while ($elapsed -lt $maxWait) {
        $statusJson = docker compose exec -T tailscale tailscale status --json 2>$null
        if ($null -ne $statusJson) {
            $status = $statusJson | ConvertFrom-Json -ErrorAction SilentlyContinue
            if ($null -ne $status -and $null -ne $status.Self) {
                # Check if it has a valid Tailscale IP and is running
                if ($status.Self.Online -and $status.Self.TailscaleIPs.Count -gt 0) {
                    $resolvedIP = $status.Self.TailscaleIPs[0]
                    break
                }
            }
        }
        Start-Sleep -Seconds 2
        $elapsed += 2
    }
```

```

    if ($null -ne $resolvedIP) {
        Write-Host "Tailscale IP resolved: $resolvedIP" -ForegroundColor Green
        Write-Host ""

        # Configure host Tailscale client to route through this exit node
        $tsCli = Get-HostTailscalePath
        if ($null -ne $tsCli) {
            Write-Host "Configuring host Tailscale to use Exit Node ($resolvedIP)..." -ForegroundColor Cyan
            & $tsCli up --exit-node=$resolvedIP --accept-dns=false >$null 2>&1
        }

        # Hijack DNS
        Hijack-HostDNS $resolvedIP
        Write-Host ""
        Write-Host "Gateway has been successfully STARTED." -ForegroundColor Green
    } else {
        Write-Host "Warning: Could not resolve gateway Tailscale IP. DNS hijacking skipped." -ForegroundColor Red
        # Fallback to .gateway_ip if it exists
        if (Test-Path ".gateway_ip") {
            $fallbackIP = Get-Content ".gateway_ip" -TotalCount 1
            if (-not [string]::IsNullOrWhiteSpace($fallbackIP)) {
                Write-Host "[Fallback] Using static IP from .gateway_ip: $fallbackIP" -ForegroundColor Yellow
                Hijack-HostDNS $fallbackIP
                Write-Host "Gateway STARTED (with static fallback IP)." -ForegroundColor Green
            }
        } else {
            Write-Host "Gateway started in offline/unauthenticated state. Run setup.sh if this is a fresh setup." -ForegroundColor Yellow
        }
    }
}

Write-Host ""
Write-Host "Press any key to exit..."
[void]$Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown")
```

32 scripts/check_circuits.py

```python
#!/usr/bin/env python3
import socket
import os
import re

def query_tor(port, command, password=""):
    try:
        s = socket.socket()
        s.settimeout(2)
        s.connect(('127.0.0.1', port))
        s.sendall(f"AUTHENTICATE \"{password}\"\r\n{command}\r\nQUIT\r\n".encode())
        response = b""
        while True:
            chunk = s.recv(4096)
            if not chunk:
                break
            response += chunk
        return response.decode()
    except Exception as e:
        return f"Error connecting to Tor Control Port: {e}"

def get_ports():
    ports = {}
    ports_file = "/tmp/nullexit-ports.env"
    if os.path.exists(ports_file):
        with open(ports_file, 'r') as f:
            for line in f:
                if line.startswith("export "):
                    # Split at first '=' to handle potential multiple '=' in value
                    parts = line.replace("export ", "").strip().split("=", 1)
                    if len(parts) == 2:
                        key, val = parts
                        try:
                            ports[key] = int(val)
```

```
                        except ValueError:
                            ports[key] = val
    return ports

def get_password(ports):
    # 1. Try ports.env
    password = ports.get("TOR_PASSWORD")
    if password:
        return password

    # 2. Try .env
    if os.path.exists(".env"):
        try:
            with open(".env", "r") as f:
                for line in f:
                    if line.startswith("ADGUARD_PASSWORD="):
                        return line.split("=", 1)[1].strip().strip(’"\’’)
                    if line.startswith("TOR_PASSWORD="):
                        return line.split("=", 1)[1].strip().strip(’"\’’)
        except Exception:
            pass

    # 3. Fallback
    return "nullexit"

def main():
    ports = get_ports()
    control_port = ports.get("TOR_CONTROL_PORT", 9051)
    password = get_password(ports)

    print(f"Connecting to Tor Control Port on 127.0.0.1:{control_port}...")
    status = query_tor(control_port, "GETINFO circuit-status", password)

    if "Error" in status:
        print(status)
        return

    print("\nActive Tor Circuits:")
    circuits = re.findall(r"(\d+)\s+BUILT\s+(.+?)\s+PURPOSE", status)
    if not circuits:
        print("No active built circuits found yet. Tor might still be bootstrapping.")
        return

    for circ_id, path_str in circuits:
        print(f"\nCircuit #{circ_id}:")
        nodes = path_str.split(",")
        for idx, node in enumerate(nodes):
            fingerprint = node.split("˜")[0].replace("$", "")
            nickname = node.split("˜")[1] if "˜" in node else "unknown"

            # Fetch node IP
            ns_info = query_tor(control_port, f"GETINFO ns/id/{fingerprint}", password)
            ip = "unknown"
            for line in ns_info.split("\n"):
                if line.startswith("r "):
                    parts = line.split(" ")
                    if len(parts) >= 7:
                        ip = parts[6]
                        break

            # Fetch country
            country = "unknown"
            if ip != "unknown":
                country_info = query_tor(control_port, f"GETINFO ip-to-country/{ip}", password)
                for line in country_info.split("\n"):
                    if line.startswith("250-ip-to-country/"):
                        country = line.split("=")[1].strip().upper()
                        break

            role = "Guard" if idx == 0 else ("Exit" if idx == len(nodes) - 1 else "Middle")
            print(f"  [{role}] {nickname} ({ip}) - Country: {country}")

if __name__ == "__main__":
    main()
```

33 scripts/diagnose-host-leak.sh

```
#!/bin/bash
# scripts/diagnose-host-leak.sh  nullexit host-routing diagnostic
#
# Symptom this addresses: tests like https://www.whatismyip.com on the
# *HOST MAC* return the underlying physical ISP/ASN
# local ISP / campus network") even though the nullexit gateway is active and remote
# tailnet clients correctly egress via Cloudflare WARP. The container and
# the gateway itself are healthy  only the host’s own routing is leaking.
#
# This script runs 7 checks in sequence, classifies the result into ONE
# of the three known leak scenarios, writes the full report to a
# timestamped file in the project root, and prints a verdict + a
# ready-to-run fix command on stdout. Pass --fix to apply the matching
# remediation in the same invocation and re-verify egress afterwards.
# See devref.md 10.30 for the deep-dive rationale.
#
# Usage:
#   bash scripts/diagnose-host-leak.sh              # diagnose + write report
#   bash scripts/diagnose-host-leak.sh --fix        # diagnose + fix + re-verify
#   bash scripts/diagnose-host-leak.sh --watch      # full baseline then loop checks every 60s
#   bash scripts/diagnose-host-leak.sh --watch 30   # same, with custom interval (seconds)
#   bash scripts/diagnose-host-leak.sh --help       # this message
#
# Multiple runs produce timestamped files (one per run) so historical
# reports can be diffed. The newest file is the most recent.

set -uo pipefail
# NOTE: errexit (‘set -e‘) is intentionally NOT enabled. Several checks
# below are EXPECTED to fail and that’s diagnostic data (e.g. ‘tailscale
# status‘ returns non-zero when the daemon is wedged). Every command
# pipes to a helper that records the exit code without killing the script.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
source "$SCRIPT_DIR/common.sh"
LOG_FILE="$PROJECT_ROOT/output.log"
REPORT_FILE="$PROJECT_ROOT/host-leak-diagnostic-$TIMESTAMP.txt"

#  Argument parsing
DO_FIX=false
DO_WATCH=false
WATCH_INTERVAL=60
WATCH_LOG="$PROJECT_ROOT/host-leak-watch.log"
case "${1:-}" in
  --fix) DO_FIX=true ;;
  --watch)
    DO_WATCH=true
    # Optional second argument: custom interval in seconds
    if [ -n "${2:-}" ] && [[ "$2" =~ ^[0-9]+$ ]]; then
      WATCH_INTERVAL="$2"
    fi
    ;;
  --help|-h)
    sed -n ’2,31p’ "$0"
    exit 0
    ;;
  "")
    : # default: diagnose only
    ;;
  *)
    echo "Unknown argument: $1" >&2
    echo "Run with --help for usage." >&2
    exit 2
    ;;
esac

#  Colours (auto-disabled when stdout is not a tty, e.g. piped)
if [ -t 1 ]; then
  RED=’\033[0;31m’; GREEN=’\033[0;32m’; YELLOW=’\033[1;33m’
  BOLD=’\033[1m’; NC=’\033[0m’
else
  RED=’’; GREEN=’’; YELLOW=’’; BOLD=’’; NC=’’
fi

#  Output helpers  write to BOTH stdout and the report file
exec 3>> "$REPORT_FILE" || {
  printf ’\n[FATAL] cannot open %s for write\n’ "$REPORT_FILE" >&2
  exit 1
}
```

```
section() {
  local title="$1"
  printf ’\n%b %s %b\n’ "$BOLD" "$title" "$NC" >&3
  printf ’\n%b %s %b\n’ "$BOLD" "$title" "$NC"
}
line()   { printf ’  %b\n’ "$1" >&3; printf ’  %b\n’ "$1"; }
ok()     { line "${GREEN}${NC} $1"; }
warn()   { line "${YELLOW}${NC} $1"; }
fail()   { line "${RED}${NC} $1"; }
note()   { line "(no ${1})"; }

#  Watch-mode header (only shown when entering watch loop)
if [ "$DO_WATCH" = "true" ]; then
  echo ""
  echo ""
  echo " n u l l e x i t   w a t c h   m o d e"
  echo ""
  echo " Interval:        ${WATCH_INTERVAL}s"
  echo " Watch log:       $WATCH_LOG"
  echo ""
else
  echo ""
  echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c"
  echo ""
  echo " Timestamp (UTC): $TIMESTAMP"
  echo " Project root:    $PROJECT_ROOT"
  echo " Report file:     $REPORT_FILE"
  echo " Mode:            $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only (pass --fix to remediate)")"
  echo ""
  echo "" >&3
  echo "" >&3
  echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c" >&3
  echo "" >&3
  echo " Timestamp (UTC): $TIMESTAMP" >&3
  echo " Mode:            $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only")" >&3
fi

#  Always add Homebrew to PATH (same as toggle.sh / recover.sh)
export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"

#  Pure-bash timeout (macOS lacks GNU ‘timeout‘)
run_with_timeout() {
  local timeout_sec="$1"
  shift
  if [ "$#" -eq 0 ]; then return 1; fi
  "$@" &
  local cmd_pid=$!
  (
    sleep "$timeout_sec"
    if kill -0 "$cmd_pid" 2>> "$LOG_FILE"; then
      kill -9 "$cmd_pid" 2>> "$LOG_FILE" || true
    fi
  ) >> "$LOG_FILE" 2>&1 &
  local watcher_pid=$!
  set +e
  wait "$cmd_pid"
  local exit_status=$?
  set -uo pipefail
  kill "$watcher_pid" 2>> "$LOG_FILE" || true
  return "$exit_status"
}

#  Resolve the active network service (same heuristic as toggle.sh)
resolve_active_service() {
  get_active_service
}

#
# Quick-check functions (used by --watch loop; return simple status strings)
# Each echoes its result so the caller can compare.
#

quick_warp() {
  # Returns: "on", "off", "unknown"
  local out
  out=$(run_with_timeout 8 curl -s --max-time 6 \
        https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null || echo "")
  local warp
  warp=$(printf ’%s’ "$out" | awk -F’=’ ’/^warp=/{print $2; exit}’)
```

```
  printf '%s' "${warp:-unknown}"
}

quick_ipv6() {
  # Returns: the IPv6 address if leaking, or empty string if clean
  local out
  out=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>/dev/null || echo "")
  if [ -n "$out" ] && printf '%s' "$out" | grep -qE '^[0-9a-fA-F:]+$'; then
    printf '%s' "$out"
  fi
}

quick_default_route() {
  # Returns: "utun" if default route is via Tailscale utun*,
  #          "wifi" if via physical Wi-Fi (192.168.x.x),
  #          "other" otherwise
  local route
  route=$(netstat -rn 2>/dev/null | awk '/^default/ {print $0; exit}')
  if [ -z "$route" ]; then
    printf '%s' "none"
  elif printf '%s' "$route" | grep -qE 'utun[0-9]+'; then
    printf '%s' "utun"
  elif printf '%s' "$route" | grep -qE '^(default|0\.0\.0\.0).*\b192\.168\.'; then
    printf '%s' "wifi"
  else
    printf '%s' "other"
  fi
}

#  Watch loop (runs after baseline diagnostic when --watch is passed)
run_watch_loop() {
  local baseline_warp="$1"
  local baseline_default="$2"
  local baseline_ipv6_leak="$3"
  local interval="$4"

  echo ""
  echo ""
  echo " Baseline established. Monitoring every ${interval}s..."
  echo "  warp:     $baseline_warp"
  echo "  default:  $baseline_default"
  echo ""

  #  Safety: warn if interval is very short (avoids hammering APIs)
  if [ "$interval" -lt 30 ]; then
    printf '%b  Short interval (%ss)  rate-limiting risk on external APIs%b\n' "$YELLOW" "$interval" "$NC"
    printf '  Consider 30s or longer for production use.\n'
  fi
  printf '[%s] WATCH START  warp=%s  default=%s  interval=%ss\n' \
    "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$baseline_warp" "$baseline_default" "$interval" \
    >> "$WATCH_LOG"

  local iteration=0
  local last_warp="$baseline_warp"
  local last_default="$baseline_default"
  local last_ipv6_ok=$([ "$baseline_ipv6_leak" = "false" ] && echo true || echo false)

  trap 'printf "\n%bWatch stopped by user. Log at: %s%b\n" "$YELLOW" "$WATCH_LOG" "$NC"; exit 0' INT TERM

  while true; do
    iteration=$((iteration + 1))

    local warp_now ipv6_now default_now
    warp_now=$(quick_warp)
    ipv6_now=$(quick_ipv6)
    default_now=$(quick_default_route)

    local ts
    ts=$(date -u +%H:%M:%S)

    #  WARP check
    if [ "$warp_now" != "$last_warp" ]; then
      if [ "$warp_now" = "off" ] || [ "$warp_now" = "unknown" ]; then
        printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
        printf '%b[%s] %bWARP FLIPPED: %s  %s%b  YOUR IP IS LEAKING!\n' \
          "$RED" "$ts" "$BOLD" "$last_warp" "$warp_now" "$NC"
        printf '[%s] ALERT  warp flipped  %s%s\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
      else
        printf '\n%b[%s]  warp recovered: %s  %s%b\n' \
          "$GREEN" "$ts" "$last_warp" "$warp_now" "$NC"
```

```
        printf ’[%s] OK     warp recovered  %s%s\n’ "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
      fi
      last_warp="$warp_now"
    fi

    #  IPv6 check
    if [ -n "$ipv6_now" ]; then
      if [ "$last_ipv6_ok" = true ]; then
        printf ’\n%b ALERT %b\n’ "$RED$BOLD" "$NC"
        printf ’%b[%s] %bIPv6 LEAK DETECTED  %s%b\n’ \
          "$RED" "$ts" "$BOLD" "$ipv6_now" "$NC"
        printf ’[%s] ALERT  IPv6 leak  %s\n’ "$ts" "$ipv6_now" >> "$WATCH_LOG"
        last_ipv6_ok=false
      fi
    else
      if [ "$last_ipv6_ok" = false ]; then
        printf ’%b[%s]  IPv6 leak resolved%b\n’ "$GREEN" "$ts" "$NC"
        printf ’[%s] OK     IPv6 resolved\n’ "$ts" >> "$WATCH_LOG"
      fi
      last_ipv6_ok=true
    fi

    #  Default route check
    if [ "$default_now" != "$last_default" ]; then
      if [ "$default_now" != "utun" ]; then
        printf ’\n%b ALERT %b\n’ "$RED$BOLD" "$NC"
        printf ’%b[%s] %bDEFAULT ROUTE CHANGED: %s  %s%b  traffic may bypass WARP!\n’ \
          "$RED" "$ts" "$BOLD" "$last_default" "$default_now" "$NC"
        printf ’[%s] ALERT  route changed  %s%s\n’ "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
      else
        printf ’%b[%s]  default route recovered: %s  %s%b\n’ \
          "$GREEN" "$ts" "$last_default" "$default_now" "$NC"
        printf ’[%s] OK     route recovered  %s%s\n’ "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
      fi
      last_default="$default_now"
    fi

    #  Heartbeat (every 10th iteration or if all OK)
    if [ $((iteration % 10)) -eq 0 ]; then
      printf ’[%s] #%d  warp=%s  ipv6=%s  route=%s\n’ \
        "$ts" "$iteration" "$warp_now" "${ipv6_now:-none}" "$default_now" >> "$WATCH_LOG"
      printf ’\r%b[%s] #%d  warp=%s  route=%s  (Ctrl+C to stop)%b’ \
        "$GREEN" "$ts" "$iteration" "$warp_now" "$default_now" "$NC"
    fi

    sleep "$interval"
  done
}

#
# Begin main diagnostic (skipped in non-watch modes if we’re just watching)
#

ACTIVE_SERVICE=$(resolve_active_service)
EN0_SERVICE=$(networksetup -listnetworkserviceorder 2>> "$LOG_FILE" \
              | grep -B 1 "Device: en0" | head -1 \
              | sed -E ’s/^\([0-9\*]+\) //’ || true)
[ -z "$EN0_SERVICE" ] && EN0_SERVICE="Wi-Fi"

# Resolve the gateway’s Tailscale IP.
GATEWAY_TS_IP=""
if [ -f "$PROJECT_ROOT/.gateway_ip" ]; then
  GATEWAY_TS_IP=$(cat "$PROJECT_ROOT/.gateway_ip" 2>> "$LOG_FILE" \
                  | tr -d ’\r’ | awk ’NR==1{print $1; exit}’)
fi
if [ -z "$GATEWAY_TS_IP" ] && command -v docker >> "$LOG_FILE" 2>&1 \
   && docker compose ps --status running 2>/dev/null | grep -q ’tailscale’; then
  GATEWAY_TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale \
                  tailscale ip -4 2>> "$LOG_FILE" \
                  | tr -d ’\r’ | awk ’NR==1{print $1; exit}’)
fi

#  Scenario classification state
SCENARIO=""
WARP_STATUS=""
SOCKS5_ENABLED=false
TS_ON_MESH=false
DEFAULT_VIA_UTUN=false
IPV6_LEAK=false

#
```

```
section "1/8  Host Tailscale mesh status"
#
if ! command -v tailscale >> "$LOG_FILE" 2>&1; then
  fail "tailscale CLI not on PATH  install via 'brew install tailscale' then re-run"
  echo "  (a Tailscale daemon is required for the exit-node path to work)"
else
  set +e
  TS_OUT=$(run_with_timeout 5 tailscale status 2>&1)
  TS_EXIT=$?
  set -uo pipefail
  TS_HEAD=$(printf '%s\n' "$TS_OUT" | head -5 | tr -d '\r')
  if [ "$TS_EXIT" -eq 137 ]; then
    fail "tailscaled daemon is wedged/unresponsive (5s timeout)"
    echo "  Fix path: 'sudo brew services restart tailscale'"
  elif printf '%s' "$TS_OUT" | grep -qE 'Logged out|not logged in|expired'; then
    fail "tailscale is logged out / auth expired on this host"
    echo "  Fix path: 'sudo tailscale up' (browser auth once)"
  elif printf '%s' "$TS_OUT" | grep -q '100\.'; then
    ok "Host is on tailnet (100.x.x.x visible in status)"
    TS_ON_MESH=true
    if printf '%s' "$TS_OUT" | grep -q "$GATEWAY_TS_IP"; then
      ok "Gateway $GATEWAY_TS_IP visible in host's peer list"
    elif [ -n "$GATEWAY_TS_IP" ]; then
      warn "Gateway $GATEWAY_TS_IP NOT in host's peer list  possible auth/key mismatch"
    fi
    line " first 5 lines of 'tailscale status' "
    line "$(printf '%s' "$TS_HEAD" | awk '{print "    "$0}')"
  else
    fail "tailscale output unrecognized:"
    line "$(printf '%s' "$TS_OUT" | head -3 | awk '{print "    "$0}')"
  fi
fi

#
section "2/8  Active SOCKS5 proxy on Wi-Fi"
#
SOCKS_OUT=$(networksetup -getsocksfirewallproxy "$ACTIVE_SERVICE" 2>> "$LOG_FILE" \
             || networksetup -getsocksfirewallproxy Wi-Fi 2>> "$LOG_FILE" \
             || echo "Could not query SOCKS5 proxy state")
if printf '%s' "$SOCKS_OUT" | grep -q "Enabled: Yes"; then
  fail "SOCKS5 proxy IS enabled on $ACTIVE_SERVICE  toggle.sh fell back to SOCKS5 mode last run"
  SOCKS5_ENABLED=true
  line "  Full state:"
  line "$(printf '%s' "$SOCKS_OUT" | awk '{print "    "$0}')"
  line "  Browse on macOS ignores system SOCKS5 by default  traffic bypasses WARP."
else
  ok "SOCKS5 proxy is OFF on $ACTIVE_SERVICE  exit-node path should be active"
fi

#
section "3/8  Egress IP + WARP tunnel verification (definitive test)"
#
CDN_OUT=$(run_with_timeout 8 curl -s --max-time 6 \
          https://www.cloudflare.com/cdn-cgi/trace 2>&1 || echo "(curl failed)")
CDN_WARP=$(printf '%s' "$CDN_OUT" | awk -F'=' '/^warp=/{print $2; exit}')
CDN_IP=$(printf '%s' "$CDN_OUT" | awk -F'=' '/^ip=/{print $2; exit}')
CDN_COLO=$(printf '%s' "$CDN_OUT" | awk -F'=' '/^colo=/{print $2; exit}')
WARP_STATUS="$CDN_WARP"
if [ "$CDN_WARP" = "on" ]; then
  ok "warp=on  tunnel is hot, host traffic IS going through Cloudflare WARP"
  line "  Public IP:   $CDN_IP"
  line "  Cloudflare:   ${CDN_COLO:-unknown colo}"
  line "   whatismyip.com's 'local ISP' result is its ISP-database error, not yours."
elif [ "$CDN_WARP" = "off" ]; then
  fail "warp=off  host traffic is NOT going through WARP"
  line "  Public IP:   $CDN_IP  (NOT Cloudflare)"
  warn "This is the actual leak."
else
  warn "could not determine warp status (curl failed or returned unexpected shape)"
  line "  Raw output: $(printf '%s' "$CDN_OUT" | head -3 | tr '\n' '|')"
fi

#
section "4/8  IPv6 leak probe (bypasses IPv4 exit-node on campus networks)"
#
IPV6_OUT=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
if [ -n "$IPV6_OUT" ] && printf '%s' "$IPV6_OUT" | grep -qE '^[0-9a-fA-F:]+$'; then
  fail "host has live IPv6 egress  $IPV6_OUT (A6 record bypasses WARP)"
  IPV6_LEAK=true
  line "  Tailscale exit-node advertisement is IPv4-only, so IPv6 traffic skips it."
  line "  Quick-fix: 'sudo networksetup -setv6off $ACTIVE_SERVICE'"
```

```
else
  ok "no IPv6 egress detected (good  IPv6 won't bypass the tunnel)"
fi

#
section "5/8  Host egress route for real traffic (route -n get 1.1.1.1)"
#
# WHY NOT 'netstat -rn | grep default':
# On macOS, Tailscale does NOT replace the physical default route installed by
# DHCP. Instead it adds a second interface-scoped route bound to the utun
# interface that the kernel prefers for real traffic forwarding. The literal
# "default" entry (192.168.x.x via en0) is left untouched in the table as a
# BSD routing quirk  it's not lying, it's just answering a different question.
# The correct question is: "where does a real destination actually resolve?"
# `route -n get 1.1.1.1` asks the kernel's forwarding logic directly.
ROUTE_GET=$(route -n get 1.1.1.1 2>/dev/null)
ROUTE_IFACE=$(echo "$ROUTE_GET" | awk '/interface:/{print $2}')
ROUTE_GATEWAY=$(echo "$ROUTE_GET" | awk '/gateway:/{print $2}')
if [ -z "$ROUTE_IFACE" ]; then
  warn "route -n get 1.1.1.1 returned no interface  routing table may be empty"
else
  line "  interface: $ROUTE_IFACE  gateway: ${ROUTE_GATEWAY:-n/a}"
  if echo "$ROUTE_IFACE" | grep -qE '^utun[0-9]+'; then
    ok "1.1.1.1 resolves via $ROUTE_IFACE  Tailscale interface-scoped route is winning"
    DEFAULT_VIA_UTUN=true
  elif echo "$ROUTE_IFACE" | grep -qE '^en[0-9]+'; then
    fail "1.1.1.1 resolves via $ROUTE_IFACE (physical Wi-Fi)  exit-node interface route not active"
  else
    warn "1.1.1.1 resolves via unexpected interface: $ROUTE_IFACE"
  fi
fi

#
section "6/8  Last toggle.sh START-relevant lines in output.log (if any)"
#
if [ ! -f "$LOG_FILE" ]; then
  note "output.log"
  line "  toggle.sh was never run from this directory. Run './toggle.sh' first,"
  line "  then re-run this diagnostic."
else
  HITS=$(grep -E 'Exit node enabled|SKIP_EXIT_NODE|Pre-flight|Falling back to SOCKS|Starting|Successfully' \
         "$LOG_FILE" 2>/dev/null | tail -8 || true)
  if [ -z "$HITS" ]; then
    note "matching lines in output.log"
  else
    line "$(printf '%s\n' "$HITS" | awk '{print "    "$0}')"
  fi
fi

#
section "7/8  Gateway IP we should be pointing at"
#
if [ -n "$GATEWAY_TS_IP" ]; then
  ok "gateway Tailscale IP: $GATEWAY_TS_IP"
else
  warn "could not resolve gateway Tailscale IP"
  line "  (looked in .gateway_ip and tried docker compose exec tailscale ip -4)"
fi

#
section "8/8  Tailscale System Extension (App Store conflict check)"
#
SE_PENDING_UNINSTALL=false
SE_ACTIVE=false
if command -v systemextensionsctl >/dev/null 2>&1; then
  SE_OUT=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale')
  if [ -n "$SE_OUT" ]; then
    line "  Tailscale System Extension(s) detected:"
    line "$(printf '%s' "$SE_OUT" | awk '{print "    "$0}')"
    if printf '%s' "$SE_OUT" | grep -q 'waiting to uninstall'; then
      SE_PENDING_UNINSTALL=true
      warn "System Extension is pending uninstall on next reboot."
    else
      SE_ACTIVE=true
      warn "Active Tailscale System Extension detected. This will conflict with Homebrew Tailscale."
    fi
  else
    ok "no Tailscale System Extension detected"
  fi
else
```

```
  note "systemextensionsctl not available (not on macOS or permission restricted)"
fi

#
section "CLASSIFICATION    applies ONE of the four known scenarios"
#

if [ "$WARP_STATUS" = "on" ] && [ "$IPV6_LEAK" = "false" ]; then
  SCENARIO="OK"
elif [ "$SE_PENDING_UNINSTALL" = "true" ]; then
  SCENARIO="SE_PENDING"
elif [ "$SOCKS5_ENABLED" = "true" ]; then
  SCENARIO="A"
elif [ "$IPV6_LEAK" = "true" ]; then
  SCENARIO="B"
elif [ "$DEFAULT_VIA_UTUN" = "false" ] && [ "$TS_ON_MESH" = "true" ]; then
  SCENARIO="C"
else
  SCENARIO="UNKNOWN"
fi

case "$SCENARIO" in
  SE_PENDING)
    echo ""
    echo -e "  ${RED}${BOLD}VERDICT:  Scenario SE_PENDING  Tailscale System Extension Pending Uninstall${NC}"
    echo "  A prior App Store Tailscale System Extension is pending uninstall."
    echo "  Since System Integrity Protection (SIP) is enabled, macOS blocks manual"
    echo "  uninstallation of terminated system extensions until the next reboot."
    echo "  Brew-managed tailscaled cannot engage the Network Extension slot until this is resolved."
    echo ""
    echo "  ${BOLD}Recommended fix:${NC}"
    echo "    sudo shutdown -r now"
    echo "    # After reboot, run:  ./toggle.sh"
    ;;
  A)
    echo ""
    echo -e "  ${RED}${BOLD}VERDICT:  Scenario A  SOCKS5 fallback lane${NC}"
    echo "  toggle.sh's pre-flight checks failed last run. The script activated"
    echo "  SOCKS5 + local DNS proxy as a fallback. DNS gets hijacked but"
    echo "  browsers on macOS ignore the system SOCKS5  traffic egresses"
    echo "  direct through the local network."
    echo ""
    echo "  ${BOLD}Recommended fix:${NC}"
    echo "    echo 'GATEWAY_BYPASS_PING=true' >> $PROJECT_ROOT/.env"
    echo "    sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \\"
    if [ -n "$GATEWAY_TS_IP" ]; then
    echo "        --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
    else
    echo "        --exit-node=\$GATEWAY_TS_IP --exit-node-allow-lan-access=true  # fill in $GATEWAY_TS_IP"
    fi
    echo "    sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
    echo "    ./toggle.sh"
    ;;
  B)
    echo ""
    echo -e "  ${RED}${BOLD}VERDICT:  Scenario B  IPv6 leak over dual-stack campus/local IPv6${NC}"
    echo "  Tailscale exit-node advertises only IPv4. macOS sends AAAA queries"
    echo "  straight out en0, which is dual-stack on most campus APs  IPv6"
    echo "  traffic bypasses the WARP tunnel entirely."
    echo ""
    echo "  ${BOLD}Recommended fix:${NC}"
    echo "    sudo networksetup -setv6off $ACTIVE_SERVICE"
    echo "    sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
    echo "    # Re-enable IPv6 later with:  sudo networksetup -setv6automatic $ACTIVE_SERVICE"
    ;;
  C)
    echo ""
    echo -e "  ${RED}${BOLD}VERDICT:  Scenario C  Tailscale route-freeze${NC}"
    echo "  Host's default route points at the Wi-Fi gateway instead of the"
    echo "  Tailscale utun* interface. The exit-node preference is set but the"
    echo "  routing-table assertion did not take effect (devref 10.26)."
    echo ""
    echo "  ${BOLD}Recommended fix:${NC}"
    echo "    sudo brew services restart tailscale"
    echo "    sudo route delete -net 0.0.0.0/1"
    echo "    sudo route delete -net 128.0.0.0/1"
    echo "    sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
    if [ -n "$GATEWAY_TS_IP" ]; then
    echo "    sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \\"
```

```
    echo "        --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
    else
    echo "    sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true --exit-node=\$TS_IP --exit-node-allow-lan-access=true"
    fi
    echo "    ./toggle.sh"
    ;;
  OK)
    echo ""
    echo -e "  ${GREEN}${BOLD}VERDICT:  No host leak detected  tunnel is working${NC}"
    echo "  The local ISP string from whatismyip.com is its IP-database"
    echo "  misclassifying the Cloudflare egress IP.  CDN-CGI's trace endpoint"
    echo "  (which is what we use here) reports warp=on because that endpoint"
    echo "  is hosted by Cloudflare itself and can see its own edge."
    echo "  If you want a third-party confirmation:"
    echo "    curl -s https://api.ipify.org; echo"
    ;;
  *)
    echo ""
    echo -e "  ${YELLOW}${BOLD}VERDICT:  Could not classify automatically${NC}"
    echo "  The diagnostic data did not match any known scenario cleanly."
    echo "  Likely causes:"
    echo "    - tailscaled is logged out / expired (see Section 1)"
    echo "    - Docker is not running (gateway containers down)"
    echo "    - .gateway_ip missing AND docker compose exec failed"
    echo "  Inspect the report at: $REPORT_FILE"
    ;;
esac
echo ""

#
# Write the verdict block to the report file
#
SECOND_BLOCK=$(cat <<EOF


 C L A S S I F I C A T I O N   R E S U L T

 Scenario: $SCENARIO
 WARP egress:  $WARP_STATUS  (cdn-cgi/trace)
 SOCKS5 lane:  $([ "$SOCKS5_ENABLED" = true ] && echo "ENABLED (browsers bypass)" || echo "off")
 IPv6 leak:    $([ "$IPV6_LEAK" = true ] && echo "YES (campus IPv6 egress)" || echo "no")
 Default via:  $([ "$DEFAULT_VIA_UTUN" = true ] && echo "utun* (Tailscale)" || echo "Wi-Fi gateway")
 Host on mesh: $([ "$TS_ON_MESH" = true ] && echo "yes" || echo "no")
 SE pending:   $([ "$SE_PENDING_UNINSTALL" = true ] && echo "yes" || echo "no")
 Gateway IP:   ${GATEWAY_TS_IP:-unresolved}
EOF
)
printf '%s\n' "$SECOND_BLOCK" >&3

#
# --fix mode
#
if [ "$DO_FIX" = "true" ] && [ "$SCENARIO" != "OK" ] && [ "$SCENARIO" != "UNKNOWN" ]; then
  echo ""
  echo ""
  echo " A P P L Y I N G   F I X  (scenario $SCENARIO)"
  echo ""
  echo ""

  case "$SCENARIO" in
    SE_PENDING)
      warn "System Extension uninstall is pending on next reboot."
      line "Please run 'sudo shutdown -r now' to reboot the system."
      ;;
    A)
      if [ -n "$GATEWAY_TS_IP" ]; then
        line " writing GATEWAY_BYPASS_PING=true to .env"
        ENV="$PROJECT_ROOT/.env"
        [ -f "$ENV" ] || touch "$ENV"
        if grep -q '^GATEWAY_BYPASS_PING=' "$ENV" 2>/dev/null; then
          sed -i '' 's/^GATEWAY_BYPASS_PING=.*/GATEWAY_BYPASS_PING=true/' "$ENV"
        else
          printf '\nGATEWAY_BYPASS_PING=true\n' >> "$ENV"
        fi
        line " re-asserting tailscale exit-node (with --reset)"
        sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
             --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
             >> "$LOG_FILE" 2>&1 \
          && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
          || warn "tailscale up did not return success (check $LOG_FILE)"
```

```
      else
        warn "no gateway Tailscale IP resolved  skipping tailscale up"
        warn "fix manually: 'sudo tailscale up --reset ... --exit-node=<TS_IP>'"
      fi
      ;;
    B)
      line " disabling IPv6 on $ACTIVE_SERVICE while gateway is up"
      sudo -n networksetup -setv6off "$ACTIVE_SERVICE" >> "$LOG_FILE" 2>&1 \
        && ok "IPv6 disabled on $ACTIVE_SERVICE (re-enable with 'setv6automatic')" \
        || warn "networksetup -setv6off failed (check $LOG_FILE)"
      ;;
    C)
      line " restarting tailscaled (route-table fix)"
      run_with_timeout 15 brew services restart tailscale >> "$LOG_FILE" 2>&1 \
        || run_with_timeout 15 sudo -n brew services restart tailscale >> "$LOG_FILE" 2>&1
      sleep 3
      line " flushing stale host routes + DNS cache"
      sudo -n route delete -net 0.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
      sudo -n route delete -net 128.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
      sudo -n dscacheutil -flushcache >> "$LOG_FILE" 2>&1 || true
      sudo -n killall -HUP mDNSResponder >> "$LOG_FILE" 2>&1 || true
      if [ -n "$GATEWAY_TS_IP" ]; then
        line " re-asserting exit-node after restart"
        sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
             --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
             >> "$LOG_FILE" 2>&1 \
          && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
          || warn "tailscale up did not return success (check $LOG_FILE)"
      fi
      ;;
  esac

  echo ""
  echo "Re-verifying after fix..."
  sleep 3
  CDN_WARP_AFTER=$(run_with_timeout 8 curl -s --max-time 6 \
      https://www.cloudflare.com/cdn-cgi/trace 2>&1 \
      | awk -F'=' '/^warp=/{print $2; exit}')
  IPV6_AFTER=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
  printf '\n[AFTER FIX]\n' >&3
  printf '  warp status:    %s\n' "$CDN_WARP_AFTER" >&3
  printf '  IPv6 egress:     %s\n' "${IPV6_AFTER:-none}" >&3
  line "  warp status:    ${CDN_WARP_AFTER:-unknown}"
  line "  IPv6 egress:    ${IPV6_AFTER:-none}"
  if [ "$CDN_WARP_AFTER" = "on" ] && [ -z "$IPV6_AFTER" ]; then
    echo ""
    ok "Fix verified  host is now routing through WARP. Remote clients unaffected."
  else
    echo ""
    warn "Fix did not fully take effect on first try. Likely causes:"
    line "   tailscaled needed a longer settling window (re-run diagnostic in 30s)"
    line "   The classified scenario was wrong; paste this report for triage."
    line "   Tailscale needs a full auth refresh: 'sudo tailscale up'"
  fi
fi

echo ""
echo ""
echo "  Full report written to:"
echo "    $REPORT_FILE"
echo ""
echo ""

#
# --watch mode: enter the continuous monitoring loop after baseline diagnostic
#
if [ "$DO_WATCH" = "true" ]; then
  # Build baseline from the full diagnostic just completed
  BASELINE_WARP="$WARP_STATUS"
  BASELINE_DEFAULT=$([ "$DEFAULT_VIA_UTUN" = "true" ] && echo "utun" || echo "wifi/other")

  if [ "$SCENARIO" != "OK" ]; then
    warn "Baseline diagnostic found issues (scenario: $SCENARIO)."
    line "  Watch loop will still run  alerts fire on any STATE CHANGE from current baseline."
    line "  Fix the issue first? Re-run with:  bash scripts/diagnose-host-leak.sh --fix"
    echo ""
  fi

  run_watch_loop "$BASELINE_WARP" "$BASELINE_DEFAULT" "$IPV6_LEAK" "$WATCH_INTERVAL"
fi
```

```
exit 0
```

34 scripts/dns-proxy.py

```python
#!/usr/bin/env python3
"""
dns-proxy.py  Local DNS proxy for nullexit gateway.

Listens on UDP port 53, receives DNS queries, forwards them over TCP
to AdGuard at 127.0.0.1:5354 (the Docker port mapping), and returns
the response via UDP. Handles the DNS-over-TCP wire format (2-byte
length prefix) correctly  unlike socat which just forwards raw bytes.
"""

import socket
import struct
import sys
import os
from concurrent.futures import ThreadPoolExecutor

LISTEN_ADDR = os.environ.get("LISTEN_ADDR", "127.0.0.1")
LISTEN_PORT = int(os.environ.get("LISTEN_PORT", 53))
TARGET_HOST = os.environ.get("TARGET_HOST", "127.0.0.1")
TARGET_PORT = int(os.environ.get("TARGET_PORT", 5354))


def handle_query(data: bytes, client_addr: tuple, sock: socket.socket) -> None:
    """Forward a single DNS query over TCP and send the response back via UDP."""
    try:
        tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        tcp.settimeout(5)
        tcp.connect((TARGET_HOST, TARGET_PORT))

        # DNS over TCP: prepend 2-byte length (big-endian)
        tcp.sendall(struct.pack("!H", len(data)) + data)

        # Read the 2-byte response length
        raw_len = tcp.recv(2)
        if len(raw_len) < 2:
            tcp.close()
            return
        resp_len = struct.unpack("!H", raw_len)[0]

        # Read the full response
        response = b""
        while len(response) < resp_len:
            chunk = tcp.recv(resp_len - len(response))
            if not chunk:
                break
            response += chunk

        tcp.close()

        # Send the response back over UDP (strip TCP length prefix)
        if response:
            sock.sendto(response, client_addr)
    except Exception:
        # Silently drop failed queries (malformed, timeout, etc.)
        pass


def main() -> None:
    # Create UDP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    try:
        sock.bind((LISTEN_ADDR, LISTEN_PORT))
    except PermissionError:
        print(f"dns-proxy: ERROR  need root to bind port {LISTEN_PORT}", flush=True)
        sys.exit(1)
    except OSError as e:
        print(f"dns-proxy: ERROR  {e}", flush=True)
        sys.exit(1)

    print(f"dns-proxy: listening on UDP {LISTEN_ADDR}:{LISTEN_PORT}  TCP {TARGET_HOST}:{TARGET_PORT}", flush=True)
```

```

    # Use a thread pool to handle concurrent DNS queries efficiently without thread exhaustion
    executor = ThreadPoolExecutor(max_workers=64)

    while True:
        try:
            data, client_addr = sock.recvfrom(4096)  # Support EDNS0 (up to 4096 bytes) to prevent truncation
            executor.submit(handle_query, data, client_addr, sock)
        except KeyboardInterrupt:
            break
        except Exception:
            pass

    executor.shutdown(wait=False)
    sock.close()


if __name__ == "__main__":
    main()
```

35 scripts/fix-docker-bridge-collision.sh

```
#!/bin/bash
# scripts/fix-docker-bridge-collision.sh  nullexit fix for devref 9.13
#
# Symptom: host default route points at Docker's bridge gateway (172.17.0.1)
# instead of the real Wi-Fi gateway. ALL host internet egress fails. Remote
# tailnet clients route through the gateway correctly (their tunnel is
# independent of the host's broken route table), so they look healthy while
# the host itself cannot reach anything.
#
# Root cause: Docker's default bridge claims 172.17.0.0/16 by default. When the
# host's physical Wi-Fi also lands in the same /16  extremely common on
# university networks which hand out 172.16.0.0/12  the kernel
# installs Docker's bridge as the default route, and every packet bound for
# the internet is silently absorbed by docker0 instead of egressing.
#
# This script:
#   1. Detects the actual collision (Docker's 172.17.0.0/16 vs the host's
#      Wi-Fi subnet, queried live via the ACTIVE interface  not hardcoded en0)
#   2. Picks a non-colliding Docker bip using /8-family routing (works for
#      172.16.0.0/12 campus networks; naive prefix-match would pick 172.26/24
#      which still lives INSIDE a /12 of 172.16.0.0/12)
#   3. Backs up ~/.colima/default/colima.yaml with an ISO-8601 timestamp
#   4. Edits the file in place, idempotent (replace existing docker.bip /
#      append new docker.bip without disturbing other keys; skip YAML comments)
#   5. Restarts Colima so the VM rebuilds with the new bridge subnet
#   6. Rebinds Wi-Fi DHCP so the host re-learns its real Wi-Fi gateway
#   7. Verifies the new default route is no longer the Docker bridge
#   8. Re-runs toggle.sh to bring the gateway back up cleanly
#   9. Re-runs scripts/diagnose-host-leak.sh and prints the new verdict
#
# Usage:
#   bash scripts/fix-docker-bridge-collision.sh        # apply the fix
#   bash scripts/fix-docker-bridge-collision.sh --dry  # show what would change, then exit
#   bash scripts/fix-docker-bridge-collision.sh --skip-toggle   # fix but don't restart gateway
#   bash scripts/fix-docker-bridge-collision.sh --help # usage

set -uo pipefail
# NOTE: errexit (‘set -e‘) is intentionally NOT enabled so intermediate
# ‘command  || warn ‘ patterns can continue past non-fatal failures.
# Every critical failure (colima restart, sed, cp backup, toggle.sh)
# uses an explicit ‘|| die‘ so a half-applied state never escapes.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
LOG_FILE="$PROJECT_ROOT/output.log"
source "$SCRIPT_DIR/common.sh"
TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
COLIMA_YAML="$HOME/.colima/default/colima.yaml"
PLATFORM="$(uname -s)"

#  Argument parsing
DRY_RUN=false
SKIP_TOGGLE=false
```

```
case "${1:-}" in
  --dry|-n)       DRY_RUN=true ;;
  --skip-toggle)  SKIP_TOGGLE=true ;;
  --help|-h)
    sed -n ’28,37p’ "$0"
    exit 0
    ;;
  "")
    : # default: apply
    ;;
  *)
    printf ’Unknown argument: %s\n’ "$1" >&2
    exit 2
    ;;
esac



# Helper: print "[dry-run] $cmd" or actually run $cmd. Wraps the redirect
# INSIDE the function so outer ‘>> $FILE‘ statements never accidentally
# append dry-run noise to the real file. (Earlier draft didn’t and the
# dry-run polluted $COLIMA_YAML.)
run() {
  if [ "$DRY_RUN" = "true" ]; then
    printf ’      [dry-run] %s\n’ "$*"
    return 0
  fi
  printf ’    $ %s\n’ "$*"
  "$@"
}

# Platform-aware sed in-place. macOS BSD sed needs an empty argument;
# GNU sed (Linux) does not. Earlier draft used ‘-i ’’‘ everywhere and
# would have failed on Linux silently.
sed_inplace() {
  local expr="$1"
  local file="$2"
  case "$PLATFORM" in
    Darwin) run sed -i ’’ "$expr" "$file" ;;
    *)      run sed -i    "$expr" "$file" ;;
  esac
}

#  0. Resolve the ACTIVE physical interface once (used everywhere)
# Replaces the earlier ‘en0‘ hardcoding. Same algorithm recover.sh uses:
# ‘route get default‘  if utun/tun, walk en0..en5 first with ‘status: active‘
# + ‘inet‘  fall back to en0. This means the script also works on hosts
# whose primary NIC is en1/en2 (Thunderbolt Ethernet, USB-LAN, etc.).
resolve_active_iface() {
  local iface
  iface=$(route get default 2>> "$LOG_FILE" | awk ’/interface:/{print $2; exit}’)
  if [ -z "$iface" ] || [ "${iface#utun}" != "$iface" ] || [ "${iface#tun}" != "$iface" ]; then
    for i in en0 en1 en2 en3 en4 en5; do
      if ifconfig "$i" 2>> "$LOG_FILE" | grep -q "status: active" \
         && ifconfig "$i" 2>> "$LOG_FILE" | grep -q "inet "; then
        iface="$i"; break
      fi
    done
  fi
  [ -z "$iface" ] && iface="en0"
  printf ’%s\n’ "$iface"
}
ACTIVE_IFACE=$(resolve_active_iface)

#  1. Detect collision
step "1/9  Detecting Docker-bridge vs Wi-Fi subnet collision"

# Host subnet on the ACTIVE interface (not hardcoded en0).
HOST_SUBNET=$(ifconfig "$ACTIVE_IFACE" 2>> "$LOG_FILE" \
              | awk ’/inet /{print $2; exit}’)
HOST_GATEWAY=$(route get default 2>> "$LOG_FILE" \
               | awk ’/gateway:/{print $2; exit}’)
DEFAULT_DEV=$(route get default 2>> "$LOG_FILE" \
              | awk ’/interface:/{print $2; exit}’)

printf ’  active iface:  %s\n’ "$ACTIVE_IFACE"
printf ’  iface inet:    %s\n’ "${HOST_SUBNET:-none}"
printf ’  default via:   %s\n’ "${HOST_GATEWAY:-none}"
printf ’  default dev:   %s\n’ "${DEFAULT_DEV:-none}"

# Collision fires when the host’s default-route gateway is 172.17.0.1 (Docker’s
```

```
# bridge) OR when the host subnet is in 172.17.0.0/16. The full-bridge-collision
# case (gateway is literally the docker bridge IP) is the smoking gun.
COLLISION=false
if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#172.17.}" != "$HOST_GATEWAY" ]; then
  COLLISION=true
  fail "default gateway $HOST_GATEWAY collides with Docker’s default bridge (172.17.0.0/16)"
elif [ -n "$HOST_SUBNET" ] && [ "${HOST_SUBNET#172.17.}" != "$HOST_SUBNET" ]; then
  COLLISION=true
  fail "host subnet $HOST_SUBNET overlaps Docker’s 172.17.0.0/16 bridge"
else
  ok "no collision detected between host Wi-Fi and Docker’s default bridge"
fi

if [ "$COLLISION" = "false" ]; then
  echo ""
  echo "Nothing to fix. If you still see egress issues, run:"
  echo "    bash scripts/diagnose-host-leak.sh"
  exit 0
fi

#  2. Pick non-colliding bip via /8-family routing
step "2/9  Picking a non-colliding Docker bip for ~/.colima/default/colima.yaml"

# Earlier draft tried naive prefix-match: "if candidate’s 3-octet prefix is
# not a literal prefix of host gateway, pick it". That breaks on /12 campus
# networks (172.16.0.0/12 covers 172.16-172.31): a candidate like
# 172.26.0.1/24 LIVES INSIDE the host’s /12 even though the prefix test
# passes. Fix: classify the host’s address family FIRST, then choose ALL
# candidates from a DIFFERENT RFC1918 family. /8 boundaries are coarse
# enough that no /24 picked this way can ever collide.
case "$HOST_GATEWAY" in
  172.16.*|172.17.*|172.18.*|172.19.*|172.20.*|172.21.*|172.22.*|172.23.*|\
  172.24.*|172.25.*|172.26.*|172.27.*|172.28.*|172.29.*|172.30.*|172.31.*)
    HOST_FAMILY="172"
    # Host is in 172.x  jump past 172.31 entirely. Pick from 10.x / 192.168.x.
    CANDIDATES="10.200.0.1/24 10.201.0.1/24 192.168.99.1/24"
    ;;
  10.*)
    HOST_FAMILY="10"
    # Host is in 10.x  pick from 172.x (well outside any commonly-deployed
    # 10.0.0.0/8 slice) / 192.168.x.
    CANDIDATES="172.26.0.1/24 172.28.0.1/24 192.168.99.1/24"
    ;;
  192.168.*)
    HOST_FAMILY="192"
    CANDIDATES="172.26.0.1/24 10.200.0.1/24 10.201.0.1/24"
    ;;
  *)
    HOST_FAMILY="unknown"
    warn "host gateway $HOST_GATEWAY is not in any RFC1918 range (/8 family)"
    warn "falling back to abstract candidate set; verify collision-free manually"
    CANDIDATES="172.26.0.1/24 10.200.0.1/24 192.168.99.1/24"
    ;;
esac

printf ’  host address family: %s\n’ "$HOST_FAMILY"
printf ’  candidate pool:      %s\n’ "$CANDIDATES"

# Sanity-check the chosen pool has at least one entry that doesn’t literally
# share a /24 with the host gateway. Naive check (just /24 prefix) is fine
# here because we already picked across families.
CHOSEN=""
for cand in $CANDIDATES; do
  PREFIX3=$(printf ’%s’ "$cand" | cut -d. -f1-3)
  if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#"$PREFIX3".}" != "$HOST_GATEWAY" ]; then
    warn "skip $cand (literal /24 collision with host gateway $HOST_GATEWAY)"
    continue
  fi
  CHOSEN="$cand"
  ok "chose bip=$CHOSEN (cross-family, no /24 literal collision)"
  break
done

if [ -z "$CHOSEN" ]; then
  CHOSEN="172.26.0.1/24"
  warn "could not find a clean candidate; defaulting to $CHOSEN"
  warn "verify with ’route get default’ after restart"
fi

#  3. Backup colima.yaml
step "3/9  Backing up $COLIMA_YAML"
```

```
mkdir -p "$(dirname "$COLIMA_YAML")"
BACKUP="$HOME/.colima/default/colima.yaml.bak.$TIMESTAMP"
if [ -f "$COLIMA_YAML" ]; then
  # ‘cp‘ is safe in dry-run because run() suppresses the inner command
  # entirely  no chance of accidentally writing a real backup.
  run cp -p "$COLIMA_YAML" "$BACKUP" || die "backup failed"
  if [ "$DRY_RUN" = "false" ]; then ok "backup: $BACKUP"; fi
else
  warn "no existing $COLIMA_YAML (will create fresh)"
fi

#  4. Edit colima.yaml in place (idempotent)
step "4/9  Setting docker.bip=$CHOSEN in colima.yaml (in place)"

# Resolve the existing docker.bip (if any) without picking up commented-out
# YAML keys. awk-based extraction; the negated-comment check (‘! /^[[:space:]]*#‘)
# ensures ‘# bip: 1.2.3.4/24‘ is treated as a comment, not the active value.
EXISTING_BIP=""
if [ -f "$COLIMA_YAML" ]; then
  EXISTING_BIP=$(awk ’
    /^[[:space:]]*#/ {next}
    /^[[:space:]]*bip:[[:space:]]*/ {print $2; exit}
  ’ "$COLIMA_YAML" 2>> "$LOG_FILE" || true)
fi

if [ -n "$EXISTING_BIP" ] && [ "$EXISTING_BIP" = "$CHOSEN" ]; then
  ok "colima.yaml already has docker.bip=$CHOSEN  no edit needed"
elif [ -n "$EXISTING_BIP" ]; then
  printf ’  (replacing existing docker.bip=%s with %s)\n’ "$EXISTING_BIP" "$CHOSEN"
  # Use sed_inplace so Linux / macOS get the right syntax. Skip YAML comment
  # lines so existing ‘# bip:‘ comments aren’t rewired.
  sed_inplace "s|^\\([[:space:]]*bip:[[:space:]]*\\).*\$|\\1$CHOSEN|" "$COLIMA_YAML" \
    || die "sed replacement failed (colima.yaml malformed?)"
  ok "colima.yaml updated"
else
  # No existing ‘bip:‘ key. Branch:
  #   - file has ‘docker:‘ block  append ‘  bip:‘ under it
  #   - file has no ‘docker:‘ block  append new docker: section
  #   - file does not exist  create fresh
  # Each branch is dry-run-aware so no branch can accidentally leak noise
  # into $COLIMA_YAML during a preview.
  HAS_DOCKER_BLOCK=false
  if [ -f "$COLIMA_YAML" ] && grep -q ’^docker:’ "$COLIMA_YAML"; then
    HAS_DOCKER_BLOCK=true
  fi

  if [ "$HAS_DOCKER_BLOCK" = "true" ]; then
    if [ "$DRY_RUN" = "true" ]; then
      printf ’    [dry-run] would append to %s:\\n  bip: %s\\n’ "$COLIMA_YAML" "$CHOSEN"
    else
      printf ’\n  bip: %s\n’ "$CHOSEN" >> "$COLIMA_YAML" \
        || die "append failed"
      ok "appended bip under existing docker: block"
    fi
  elif [ -f "$COLIMA_YAML" ]; then
    if [ "$DRY_RUN" = "true" ]; then
      printf ’    [dry-run] would append to %s:\\ndocker:\\n  bip: %s\\n’ "$COLIMA_YAML" "$CHOSEN"
    else
      printf ’\ndocker:\n  bip: %s\n’ "$CHOSEN" >> "$COLIMA_YAML" \
        || die "append failed"
      ok "appended new docker: block at end of file"
    fi
  else
    if [ "$DRY_RUN" = "true" ]; then
      printf ’    [dry-run] would create %s with:\\ndocker:\\n  bip: %s\\n’ "$COLIMA_YAML" "$CHOSEN"
    else
      printf ’docker:\n  bip: %s\n’ "$CHOSEN" > "$COLIMA_YAML" \
        || die "create failed"
      ok "created fresh $COLIMA_YAML with docker.bip=$CHOSEN"
    fi
  fi
fi

# Always show the final colima.yaml docker section for visibility
printf ’\n%b  colima.yaml ‘docker:‘ section now reads:%b\n’ "$BOLD" "$NC"
awk ’
  /^docker:/{p=1}
  p && /^[^ ]/ && !/^docker:/{exit}
  p
’ "$COLIMA_YAML" 2>/dev/null | sed ’s/^/    /’ || true

```

```
if [ "$DRY_RUN" = "true" ]; then
  printf '\n  %b(dry-run: exiting before colima restart / dhcp rebind / toggle.sh)%b\n' "$YELLOW" "$NC"
  exit 0
fi

#  5. Restart Colima
step "5/9  Restarting Colima VM with new bip"
if ! command -v colima >> "$LOG_FILE" 2>&1; then
  die "colima not on PATH  install with 'brew install colima' first"
fi
# No timeout wrapper (would mask actual VM startup time). If colima hangs,
# the user kills with Ctrl+C and can manually `colima restart` from a fresh
# terminal  the rest of this script is idempotent so re-running works.
colima restart >> "$LOG_FILE" 2>&1 || die "colima restart failed; check $LOG_FILE"
ok "colima restarted"

#  6. Rebind DHCP on the ACTIVE interface (not hardcoded en0)
step "6/9  Rebinding DHCP on $ACTIVE_IFACE so host re-learns its real gateway"

# Map ifname  macOS network service name (Wi-Fi, USB 10/100 LAN, etc.).
# Same logic as the diagnostic + toggle.sh.
ACTIVE_SVC=""
if [ "$PLATFORM" = "Darwin" ]; then
  ACTIVE_SVC=$(get_active_service)
fi

case "$PLATFORM" in
  Darwin)
    sudo -n networksetup -setdhcp "$ACTIVE_SVC" renew >> "$LOG_FILE" 2>&1 \
      && ok "DHCP rebind issued on $ACTIVE_SVC" \
      || warn "setdhcp renew failed; try 'sudo ipconfig set $ACTIVE_IFACE DHCP' manually"
    ;;
  Linux)
    sudo -n dhclient -r "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 || true
    sudo -n dhclient "$ACTIVE_IFACE"     >> "$LOG_FILE" 2>&1 \
      && ok "dhclient rebind on $ACTIVE_IFACE" \
      || warn "dhclient rebind failed"
    ;;
  *)
    warn "unsupported platform $PLATFORM; skip DHCP rebind"
    ;;
esac

#  7. Verify new default route
step "7/9  Verifying the new default route is NOT the Docker bridge"
sleep 2  # give DHCP + kernel a beat
NEW_DEFAULT=$(netstat -rn 2>> "$LOG_FILE" | awk '/^default/ {print $0; exit}')
printf '  %s\n' "${NEW_DEFAULT:-no default route detected}"
NEW_GATEWAY=$(printf '%s' "$NEW_DEFAULT" | awk '{print $2; exit}')

if [ -n "$NEW_GATEWAY" ] && [ "${NEW_GATEWAY#172.17.}" != "$NEW_GATEWAY" ]; then
  fail "default route STILL points at 172.17.x.x  DHCP rebind may need manual touch"
  warn "try: sudo ipconfig set $ACTIVE_IFACE DHCP  (or reconnect Wi-Fi in the menu bar)"
elif [ -n "$NEW_GATEWAY" ]; then
  ok "default route is now via $NEW_GATEWAY (no longer Docker's bridge)"
else
  warn "could not determine new gateway; verify with 'route get default'"
fi

#  8. Re-run toggle.sh to bring the gateway back up
if [ "$SKIP_TOGGLE" = "true" ]; then
  step "8/9  Skipping toggle.sh (--skip-toggle)"
  warn "gateway is currently down  run ./toggle.sh manually when ready"
else
  step "8/9  Re-running toggle.sh to bring the gateway back up"
  if bash "$PROJECT_ROOT/toggle.sh" 2>&1 | tee -a "$LOG_FILE"; then
    ok "toggle.sh completed"
  else
    warn "toggle.sh returned non-zero  see $LOG_FILE for details"
  fi
fi

#  9. Re-run the diagnostic + show verdict
step "9/9  Re-running host-leak diagnostic to confirm fix"
if bash "$SCRIPT_DIR/diagnose-host-leak.sh" 2>&1 | tee -a "$LOG_FILE"; then
  LATEST_REPORT=$(ls -t "$PROJECT_ROOT"/host-leak-diagnostic-*.txt 2>/dev/null | head -1)
  if [ -n "$LATEST_REPORT" ]; then
    VERDICT=$(awk '/^ Scenario:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
    WARP=$(awk -F': *' '/^ WARP egress:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
    printf '\n  Most recent verdict: Scenario=%s  WARP=%s\n' \
           "${VERDICT:-?}" "${WARP:-?}"
```

```
    if [ "$VERDICT" = "OK" ]; then
      ok "fix confirmed by diagnostic  host traffic is going through WARP"
    fi
  fi
else
  warn "diagnostic exited non-zero; verify manually"
fi

printf '\n%b%b\n' "$BOLD" "$NC"
printf '%bDone.%b Full transcript at %s\n' "$BOLD" "$NC" "$LOG_FILE"
printf '%b%b\n' "$BOLD" "$NC"
exit 0
```

36 scripts/fix-ssh-delay.sh

```
#!/bin/bash
# scripts/fix-ssh-delay.sh  Fix ~20s SSH connection delay caused by UseDNS
#
# macOS OpenSSH defaults UseDNS to "yes". When Tailscale SSH connects from
# a peer (phone/laptop), sshd performs a reverse-DNS (PTR) lookup on the
# client's 100.x.x.x Tailscale IP. Since DNS is hijacked to the nullexit
# gateway  AdGuard  Cloudflare WARP, and 100.64.0.0/10 is CGNAT private
# space, the PTR query times out (~15-20s) before SSH proceeds.
#
# This script uncomments "UseDNS no" in /etc/ssh/sshd_config and reloads
# sshd. Existing SSH sessions are NOT dropped.
#
# Usage:
#   sudo bash scripts/fix-ssh-delay.sh

set -euo pipefail

SSHD_CONFIG="/etc/ssh/sshd_config"

#  Check we're root
if [ "$(id -u)" -ne 0 ]; then
    echo "ERROR: This script must be run with sudo (it edits $SSHD_CONFIG)"
    echo "Usage: sudo bash scripts/fix-ssh-delay.sh"
    exit 1
fi

#  Apply config (idempotent)
if grep -qE '^UseDNS no' "$SSHD_CONFIG" 2>/dev/null; then
    echo " UseDNS is already set to 'no' in $SSHD_CONFIG"
elif grep -qE '^#UseDNS' "$SSHD_CONFIG" 2>/dev/null; then
    echo " Uncommenting 'UseDNS no' in $SSHD_CONFIG"
    sed -i '' 's/^#UseDNS no/UseDNS no/' "$SSHD_CONFIG"
elif grep -qEi '^UseDNS' "$SSHD_CONFIG" 2>/dev/null; then
    echo " Changing existing UseDNS line to 'UseDNS no'"
    sed -i '' 's/^UseDNS.*/UseDNS no/' "$SSHD_CONFIG"
else
    echo " Appending 'UseDNS no' to $SSHD_CONFIG"
    echo "" >> "$SSHD_CONFIG"
    echo "UseDNS no" >> "$SSHD_CONFIG"
fi

#  Verify
if ! grep -qE '^UseDNS no' "$SSHD_CONFIG"; then
    echo " Failed to apply UseDNS no  check $SSHD_CONFIG manually"
    exit 1
fi

#  Reload sshd (does NOT drop existing connections)
echo " Reloading sshd..."

SSHD_PID=$(pgrep -f /usr/sbin/sshd 2>/dev/null | head -1 || true)
if [ -n "$SSHD_PID" ]; then
    kill -HUP "$SSHD_PID"
    echo " Sent SIGHUP to sshd (PID $SSHD_PID)  config reloaded"
else
    echo "  No running sshd found. macOS starts sshd on-demand per connection."
    echo "  The new 'UseDNS no' config will take effect on the next SSH connection."
fi

echo ""
echo " Done. SSH connections should now connect instantly."
```

```
echo "  Existing sessions were not interrupted."
```

37 scripts/generate_tex.py

```
#!/usr/bin/env python3
import os

PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ’..’))

IGNORE_DIRS = {’.git’, ’.github’, ’adguard’, ’__pycache__’, ’Recover Gateway.app’, ’Toggle Gateway.app’}
IGNORE_FILES = {’.env’, ’nullexit_unified.tex’, ’nullexit_unified.pdf’, ’output.log’, ’blocked.log’, ’.DS_Store’, ’.lan_p2p_detected’, ’.signatures’, ’.gateway_ip’, ’TUNNEL_FAILED_CLOSED.marker’}
IGNORE_EXTS = {’.pdf’, ’.tex’, ’.png’, ’.jpg’, ’.jpeg’, ’.zip’, ’.tar’, ’.gz’, ’.log’, ’.aux’, ’.fls’, ’.fdb_latexmk’, ’.out’, ’.toc’, ’.xdv’, ’.synctex(busy)’}

# Exclude from code block inclusion, but keep in tree
SKIP_CONTENT_FILES = {’LICENSE’}

PRIORITY_FILES = [
    ’README.md’,
    ’agent.md’,
    ’devref.md’,
    ’diagrams.md’,
    ’toggle.sh’,
    ’recover.sh’,
    ’setup.sh’,
    ’scripts/toggle-linux.sh’,
    ’scripts/recover-linux.sh’,
    ’scripts/setup-linux.sh’,
    ’docker-compose.yml’,
    ’scripts/common.sh’,
    ’scripts/watcher.sh’,
    ’scripts/crypto.sh’
]

def get_language(filename):
    ext = os.path.splitext(filename)[1].lower()
    if ext == ’.sh’ or ext == ’.applescript’:
        return ’bash’
    elif ext == ’.py’:
        return ’Python’
    elif ext == ’.go’:
        return ’Go’
    elif ext in {’.yml’, ’.yaml’}:
        return ’bash’
    elif ext == ’.plist’:
        return ’XML’
    elif ext == ’.md’:
        return ’’
    return ’’

def escape_tex(text):
    return text.replace(’_’, ’\\_’)

def generate_tree(dir_path, prefix=""):
    tree_str = ""
    entries = sorted(os.listdir(dir_path))
    entries = [e for e in entries if e not in IGNORE_DIRS and e not in IGNORE_FILES and os.path.splitext(e)[1].lower() not in IGNORE_EXTS]

    for i, entry in enumerate(entries):
        path = os.path.join(dir_path, entry)
        is_last = (i == len(entries) - 1)
        connector = "‘-- " if is_last else "|-- "
        tree_str += f"{prefix}{connector}{entry}\n"

        if os.path.isdir(path):
            extension = "    " if is_last else "|   "
            tree_str += generate_tree(path, prefix + extension)

    return tree_str

def main():
    tex_path = os.path.join(PROJECT_ROOT, ’nullexit_unified.tex’)

    preamble = r"""\documentclass{article}
```

```
\usepackage[utf8]{inputenc}
\usepackage[margin=1in]{geometry}
\usepackage{listings}
\usepackage{xcolor}
\usepackage{hyperref}
\usepackage{tocloft}

\definecolor{codegreen}{rgb}{0,0.6,0}
\definecolor{codegray}{rgb}{0.5,0.5,0.5}
\definecolor{codepurple}{rgb}{0.58,0,0.82}
\definecolor{backcolour}{rgb}{0.97,0.97,0.96}

\lstdefinestyle{mystyle}{
    backgroundcolor=\color{backcolour},
    commentstyle=\color{codegreen},
    keywordstyle=\color{blue}\bfseries,
    numberstyle=\tiny\color{codegray},
    stringstyle=\color{codepurple},
    basicstyle=\ttfamily\scriptsize,
    breakatwhitespace=false,
    breaklines=true,
    captionpos=b,
    keepspaces=true,
    numbers=left,
    numbersep=5pt,
    showspaces=false,
    showstringspaces=false,
    showtabs=false,
    tabsize=2,
    frame=single,
    rulecolor=\color{codegray},
    extendedchars=true,
    literate={}{-}1 {}{=}1 {}{{OK}}2 {}{-}1 {}{|}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{+}1 {}{v}1
}
\lstset{style=mystyle}

\title{Nullexit: Unified Project Document}
\author{Omar Alaaeldein}
\date{\today}

\begin{document}
\maketitle

\tableofcontents
\newpage

\section{Project Directory Tree}
\begin{verbatim}
nullexit/
"""

    tree = generate_tree(PROJECT_ROOT)
    midamble = r"""\end{verbatim}
\newpage
"""

    all_relpaths = []
    for root, dirs, files in os.walk(PROJECT_ROOT):
        dirs[:] = sorted([d for d in dirs if d not in IGNORE_DIRS])
        for file in sorted(files):
            if file in IGNORE_FILES or file in SKIP_CONTENT_FILES or os.path.splitext(file)[1].lower() in IGNORE_EXTS:
                continue

            filepath = os.path.join(root, file)
            relpath = os.path.relpath(filepath, PROJECT_ROOT)
            all_relpaths.append(relpath)

    def sort_key(path):
        if path in PRIORITY_FILES:
            return (0, PRIORITY_FILES.index(path))
        return (1, path)

    all_relpaths.sort(key=sort_key)

    with open(tex_path, 'w', encoding='utf-8') as f:
        f.write(preamble)
        f.write(tree)
        f.write(midamble)

```

```
        for relpath in all_relpaths:
            lang = get_language(relpath)
            lang_attr = f"[language={lang}]" if lang else ""

            f.write(f"\\section{{{escape_tex(relpath)}}}\n")
            f.write(f"\\begin{{lstlisting}}{lang_attr}\n")
            try:
                with open(os.path.join(PROJECT_ROOT, relpath), ’r’, encoding=’utf-8’, errors=’ignore’) as src:
                    content = src.read()
                    import re
                    # Strip all non-ASCII characters (emojis, box drawings, em dashes, etc)
                    content = re.sub(r’[^\x00-\x7F]+’, ’’, content)
                    f.write(content)
                    if not content.endswith(’\n’):
                        f.write(’\n’)
            except Exception as e:
                f.write(f"Error reading file: {e}\n")
            f.write("\\end{lstlisting}\n\n")

        f.write("\\end{document}\n")

    print(f"Generated {tex_path}")

if __name__ == ’__main__’:
    main()
```

38 scripts/logger/go.mod

```
module nullexit/logger

go 1.22
```

39 scripts/logger/main.go

```
package main

import (
  "bufio"
  "encoding/json"
  "fmt"
  "io"
  "log"
  "os"
  "regexp"
  "strings"
  "time"
)

const (
  queryLogPath  = "/adguard_work/data/querylog.json"
  procKmsgPath  = "/proc/kmsg"
  outputLogPath = "/app/blocked.log"
)

func logMessage(msg string) {
  timestamp := time.Now().Format("2006-01-02 15:04:05")
  formattedMsg := fmt.Sprintf("%s %s\n", timestamp, msg)
  fmt.Print(formattedMsg)

  f, err := os.OpenFile(outputLogPath, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
  if err != nil {
    log.Printf("Error writing to log file: %v\n", err)
    return
  }
  defer f.Close()
  f.WriteString(formattedMsg)
}

func tailFile(filepath string, out chan<- string) {
  for {
    if _, err := os.Stat(filepath); os.IsNotExist(err) {
```

```
      time.Sleep(1 * time.Second)
      continue
    }
    break
  }

  f, err := os.Open(filepath)
  if err != nil {
    log.Printf("Error opening file %s: %v\n", filepath, err)
    return
  }
  defer func() {
    if f != nil {
      f.Close()
    }
  }()

  // Seek to end
  f.Seek(0, io.SeekEnd)
  reader := bufio.NewReader(f)

  for {
    line, err := reader.ReadString('\n')
    if err != nil {
      if err == io.EOF {
        stat, err := os.Stat(filepath)
        if err == nil {
          currentOffset, _ := f.Seek(0, io.SeekCurrent)
          if stat.Size() < currentOffset {
            // File truncated or rotated
            f.Close()
            for {
              if _, err := os.Stat(filepath); os.IsNotExist(err) {
                time.Sleep(1 * time.Second)
                continue
              }
              break
            }
            f, _ = os.Open(filepath)
            reader = bufio.NewReader(f)
            continue
          }
        }
        time.Sleep(500 * time.Millisecond)
        continue
      }
      log.Printf("Error reading file %s: %v\n", filepath, err)
      time.Sleep(1 * time.Second)
      continue
    }
    out <- line
  }
}

type adguardLogEntry struct {
  IP     string `json:"IP"`
  QH     string `json:"QH"`
  QT     string `json:"QT"`
  Result struct {
    IsFiltered bool `json:"IsFiltered"`
    Reason     int  `json:"Reason"`
    Rules      []struct {
      Text string `json:"Text"`
    } `json:"Rules"`
  } `json:"Result"`
}

func monitorDNS() {
  logMessage("[System] Starting DNS block logger...")
  lines := make(chan string)
  go tailFile(queryLogPath, lines)

  reasonMap := map[int]string{
    2: "CustomRule",
    3: "BlockList",
    4: "SafeBrowsing",
    5: "ParentalControl",
    6: "SafeSearch",
    7: "BlockedService",
  }

```

```
  for line := range lines {
    var data adguardLogEntry
    if err := json.Unmarshal([]byte(line), &data); err != nil {
      continue
    }

    res := data.Result
    if res.IsFiltered || (res.Reason != 0 && res.Reason != 1) {
      qh := data.QH
      if qh == "" {
        qh = "unknown"
      }
      qt := data.QT
      if qt == "" {
        qt = "unknown"
      }
      clientIP := data.IP
      if clientIP == "" {
        clientIP = "unknown"
      }

      ruleText := "unknown rule"
      if len(res.Rules) > 0 && res.Rules[0].Text != "" {
        ruleText = res.Rules[0].Text
      }

      reasonStr, ok := reasonMap[res.Reason]
      if !ok {
        reasonStr = fmt.Sprintf("Reason-%d", res.Reason)
      }

      logMessage(fmt.Sprintf("[DNS] Blocked %s (Type: %s) for client %s | Reason: %s | Rule: %s", qh, qt,
      clientIP, reasonStr, ruleText))
    }
  }
}

func monitorIPs() {
  logMessage("[System] Starting IP block logger...")

  prefixRe := regexp.MustCompile(‘(IP_BLOCK_[A-Z_]+):‘)
  srcRe := regexp.MustCompile(‘SRC=([0-9a-fA-F\.:]+)‘)
  dstRe := regexp.MustCompile(‘DST=([0-9a-fA-F\.:]+)‘)
  protoRe := regexp.MustCompile(‘PROTO=([A-Z0-9]+)‘)
  sptRe := regexp.MustCompile(‘SPT=(\d+)‘)
  dptRe := regexp.MustCompile(‘DPT=(\d+)‘)
  inRe := regexp.MustCompile(‘IN=([a-zA-Z0-9\.\-_]+)‘)
  outRe := regexp.MustCompile(‘OUT=([a-zA-Z0-9\.\-_]+)‘)

  f, err := os.Open(procKmsgPath)
  if err != nil {
    if os.IsPermission(err) {
      logMessage("[System] ERROR: Insufficient permissions to read /proc/kmsg. Enable CAP_SYSLOG.")
    } else {
      logMessage(fmt.Sprintf("[System] ERROR in IP logger: %v", err))
    }
    return
  }
  defer f.Close()

  reader := bufio.NewReader(f)
  for {
    line, err := reader.ReadString(’\n’)
    if err != nil {
      if err == io.EOF {
        time.Sleep(100 * time.Millisecond)
        continue
      }
      logMessage(fmt.Sprintf("[System] ERROR reading kmsg: %v", err))
      time.Sleep(1 * time.Second)
      continue
    }

    if strings.Contains(line, "IP_BLOCK") {
      match := prefixRe.FindStringSubmatch(line)
      if len(match) > 1 {
        prefix := match[1]

        src := "unknown"
        if m := srcRe.FindStringSubmatch(line); len(m) > 1 {
          src = m[1]
```

```go
        }

        dst := "unknown"
        if m := dstRe.FindStringSubmatch(line); len(m) > 1 {
          dst = m[1]
        }

        proto := "unknown"
        if m := protoRe.FindStringSubmatch(line); len(m) > 1 {
          proto = m[1]
        }

        spt := ""
        if m := sptRe.FindStringSubmatch(line); len(m) > 1 {
          spt = m[1]
        }

        dpt := ""
        if m := dptRe.FindStringSubmatch(line); len(m) > 1 {
          dpt = m[1]
        }

        inIf := ""
        if m := inRe.FindStringSubmatch(line); len(m) > 1 {
          inIf = m[1]
        }

        outIf := ""
        if m := outRe.FindStringSubmatch(line); len(m) > 1 {
          outIf = m[1]
        }

        direction := "inbound"
        if strings.HasSuffix(prefix, "DST") {
          direction = "outbound"
        }

        listType := "MALICIOUS"
        if !strings.Contains(prefix, "MALICIOUS") {
          parts := strings.Split(prefix, "_")
          if len(parts) >= 3 {
            listType = "GEO_" + parts[2]
          }
        }

        portStr := ""
        if dpt != "" {
          portStr = ":" + dpt
        }

        srcPortStr := ""
        if spt != "" {
          srcPortStr = ":" + spt
        }

        ifStr := ""
        if inIf != "" && outIf != "" {
          ifStr = fmt.Sprintf("IF: %s->%s", inIf, outIf)
        } else if inIf != "" || outIf != "" {
          ifStr = fmt.Sprintf("IF: %s%s", inIf, outIf)
        }

        logMessage(fmt.Sprintf("[IP] Blocked %s to %s%s (Proto: %s) from %s%s (%s) | List: %s", direction, dst, portStr, proto, src, srcPortStr, ifStr, listType))
      }
    }
  }
}

func main() {
  logMessage("[System] Nullexit Block Logger starting...")

  go monitorDNS()
  monitorIPs()
}
```

40 scripts/pf.conf

```
# scripts/pf.conf - Packet Filter configuration for nullexit killswitch
# Dynamically parameterized by pfctl macro override

# ext_if must be passed via command line, e.g. -D ext_if=en0
# If not passed, default to en0
ext_if = "en0"
ts_if = "utun+"

# Tables populated dynamically via pfctl
table <vpn_endpoints> persist
table <derp_relays> persist

# Core Rules
set skip on lo0                # Ensure local loopback is never blocked
scrub out all max-mss 1160     # TCP MSS Clamping: Prevents fragmentation on host TCP flows (like Tailscale control plane)
pass quick on $ts_if all       # Allow all Tailscale traffic (essential to prevent SSH lockout)

# Default deny outbound WAN traffic
block out on $ext_if all

# Block inbound Screen Sharing (VNC) and SSH from local Wi-Fi, forcing them to use Tailscale
block in on $ext_if proto tcp from any to any port { 22, 5900 }

# Exceptions for tunnel handshakes and coordination
pass out quick on $ext_if proto udp to <vpn_endpoints> port 2408
pass out quick on $ext_if proto udp to <derp_relays>
pass out quick on $ext_if proto tcp to 192.200.0.0/24 port 443

# Allow local LAN traffic (AirDrop, local Tailscale P2P, etc)
pass out quick on $ext_if proto tcp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
pass out quick on $ext_if proto udp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
pass out quick on $ext_if proto icmp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }

# Allow DHCP outbound (client port 68 to server port 67) so we can obtain/renew IP address
pass out quick on $ext_if proto udp from any port 68 to any port 67
```

41 scripts/reinstall-host-tailscale.sh

```
#!/usr/bin/env bash
#
# scripts/reinstall-host-tailscale.sh
#
# Complete uninstall + reinstall of the host-side Homebrew Tailscale.
#
# Use this when the brew LaunchAgent is wedged, Login Items entries are
# duplicated, "Tailscale is stopped" persists despite
# ‘brew services restart tailscale‘, or you simply want a clean baseline
# before re-engaging the gateway as exit node.
#
# Idempotent. Safe to re-run. Use --dry to preview every step with zero
# changes. Use --yes to skip the per-step confirmation prompts.
#
# This script ONLY touches the HOST-side Tailscale (the one installed by
# Homebrew ‘brew install tailscale‘ and managed by launchd as
# homebrew.mxcl.tailscale). The nullexit gateway container’s tailscaled
# (which lives inside the Colima VM at 100.100.21.8 and offers the exit
# node to this host) is unaffected.

set -u

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

DRY=0
YES=0

for arg in "$@"; do
  case "$arg" in
    --dry)  DRY=1 ;;
    --yes)  YES=1 ;;
    --help|-h)      # NOTE: heredoc is UNQUOTED so $SCRIPT_DIR expands in the body.  If you
      # add new $-style placeholders here, they’ll expand the same way.
      cat <<USAGE
```

```
Usage: bash "$SCRIPT_DIR/reinstall-host-tailscale.sh" [--dry] [--yes]

  --dry  Print every step, make zero changes. Use to audit before
         committing.
  --yes  Auto-confirm the prompts at every destructive step. Useful in
         CI / fully-scripted recovery; do not use until you’ve read
         the script.

After this script completes cleanly, you still must (MANUALLY):

  1. Open System Settings  Privacy & Security  Local Network and
     enable Tailscale. macOS will prompt automatically the first time
     the fresh install of ‘tailscale‘ is invoked.

  2. Re-engage the tailnet + exit node:
         sudo tailscale up --ssh=true --accept-dns=false \
           --exit-node="$(cat .gateway_ip | tr -d ’\r’ | awk ’NR==1{print $1; exit}’)" \
           --exit-node-allow-lan-access=true

  3. Re-run ./toggle.sh from the project root.

  4. Final verification:
         curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E ’^(warp|ip|colo)=’
     expected: warp=on, ip=<Cloudflare>, colo=YYZ/YUL
USAGE
      exit 0
      ;;
    *)
      echo "Unknown flag: $arg" >&2
      exit 2
      ;;
  esac
done

# ---------- helpers ----------

confirm() {
  if [ "$YES" = 1 ] || [ "$DRY" = 1 ]; then
    return 0
  fi
  printf "      %s  [y/N] " "$1"
  read -r ans
  case "$ans" in
    y|Y|yes|YES) return 0 ;;
    *)
      echo "     aborted by user"
      exit 1
      ;;
  esac
}

header() {
  printf ’\n %s\n’ "$1"
}

# with_timeout <seconds> <cmd...>
# Run a command in the background and kill it if it exceeds <seconds>.
# Returns the command’s exit code if it finished in time, 124 (matching
# GNU ‘timeout‘ convention) if it timed out. macOS doesn’t ship GNU
# coreutils ‘timeout‘, so this is the bash-3.2-friendly replacement.
# stdout + stderr are discarded (we only care about exit code); redirect
# or pipe externally if you need the output.
with_timeout() {
  local timeout_s="${1:-30}"
  shift
  "$@" >/dev/null 2>&1 &
  local cmdpid=$!
  local elapsed=0
  while kill -0 "$cmdpid" 2>/dev/null; do
    if [ "$elapsed" -ge "$timeout_s" ]; then
      kill -9 "$cmdpid" 2>/dev/null
      wait "$cmdpid" 2>/dev/null
      return 124
    fi
    sleep 1
    elapsed=$((elapsed+1))
  done
  wait "$cmdpid" 2>/dev/null
  return $?
}

```

```
# ---------- 0. Pre-flight ----------
header "Pre-flight  current host tailscale state"
echo "    brew:                   $(command -v brew >/dev/null 2>&1 && brew --version | head -1 || echo ’not on PATH’)"
echo "    tailscaled running:     $(pgrep -lf tailscaled 2>/dev/null | wc -l | tr -d ’ ’) process(es)"
echo "    brew service tailscale: $(brew services list 2>/dev/null | awk ’$1=="tailscale"{print $2, $3}’ | tr ’\n’ ’ ’ | sed ’s/ $//’)"
echo "    LaunchAgent plists (under ~/Library/LaunchAgents/):"
ls -1 ~/Library/LaunchAgents/ 2>/dev/null | grep -i ’tail’ | sed ’s/^/      /’ || echo "      (none)"

# Detect Tailscale’s macOS Network Extension (residual from prior App Store
# Tailscale.app installs). Lives under /Library/Apple/SystemExtensions/ and
# is managed by ‘systemextensionsctl‘  NOT launchd. If loaded, brew
# tailscaled will NOT engage the Network Extension framework slot because
# the SE still has it claimed. Symptom: ’sudo tailscale status’ returns
# "Tailscale is stopped" even when the daemon is alive and listening.
se_found=0
if command -v systemextensionsctl >/dev/null 2>&1; then
  # Grab any line matching tailscale and network-extension
  se_line=$(systemextensionsctl list 2>/dev/null | grep -iE ’(io|com)\.tailscale\..*network-extension’ | head -n 1)
  if [ -n "$se_line" ]; then
    se_found=1
    # Extract the teamID (10-char uppercase/numeric word)
    se_teamID=$(echo "$se_line" | tr -s ’ \t’ ’\n’ | grep -E ’^[A-Z0-9]{10}$’ | head -n 1)
    # Extract the bundleID (starts with io.tailscale or com.tailscale)
    se_bundleID=$(echo "$se_line" | tr -s ’ \t’ ’\n’ | grep -E ’^(io|com)\.tailscale\.’ | head -n 1 | sed ’s/(.*//’)

    # If teamID is not found, fallback to ’-’
    if [ -z "$se_teamID" ]; then
      se_teamID="-"
    fi
  fi
fi

if [ "$se_found" = 1 ] && [ -n "$se_bundleID" ]; then
  echo "     Tailscale Network Extension System Extension is loaded (residual from App Store Tailscale.app install)"
  echo "      Team ID: $se_teamID, Bundle ID: $se_bundleID"
  echo "      brew tailscaled cannot engage the NE slot while this SE has it claimed."
  if [ "$DRY" = 1 ]; then
    echo "    [dry] (would prompt) sudo systemextensionsctl uninstall $se_teamID $se_bundleID"
  else
    confirm "Run ’sudo systemextensionsctl uninstall $se_teamID $se_bundleID’? (frees the NE slot for brew tailscale; non-fatal if it errors)"
    sudo systemextensionsctl uninstall "$se_teamID" "$se_bundleID" 2>&1 \
      || echo "    ! uninstall returned non-zero (may require reboot, non-fatal)"
  fi
else
  echo "     no Tailscale System Extension loaded"
fi

# Pattern used by the Step-1 bootout loops, in both dry + real branches.
# Widened from a bare /tailscale/ so the same loops also pick up any
# nullexit-labeled LaunchAgents (e.g., com.nullexit.wake-recovery, the
# caffeinate + post-wake recovery hook). The wake-recovery LaunchAgent
# is re-installed by ./toggle.sh / setup.sh on re-engage, so wiping it
# here is non-fatal and is part of restoring a clean baseline.
TAILSCALE_LABEL_RE=’tailscale’
NULLEXIT_LABEL_RE=’nullexit’

# ---------- 1. brew services stop ----------
header "Step 1  Stop the brew-managed service + unload LaunchAgents"
# Detect whether the brew-managed service is registered as $USER (gui/$UID
# domain) or as root (system domain). NOPASSWD sudoers can leave a stale
# root-owned registration after a prior ’sudo brew services ...’ round;
# ’brew services stop’ without sudo refuses to touch a root-owned plist
# (Error: Service ’tailscale’ is started as ‘root‘).
brew_status_line="$(brew services list 2>/dev/null | awk ’$1=="tailscale"’)"
brew_status_user="$(echo "$brew_status_line" | awk ’{print $3}’)"
if [ -n "$brew_status_line" ]; then
  if [ "$DRY" = 1 ]; then
    echo "    [dry] brew services stop tailscale (user=$brew_status_user)"
  else
    if [ "$brew_status_user" = "root" ]; then
      confirm "Run ’sudo brew services stop tailscale’ (service is registered as root)?"
      sudo brew services stop tailscale 2>&1 || echo "    (sudo brew services stop returned non-zero  non-fatal)"
    else
      echo "     brew services stop tailscale (user=$brew_status_user, no sudo needed)"
```

```
      brew services stop tailscale 2>&1 || echo "    (brew reported it wasn’t actually running)"
    fi
  fi
  echo "     unload any tailscale-related LaunchAgents (prevents launchd from respawning after kill)"
  if [ "$DRY" = 1 ]; then
    echo "      [dry] for each ’tailscale|nullexit’-matched label in gui/$UID domain:"
    while IFS= read -r label; do
      [ -z "$label" ] && continue
      echo "             [dry] launchctl bootout gui/$UID/$label"
    done < <(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" ’NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}’)
    echo "      [dry] for each ’tailscale|nullexit’-matched label in system domain (root-owned brew plist):"
    while IFS= read -r label; do
      [ -z "$label" ] && continue
      echo "             [dry] (would prompt) sudo launchctl bootout system/$label"
    done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" ’NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}’)
    if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
      echo "      [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macos"
    fi
    if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
      echo "      [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macsys"
    fi
  else
    # Bootout every gui-domain LaunchAgent whose label matches ’tailscale’
    # OR ’nullexit’. The nullexit arm catches com.nullexit.wake-recovery,
    # which holds caffeinate/watcher.sh alive across a daemon reset  not
    # desired while the script is establishing a fresh baseline. Awake-
    # recovery is re-installed by ./toggle.sh / setup.sh on re-engage, so
    # wiping here is non-fatal. The pattern stays narrower than a bare
    # /tail/ or /null/ so unrelated Apple services are not unloaded.
    while IFS= read -r label; do
      [ -z "$label" ] && continue
      echo "       launchctl bootout gui/$UID/$label"
      launchctl bootout "gui/$UID/$label" 2>/dev/null || true
    done < <(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" ’NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}’)
    # Bootout every system-domain LaunchDaemon/Agent whose label matches
    # ’tailscale’ OR ’nullexit’. This catches the root-owned brew plist left
    # behind by prior ’sudo brew services ...’ invocations  that was the
    # case that left tailscaled PID 47447  57734 respawning between sudo
    # pkill runs. Defensive nullexit catch in case the user previously
    # ‘sudo ln -s‘’d the wake-recovery plist into /Library/LaunchDaemons.
    while IFS= read -r label; do
      [ -z "$label" ] && continue
      confirm "Run ’sudo launchctl bootout system/$label’?"
      sudo launchctl bootout "system/$label" 2>/dev/null || true
    done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" ’NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}’)
    # Legacy Tailscale.app system-domain entries. Each one is checked and
    # confirmed separately so we don’t sudo-promp or sudo-bootout for nothing.
    if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
      confirm "Run ’sudo launchctl bootout system/com.tailscale.ipn.macos’?"
      sudo launchctl bootout system/com.tailscale.ipn.macos 2>/dev/null || true
    fi
    if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
      confirm "Run ’sudo launchctl bootout system/com.tailscale.ipn.macsys’?"
      sudo launchctl bootout system/com.tailscale.ipn.macsys 2>/dev/null || true
    fi
  fi
  echo "     waiting up to 10s for tailscaled to exit"
  for i in 1 2 3 4 5 6 7 8 9 10; do
    if [ "$DRY" = 1 ]; then
      echo "    [dry] sleep 1; loop check would terminate here"
      break
    fi
    sleep 1
    if ! pgrep -lf tailscaled >/dev/null 2>&1; then
      echo "     tailscaled exited in ${i}s"
      break
    fi
    [ "$i" = 10 ] && echo "     tailscaled still alive after 10s  Step 2 will deal with it"
  done
else
  echo "     brew reports no tailscale service loaded"
fi

# ---------- 2. Kill any orphan tailscaled ----------
header "Step 2  Kill any orphan tailscaled process(es) (may escalate to sudo)"
if ! pgrep -lf tailscaled >/dev/null 2>&1; then
```

```
  echo "     no orphans"
else
  echo "    Stray process(es):"
  pgrep -lf tailscaled | sed 's/^/      /'
  echo "     no-sudo: pkill tailscaled"
  if [ "$DRY" = 1 ]; then
    echo "    [dry] pkill tailscaled  on failure: sudo pkill -9 tailscaled"
  else
    if pkill tailscaled 2>/dev/null; then
      echo "     no-sudo pkill succeeded"
    else
      echo "    ! no-sudo pkill insufficient; cannot escalate yet"
    fi
    sleep 1
  fi

  # Recheck; if still alive, offer sudo escalation
  if { [ "$DRY" = 0 ] && pgrep -lf tailscaled >/dev/null 2>&1; } || [ "$DRY" = 1 ]; then
    if [ "$DRY" = 1 ]; then
      echo "    [dry] sudo pkill -9 tailscaled  (escalation would fire here)"
    else
      confirm "Run 'sudo pkill -9 tailscaled' to escalate?"
      sudo pkill -9 tailscaled 2>&1
      sleep 1
    fi
  fi

  if [ "$DRY" = 0 ]; then
    if pgrep -lf tailscaled >/dev/null 2>&1; then
      echo "     Failed to kill stragglers  bailing out"
      pgrep -lf tailscaled | sed 's/^/      /'
      exit 3
    fi
    echo "     all tailscaled processes gone"
  fi
fi

# ---------- 3. brew uninstall ----------
header "Step 3  brew uninstall tailscale"
if ! brew list tailscale >/dev/null 2>&1; then
  echo "     not currently installed via brew  skip"
else
  # Detect whether the keg directory is owner=$USER or owner=root. If the
  # keg is root-owned (typically a residual of some prior 'sudo brew ...'
  # cycle that did perms fixups), 'brew uninstall' without sudo refuses
  # to remove the files and prints: 'Error: Could not remove tailscale
  # keg! Do so manually: sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>'.
  # Detect that and escalate to a sudoed uninstall, with a manual rm -rf
  # fallback if `sudo brew uninstall` itself errors out (e.g., partial
  # state where the keg dir is gone but brew still thinks it's installed).
  # Detect keg state. Treat `(dir absent)` AND `(dir present + empty)` as the
  # same `<unknown>` bucket so both escalate to sudo. The empty-parent case
  # is what you get after a manual `sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>`
  # from inside: brew's parent dir is left present-but-empty and brew still
  # thinks the package is installed.
  keg_dir="/opt/homebrew/Cellar/tailscale"
  if [ -d "$keg_dir" ] && [ -n "$(ls -A "$keg_dir" 2>/dev/null)" ]; then
    keg_owner="$(stat -f '%Su' "$keg_dir" 2>/dev/null || echo '<unknown>')"
  else
    keg_owner="<unknown>"
  fi
  if [ "$DRY" = 1 ]; then
    echo "    [dry] brew uninstall tailscale (keg_owner=$keg_owner)"
  else
    # 'root' AND '<unknown>' both require sudo escalation. The '<unknown>'
    # case is the partial-state path: keg dir was already removed (manually
    # or by a prior failed uninstall) but brew's internal state still says
    # installed. Without this branch the script would fall through to a
    # non-sudo brew uninstall that fails again with the same 'Could not
    # remove tailscale keg!' error.
    if [ "$keg_owner" = "root" ] || [ "$keg_owner" = "<unknown>" ]; then
      confirm "Run 'sudo brew uninstall tailscale' (keg=$keg_owner)?"
      sudo brew uninstall tailscale 2>&1 || {
        echo "    ! sudo brew uninstall failed; offering manual rm -rf fallback"
        # Guard the glob so an empty/absent dir doesn't literal-expand the
        # '*' and confuse the user with a sudo error about a non-existent path.
        if [ -d /opt/homebrew/Cellar/tailscale ] && [ -n "$(ls -A /opt/homebrew/Cellar/tailscale 2>/dev/null)" ]; then
          confirm "Run 'sudo rm -rf /opt/homebrew/Cellar/tailscale/*'?"
          sudo rm -rf /opt/homebrew/Cellar/tailscale/* 2>&1
        else
```

```
          echo "     keg dir already empty or absent  nothing more to rm"
        fi
      }
    else
      confirm "Run 'brew uninstall tailscale' (removes LaunchAgent plist + linked symlinks)?"
      brew uninstall tailscale 2>&1
    fi
  fi
fi

# ---------- 4. Wipe state directories ----------
header "Step 4  Wipe stale state"
# User-owned
declare -a USER_PATHS
USER_PATHS=(
  "$HOME/.local/share/tailscale"
  "$HOME/.local/run/tailscaled.socket"
  "$HOME/.local/run/tailscaled.state"
  "$HOME/.local/state/tailscale"
  "$HOME/Library/Application Support/Tailscale"
  "$HOME/Library/Caches/com.tailscale.ipn.macos"
  "$HOME/Library/Caches/Tailscale"
  "$HOME/Library/Logs/Tailscale"
)
echo "    User-owned paths (no sudo):"
for p in "${USER_PATHS[@]}"; do
  if [ -e "$p" ]; then
    if [ "$DRY" = 1 ]; then
      echo "      [dry] rm -rf '$p'"
    else
      rm -rf "$p"
      echo "       rm -rf '$p'"
    fi
  fi
done

# System
declare -a SUDO_PATHS
SUDO_PATHS=(
  "/var/lib/tailscale"
  "/var/cache/tailscale"
)
echo "    System paths under /var (will prompt for sudo if any are present):"
eligible_sudo=0
for p in "${SUDO_PATHS[@]}"; do
  if [ -e "$p" ]; then
    eligible_sudo=1
    if [ "$DRY" = 1 ]; then
      echo "      [dry] sudo rm -rf '$p'"
    else
      if confirm "Run 'sudo rm -rf $p'?"; then
        sudo rm -rf "$p" && echo "       rm -rf '$p'"
      fi
    fi
  fi
done
[ "$eligible_sudo" = 0 ] && echo "      (none of /var/lib/tailscale, /var/cache/tailscale are present)"

# ---------- 5. Wipe stale LaunchAgent plists ----------
header "Step 5  Wipe stale LaunchAgent plist files"
declare -a PLIST_FILES
PLIST_FILES=(
  "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
  "$HOME/Library/LaunchAgents/com.tailscale.ipn.macos.plist"
  "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist.bak"
  "$HOME/Library/LaunchAgents/com.nullexit.wake-recovery.plist"
)
for f in "${PLIST_FILES[@]}"; do
  if [ -e "$f" ]; then
    if [ "$DRY" = 1 ]; then
      echo "    [dry] rm -f '$f'"
    else
      rm -f "$f"
      echo "     rm -f '$f'"
    fi
  fi
done
echo "    Remaining tailscale|nullexit plists (should be empty):"
ls -1 ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/^/      /' || echo "      (none)"

# ---------- 5.5. Drain Background-Items (Login Items) ----------
```

```
header "Step 5.5  Drain Background-Items (Login Items)"
# macOS 13+ keeps a parallel "Background Items" database alongside
# classic LaunchAgents. Even when we wipe ~/Library/LaunchAgents/homebrew.mxcl.tailscale.plist
# in Step 5, the .btm file at
# ~/Library/Application Support/com.apple.backgroundtaskmanagementagent.backgrounditems.btm
# can still hold a Background-Items entry pointing at the (now-dead) plist
# path. Next login: macOS sees the entry, tries to bootstrap, finds the
# plist missing, and either silently regenerates it via brew’s post-install
# hook (causing tailscaled respawn at next login) or refuses outright.
# ‘sfltool reset-login-items‘ is Apple’s canonical API for wiping the
# entire .btm file. It DOES wipe all Login Items  the user re-adds what
# they want via System Settings  General  Login Items. Acceptable here
# because the goal of this script is "fresh baseline for Tailscale".
BTM="$HOME/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm"
if [ -f "$BTM" ]; then
  if [ "$DRY" = 1 ]; then
    echo "    [dry] found $BTM; would prompt sfltool reset-login-items"
  else
    confirm "Run ’sfltool reset-login-items’ (wipes ALL Login Items; rebuild via System Settings  General  Login Items)?"
    if sfltool reset-login-items 2>&1; then
      echo "     Login Items drained"
    else
      echo "    ! sfltool reset-login-items failed (non-fatal; do manually: System Settings  General  Login Items)"
    fi
  fi
else
  echo "     no backgrounditems.btm present"
fi

# ---------- 6. brew install ----------
header "Step 6  brew install tailscale (fresh)"
if brew list tailscale >/dev/null 2>&1; then
  echo "     already installed  skip"
else
  confirm "Run ’brew install tailscale’?"
  if [ "$DRY" = 1 ]; then
    echo "    [dry] brew install tailscale"
  else
    brew install tailscale 2>&1
  fi
fi

# ---------- 6.5. Strip quarantine xattr from keg ----------
header "Step 6.5  Strip quarantine xattr from tailscale install"
# brew’s freshly poured bottles don’t carry com.apple.quarantine (the
# bottles are content-addressed and Homebrew-verified), but the upstream
# installer (Tailscale.app from App Store) and any manual download do.
# Strip the xattr from the entire keg as a defensive safety net so the
# freshly-installed tailscaled binary and any helper binaries are not
# Gatekeeper-blocked on first launch. Idempotent  no-op if no quarantine
# xattr is present.
quarantine_count="$(xattr -lr /opt/homebrew/Cellar/tailscale 2>/dev/null | grep -c ’^[^ ]* com.apple.quarantine’ || echo 0)"
if [ "${quarantine_count:-0}" -gt 0 ]; then
  if [ "$DRY" = 1 ]; then
    echo "    [dry] found $quarantine_count file(s) with com.apple.quarantine; would prompt xattr -dr"
  else
    confirm "Run ’xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale’? (strips quarantine xattr from $quarantine_count file(s); non-fatal if it errors)"
    if xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale 2>&1; then
      echo "     quarantine xattr stripped"
    else
      echo "    ! xattr -dr failed (non-fatal; open /opt/homebrew/Cellar/tailscale in Finder, right-click tailscaled  Open Anyway)"
    fi
  fi
else
  echo "     no com.apple.quarantine xattr on tailscale install"
fi

# ---------- 7. brew services start (NO sudo!) ----------
header "Step 7  Start the service as $USER (NO sudo, with multi-tier self-healing)"
if ! command -v tailscale >/dev/null 2>&1; then
  echo "     tailscale not on PATH after install  brew install errored; aborting"
  exit 4
fi

# Aliveness fan-out: ANY one of these is taken as proof of life. None of
# them alone is reliable  pgrep misses tailscaled if it does setproctitle
```

```
# or fork-detach; launchctl's reported PID can be a shim that exited
# within 1s; tailscale-cli hangs while macOS Local-Network permission is
# unresolved. Combined, they cover each other's blind spots.

tailscaled_api_up() {
  # Strategy A: tailscale CLI can talk to the local API. Canonical proof.
  # Bound at 8s  tailscale status itself returns quickly on success OR
  # failure, but if the post-install state is wedged it can hang.
  with_timeout 8 /opt/homebrew/bin/tailscale status >/dev/null 2>&1
}

tailscaled_proc_up() {
  # Strategy B: any process has 'tailscaled' substring in argv.
  pgrep -lf tailscaled >/dev/null 2>&1
}

tailscaled_lc_up() {
  # Strategy C: launchctl-managed service has a non-dash, non-zero PID.
  local lc_pid
  lc_pid="$(launchctl list 2>/dev/null | awk '$3 == "homebrew.mxcl.tailscale" {print $1; exit}')"
  if [ -n "$lc_pid" ] && [ "$lc_pid" != "-" ] && [ "$lc_pid" -gt 0 ] 2>/dev/null; then
    return 0
  fi
  return 1
}

tailscaled_is_alive() {
  tailscaled_api_up  && return 0
  tailscaled_proc_up && return 0
  tailscaled_lc_up   && return 0
  return 1
}

confirm "Run 'brew services start tailscale'?"
if [ "$DRY" = 1 ]; then
  echo "    [dry] brew services start tailscale"
  echo "    [dry] (then poll up to 30s for tailscaled_is_alive)"
  echo "    [dry] (recovery tier 1: launchctl kickstart)"
  echo "    [dry] (recovery tier 2: launchctl bootout + bootstrap of explicit plist)"
  echo "    [dry] (recovery tier 3: brew services restart (stop+start cycle))"
  echo "    [dry] (recovery tier 4: direct /opt/homebrew/bin/tailscaled spawn (bypasses launchd))"
else
  brew services start tailscale 2>&1
  # Primary poll  tailscaled_is_alive covers all 3 detection strategies.
  echo "     polling for tailscaled to come up (up to ~30s)"
  tailscaled_up=0
  for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; do
    if tailscaled_is_alive; then
      tailscaled_up=1
      echo "       tailscaled alive after ${i}*2s"
      break
    fi
    sleep 2
  done

  # Recovery tier 1: kickstart the launchd-managed service. This is the
  # cheapest and most-aligned-with-launchd fix when brew has loaded the
  # LaunchAgent but tailscaled didn't fork. Kickstart tells launchd
  # "re-spawn if not running" without forcing a teardown.
  if [ "$tailscaled_up" = 0 ]; then
    echo "    ! Recovery tier 1: launchctl kickstart"
    if launchctl kickstart -kp "gui/$UID/homebrew.mxcl.tailscale" 2>&1; then
      for i in 1 2 3 4 5 6 7 8 9 10; do
        if tailscaled_is_alive; then
          tailscaled_up=1
          echo "       tailscaled alive after kickstart (${i}*2s)"
          break
        fi
        sleep 2
      done
    else
      echo "      ! kickstart returned non-zero (LaunchAgent may not be loaded)"
    fi
  fi

  # Recovery tier 2: explicit plist bootout + bootstrap. Forces a fresh
  # launchd registration cycle for the brew-managed service, which fixes
  # the wider class of "brew says started but daemon never spawned"
  # wedge states that kickstart alone can't dig out of.
  if [ "$tailscaled_up" = 0 ]; then
    echo "    ! Recovery tier 2: launchctl bootout + bootstrap of explicit plist"
```

```
    launchctl bootout "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null || true
    sleep 1
    # Brew installs the canonical plist template at $HOMEBREW_PREFIX/Library/.
    # On Apple Silicon that's /opt/homebrew/Library/LaunchAgents/. On Intel
    # it's /usr/local/Library/LaunchAgents/. Detect which and pick the
    # right one so this script works across both architectures.
    if [ -f /opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
      plist_path="/opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
    elif [ -f /usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
      plist_path="/usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
    else
      plist_path=""
    fi
    if [ -n "$plist_path" ]; then
      if launchctl bootstrap "gui/$UID" "$plist_path" 2>&1; then
        for i in 1 2 3 4 5 6 7 8 9 10; do
          if tailscaled_is_alive; then
            tailscaled_up=1
            echo "       tailscaled alive after bootstrap (${i}*2s)"
            break
          fi
          sleep 2
        done
      else
        echo "      ! explicit bootstrap returned non-zero"
      fi
    else
      echo "      ! could not locate brew plist template; skipping bootstrap tier"
    fi
  fi

  # Recovery tier 3: brew services restart (forces a stop + start cycle,
  # which unsticks LaunchAgents with stale 'started' state from prior
  # half-cycles). This is the canonical brew-doc fix for 'services don't
  # actually start even though brew says started'.
  if [ "$tailscaled_up" = 0 ]; then
    echo "    ! Recovery tier 3: brew services restart (forces stop+start cycle)"
    if brew services restart tailscale 2>&1; then
      for i in 1 2 3 4 5 6 7 8 9 10; do
        if tailscaled_is_alive; then
          tailscaled_up=1
          echo "       tailscaled alive after brew restart (${i}*2s)"
          break
        fi
        sleep 2
      done
    else
      echo "      ! brew services restart returned non-zero"
    fi
  fi

  # Recovery tier 4: last-resort direct spawn of the daemon binary,
  # bypassing launchd entirely. If tailscaled crashes here too, we'll
  # capture its actual stderr in /tmp for the user to read (NOPASSWD
  # sudoers often have local-network permission denied  the captured
  # stderr makes it obvious what to fix in System Settings).
  if [ "$tailscaled_up" = 0 ]; then
    echo "    ! Recovery tier 4: direct spawn (bypasses launchd; tail stderr to /tmp)"
    /opt/homebrew/bin/tailscaled >/tmp/nullexit-tailscaled-spawn.log 2>&1 &
    spawned_pid=$!
    disown "$spawned_pid" 2>/dev/null || true
    sleep 3
    if tailscaled_is_alive; then
      tailscaled_up=1
      echo "       tailscaled alive via direct spawn (pid=$spawned_pid)"
      echo "      (NOTE: not under launchd supervision  a reboot, re-run of"
      echo "       this script, or ./toggle.sh is required to recreate."
      echo "       Last 5 lines of stderr captured to /tmp/nullexit-tailscaled-spawn.log:)"
      tail -n 5 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/^/        /' || true
    fi
  fi

  if [ "$tailscaled_up" = 0 ]; then
    echo "     HARD FAIL: tailscaled refuses to stay alive across every recovery tier"
    echo "      launchctl-managed state (if available):"
    launchctl print "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null \
      | grep -E 'state =|last exit code =|runs =|path =' \
      | sed 's/^/        /' \
      || echo "        (no launchctl state available for homebrew.mxcl.tailscale)"
    echo "      Last 10 lines of /tmp/nullexit-tailscaled-spawn.log (if any):"
```

```
    tail -n 10 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/^/        /' || echo "        (no
     log)"
    echo "      Most likely remaining cause: macOS Local Network permission denied."
    echo "      System Settings  Privacy & Security  Local Network  toggle Tailscale ON."
    exit 7
  fi
fi
sleep 3

# ---------- 8. Verify ----------
header "Step 8  Verify fresh install"
echo "    brew service tailscale: $(brew services list 2>/dev/null | awk '$1=="tailscale"{print $2, $3}'
     | tr '\n' ' ' | sed 's/ $//')"
echo "    tailscaled process:     $(pgrep -lf tailscaled 2>/dev/null | head -1 || echo 'NONE (check brew
     logs)')"
ts_ver=""
if [ -x /opt/homebrew/bin/tailscale ]; then
  ts_ver="$(/opt/homebrew/bin/tailscale version 2>/dev/null | head -1 | tr -d '\n')"
elif command -v tailscale >/dev/null 2>&1; then
  ts_ver="$(tailscale version 2>/dev/null | head -1 | tr -d '\n')"
fi
echo "    tailscale version:      ${ts_ver:-binary missing}"
ts_status=""
if [ -x /opt/homebrew/bin/tailscale ]; then
  ts_status="$(/opt/homebrew/bin/tailscale status 2>/dev/null | head -3)"
fi
if [ -n "$ts_status" ]; then
  echo "    tailscale status:"
  printf '%s\n' "$ts_status" | sed 's/^/      /'
else
  echo "    tailscale status:       (binary missing or errored)"
fi

if { [ "$DRY" = 0 ] && /opt/homebrew/bin/tailscale status 2>/dev/null | grep -q '^Tailscale is stopped';
     }; then
  echo
  echo "     NOTICE: 'tailscale status' still reports stopped after a fresh install."
  echo "       Two equally likely causes:"
  echo "         (a) macOS hasn't yet shown the first-run 'Local Network' prompt for"
  echo "             Tailscale  open System Settings  Privacy & Security  Local"
  echo "             Network. Toggle Tailscale ON."
  echo "         (b) macOS denied Local Network permission at some prior point."
  echo "       Either way: System Settings  Privacy & Security  Local Network."
  echo "       If the entry isn't present, invoking any tailscale command will"
  echo "       re-trigger the prompt automatically."
fi

# ---------- 9. Done ----------
header "Done"
if [ "$DRY" = 1 ]; then
  echo "    (dry-mode finished; no follow-up commands were issued.)"
  exit 0
fi

cat <<'DONE'

  Manual follow-ups (all of these; the script stops short of running sudo
   tailscale up so you can verify the brew install is actually live first):

   1.  System Settings  Privacy & Security  Local Network  Tailscale ON
      macOS will normally pop the prompt the first time you run `tailscale`.
      If it didn't, trigger it via the menu bar  check.

   2.  Re-engage the tailnet + exit node:
          sudo tailscale up --ssh=true --accept-dns=false \
            --exit-node="$(cat .gateway_ip | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
            --exit-node-allow-lan-access=true

   3.  Re-run: ./toggle.sh
      (re-installs the com.nullexit.wake-recovery LaunchAgent  wiped
      in Step 5 to clear the slate  and re-engages the gateway)

   4.  Final verification:
          curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
        expect: warp=on, ip=<Cloudflare IP>, colo=YYZ/YUL

   Container-side (gateway / 100.100.21.8) Tailscale is unaffected by this
   script. Other tailnet devices (S24, iPhone, Windows) are unaffected.

DONE
```

42 scripts/routing-fix.sh

```
#!/bin/sh
# routing-fix: Maintains routing for SOCKS5 proxy traffic through WARP (tun0)
#              and FORWARD rules for Tailscale exit-node return traffic.
#
# Runs inside the warp container’s network namespace. This script ensures:
# 1. Table 200 has default via tun0 (for SOCKS5 proxy traffic from 172.18.0.2)
#    with the WARP endpoint exception via eth0 (prevents tunnel loop)
# 2. The CGNAT ip rule (100.64.0.0/10  table 52  tailscale routes) is maintained for Tailscale
# 3. FORWARD RELATED,ESTABLISHED rule allows return traffic for exit-node
#    forwarded connections (S24  tailscale0  eth0  host  internet)
#
# CRITICAL: Does NOT touch the main table’s default route. Gluetun handles that
# internally via its own routing rules and WireGuard fwmark (0xca6c  table 51820).
# Overriding the main table default would create a loop: encrypted WireGuard
# packets exiting tun0 would match default dev tun0 and re-enter the tunnel.

set -e

trap ’echo "routing-fix: Caught signal, exiting..."; exit 0’ TERM INT QUIT

sleep 15 &
wait $! || true

echo "routing-fix: Setting up routing for SOCKS5 proxy through WARP (tun0)..."

# Dynamically detect Docker subnet from eth0
DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk ’{print $1}’)
DOCKER_GW=$(ip route show default | head -1 | awk ’{print $3}’)
WARP_ENDPOINT="${WARP_ENDPOINT}"
TAILSCALE_ALLOW_LAN_P2P="${TAILSCALE_ALLOW_LAN_P2P:-false}"
IP_BLOCKLIST_FILE="/userfilters/ip_blocklist.ipset"
LAST_IP_MTIME=""
echo "routing-fix: Docker network: ${DOCKER_NET}, gateway: ${DOCKER_GW}"

# Table 200: Used by ip rule 100 (from 172.18.0.2) for SOCKS5 proxy traffic
# WARP endpoint goes via eth0 to avoid tunnel loop, everything else via tun0
ip route add "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || \
  ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
ip route replace default dev tun0 table 200 2>/dev/null || true

# Main table: Just ensure the Docker subnet route exists via eth0
# (gluetun owns the default route here  do NOT touch it)
ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true

# FORWARD RELATED,ESTABLISHED: Allow return traffic for exit-node connections.
# The container has BOTH iptables (nftables backend) and iptables-legacy
# loaded. Gluetun’s post-rules.txt uses the nftables backend, while Tailscale
# uses the legacy backend. Packets go through BOTH stacks, so we add the rule
# to both to ensure it survives regardless of which backend has policy DROP.
#
# Packet flow: S24 outgoing (tailscale0->eth0) ACCEPTed by first rule.
# Return traffic (eth0->eth0 via table 52 to host) needs RELATED,ESTABLISHED
# to match the conntrack entry created by the outgoing flow.
add_fwd_related_established() {
  # nftables backend (used by gluetun’s post-rules.txt)
  if ! iptables -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
    iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
  fi
  # legacy backend (used by Tailscale’s ts-forward)
  if command -v iptables-legacy >/dev/null 2>&1; then
    if ! iptables-legacy -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
      iptables-legacy -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
    fi
  fi
}

add_fwd_related_established

# Policy Routing for Tailscale P2P:
# Tailscale sends its encrypted mesh UDP packets from port 41642.
# If these go out tun0 (WARP), Cloudflare’s strict NAT drops the returning
# hole-punch packets, causing Tailscale to fail P2P and fall back to DERP relays (high latency).
# We mark packets originating from port 41642 and force them out the main table (eth0).
#
# LAN P2P is controlled by two sources (in priority order):
#   1. /app/.lan_p2p_detected  written by watcher.sh on macOS on every network change
#      using airport -I (802.1x detection) + AP isolation probe. Overrides .env.
```

```
#   2. TAILSCALE_ALLOW_LAN_P2P env var (from .env)  static fallback.
# When disabled, we explicitly DROP local RFC1918-destined Tailscale UDP so the
# phone cleanly falls back to DERP rather than getting poisoned by macOS gvproxy SNAT.
add_tailscale_p2p_bypass() {
  # Dynamic override from watcher.sh (network auto-detection) takes priority over .env
  local allow_p2p="${TAILSCALE_ALLOW_LAN_P2P:-false}"
  if [ -f "/app/.lan_p2p_detected" ]; then
    allow_p2p=$(cat "/app/.lan_p2p_detected" 2>/dev/null | tr -d ’[:space:]’)
  fi

  # Always allow P2P directly to the Docker host gateway (the Mac running this container).
  # The gateway container is always co-located with the host Mac on the same machine,
  # so direct Tailscale P2P via the Docker bridge is always safe and avoids DERP overhead.
  # This RETURN rule must be inserted BEFORE the general DROP rules so it takes priority.
  local default_gw
  default_gw=$(ip route show default | awk ’{print $3}’)
  if [ -n "$default_gw" ]; then
    if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null; then
      iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null || true
    fi
  fi

  # Also explicitly whitelist all physical IPs of the host Mac (e.g. 172.17.52.223).
  # The Tailscale control plane registers the Mac using its physical IP, so the
  # container will attempt P2P handshakes using that physical IP instead of the
  # Docker bridge IP. If this physical IP falls within 172.16.0.0/12, it would be dropped.
  if [ -f "/app/.host_ips" ]; then
    while IFS= read -r host_ip; do
      [ -z "$host_ip" ] && continue
      if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null; then
        iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null || true
      fi
    done < "/app/.host_ips"
  fi

  if [ "${allow_p2p}" != "true" ]; then
    # Drop Tailscale UDP packets destined for local subnets to prevent macOS gvproxy SNAT
    # endpoint poisoning. When disabled (campus/enterprise WPA2-Enterprise with AP isolation),
    # dropping these forces the S24 to use DERP relays reliably.
    for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
      if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; then
        iptables -t mangle -A OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null || true
      fi
    done
  else
    # Remove DROP rules if they exist (switching from restricted to allowed)
    for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
      while iptables -t mangle -D OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; do :; done
    done
  fi

  # Bypass STUN and P2P packets to public IPs (DERP relays and remote peers) out eth0
  if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null; then
    iptables -t mangle -A OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null || true
  fi
  if ! ip rule show | grep -q ’fwmark 0x8888’; then
    ip rule add fwmark 0x8888 lookup 254 pref 98 2>/dev/null || true
  fi
}

add_tailscale_p2p_bypass

add_onion_transparent_proxy() {
  for ipt in iptables iptables-legacy; do
    command -v "$ipt" >/dev/null 2>&1 || continue
    if ! $ipt -t nat -C PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null; then
      $ipt -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null || true
    fi
    if ! $ipt -C FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null; then
      $ipt -I FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null || true
    fi
  done
}

add_tailscale_p2p_bypass
add_onion_transparent_proxy
```

```

create_log_drop_chain() {
  local ipt="$1"
  local chain="$2"
  local prefix="$3"

  if ! $ipt -N "$chain" 2>/dev/null; then
    $ipt -F "$chain" 2>/dev/null || true
  fi
  $ipt -A "$chain" -j LOG --log-prefix "$prefix"
  $ipt -A "$chain" -j DROP 2>/dev/null || true
}

add_country_block() {
  # To add a country to the blocklist, simply define the ’BLOCKED_COUNTRIES’ variable in .env with 2-letter ISO country codes.
  # You can find the country code by looking at the filename on http://www.ipdeny.com/ipblocks/data/countries/
  # For example, to block China (cn) and Russia (ru), set: BLOCKED_COUNTRIES="cn ru" in your .env file
  for zone in $BLOCKED_COUNTRIES; do
    set_name="block_${zone}"
    zone_upper=$(echo "$zone" | tr ’a-z’ ’A-Z’)
    chain_name="LOG_DROP_${zone_upper}"
    log_prefix="IP_BLOCK_${zone_upper}: "

    # Create the ipset if it doesn’t exist
    if ! ipset list "$set_name" >/dev/null 2>&1; then
      ipset create "$set_name" hash:net 2>/dev/null || true
      echo "routing-fix: Downloading ${zone_upper} IPs for blocklist..."
      curl -sS "http://www.ipdeny.com/ipblocks/data/countries/${zone}.zone" | grep -v ’^#’ | while read -r subnet; do
        [ -n "$subnet" ] && ipset add "$set_name" "$subnet" 2>/dev/null || true
      done
    fi

    # Apply LOG_DROP rule to both backends
    for ipt in iptables iptables-legacy; do
      command -v "$ipt" >/dev/null 2>&1 || continue

      create_log_drop_chain "$ipt" "$chain_name" "$log_prefix"

      # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
      while $ipt -D FORWARD -m set --match-set "$set_name" dst -j DROP 2>/dev/null; do :; done

      if ! $ipt -C FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null; then
        $ipt -I FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null || true
      fi
    done
  done
}

add_ip_blocklist() {
  # Only reload when the compiled file has actually changed (mtime check).
  # This prevents a pointless ipset restore on every 30-second loop tick.
  if [ ! -f "$IP_BLOCKLIST_FILE" ]; then
    return 0
  fi

  CURRENT_MTIME=$(stat -c %Y "$IP_BLOCKLIST_FILE" 2>/dev/null || echo "0")
  if [ "$CURRENT_MTIME" != "$LAST_IP_MTIME" ]; then
    echo "routing-fix: IP blocklist changed, reloading..."
    # ipset restore handles the atomic swap internally:
    # create_new  populate  swap with live  destroy_new
    if ipset restore < "$IP_BLOCKLIST_FILE" 2>/dev/null; then
      LAST_IP_MTIME="$CURRENT_MTIME"
      echo "routing-fix: IP blocklist loaded ($(ipset list block_malicious | grep -c ’^[0-9]’) entries)."
    else
      echo "routing-fix: Warning: ipset restore failed. Retrying next cycle."
      return 1
    fi
  fi

  # Apply FORWARD rules in BOTH iptables backends.
  for ipt in iptables iptables-legacy; do
    command -v "$ipt" >/dev/null 2>&1 || continue

    create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_DST "IP_BLOCK_MALICIOUS_DST: "
    create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_SRC "IP_BLOCK_MALICIOUS_SRC: "

    # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
    while $ipt -D FORWARD -m set --match-set block_malicious dst -j DROP 2>/dev/null; do :; done
```

```
    while $ipt -D FORWARD -m set --match-set block_malicious src -j DROP 2>/dev/null; do :; done

    if ! $ipt -C FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null; then
      $ipt -I FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null || true
    fi

    if ! $ipt -C FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null; then
      $ipt -I FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null || true
    fi
  done
}

# Start background block logger (DNS + IP)
if ! pgrep -f "nullexit-logger" >/dev/null; then
  echo "routing-fix: Starting background block logger..."
  nullexit-logger &
fi

add_country_block
add_ip_blocklist

echo 'routing-fix: Routes applied.'

UNHEALTHY_COUNT=0

# Re-assert loop (every 30 seconds)
while true; do
  # Table 200 routes
  ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true

  # Tunnel health check
  if ! curl -m 3 -sS --interface tun0 https://cloudflare.com/cdn-cgi/trace >/dev/null 2>&1; then
    UNHEALTHY_COUNT=$((UNHEALTHY_COUNT + 1))
  else
    UNHEALTHY_COUNT=0
    if [ -f "/app/TUNNEL_FAILED_CLOSED.marker" ]; then
      rm -f "/app/TUNNEL_FAILED_CLOSED.marker"
    fi
  fi

  if [ "$UNHEALTHY_COUNT" -ge 3 ]; then
    echo "routing-fix: Tunnel health check failed $UNHEALTHY_COUNT times. Failing closed."
    ip route del default dev tun0 table 200 2>/dev/null || true
    ip route replace blackhole default table 200 2>/dev/null || true
    touch "/app/TUNNEL_FAILED_CLOSED.marker"
  else
    ip route replace default dev tun0 table 200 2>/dev/null || true
  fi

  # Main table: keep Docker subnet route
  ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true

  # CGNAT ip rule for Tailscale
  if ! ip rule show | grep -q '100.64.0.0/10'; then
    ip rule add to 100.64.0.0/10 lookup 52 pref 99 2>/dev/null || true
    echo 'routing-fix: CGNAT rule missing, re-injected'
  fi

  # FORWARD RELATED,ESTABLISHED: Allow return traffic in both iptables
  # backends. Gluetun may reset the nftables ruleset on VPN reconnect,
  # and Tailscale may reset the legacy ruleset on auth refresh.
  add_fwd_related_established

  # Ensure P2P packets bypass WARP to maintain low latency
  add_tailscale_p2p_bypass

  # Ensure transparent proxying for .onion is active
  add_onion_transparent_proxy

  # Enforce country blocklist
  add_country_block

  # Enforce IP blocklist (reloads automatically when file changes)
  add_ip_blocklist

  sleep 30 &
  wait $! || true
```

```
done
```

43 scripts/rule-compiler/go.mod

```
module nullexit/rule-compiler

go 1.22
```

44 scripts/rule-compiler/main.go

```
package main

import (
  "bufio"
  "bytes"
  "crypto/md5"
  "encoding/base64"
  "encoding/hex"
  "fmt"
  "io"
  "net/http"
  "net/netip"
  "os"
  "os/exec"
  "path/filepath"
  "regexp"
  "sort"
  "strings"
  "sync"
  "time"
)

const (
  BlackListPath = "black_list.txt"
  WhiteListPath = "white_list.txt"
  OutputPath    = "adguard/work/userfilters/compiled_rules.txt"
  IpOutputPath  = "adguard/work/userfilters/ip_blocklist.ipset"
  IpCacheDir    = "adguard/work/userfilters/cache/ip"
  CacheDir      = "adguard/work/userfilters/cache"
)

var CoreLists = []string{
  "https://raw.githubusercontent.com/anudeepND/blacklist/master/adservers.txt",
  "https://raw.githubusercontent.com/anudeepND/blacklist/master/facebook.txt",
  "https://raw.githubusercontent.com/crazy-max/WindowsSpyBlocker/master/data/hosts/spy.txt",
  "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.samsung.txt",
  "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.apple.txt",
  "https://abp.oisd.nl/basic/",
  "https://adaway.org/hosts.txt",
  "https://pgl.yoyo.org/adservers/serverlist.php?hostformat=adblockplus&showintro=1&mimetype=plaintext",
  "https://raw.githubusercontent.com/nextdns/cname-cloaking-blocklist/master/domains",
}

var MediumAdditions = []string{
  "https://raw.githubusercontent.com/lightswitch05/hosts/master/docs/lists/facebook-extended.txt",
  "https://someonewhocares.org/hosts/zero/hosts",
  "https://urlhaus.abuse.ch/downloads/hostfile/",
}

var HeavyAdditions = []string{
  "https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts",
  "https://raw.githubusercontent.com/Perflyst/PiHoleBlocklist/master/SmartTV-AGH.txt",
  "https://raw.githubusercontent.com/DandelionSprout/adfilt/master/GameConsoleAdblockList.txt",
}

var IpBlockLists = []string{
  "https://feodotracker.abuse.ch/downloads/ipblocklist.txt",
  "https://www.spamhaus.org/drop/drop.txt",
  "https://rules.emergingthreats.net/fwrules/emerging-Block-IPs.txt",
  "https://cinsscore.com/list/ci-badguys.txt",
}

```

```go
func getProfiles() map[string][]string {
  profiles := make(map[string][]string)

  light := append([]string(nil), CoreLists...)
  light = append(light, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/light.txt")
  profiles["light"] = light

  medium := append([]string(nil), CoreLists...)
  medium = append(medium, MediumAdditions...)
  medium = append(medium, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/multi.txt")
  profiles["medium"] = medium

  heavy := append([]string(nil), CoreLists...)
  heavy = append(heavy, MediumAdditions...)
  heavy = append(heavy, HeavyAdditions...)
  heavy = append(heavy, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/pro.txt")
  profiles["heavy"] = heavy

  return profiles
}

func loadEnvProfile() string {
  profile := "medium"

  if bytesData, err := os.ReadFile(".env"); err == nil {
    scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
    for scanner.Scan() {
      line := strings.TrimSpace(scanner.Text())
      if line != "" && !strings.HasPrefix(line, "#") {
        parts := strings.SplitN(line, "=", 2)
        if len(parts) == 2 {
          key := strings.TrimSpace(parts[0])
          val := strings.TrimSpace(parts[1])
          if key == "GATEWAY_RULE_PROFILE" {
            val = strings.Trim(val, "\"’")
            profile = strings.ToLower(val)
          }
        }
      }
    }
  } else if !os.IsNotExist(err) {
    fmt.Printf("Warning: Failed to read .env file for profile configuration (%v). Using default.\n", err)
  }

  if envVal := os.Getenv("GATEWAY_RULE_PROFILE"); envVal != "" {
    profile = strings.ToLower(envVal)
  }

  profiles := getProfiles()
  if _, exists := profiles[profile]; !exists {
    fmt.Printf("Warning: Profile ’%s’ is invalid. Falling back to ’medium’.\n", profile)
    profile = "medium"
  }
  return profile
}

func loadDomains(filepath string) map[string]bool {
  domains := make(map[string]bool)
  if bytesData, err := os.ReadFile(filepath); err == nil {
    scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
    for scanner.Scan() {
      line := strings.TrimSpace(scanner.Text())
      if line != "" && !strings.HasPrefix(line, "#") {
        domains[strings.ToLower(line)] = true
      }
    }
  }
  return domains
}

func parseDomainsFromContent(content string) map[string]bool {
  domains := make(map[string]bool)
  scanner := bufio.NewScanner(strings.NewReader(content))
  ipRegex := regexp.MustCompile(‘^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$‘)
  agRegex := regexp.MustCompile(‘^\|\|([a-z0-9.-]+)\^$‘)
  domainRegex := regexp.MustCompile(‘^[a-z0-9.-]+$‘)

  for scanner.Scan() {
    line := strings.TrimSpace(scanner.Text())
    if line == "" || strings.HasPrefix(line, "!") || strings.HasPrefix(line, "[") {
```

```go
      continue
    }
    parts := strings.SplitN(line, "#", 2)
    line = strings.TrimSpace(parts[0])
    if line == "" {
      continue
    }

    fields := strings.Fields(line)
    var rule string
    if len(fields) >= 2 {
      if ipRegex.MatchString(fields[0]) {
        rule = fields[1]
      } else {
        rule = fields[0]
      }
    } else {
      rule = fields[0]
    }

    rule = strings.TrimSpace(strings.ToLower(rule))

    var domain string
    if m := agRegex.FindStringSubmatch(rule); m != nil {
      domain = m[1]
    } else if domainRegex.MatchString(rule) {
      domain = rule
    } else {
      domain = rule
    }

    if domain != "" && domain != "localhost" && domain != "0.0.0.0" && domain != "127.0.0.1" && domain != "broadcasthost" {
      domains[domain] = true
    }
  }
  return domains
}

// WARNING: Parsing AdGuardHome.yaml using regex instead of a proper YAML parser
// is brittle and assumes the exact format currently emitted by AdGuard.
// If an AdGuard version bump updates the configuration schema format,
// this parser will fail silently or loudly and should be refactored to use gopkg.in/yaml.v3.
func getAdguardNativeLists() []string {
  yamlPath := "adguard/conf/AdGuardHome.yaml"
  var enabledUrls []string
  if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
    return enabledUrls
  }

  contentBytes, err := os.ReadFile(yamlPath)
  if err != nil {
    fmt.Printf("Warning: Failed to parse AdGuardHome.yaml for native lists (%v)\n", err)
    return enabledUrls
  }
  content := string(contentBytes)

  filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*?)(?:^[a-zA-Z_]+:|\z)`)
  match := filtersRegex.FindStringSubmatch(content)
  if len(match) > 1 {
    filtersText := match[1]
    blocks := strings.Split(filtersText, "\n  - ")
    urlRegex := regexp.MustCompile(`url:\s*(\S+)`)

    for _, block := range blocks {
      if strings.TrimSpace(block) == "" {
        continue
      }
      if strings.Contains(block, "enabled: true") {
        urlMatch := urlRegex.FindStringSubmatch(block)
        if len(urlMatch) > 1 {
          url := urlMatch[1]
          if !strings.Contains(url, "compiled_rules.txt") {
            enabledUrls = append(enabledUrls, url)
          }
        }
      }
    }
  }
  return enabledUrls
}
```

```go

func getAdguardCompiledRulesCachePath() string {
  yamlPath := "adguard/conf/AdGuardHome.yaml"
  if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
    return ""
  }

  contentBytes, err := os.ReadFile(yamlPath)
  if err != nil {
    fmt.Printf("Warning: Failed to resolve compiled_rules.txt cache path (%v)\n", err)
    return ""
  }
  content := string(contentBytes)

  filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*?)(?:^[a-zA-Z_]+:|\z)`)
  match := filtersRegex.FindStringSubmatch(content)
  if len(match) > 1 {
    blocks := strings.Split(match[1], "\n  - ")
    idRegex := regexp.MustCompile(`id:\s*(\d+)`)
    for _, block := range blocks {
      if !strings.Contains(block, "compiled_rules.txt") {
        continue
      }
      idMatch := idRegex.FindStringSubmatch(block)
      if len(idMatch) > 1 {
        return fmt.Sprintf("adguard/work/data/filters/%s.txt", idMatch[1])
      }
    }
  }
  return ""
}

func handleFetchError(urlStr string, cacheFile string, err error) map[string]bool {
  if _, statErr := os.Stat(cacheFile); statErr == nil {
    fmt.Printf(" -> Warning: Failed to fetch %s (%v). Falling back to expired local cache.\n", urlStr, err)
    content, readErr := os.ReadFile(cacheFile)
    if readErr == nil {
      domains := parseDomainsFromContent(string(content))
      fmt.Printf(" -> Loaded from expired cache (%d domains).\n", len(domains))
      return domains
    }
    fmt.Printf(" -> Warning: Failed to read expired cache (%v). Using local lists only.\n", readErr)
  } else {
    fmt.Printf(" -> Warning: Failed to fetch %s (%v). Using local lists only for this source.\n", urlStr, err)
  }
  return make(map[string]bool)
}

func fetchRemoteDomains(urlStr string) map[string]bool {
  os.MkdirAll(CacheDir, 0755)

  hash := md5.Sum([]byte(urlStr))
  urlHash := hex.EncodeToString(hash[:])
  cacheFile := filepath.Join(CacheDir, fmt.Sprintf("%s.txt", urlHash))

  if stat, err := os.Stat(cacheFile); err == nil {
    fileAge := time.Since(stat.ModTime()).Seconds()
    if fileAge < 86400 {
      fmt.Printf("Loading remote blacklist from cache: %s\n", urlStr)
      content, err := os.ReadFile(cacheFile)
      if err == nil {
        domains := parseDomainsFromContent(string(content))
        fmt.Printf(" -> Loaded from cache (copy is %.1f hours old, %d domains).\n", fileAge/3600.0, len(domains))
        return domains
      }
      fmt.Printf(" -> Warning: Failed to read cache file %s (%v). Re-fetching...\n", cacheFile, err)
    }
  }

  fmt.Printf("Fetching remote blacklist from: %s ...\n", urlStr)

  req, err := http.NewRequest("GET", urlStr, nil)
  if err != nil {
    return handleFetchError(urlStr, cacheFile, err)
  }
  req.Header.Set("User-Agent", "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)")

  client := &http.Client{Timeout: 15 * time.Second}
```

```go
	resp, err := client.Do(req)
	if err != nil {
		return handleFetchError(urlStr, cacheFile, err)
	}
	defer resp.Body.Close()

	if resp.StatusCode != 200 {
		return handleFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
	}

	bodyBytes, err := io.ReadAll(resp.Body)
	if err != nil {
		return handleFetchError(urlStr, cacheFile, err)
	}

	content := string(bodyBytes)
	domains := parseDomainsFromContent(content)

	if len(domains) < 10 {
		return handleFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only found %d domains. Possible 404 or bad URL", len(domains)))
	}

	err = os.WriteFile(cacheFile, bodyBytes, 0644)
	if err != nil {
		fmt.Printf(" -> Warning: Failed to save cache file (%v)\n", err)
	}

	fmt.Printf(" -> Successfully fetched and cached %d domains.\n", len(domains))
	return domains
}

func optimizeSubdomains(domains map[string]bool, listName string) map[string]bool {
	fmt.Printf("Optimizing %s by removing redundant subdomains...\n", listName)
	rawCount := len(domains)
	if rawCount == 0 {
		return make(map[string]bool)
	}

	optimized := make(map[string]bool)
	for domain := range domains {
		if strings.ContainsAny(domain, "|^@$/") {
			optimized[domain] = true
			continue
		}

		parts := strings.Split(domain, ".")
		hasParent := false

		for i := 1; i < len(parts)-1; i++ {
			parent := strings.Join(parts[i:], ".")
			if domains[parent] {
				hasParent = true
				break
			}
		}

		if !hasParent {
			optimized[domain] = true
		}
	}

	saved := rawCount - len(optimized)
	reduction := float64(0)
	if rawCount > 0 {
		reduction = (float64(saved) / float64(rawCount)) * 100
	}
	fmt.Printf(" -> Reduced %s from %d to %d domains (-%d / %.1f%% reduction).\n", listName, rawCount, len(optimized), saved, reduction)
	return optimized
}

func parseIpsFromContent(content string) map[string]bool {
	ips := make(map[string]bool)
	scanner := bufio.NewScanner(strings.NewReader(content))

	for scanner.Scan() {
		line := strings.TrimSpace(scanner.Text())
		if line == "" || strings.HasPrefix(line, "#") || strings.HasPrefix(line, ";") {
			continue
		}
```

```go

    fields := strings.Fields(line)
    if len(fields) > 0 {
      token := strings.TrimRight(fields[0], ";,")
      if _, err := netip.ParsePrefix(token); err == nil {
        ips[token] = true
      } else if _, err := netip.ParseAddr(token); err == nil {
        ips[token] = true
      }
    }
  }
  return ips
}

func handleIpFetchError(urlStr string, cacheFile string, err error) map[string]bool {
  if _, statErr := os.Stat(cacheFile); statErr == nil {
    fmt.Printf(" -> Warning: fetch failed (%v). Falling back to stale cache.\n", err)
    content, readErr := os.ReadFile(cacheFile)
    if readErr == nil {
      ips := parseIpsFromContent(string(content))
      fmt.Printf(" -> %d IPs loaded from stale cache.\n", len(ips))
      return ips
    }
    fmt.Printf(" -> Warning: stale cache also failed (%v).\n", readErr)
  } else {
    fmt.Printf(" -> Warning: %s failed (%v). No cache available. Skipping.\n", urlStr, err)
  }
  return make(map[string]bool)
}

func fetchRemoteIps(urlStr string) map[string]bool {
  os.MkdirAll(IpCacheDir, 0755)

  hash := md5.Sum([]byte(urlStr))
  urlHash := hex.EncodeToString(hash[:])
  cacheFile := filepath.Join(IpCacheDir, fmt.Sprintf("%s.txt", urlHash))

  if stat, err := os.Stat(cacheFile); err == nil {
    fileAge := time.Since(stat.ModTime()).Seconds()
    if fileAge < 86400 {
      fmt.Printf("Loading IP feed from cache: %s\n", urlStr)
      content, err := os.ReadFile(cacheFile)
      if err == nil {
        ips := parseIpsFromContent(string(content))
        fmt.Printf(" -> %d IPs (cache is %.1fh old).\n", len(ips), fileAge/3600.0)
        return ips
      }
      fmt.Printf(" -> Warning: cache read failed (%v). Re-fetching...\n", err)
    }
  }

  fmt.Printf("Fetching IP feed: %s ...\n", urlStr)
  req, err := http.NewRequest("GET", urlStr, nil)
  if err != nil {
    return handleIpFetchError(urlStr, cacheFile, err)
  }
  req.Header.Set("User-Agent", "Mozilla/5.0")

  client := &http.Client{Timeout: 15 * time.Second}
  resp, err := client.Do(req)
  if err != nil {
    return handleIpFetchError(urlStr, cacheFile, err)
  }
  defer resp.Body.Close()

  if resp.StatusCode != 200 {
    return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
  }

  bodyBytes, err := io.ReadAll(resp.Body)
  if err != nil {
    return handleIpFetchError(urlStr, cacheFile, err)
  }

  content := string(bodyBytes)
  ips := parseIpsFromContent(content)
  if len(ips) < 1 {
    return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only %d IPs found. Possible 404", len(ips)))
  }

```

```go
  err = os.WriteFile(cacheFile, bodyBytes, 0644)
  if err != nil {
    fmt.Printf(" -> Warning: cache write failed (%v)\n", err)
  }

  fmt.Printf(" -> %d IPs fetched and cached.\n", len(ips))
  return ips
}

func compileIpBlocklist() int {
  fmt.Println("\n IP Blocklist Compilation ")

  allIps := make(map[string]bool)
  var mu sync.Mutex
  var wg sync.WaitGroup

  maxThreads := 8
  if len(IpBlockLists) < 8 {
    maxThreads = len(IpBlockLists)
  }
  semaphore := make(chan struct{}, maxThreads)

  for _, urlStr := range IpBlockLists {
    wg.Add(1)
    go func(u string) {
      defer wg.Done()
      semaphore <- struct{}{}
      defer func() { <-semaphore }()

      ips := fetchRemoteIps(u)
      mu.Lock()
      for ip := range ips {
        allIps[ip] = true
      }
      mu.Unlock()
    }(urlStr)
  }
  wg.Wait()

  fmt.Printf("Total unique entries before filtering: %d\n", len(allIps))

  reservedStr := []string{
    "10.0.0.0/8",
    "172.16.0.0/12",
    "192.168.0.0/16",
    "127.0.0.0/8",
    "169.254.0.0/16",
    "100.64.0.0/10",
    "0.0.0.0/8",
  }
  var reserved []netip.Prefix
  for _, s := range reservedStr {
    p, _ := netip.ParsePrefix(s)
    reserved = append(reserved, p)
  }

  cleanIps := make(map[string]bool)
  for entry := range allIps {
    var prefix netip.Prefix
    p, err := netip.ParsePrefix(entry)
    if err != nil {
      addr, err2 := netip.ParseAddr(entry)
      if err2 != nil {
        continue
      }
      prefix = netip.PrefixFrom(addr, addr.BitLen())
    } else {
      prefix = p
    }

    overlaps := false
    for _, r := range reserved {
      if r.Overlaps(prefix) {
        overlaps = true
        break
      }
    }
    if !overlaps {
      cleanIps[entry] = true
    }
  }
```

```go

  removed := len(allIps) - len(cleanIps)
  if removed > 0 {
    fmt.Printf(" -> Removed %d private/reserved entries.\n", removed)
  }
  fmt.Printf(" -> Final IP blocklist: %d entries.\n", len(cleanIps))

  os.MkdirAll(filepath.Dir(IpOutputPath), 0755)

  f, err := os.Create(IpOutputPath)
  if err != nil {
    fmt.Printf("Error writing IP output file: %v\n", err)
    return len(cleanIps)
  }
  defer f.Close()

  f.WriteString("# nullexit Compiled IP Blocklist\n")
  f.WriteString("# Sources: Feodo Tracker, Spamhaus DROP/EDROP, Emerging Threats, CINS\n")
  f.WriteString(fmt.Sprintf("# Entries: %d\n\n", len(cleanIps)))

  f.WriteString("create block_malicious_new hash:net maxelem 200000 -exist\n")

  var sortedIps []string
  for ip := range cleanIps {
    sortedIps = append(sortedIps, ip)
  }
  sort.Strings(sortedIps)

  for _, ip := range sortedIps {
    f.WriteString(fmt.Sprintf("add block_malicious_new %s -exist\n", ip))
  }

  f.WriteString("create block_malicious hash:net maxelem 200000 -exist\n")
  f.WriteString("swap block_malicious block_malicious_new\n")
  f.WriteString("destroy block_malicious_new\n")

  fmt.Printf("IP blocklist written to %s\n", IpOutputPath)
  return len(cleanIps)
}

func main() {
  profile := loadEnvProfile()
  fmt.Printf("Active memory profile: ’%s’\n", strings.ToUpper(profile))

  localBlackList := loadDomains(BlackListPath)
  whiteList := loadDomains(WhiteListPath)

  blackList := make(map[string]bool)
  for k := range localBlackList {
    blackList[k] = true
  }

  profiles := getProfiles()
  urlsToFetch := profiles[profile]

  fmt.Printf("\nStarting concurrent downloads for %d lists...\n", len(urlsToFetch))
  startTime := time.Now()

  var mu sync.Mutex
  var wg sync.WaitGroup

  maxThreads := 16
  if len(urlsToFetch) < 16 {
    maxThreads = len(urlsToFetch)
  }
  semaphore := make(chan struct{}, maxThreads)

  for _, urlStr := range urlsToFetch {
    wg.Add(1)
    go func(u string) {
      defer wg.Done()
      semaphore <- struct{}{}
      defer func() { <-semaphore }()

      domains := fetchRemoteDomains(u)
      mu.Lock()
      for d := range domains {
        blackList[d] = true
      }
      mu.Unlock()
    }(urlStr)
```

```go
  }
  wg.Wait()

  fmt.Printf("Finished concurrent downloads in %.2f seconds using %d threads.\n", time.Since(startTime).Seconds(), maxThreads)

  adguardNativeUrls := getAdguardNativeLists()
  var adguardNativeDomains map[string]bool
  if len(adguardNativeUrls) > 0 {
    fmt.Printf("\nFetching %d AdGuard native list(s) for deduplication...\n", len(adguardNativeUrls))
    adguardNativeDomains = make(map[string]bool)

    maxNativeThreads := 4
    if len(adguardNativeUrls) < 4 {
      maxNativeThreads = len(adguardNativeUrls)
    }
    nativeSemaphore := make(chan struct{}, maxNativeThreads)

    for _, urlStr := range adguardNativeUrls {
      wg.Add(1)
      go func(u string) {
        defer wg.Done()
        nativeSemaphore <- struct{}{}
        defer func() { <-nativeSemaphore }()

        domains := fetchRemoteDomains(u)
        mu.Lock()
        for d := range domains {
          adguardNativeDomains[d] = true
        }
        mu.Unlock()
      }(urlStr)
    }
    wg.Wait()

    if len(adguardNativeDomains) > 0 {
      fmt.Println("Deduplicating compiled rules against AdGuard native lists...")
      originalSize := len(blackList)
      for d := range adguardNativeDomains {
        delete(blackList, d)
      }
      fmt.Printf(" -> Removed %d redundant rules already covered by AdGuard.\n", originalSize-len(blackList))
    }
  }

  var contradictions []string
  for d := range whiteList {
    if blackList[d] {
      contradictions = append(contradictions, d)
    }
  }

  if len(contradictions) > 0 {
    fmt.Printf("Detected %d contradictions. Whitelist taking priority.\n", len(contradictions))
    sort.Strings(contradictions)
    for _, domain := range contradictions {
      if localBlackList[domain] {
        fmt.Printf(" -> Removed local blacklist domain: %s\n", domain)
      }
      delete(blackList, domain)
    }
  }

  blackList = optimizeSubdomains(blackList, "blacklist")
  whiteList = optimizeSubdomains(whiteList, "whitelist")

  os.MkdirAll(filepath.Dir(OutputPath), 0755)
  outFile, err := os.Create(OutputPath)
  if err != nil {
    fmt.Printf("Error creating output file: %v\n", err)
    return
  }
  defer outFile.Close()

  outFile.WriteString("! Custom Compiled Rules (Auto-Generated)\n")
  outFile.WriteString(fmt.Sprintf("! Memory Profile: %s\n", strings.ToUpper(profile)))
  outFile.WriteString(fmt.Sprintf("! Total Block Rules: %d\n", len(blackList)))
  outFile.WriteString(fmt.Sprintf("! Native AdGuard Rules: %d\n", len(adguardNativeDomains)))
  outFile.WriteString(fmt.Sprintf("! Total Whitelist Rules: %d\n\n", len(whiteList)))

```

```go
	outFile.WriteString("! --- Blacklist Rules ---\n")

	var sortedBlacklist []string
	for d := range blackList {
		sortedBlacklist = append(sortedBlacklist, d)
	}
	sort.Strings(sortedBlacklist)

	for _, domain := range sortedBlacklist {
		if strings.Contains(domain, "$") {
			parts := strings.SplitN(domain, "$", 2)
			dom, mod := parts[0], parts[1]
			if strings.HasPrefix(dom, "||") {
				if strings.HasSuffix(dom, "^") {
					outFile.WriteString(fmt.Sprintf("%s$%s\n", dom, mod))
				} else {
					outFile.WriteString(fmt.Sprintf("%s^$%s\n", dom, mod))
				}
			} else {
				outFile.WriteString(fmt.Sprintf("||%s^$%s\n", dom, mod))
			}
		} else if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(
		 domain, "|") || strings.HasSuffix(domain, "^") {
			outFile.WriteString(fmt.Sprintf("%s\n", domain))
		} else {
			outFile.WriteString(fmt.Sprintf("||%s^\n", domain))
		}
	}

	outFile.WriteString("\n! --- Whitelist Rules ---\n")
	var sortedWhitelist []string
	for d := range whiteList {
		sortedWhitelist = append(sortedWhitelist, d)
	}
	sort.Strings(sortedWhitelist)

	for _, domain := range sortedWhitelist {
		if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|")
		  || strings.HasSuffix(domain, "^") || strings.HasPrefix(domain, "@@") {
			rule := domain
			if !strings.HasPrefix(domain, "@@") {
				rule = "@@" + domain
			}
			outFile.WriteString(fmt.Sprintf("%s\n", rule))
		} else {
			outFile.WriteString(fmt.Sprintf("@@||%s^\n", domain))
		}
	}

	fmt.Printf("\nSuccessfully compiled %d block rules and %d allow rules to %s\n", len(blackList), len(
	 whiteList), OutputPath)

	cachedFilterPath := getAdguardCompiledRulesCachePath()
	if cachedFilterPath != "" {
		input, err := os.ReadFile(OutputPath)
		if err == nil {
			err = os.WriteFile(cachedFilterPath, input, 0644)
			if err == nil {
				fmt.Printf("Updated AdGuard filter cache: %s\n", cachedFilterPath)
			} else {
				fmt.Printf("Warning: Could not update AdGuard filter cache %s (%v)\n", cachedFilterPath, err)
			}
		} else {
			fmt.Printf("Warning: Could not read OutputPath to copy to cache (%v)\n", err)
		}
	} else {
		fmt.Println("Warning: Could not determine compiled_rules.txt cache path from AdGuardHome.yaml;
		 skipping cache update.")
	}

	adguardReachable := false

	dockerComposeYamlExists := false
	if _, err := os.Stat("docker-compose.yml"); err == nil {
		dockerComposeYamlExists = true
	}
	dockerPath, err := exec.LookPath("docker")
	if err == nil && dockerComposeYamlExists {
		cmd := exec.Command(dockerPath, "compose", "ps", "--status", "running", "-q", "adguardhome")
		out, err := cmd.Output()
		if err == nil && strings.TrimSpace(string(out)) != "" {
```

```go
      fmt.Println("Force-recreating AdGuard Home container to drop persisted filter state...")
      upCmd := exec.Command(dockerPath, "compose", "up", "-d", "--force-recreate", "--no-deps", "adguardhome")
      err := upCmd.Run()
      if err == nil {
        fmt.Println("AdGuard Home force-recreated successfully.")
        time.Sleep(8 * time.Second)
        adguardReachable = true
      } else {
        fmt.Println("Failed to restart AdGuard Home. Is it running?")
      }
    }
  }

  if adguardReachable {
    agPass := "nullexit"
    if bytesData, err := os.ReadFile(".env"); err == nil {
      scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
      for scanner.Scan() {
        line := scanner.Text()
        if strings.HasPrefix(line, "ADGUARD_PASSWORD=") {
          parts := strings.SplitN(line, "=", 2)
          if len(parts) == 2 {
            agPass = strings.Trim(strings.TrimSpace(parts[1]), "\"'")
          }
          break
        }
      }
    }

    creds := base64.StdEncoding.EncodeToString([]byte(fmt.Sprintf("admin:%s", agPass)))
    req, err := http.NewRequest("POST", "http://127.0.0.1:3000/control/filtering/refresh", bytes.NewBuffer([]byte("{}")))
    if err == nil {
      req.Header.Set("Authorization", fmt.Sprintf("Basic %s", creds))
      req.Header.Set("Content-Type", "application/json")
      client := &http.Client{Timeout: 15 * time.Second}
      resp, err := client.Do(req)
      if err == nil {
        fmt.Printf("Triggered AdGuard filter refresh via REST API (HTTP=%d).\n", resp.StatusCode)
        resp.Body.Close()
      } else {
        fmt.Printf("Warning: AdGuard filter refresh API call failed (%v). New whitelist will not load until next refresh tick (~24h) or manual refresh.\n", err)
      }
    }
  }

  compileIpBlocklist()
}
```

45 scripts/setup-common.sh

```bash
#!/bin/bash
#  1. Docker
step "Checking Docker"

if ! command -v docker >> output.log 2>&1; then
    echo ""
    if [[ "$OSTYPE" == "darwin"* ]]; then
        echo "  Docker is not installed."
        # Note: Homebrew aggressively rolls forward and discourages version pinning.
        # This setup is confirmed stable on Colima v0.10.3 and Docker v29.6.1.
        read -rp "  Would you like to automatically install Colima & Docker via Homebrew? [y/N]: " INSTALL_DOCKER
        if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
            step "Installing Colima and Docker..."
            brew install colima docker docker-compose
            step "Starting Colima VM..."
            colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged
        else
            echo ""
            echo "  Please install Docker manually. Two options:"
            echo "  A) Colima (Recommended): brew install colima docker docker-compose && colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged"
            echo "  B) Docker Desktop: https://www.docker.com/products/docker-desktop/"
```

```
            die "Install Docker, then re-run this script."
        fi
    else
        echo "  Docker is not installed."
        # Note: The Docker convenience script always fetches the latest stable release.
        # This setup is confirmed stable on Docker Engine v29.6.1.
        read -rp "  Would you like to automatically install Docker? (Requires sudo) [y/N]: " INSTALL_DOCKER
        if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
            step "Installing Docker..."
            curl -fsSL https://get.docker.com | sh
            sudo usermod -aG docker "$USER"
            echo -e "\n\033[0;32m[OK] Docker installed successfully!\033[0m"
            echo -e "\033[0;33m[!] CRITICAL: You must log out of your SSH session and log back in for Docker permissions to apply.\033[0m"
            die "Please log out, log back in, and re-run setup.sh."
        else
            echo "  Please install Docker manually:"
            echo "       curl -fsSL https://get.docker.com | sh"
            echo "       sudo usermod -aG docker \$USER && newgrp docker"
            die "Install Docker, then re-run this script."
        fi
    fi
fi

if ! docker info >> output.log 2>&1; then
    echo ""
    if [[ "$OSTYPE" == "darwin"* ]]; then
        echo "  Docker is installed but not running."
        echo "  If using Colima:        colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged"
        echo "  If using Docker Desktop: open the app."
    else
        echo "  Docker is installed but not running."
        echo "  Run: sudo systemctl start docker"
    fi
    die "Start Docker, then re-run this script."
fi

if ! docker compose version >> output.log 2>&1; then
    die "Docker Compose v2 not found. Update Docker or install the compose plugin:\n  https://docs.docker.com/compose/install/"
fi

ok "Docker $(docker --version | grep -oE ’[0-9.]+’ | head -1) is running."

#  1.5 Python 3
step "Checking Python 3"

if ! command -v python3 >> output.log 2>&1; then
    echo ""
    if [[ "$OSTYPE" == "darwin"* ]]; then
        echo "  Python 3 is not installed."
        echo "  macOS includes it via Xcode Command Line Tools, or you can install it via Homebrew:"
        echo "       brew install python3"
        die "Install Python 3, then re-run this script."
    else
        echo "  Python 3 is not installed."
        echo "  Please install it using your system’s package manager. For example:"
        echo "       Debian/Ubuntu:  sudo apt install python3"
        echo "       Fedora:         sudo dnf install python3"
        echo "       Arch:           sudo pacman -S python"
        die "Install Python 3, then re-run this script."
    fi
fi

# Note: This setup is confirmed stable on Python 3.9.6 (macOS default).
PYTHON_VER=$(python3 --version 2>&1 | head -n 1)
ok "$PYTHON_VER is installed."

#  2. Tailscale (host)
step "Checking Tailscale (host)"
# The host needs its own Tailscale installation  separate from the Docker
# container  so Toggle-Gateway can run ’tailscale down’ before stopping
# containers and ’tailscale up’ after starting them. Without this, the
# exit-node deadlock that setup.sh is designed to prevent can still occur.

# Resolve the tailscale binary: CLI (brew/package) or bundled macOS app
TAILSCALE_BIN=""
if command -v tailscale >> output.log 2>&1; then
    TAILSCALE_BIN="tailscale"
```

```
elif [[ -x "/Applications/Tailscale.app/Contents/MacOS/Tailscale" ]]; then
    TAILSCALE_BIN="/Applications/Tailscale.app/Contents/MacOS/Tailscale"
fi

RECOMMENDED_TS_VERSION="1.98.5"

if [[ -z "$TAILSCALE_BIN" ]]; then
    warn "Tailscale not found on host  installing..."

    if [[ "$OSTYPE" == "darwin"* ]]; then
        if command -v brew >> output.log 2>&1; then
            step "Installing Tailscale CLI formula via Homebrew (standalone, no .app GUI)..."
            # Note: We install Tailscale via Homebrew, targeting the stable version (recommended: 1.98.5)
            brew install tailscale -q
            TAILSCALE_BIN="tailscale"

            step "Starting tailscaled as a per-user LaunchAgent (auto-starts at login)..."
            if brew services start tailscale 2>&1 | grep -qE 'Successfully|started'; then
                ok "tailscaled LaunchAgent registered."
            else
                warn "Could not auto-start tailscaled. Run 'brew services start tailscale' manually."
                warn "For a system-wide LaunchDaemon that runs before login, run instead:"
                warn "    sudo brew services start tailscale"
            fi
            echo ""
        else
            die "Homebrew not found. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):\n  https://tailscale.com/download/mac\n  Then re-run this script."
        fi

    elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
        curl -fsSL https://tailscale.com/install.sh | sh
        sudo systemctl enable --now tailscaled 2>> output.log || true
        TAILSCALE_BIN="tailscale"

    else
        die "Unsupported OS. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):\n  https://tailscale.com/download\n  Then re-run this script."
    fi

    TAILSCALE_VER=$(${TAILSCALE_BIN} version 2>> output.log | head -n 1)
    if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
        warn "Tailscale installed, but version ($TAILSCALE_VER) differs from recommended version ($RECOMMENDED_TS_VERSION)."
    else
        ok "Tailscale installed (version $TAILSCALE_VER)."
    fi
else
    TAILSCALE_VER=$(${TAILSCALE_BIN} version 2>> output.log | head -n 1)
    if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
        warn "Tailscale is already installed ($TAILSCALE_VER), but recommended version is $RECOMMENDED_TS_VERSION for host/container consistency."
    else
        ok "Tailscale is already installed at recommended version ($TAILSCALE_VER)."
    fi
fi

# Authenticate the host machine if it isn't already connected.
# This is an interactive step  it opens a browser login URL.
# We do it now, before containers start, so Toggle-Gateway can
# safely run 'tailscale down/up' from day one.
if ! ${TAILSCALE_BIN} status >> output.log 2>&1; then
    echo ""
    echo "  Your host machine needs to join your Tailscale network."
    echo "  A browser window or login URL will appear  authenticate, then"
    echo "  return here. The script will continue automatically."
    echo ""
    if [[ "$OSTYPE" == "linux-gnu"* ]]; then
        sudo ${TAILSCALE_BIN} up
    else
        ${TAILSCALE_BIN} up
    fi
    ok "Host machine connected to Tailscale."
else
    ok "Host machine is already connected to Tailscale."
fi

#  3. wgcf
step "Checking wgcf (Cloudflare WARP key tool)"

if ! command -v wgcf >> output.log 2>&1; then
```

```
    warn "wgcf not found  installing..."

    if [[ "$OSTYPE" == "darwin"* ]]; then
        command -v brew >> output.log 2>&1 || die "Homebrew not found.\nInstall wgcf manually: https://github.com/ViRb3/wgcf/releases"
        brew install wgcf -q

    elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
        ARCH=$(uname -m)
        case $ARCH in
            x86_64)  WGCF_ARCH="amd64" ;;
            aarch64) WGCF_ARCH="arm64" ;;
            armv7l)  WGCF_ARCH="armv7" ;;
            *) die "Unsupported architecture: $ARCH.\nInstall wgcf manually: https://github.com/ViRb3/wgcf/releases" ;;
        esac

        echo "  Fetching latest release..."
        WGCF_VERSION=$(curl -sf https://api.github.com/repos/ViRb3/wgcf/releases/latest \
            | grep '"tag_name"' | cut -d'"' -f4)
        [[ -z "$WGCF_VERSION" ]] && die "Could not fetch wgcf version. Check your internet connection."

        sudo curl -sfL \
            "https://github.com/ViRb3/wgcf/releases/download/${WGCF_VERSION}/wgcf_${WGCF_VERSION#v}_linux_${WGCF_ARCH}" \
            -o /usr/local/bin/wgcf
        sudo chmod +x /usr/local/bin/wgcf
    else
        die "Unsupported OS. Install wgcf manually: https://github.com/ViRb3/wgcf/releases"
    fi
fi

ok "wgcf is ready."

#  4. WARP profile
step "Generating Cloudflare WARP profile"

if [[ -f "wgcf-profile.conf" ]]; then
    warn "wgcf-profile.conf already exists  skipping key generation."
else
    wgcf register --accept-tos 2>> output.log || die "wgcf register failed. Check your internet connection."
    wgcf generate 2>> output.log           || die "wgcf generate failed."
    ok "WARP profile generated."
fi

# Parse keys (IPv4 address only  gluetun doesn't need the IPv6 one)
PRIVATE_KEY=$(awk '/PrivateKey/{print $3}' wgcf-profile.conf)
PUBLIC_KEY=$(awk '/PublicKey/{print $3}'   wgcf-profile.conf)
ADDRESSES=$(awk '/Address/{print $3}'       wgcf-profile.conf | grep -v ':' | head -1)

[[ -z "$PRIVATE_KEY" ]] && die "Could not parse PrivateKey from wgcf-profile.conf"
[[ -z "$PUBLIC_KEY" ]]  && die "Could not parse PublicKey from wgcf-profile.conf"
[[ -z "$ADDRESSES" ]]   && die "Could not parse IPv4 Address from wgcf-profile.conf"

ok "WARP keys extracted."

#  5. Tailscale auth key
step "Tailscale authentication"

TS_AUTHKEY=""
if [[ -f ".env" ]]; then
    EXISTING_TS_KEY=$(read_env_var TS_AUTHKEY)
    if [[ -n "$EXISTING_TS_KEY" ]]; then
        TS_AUTHKEY="$EXISTING_TS_KEY"
        warn "Found existing TS_AUTHKEY in .env. Skipping prompt."
    fi
fi

if [[ -z "$TS_AUTHKEY" ]]; then
    echo ""
    echo "  Generate a key at: https://login.tailscale.com/admin/settings/keys"
    echo "   'Generate auth key'  enable Reusable if you plan to re-run setup"
    echo ""
    read -rp "  Paste your Tailscale auth key: " TS_AUTHKEY
    echo ""
fi

[[ -z "$TS_AUTHKEY" ]] && die "Tailscale auth key is required."
[[ "$TS_AUTHKEY" != tskey-auth-* ]] && warn "Key doesn't look like a Tailscale auth key  proceeding anyway."
```

```
ok "Auth key accepted."

#  6. AdGuard Home admin password
step "AdGuard Home admin account"

# Hardcoded default credentials to prevent lockout
# Username: admin
# Password: nullexit
if [[ -f ".env" ]]; then
    EXISTING_AG_PASS=$(read_env_var ADGUARD_PASSWORD)
    if [[ -n "$EXISTING_AG_PASS" ]]; then
        ADGUARD_PASSWORD="$EXISTING_AG_PASS"
        ADGUARD_PWD_ESC="$EXISTING_AG_PASS"
        ok "Found existing ADGUARD_PASSWORD in .env."
    else
        ADGUARD_PASSWORD="nullexit"
        ADGUARD_PWD_ESC="nullexit"
        ok "Default password set to ’nullexit’."
    fi
else
    ADGUARD_PASSWORD="nullexit"
    ADGUARD_PWD_ESC="nullexit"
    ok "Default password set to ’nullexit’."
fi


#  7. Write .env
step "Writing .env"

if [[ -f ".env" ]]; then
    read -rp "  .env already exists. Overwrite? [y/N]: " OVERWRITE
    if [[ "$OVERWRITE" != "y" && "$OVERWRITE" != "Y" ]]; then
        warn "Keeping existing .env."
    else
        WRITE_ENV=1
    fi
else
    WRITE_ENV=1
fi

if [[ "${WRITE_ENV:-0}" == "1" ]]; then
    # Preserve existing configuration flags if .env already exists
    EXISTING_PROFILE="medium"
    EXISTING_MSS="1120"
    EXISTING_HIJACK="true"
    EXISTING_EXIT="true"
    EXISTING_THRESH="6"
    EXISTING_KILL="false"
    EXISTING_BLOCKED="kp il"
    EXISTING_HOST_MTU="1200"

    if [[ -f ".env" ]]; then
        [[ -n "$(read_env_var GATEWAY_RULE_PROFILE)" ]] && EXISTING_PROFILE=$(read_env_var GATEWAY_RULE_PROFILE)
        [[ -n "$(read_env_var GATEWAY_MSS)" ]] && EXISTING_MSS=$(read_env_var GATEWAY_MSS)
        [[ -n "$(read_env_var GATEWAY_HIJACK_HOST)" ]] && EXISTING_HIJACK=$(read_env_var GATEWAY_HIJACK_HOST)
        [[ -n "$(read_env_var GATEWAY_USE_EXIT_NODE)" ]] && EXISTING_EXIT=$(read_env_var GATEWAY_USE_EXIT_NODE)
        [[ -n "$(read_env_var WARP_FAIL_THRESHOLD)" ]] && EXISTING_THRESH=$(read_env_var WARP_FAIL_THRESHOLD)
        [[ -n "$(read_env_var KILL_SWITCH)" ]] && EXISTING_KILL=$(read_env_var KILL_SWITCH)
        [[ -n "$(read_env_var BLOCKED_COUNTRIES)" ]] && EXISTING_BLOCKED=$(read_env_var BLOCKED_COUNTRIES)
        [[ -n "$(read_env_var HOST_MTU)" ]] && EXISTING_HOST_MTU=$(read_env_var HOST_MTU)
    fi

    cat > .env <<EOF
WIREGUARD_PRIVATE_KEY=${PRIVATE_KEY}
WIREGUARD_PUBLIC_KEY=${PUBLIC_KEY}
WIREGUARD_ADDRESSES=${ADDRESSES}
TS_AUTHKEY=${TS_AUTHKEY}
GATEWAY_RULE_PROFILE=${EXISTING_PROFILE}
# Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds and have a healthy path.
GATEWAY_MSS=${EXISTING_MSS}
# Lower the host Tailscale MTU to 1200 to prevent fragmentation when passing through the 1280-byte WARP tunnel.
HOST_MTU=${EXISTING_HOST_MTU}
# Set to false to run as a ’Headless Server’ (Docker acts as an exit node, but the host Mac/Linux’s own internet is NOT hijacked).
```

```
GATEWAY_HIJACK_HOST=${EXISTING_HIJACK}
GATEWAY_USE_EXIT_NODE=${EXISTING_EXIT}
WARP_FAIL_THRESHOLD=${EXISTING_THRESH}
KILL_SWITCH=${EXISTING_KILL}
ADGUARD_PASSWORD=${ADGUARD_PASSWORD}
BLOCKED_COUNTRIES="${EXISTING_BLOCKED}"
EOF
    ok ".env written."
fi

#  8. Prepare directories
step "Preparing AdGuard directories"

mkdir -p adguard/conf adguard/work/userfilters

# Remove empty AdGuardHome.yaml that would cause a crash loop (same logic as Toggle-Gateway scripts)
if [[ -f "adguard/conf/AdGuardHome.yaml" && ! -s "adguard/conf/AdGuardHome.yaml" ]]; then
    rm "adguard/conf/AdGuardHome.yaml"
    warn "Removed empty AdGuardHome.yaml to prevent crash loop."
fi

ok "Directories ready."

#  9. Compile DNS filter rules
step "Compiling DNS filter rules (this may take a minute)"


#  10. Start containers
step "Starting containers"
# Note: tailscale depends on adguardhome being healthy (port 5335 up),
# so Docker Compose will hold it back automatically until the wizard is done.
docker compose up -d
ok "Containers starting."

#  11. Wait for AdGuard Home setup wizard
step "Waiting for AdGuard Home setup wizard"
echo "  (up to 60 seconds...)"

MAX=60; ELAPSED=0
until docker exec routing-fix curl -sf http://127.0.0.1:3000 >> output.log 2>&1; do
    sleep 2; ELAPSED=$((ELAPSED + 2))
    [[ $ELAPSED -ge $MAX ]] && die "AdGuard Home didn’t come up.\nDebug with: docker compose logs adguardhome"
done

ok "AdGuard Home is up."

#  12. Configure AdGuard Home via API
step "Configuring AdGuard Home"

# Run all API calls in a single docker exec sh -c so the session cookie file
# is shared across calls without leaving the container.
docker exec routing-fix sh -c "
  set -e

  # Step 1: Complete the setup wizard (sets admin creds + DNS port to 5335)
  curl -sf -X POST http://127.0.0.1:3000/control/install/configure \
    -H ’Content-Type: application/json’ \
    -d ’{\"web\":{\"ip\":\"0.0.0.0\",\"port\":3000},\"dns\":{\"ip\":\"0.0.0.0\",\"port\":5335},\"username\":\"admin\",\"password\":\"${ADGUARD_PWD_ESC}\"}’ \
    >/dev/null || true

  # Step 2: Wait for AdGuard to restart after wizard
  sleep 4
  WAITED=0
  until curl -sf http://127.0.0.1:3000 >/dev/null 2>&1; do
    sleep 2; WAITED=\$((WAITED + 2))
    [ \$WAITED -ge 30 ] && { echo ’AdGuard did not restart in time’; exit 1; }
  done

  # Step 3: Login to get session cookie
  curl -sf -X POST http://127.0.0.1:3000/control/login \
    -H ’Content-Type: application/json’ \
    -d ’{\"name\":\"admin\",\"password\":\"${ADGUARD_PWD_ESC}\"}’ \
    -c /tmp/agh.cookie >/dev/null

  # Step 4: Point upstream DNS back through the WARP tunnel
  curl -sf -X POST http://127.0.0.1:3000/control/dns_config \
    -H ’Content-Type: application/json’ \
    -b /tmp/agh.cookie \
```

```
    -d ’{\"upstream_dns\":[\"127.0.0.1:53\",\"[/onion/]127.0.0.1:5353\"],\"bootstrap_dns\":[\"1.1.1.1\",\"8.8.8.8\"],\"upstream_mode\":\"load_balance\"}’ \
    >/dev/null

  # Step 5: Register the compiled rules file as a filter list
  curl -sf -X POST http://127.0.0.1:3000/control/filtering/add_url \
    -H ’Content-Type: application/json’ \
    -b /tmp/agh.cookie \
    -d ’{\"name\":\"Custom Compiled Rules\",\"url\":\"/opt/adguardhome/work/userfilters/compiled_rules.txt\"}’ \
    >/dev/null || true

  # Step 6: Force a filter refresh to load the rules immediately
  curl -sf -X POST http://127.0.0.1:3000/control/filtering/refresh \
    -H ’Content-Type: application/json’ \
    -b /tmp/agh.cookie \
    -d ’{\"whitelist\":false}’ \
    >/dev/null || true

  rm -f /tmp/agh.cookie
"

ok "AdGuard Home configured."

#  13. Wait for Tailscale
step "Waiting for Tailscale to authenticate"
echo "  (Tailscale only starts once AdGuard’s healthcheck passes  up to 90s)"

MAX=90; ELAPSED=0; TS_IP=""
until [[ -n "$TS_IP" ]]; do
    TS_IP=$(docker exec tailscale tailscale ip -4 2>> output.log || true)
    if [[ -z "$TS_IP" ]]; then
        sleep 3; ELAPSED=$((ELAPSED + 3))
        if [[ $ELAPSED -ge $MAX ]]; then
            warn "Tailscale hasn’t authenticated yet."
            warn "Once it does, re-run the toggle script  it will resolve the IP dynamically."
            break
        fi
    fi
done

if [[ -n "$TS_IP" ]]; then
    ok "Tailscale IP: ${TS_IP} (resolved dynamically by the toggle script on each startup)"
fi
```

46 scripts/unlock-files.sh

```
#!/bin/bash
# Resolve project root relative to this script’s location, so the script
# works regardless of the caller’s CWD.
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

echo "Unlocking .env..."
if [ -f "$PROJECT_ROOT/.env" ]; then
  sudo cat "$PROJECT_ROOT/.env" > "$PROJECT_ROOT/.env.tmp" && mv "$PROJECT_ROOT/.env.tmp" "$PROJECT_ROOT/.env"
  echo " .env unlocked"
fi

echo "Unlocking AdGuard cache files..."
for f in "$PROJECT_ROOT"/adguard/work/userfilters/compiled_rules.txt \
         "$PROJECT_ROOT"/adguard/work/userfilters/ip_blocklist.ipset \
         "$PROJECT_ROOT"/adguard/work/userfilters/cache/*.txt \
         "$PROJECT_ROOT"/adguard/work/userfilters/cache/ip/*.txt \
         "$PROJECT_ROOT"/adguard/work/data/filters/*.txt; do
  if [ -f "$f" ]; then
    cat "$f" > "$f.tmp" && mv "$f.tmp" "$f"
    echo " Unlocked $f"
  fi
done

echo ""
echo "All locked files have been successfully bypassed via atomic rename!"
```

47 tor-bridges.txt

```
```

48 white_list.txt

```
0.client-channel.google.com
1drv.com
2.android.pool.ntp.org
akamaihd.net
akamaitechnologies.com
akamaized.net
amazonaws.com
android.clients.google.com
api-adservices.apple.com
api.ipify.org
api.rlje.net
app-api.ted.com
appleid.apple.com
apps.skype.com
appsbackup-pa.clients6.google.com
appsbackup-pa.googleapis.com
appspot-preview.l.google.com
apt.sonarr.tv
aspnetcdn.com
attestation.xboxlive.com
ax.phobos.apple.com.edgesuite.net
books-analytics-events.apple.com
brightcove.net
c.s-microsoft.com
cdn.cloudflare.net
cdn.embedly.com
cdn.optimizely.com
cdn.vidible.tv
cdn2.optimizely.com
cdn3.optimizely.com
cdnjs.cloudflare.com
cert.mgt.xboxlive.com
clientconfig.passport.net
clients1.google.com
clients2.google.com
clients3.google.com
clients4.google.com
clients5.google.com
clients6.google.com
connectivitycheck.android.com
connectivitycheck.gstatic.com
continuum.dds.microsoft.com
cpms.spop10.ams.plex.bz
cpms35.spop10.ams.plex.bz
cse.google.com
ctldl.windowsupdate.com
cws.conviva.com
d2c8v52ll5s99u.cloudfront.net
d2gatte9o95jao.cloudfront.net
dashboard.plex.tv
dataplicity.com
def-vef.xboxlive.com
delivery.vidible.tv
dev.virtualearth.net
device.auth.xboxlive.com
display.ugc.bazaarvoice.com
displaycatalog.mp.microsoft.com
dl.delivery.mp.microsoft.com
dl.dropbox.com
dl.dropboxusercontent.com
dns.msftncsi.com
download.sonarr.tv
drift.com
driftt.com
dynupdate.no-ip.com
ecn.dev.virtualearth.net
edge.api.brightcove.com
eds.xboxlive.com
fonts.gstatic.com
forums.sonarr.tv
```

```
g.live.com
geo-prod.do.dsp.mp.microsoft.com
geo3.ggpht.com
gfwsl.geforce.com
giphy.com
github.com
github.io
googleapis.com
gravatar.com
gstatic.com
help.ui.xboxlive.com
hls.ted.com
i.ytimg.com
i1.ytimg.com
iadsdk.apple.com
imagesak.secureserver.net
img.vidible.tv
imgix.net
imgs.xkcd.com
instantmessaging-pa.googleapis.com
intercom.io
jquery.com
jsdelivr.net
keystone.mwbsys.com
lastfm-img2.akamaized.net
licensing.xboxlive.com
live.com
livepassdl.conviva.com
login.live.com
login.microsoftonline.com
manifest.googlevideo.com
meta-db-worker02.pop.ric.plex.bz
meta.plex.bz
meta.plex.tv
microsoftonline.com
msftncsi.com
my.plexapp.com
nexusrules.officeapps.live.com
nine.plugins.plexapp.com
no-ip.com
node.plexapp.com
notes-analytics-events.apple.com
notify.xboxlive.com
npr-news.streaming.adswizz.com
ns1.dropbox.com
ns2.dropbox.com
o1.email.plex.tv
o2.sg0.plex.tv
ocsp.apple.com
office.com
office.net
office365.com
officeclient.microsoft.com
om.cbsi.com
onedrive.live.com
outlook.live.com
outlook.office365.com
pings.conviva.com
placehold.it
placeholdit.imgix.net
players.brightcove.net
pricelist.skype.com
products.office.com
proxy.plex.bz
proxy.plex.tv
proxy02.pop.ord.plex.bz
pubsub.plex.bz
pubsub.plex.tv
raw.githubusercontent.com
redirector.googlevideo.com
res.cloudinary.com
s.gateway.messenger.live.com
s.marketwatch.com
s.youtube.com
s.ytimg.com
s1.wp.com
s2.youtube.com
s3.amazonaws.com
sa.symcb.com
secure.avangate.com
secure.brightcove.com
```

```
secure.surveymonkey.com
services.sonarr.tv
skyhook.sonarr.tv
spclient.wg.spotify.com
ssl.p.jwpcdn.com
staging.plex.tv
status.plex.tv
t.co
t0.ssl.ak.dynamic.tiles.virtualearth.net
t0.ssl.ak.tiles.virtualearth.net
tawk.to
tedcdn.com
themoviedb.com
thetvdb.com
tinyurl.com
title.auth.xboxlive.com
title.mgt.xboxlive.com
traffic.libsyn.com
tvdb2.plex.tv
tvthemes.plexapp.com
twimg.com
ui.skype.com
video-stats.l.google.com
videos.vidible.tv
vidtech.cbsinteractive.com
weather-analytics-events.apple.com
widget-cdn.rpxnow.com
win10.ipv6.microsoft.com
wp.com
ws.audioscrobbler.com
www.dataplicity.com
www.googleapis.com
www.msftconnecttest.com
www.msftncsi.com
www.no-ip.com
www.youtube-nocookie.com
xbox.ipv6.microsoft.com
xboxexperiencesprod.experimentation.xboxlive.com
xflight.xboxlive.com
xkms.xboxlive.com
xsts.auth.xboxlive.com
youtu.be
youtube-nocookie.com
yt3.ggpht.com
zee.cws.conviva.com

# YouTube family  base domains so every subdomain (video stream servers on
# *.googlevideo.com, image hosts on *.ytimg.com, profile thumbnails on
# *.ggpht.com, etc.) is automatically allowlisted via the AdGuard ’||domain^’
# wildcard. The specific s.youtube.com / manifest.googlevideo.com entries
# above are kept for clarity but will be optimized away by rule-compiler
# because their parents are now in this list.
youtube.com
googlevideo.com
ytimg.com
ggpht.com
gvt1.com
gvt2.com
gvt3.com
gvt1.net
gvt2.net
gvt3.net

edge-mqtt.facebook.com
gateway.instagram.com
i.instagram.com

facebook.com
fb.com
fbcdn.net
fbsbx.com
messenger.com
whatsapp.com
web.whatsapp.com
instagram.com
cdninstagram.com
```